基于原始感官评分与缺陷约束的啤酒风味预测和风格迁移

摘要

针对啤酒感官分级、关键化合物识别和风格迁移问题，本文结合竞赛附件与公开的逐人感官评分，建立从评分重建到迁移可行性分析的研究流程。首先核查 250 款啤酒的样本对应关系，利用 2197 条原始评分构造各感官属性的 0—6 分面板均分和七级评分分布。以啤酒为单位进行三折五次重复验证，极端随机树与梯度提升树的等权融合在 30 个 Aroma/Flavour 核心属性上的均分预测平均绝对误差为 0.50222，在全部 50 项上的误差为 0.43560；核心属性的风格中位数基线误差为 0.50969，二者差值的固定预测条件下 95% 重采样区间为 [-0.01914，0.00418]。七级分布模型在核心属性上的累计分布平方距离为 0.02178，对应的个人评分期望 RPS 为 0.11044；核心属性的面板众数一致率为 80.52%，非零众数一致率为 24.33%。其次，构建化合物相关图并比较图约束与普通岭回归：真实图的单折平均误差为 0.537035，无图为 0.536911；按酒聚合重复预测后分别为 0.524865 和 0.526148。最终因素模型选择图惩罚系数 0，150 条 Top-3 结果中 36 条满足描述性支持条件。最后，对 Tripel、Kriek 和 Lambic 生成风味谱，并在冻结候选后进行独立角色复核；真实留出来源的 10 条重复迁移记录涉及 7 款酒，其中 1 条满足全部代理条件。浓度域混配在四项浓度均不增加的严格约束下，对 55 条来源与划分记录均未找到正的目标风格份额；改用阈值超额不增加的化学风险约束后，参考阈值下 30 条记录允许正份额。研究结果为感官预测、因素解释及迁移约束设计提供了可复核的定量依据。

关键词：啤酒风味；感官评分；有序分布；化合物相关网络；风格迁移；缺陷约束

目录

1 问题分析 **1**

2 数据处理与评分端点 **1**

2.1 样本与字段核查 1

2.2 原始评分对应与人员权重 1

2.3 化学尺度与异常 1

3 第一问：感官等级与面板均分预测 **2**

3.1 七级分布与评价指标 2

3.2 面板均分预测 2

3.3 预测区间与缺失压力 2

4 第二问：化合物图与关键因素 **3**

4.1 图构建和模型 3

4.2 因素稳定性与化学解释 3

5 第三问：风味生成与迁移可行性 **3**

5.1 风格条件生成 3

5.2 冻结候选与代理复核 4

5.3 严格浓度约束与阈值风险约束 4

6 验证范围与应用建议 **4**

7 结论 **5**

参考文献 **5**

A 全部 50 项 Top-3 与结果索引 **6**

B 完整源程序 **8**

1 问题分析

啤酒风味由挥发性化合物、理化条件及其相互作用共同影响。题目要求完成三项任务：根据化合物数据预测 0—6 分的感官等级；利用化合物相关关系识别各感官维度的三个关键因素，并论证稳定性和化学合理性；在给定风格下生成化合物谱与感官画像，研究 Pils/Lager 向 Tripel 的迁移及缺陷控制。

题面描述数据约 226 款酒、Aroma 和 Flavour 合计约 40 项；实际用于建模的附件包含 250 款酒、231 个数值输入和 50 个感官目标，其中 Aroma 与 Flavour 前缀各 15 项。本文对全部 50 项目标建模与输出，同时单独报告 30 个 Aroma/Flavour 核心属性。原始个人评分来自作者公开数据 [1][2]，附件中的标准化分值用于样本对应核查，不作为 0—6 分值直接使用。

三问共用同一数据与验证框架。第一问分别建立面板均分和个人评分分布模型；第二问将训练折内的化合物网络引入多目标回归，并以无图和打乱图作对照；第三问将生成、候选搜索、代理评价和浓度域可行性分析依次连接。各方法基于极端随机树、梯度提升、岭回归、图正则化和保形预测等已有方法 [3][4][5][6][7]，本文的工作重点是将这些方法与题目的评分尺度、缺陷约束和验证条件对应起来。

2 数据处理与评分端点

2.1 样本与字段核查

附件 1 包含 250 行、234 列，其中 3 列为样本元信息；附件 4 包含 250 行、53 列。两表按唯一的 beer 字段对齐，并核对风格标签。评分卡与监督表中存在命名差异，例如 A/F_ester_fruity 对应 A/F_esters_fruity，4vg 对应 X4vg。评分卡的乙醛 Aroma/Flavour 位置没有独立监督列，Barnyard 汇总公式存在疑似引用偏差，相关字段按原始记录保留。

将附件 2 的相关矩阵与附件 1 重算结果比较，矩阵与 Spearman 秩相关的平均绝对差约为 1.58×10^{-16}，与 Pearson 相关的平均绝对差约为 0.0873。因此，本文将附件 2 解释为秩相关矩阵。对建模输入，所有截尾界限、中位数和标准化参数均只由训练数据估计；测试样本使用对应训练参数变换。部分化合物字段为对数浓度，负值可对应小于 1 的原始浓度。

2.2 原始评分对应与人员权重

作者公开数据含 2197 条逐人评分、16 名匿名评分人员和 250 款啤酒，评分取值为 0—6。根据公开补充表行序建立原始编号与附件 beer 的对应关系，再把 50 个属性分成互不重叠的两组进行一对一匹配核查，两组结果均与行序对应一致。38 个无人员内零方差问题的属性重现公开连续标签，最大误差为 2.8×10^{-8}；其余 12 项涉及零方差，采用部分样本估计替代常数并在其余样本上核验。该对应关系用于数据整合，预测测试集仍按啤酒划分。

同一人员对同一款酒的重复评分先取均值，再对人员等权平均。记第 b 款酒、第 j 个属性中人员 t 的第 r 次评分为 s_{btrj}，则面板均分为

$$\bar{s}_{bj} = \frac{1}{|T_b|}\sum_{t\in T_b}\frac{1}{n_{bt}}\sum_{r=1}^{n_{bt}} s_{btrj}. \tag{1}$$

每款酒对应 3—14 名不同人员。采用相同的人员权重，七级评分分布定义为

$$q_{bjk} = \frac{1}{|T_b|}\sum_{t\in T_b}\frac{1}{n_{bt}}\sum_{r=1}^{n_{bt}}\mathbf{1}(s_{btrj}=k), \quad k=0,\dots,6. \tag{2}$$

面板均分、个人评分分布和面板众数分别描述平均强度、评分离散情况和最常见等级。后续预测和评价保持这三种端点的区别。

2.3 化学尺度与异常

公开校准化学表提供 70 个数值参考字段。比较恒等、半最小正值替换及替换后取常用对数等规则后，38 列在全部可比记录中一致；其余 32 列存在 80 处数值差异，保留供敏感性分析。DMS 和 4VG 的零值采用半最小正值替换后可重现对应变换，该规则的仪器检出限尚未取得。

浓度混配分析选用来源标注单位为 mg/L、g/L、μg/L 或体积分数的 61 个字段。五个综合量的 13 种候选重算公式均未达到预设重现精度，故条件生成模型排除这五个综合量，使用 226 个非综合字段。浓度单位与历史来源可以追踪，但完整标准曲线、回收率及新批次测量信息仍需补充。

3 第一问：感官等级与面板均分预测

3.1 七级分布与评价指标

以式（2）的七级分布为监督目标，使用多输出极端随机树估计各等级概率 $\hat{p}_{bjk}$。预测众数用于精确等级决策，预测中位数用于绝对等级损失决策：

$$\hat{g}_{\mathrm{mode}} = \arg\max_k \hat{p}_k, \qquad \hat{g}_{\mathrm{med}} = \min\left\{k : \sum_{h=0}^{k} \hat{p}_h \geq 0.5\right\}. \tag{3}$$

记预测与观测分布在等级边界 k 的累计概率为 P_k 和 Q_k，累计分布平方距离定义为

$$D = \frac{1}{6}\sum_{k=0}^{5}(P_k - Q_k)^2. \tag{4}$$

该指标衡量预测分布与历史面板经验分布之间的距离。对个人实际等级 Y 计算的标准有序概率评分为 $\mathrm{RPS}(P,Y) = \frac{1}{6}\sum_{k=0}^{5}(P_k - \mathbf{1}(Y \leq k))^2$[8]。按经验分布取期望后有

$$\mathbb{E}_{Y\sim q}[\mathrm{RPS}(P,Y)] = D + \frac{1}{6}\sum_{k=0}^{5} Q_k(1-Q_k). \tag{5}$$

式（5）的第二项反映面板评分分歧，因此分布距离和个人期望 RPS 需要分别报告。面板众数一致率以每款酒的预测众数与观测面板众数是否相同计算；非零众数一致率仅在观测面板众数大于 0 的记录上计算。

核心 30 项 方法	累计分布平方距离 D	个人期望 RPS	面板众数一致率	非零众数一致率
训练总体分布	0.02820	0.11685	79.31%	21.30%
风格分布基线	0.02313	0.11179	80.72%	26.58%
频率极端随机树	0.02178	0.11044	80.52%	24.33%
累计概率极端随机树	0.02170	0.11036	80.65%	24.88%

核心属性观测面板众数为 0 的比例为 78.03%。频率极端随机树相对总体分布基线的 D 下降 22.74%，个人期望 RPS 下降 5.49%；风格分布基线的非零众数一致率高于两种树模型。累计概率模型相对频率模型的核心 D 差值为-0.000081，固定预测条件下的 95% 重采样区间为 [-0.000169，0.000004]。现有证据支持报告各方法的差异，同时显示非零等级识别仍是主要改进方向。

全部 50 项中，频率极端随机树的面板众数一致率为 79.10%，非零众数一致率为 31.33%；总体分布基线分别为 77.84% 和 26.92%。生成的概率分布、众数和中位数结果按属性分别保存，以适配不同的决策损失。

3.2 面板均分预测

对面板均分，比较训练均值、训练中位数、风格中位数、岭回归、极端随机树、梯度提升树及固定等权融合。外层采用按风格分层的三折五次重复划分；对同一啤酒的五次留出预测取平均，再计算平均绝对误差。岭回归参数在训练数据内选择，其他方法使用预先固定的设置。

$$\mathrm{MAE} = \frac{1}{nq}\sum_{b=1}^{n}\sum_{j=1}^{q}|\bar{s}_{bj} - \hat{s}_{bj}|. \tag{6}$$

方法	核心 30 项 MAE/分	全 50 项 MAE/分
训练均值	0.60450	0.54879
训练总体中位数	0.58785	0.52275
风格中位数	0.50969	0.44712
岭回归	0.52270	0.45708
极端随机树	0.50589	0.44336
梯度提升树	0.50819	0.43780
固定等权融合	0.50222	0.43560

融合模型核心误差比训练均值低 0.10228 分，比风格中位数低 0.00747 分。以酒为单位、按风格分层，对固定的留出预测重采样 5000 次，融合与风格中位数的误差差值区间为 [-0.01914，0.00418]；融合与极端随机树的差值为-0.00367，区间为 [-0.00668，-0.00056]。这些区间反映当前留出预测的样本重采样变化，未纳入模型重训和选择过程的不确定性。风格中位数依赖已知风格标签，适合用于分析风格信息提供的基线。

3.3 预测区间与缺失压力

另用极端随机树进行五次分割保形实验，每次分别使用 150 款酒训练、50 款校准、50 款测试，按属性校准绝对残差。五次分割的逐属性平均覆盖率为 90.06%，平均区间宽度为 1.7342 分。覆盖率反映该实验中的逐维面板均分区间；不同属性存在差异，例如 barnyard、F_malt_all 和 stale hops 的平均覆盖率分别为 84.0%、84.8% 和 85.2%。

为考察输入缺失，测试阶段在原有缺失基础上增加 0%、5%、10% 和 20% 的随机缺失，使用训练阶段拟合的插补参数。该实验反映随机丢失输入的压力；

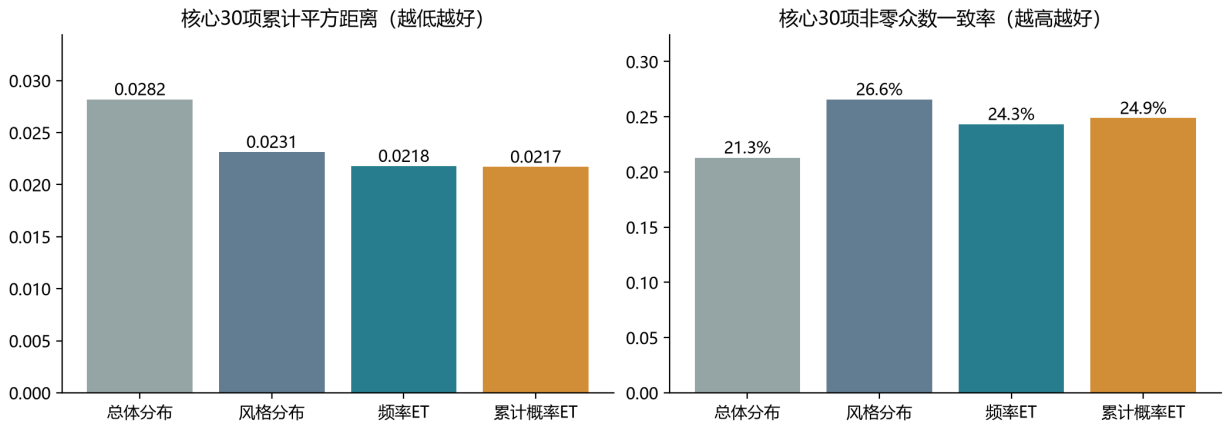


图 1: 等级分布预测的累计距离与非零众数识别比较。

低浓度未检出或整类检测通道缺失的影响，需要另设缺失机制实验。

4 第二问：化合物图与关键因素

4.1 图构建和模型

在全部数据上描述化合物相关结构时，取 $|r| \geq 0.7$ 得到 231 个节点、109 条边。模型验证时，每个训练折单独计算相关图。令 W 为带符号权重矩阵，$L = \mathrm{diag}(\sum_j |W_{ij}|) - W$，多目标系数矩阵 B 由下式估计：

$$\hat{B} = \arg\min_B \{\|Y_c - ZB\|_F^2 + \alpha\|B\|_F^2 + \lambda\,\mathrm{tr}(B^T L B)\}. \tag{7}$$

对单个系数向量 b，有 $b^T L b = \sum_{i<j} |W_{ij}|(b_i - \mathrm{sign}(W_{ij})b_j)^2 \geq 0$。因此当 $\alpha > 0$ 且 $\lambda \geq 0$ 时，目标函数具有唯一解。正相关边鼓励系数接近，负相关边鼓励系数方向相反。α、λ 和相关阈值均在训练折内部选择；节点标签打乱图用于检验具体网络结构的作用。

核心 30 项评价口径	无图 MAE	真实图 MAE	真实图减无图
15 个留出折误差的平均	0.536911	0.537035	+0.000124
每酒五次预测平均后的误差	0.526148	0.524865	-0.001283

两种口径分别衡量单次拟合与重复预测聚合后的误差，真实图的差异均较小。聚合口径下，以酒为单位的固定预测条件重采样差值区间为 [-0.002392，-0.000177]。最终全数据描述模型选得 $\lambda = 0$，因此后续 Top-3 由岭回归系数确定。

4.2 因素稳定性与化学解释

对每个感官目标，按标准化输入的岭回归系数绝对值选取三个化合物或元素，共得到 150 条因素记录。采用 200 次以酒为单位的 bootstrap，重新建立预处理与相关图，并保持全数据选定的参数；同时计算留出数据上的普通置换和风格内置换贡献。某因素被置换前后的误差增加量用于衡量该因素对预测的贡献。

描述性支持条件为：bootstrap 入选频率至少 0.5、至少在 5 个训练折入选，且所选折的普通与风格内置换平均贡献均为正。150 条记录中有 36 条满足条件。该频率以固定参数为前提；重采样中重新选参可进一步评估完整选择流程的稳定性。

化学解释结合统计证据逐项给出。X4VG 与 4VG 感官属性的关联，与 4VG 相关香气研究方向一致 [9]；iron 与金属感的关联，可结合金属诱发口腔氧化的研究讨论 [10]；linalool 与芫荽风味存在文献依据 [11]，其当前入选频率低于 0.5。对于 A_hops_all 的 iron，风格中心化后的 Top-3 保留比例为 0.1，说明其排名对风格差异敏感。相关性较强的化合物还会在模型中相互替代，因此完整因素表同时保留频率、置换贡献、单位和文献对应信息。

5 第三问：风味生成与迁移可行性

5.1 风格条件生成

以 tasting_category_fine 为风格标签，比较风格内重采样、局部凸组合、条件 PCA 高斯生成和受约束局部凸组合。每次分层划分设搜索训练集 G 为 100 款、评价训练集 J 为 100 款、真实留出集 T 为 50 款；对 Tripel、Kriek、Lambic，每种方法在每次划分下生成 40 个谱。候选在 G 中形成并冻结，再由 J 模型复

核；T 用于检验留出来源的表现。

局部凸组合的形式为 $x_{\mathrm{new}} = ux_a + (1-u)x_b$，其中 x_a, x_b 为同风格邻近样本，$0 \le u \le 1$。生成谱使用最大均值差异 MMD、范围检查、复制比例和 J 风格分类结果评价 [12]。对于 Tripel，受约束局部凸组合的 J 目标风格识别率为 69.5%，高于风格内重采样的 56.5%；其 MMD 平方为 0.1158，高于重采样的 0.1034。分类识别率与分布距离反映不同性质，应结合覆盖度和重复率共同判断生成质量。已有 MMD 采用包含对角项的 V 统计量，结果受有限样本影响。

在冻结生成谱上，利用 G 和 J 分别训练的原始评分模型生成 50 项面板均分和整数等级画像。画像表示模型对化合物谱的预测；实际风味需通过制备和品评测量。

5.2 冻结候选与代理复核

迁移搜索限制特征变动半径、目标风格支持及四项化学缺陷字段，同时要求陈旧酒花感官代理不增加。在候选冻结后，补充使用 G 和 J 分别训练的原始评分尺度模型复核 stale hops，候选及排序保持固定。

半径为 1 的迁移记录	全部来源与划分：55 条	T 留出来源：10 条
G 搜索可行	51	10
原 J 代理支持	11	2
同时满足新增 G/J 原始尺度 stale hops 条件	6	1

T 来源的 10 条记录涉及 7 款不同酒；重复划分形成的记录不是新增酒样。G 和 J 的评分代理对部分候选给出不同判断，说明仅凭搜索模型的可行结果不足以确定迁移效果。

以第 1 次划分的来源酒 81 为例，半径 1 的冻结统计候选在 G 模型下的 stale hops 预测由 0.8333 降至 0.3072，在 J 模型下由 0.2629 升至 0.3098；该候选未获得原 J 代理支持。另一项浓度混配分析中，同一来源和划分在严格协议下的最大 Tripel 份额为 0；参考阈值风险协议下，四项化学约束允许的最大份额为 0.3514。这两项分析分别作用于统计候选和浓度域混配，前者展示评价模型分歧，后者展示约束定义对可行域的影响。

5.3 严格浓度约束与阈值风险约束

在 61 个报告浓度字段构成的次级分析中，以 Pils 来源酒和 G 中的 Tripel 锚点进行浓度域线性混配。设来源浓度为 c_s、锚点浓度矩阵为 C_A、锚点份额为 w，则

$$c_{\mathrm{mix}} = (1 - \mathbf{1}^T w)c_s + C_A w, \quad w \ge 0, \quad \mathbf{1}^T w \le 0.75. \tag{8}$$

协议 A 要求 DMS、双乙酰、4VG 和 trans-2-nonenal 四项浓度逐项不增加。令 D 为各锚点与来源在这四项上的浓度差，最大目标份额问题为

$$\max_{w} \ \mathbf{1}^T w, \qquad Dw \le 0, \quad w \ge 0, \quad \mathbf{1}^T w \le 0.75. \tag{9}$$

对 55 条来源与划分记录，协议 A 所得最大目标份额均为 0。对各行按正尺度归一化得到 D_{norm}；若存在 $v \ge 0$，使 $D_{\mathrm{norm}}^T v$ 对每个锚点均严格为正，则任意非零 $w \ge 0$ 都有 $v^T D_{\mathrm{norm}} w > 0$，与协议 A 矛盾。55 条记录均找到正余量证书，其中 39 条可由单项约束阻塞解释，16 条需要联合约束。分别删除 DMS、双乙酰、4VG 和 nonenal 约束后，允许正份额的记录数为 30、4、0 和 4；该消融用于定位阻塞来源。

题目强调超阈值缺陷，本文进一步设风险函数 $R_d(c) = \max(c/t_d - 1, 0)$，以协议 B 要求 $R_d(c'_d) \le R_d(c_d)$，等价于 $c'_d \le \max(c_d, t_d)$。参考阈值取 DMS 19 μg/L、双乙酰 0.09 mg/L、4VG 0.2 mg/L、nonenal 0.1 μg/L[13][14]。以参考阈值的 0.5、1 和 2 倍计算时，55 条记录中分别有 15、30 和 34 条允许正的 Tripel 份额。该结果仅检验四项化学风险约束；目标风格提升和 stale hops 仍需按冻结候选流程评价。阈值与啤酒基质、测量方法相关，零浓度记录还受到未知检出限影响。

6 验证范围与应用建议

本文以现有 250 款酒及历史面板为基础，按酒划分评价新酒预测；评分人员在训练与测试之间存在重叠。后续可采用人员留出设计，评价模型对新面板的适用性。对于非零等级，可在训练内部比较累计概率模型、零与非零两段模型及校准方法，并按目标和风格报告混淆矩阵与等级距离误差。

当前预测结果由验证模型生成。实际使用时，需要基于全部可用训练数据拟合并保存最终模型、预处理参数、输入字段顺序和单位，建立只需化合物输入的预测入口，并由独立队员在新环境验证。

实物验证可先将候选转化为具有批号的可制备小试，对四项化合物进行重复测量并记录检出限、定量限和回收率，再采用随机编码、平衡顺序的配对盲评。一个可执行的试验方案为 30 名训练人员、3 个独立批次；以人员为单位汇总批次差异，对五项缺陷预先设

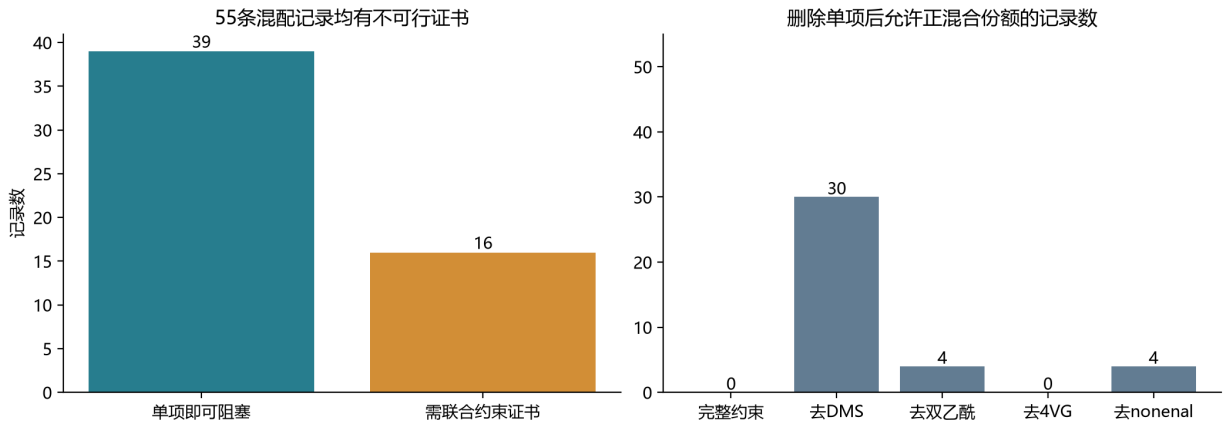


图 2: 严格混配约束的不可行证书及单项约束消融。

定评价界限。新批次化学测量、盲评结果和制备条件将决定统计候选能否用于实际配方。

7 结论

基于公开逐人评分与附件样本核查，本文构造了 50 项 0—6 分面板均分和七级评分分布。固定融合模型在核心 30 项上的均分 MAE 为 0.50222，较风格中位数基线低 0.00747 分；等级分布预测优于总体分布基线，但非零众数一致率为 24.33%，仍有明显提升空间。

化合物图对预测误差的影响随汇总口径变化，最终 Top-3 由普通岭模型产生。150 条因素记录中 36 条满足描述性支持条件；结合风格内置换和化学文献，可以区分稳定关联与依赖风格结构的候选。

条件生成模型能够产生各目标风格的统计谱，独立代理复核显示迁移结果对评价模型敏感。严格浓度不增加约束下，55 条记录均无正的 Tripel 混合份额；阈值超额风险约束下，参考阈值允许其中 30 条记录获得正份额。下一步应在同一冻结候选上联合检查风格、五项缺陷与制备条件，并通过新批次测量和盲评确认实际效果。

本文的数据核查、分析实现和文字整理使用了 OpenAI Codex 辅助；工具使用详情与队员实际复核记录另行提供。

参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.

[2] Schreurs M, et al. Data from: Predicting and improving complex beer flavor through machine learning, Version v4. Zenodo, 2026. DOI:10.5281/zenodo.23624915.

[3] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3–42, 2006.

[4] Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189–1232, 2001.

[5] Breiman L. Stacked Regressions. Machine Learning, 24:49–64, 1996.

[6] Li C, Li H. Variable selection and regression analysis for graph-structured covariates with an application to genomics. The Annals of Applied Statistics, 4(3):1498–1516, 2010.

[7] Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094–1111, 2018.

[8] Gneiting T, Raftery A E. Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association, 102(477):359–378, 2007.

[9] Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983–11988, 2008.

[10] Omur-Ozbek P, Dietrich A M, Duncan S E, Lee Y. Role of lipid oxidation, chelating agents, and antioxidants in metallic flavor development in the oral cavity. Journal of Agricultural and Food Chemistry,

60(9):2274–2280, 2012.

[11] Pater A, Satora P, Januszek M. Effect of Coriander Seed Addition at Different Stages of Brewing on Selected Parameters of Low-Alcohol Wheat Beers. Molecules, 29(4):844, 2024.

[12] Gretton A, Borgwardt K M, Rasch M J, et al. A Kernel Two-Sample Test. Journal of Machine Learning Research, 13:723–773, 2012.

[13] Meilgaard M C. Individual differences in sensory threshold for aroma chemicals added to beer. Food Quality and Preference, 4(3):153–167, 1993.

[14] Santos J R, et al. Determination of E-2-nonenal by high-performance liquid chromatography with UV detection assay for the evaluation of beer ageing. Journal of Chromatography A, 985(1 – 2):395 – 402, 2003.

A 全部 50 项 Top-3 与结果索引

以下为原始面板均分的最终描述模型 Top-3，括号为固定全数据参数条件下的 200 次 bootstrap 入选频率。λ=0，实际为普通岭回归；不是因果概率、独立检验 p 值或新化学试验。

目标	Top-3 字段（条件入选频率）
A_malt_all	X2.acetylfuran (0.77)；ethyl.furoate (0.42)；phenethyl_alcohol (0.36)
A_malt_grain	gamma.nonalactone (0.30)；car.3.en.2.one (0.36)；X5.methylfurfural (0.20)
A_malt_bread	furfural (0.36)；ethyl.furoate (0.28)；ethyl.tiglate (0.13)
A_malt_cara	X2.acetylfuran (0.60)；phenethyl_alcohol (0.45)；octyl_acetate (0.31)
A_malt_burn	methylpyrazine (0.75)；X2.acetylfuran (0.60)；X4.methyl.1.pentanol (0.11)
A_hops_all	iron (0.56)；DMS (0.34)；cis.dihydro.beta.terpineol (0.20)
A_hops_citrus	X6.methyl.5.hepten.2.one (0.53)；terpinolene (0.19)；geraniol (0.26)
A_hops_tropical	DMS (0.55)；beta.phellandrene (0.26)；citronellol (0.29)
A_hops_noble	iron (0.36)；ethyl_octanoate (0.23)；ethyl_decanoate (0.16)
A_hops_woody	iron (0.27)；ethyl_2.methyl_butyrate (0.15)；X2Z.6Z.farnesol (0.23)
A_esters_all	acetaldehyde (0.78)；X.E..beta.ionone (0.23)；n.hexanol (0.23)
A_esters_ethac	acetaldehyde (0.40)；ethyl_acetate (0.21)；cis.linalool.oxide (0.17)
A_esters_isoaa	n.hexanol (0.40)；ambroxide (0.46)；ethyl.hexadecanoate (0.20)
A_esters_flower	beta.phellandrene (0.56)；X.E..alpha.ionone (0.40)；terpinolene (0.14)
A_esters_fruity	beta.phellandrene (0.86)；X1.10.di.epi.cubenol (0.18)；benzyl.alcohol (0.30)
F_malt_all	acetaldehyde (0.57)；X2.acetylfuran (0.47)；X.E..alpha.ionone (0.44)
F_malt_grain	X2.3.butanedione (0.49)；gamma.nonalactone (0.14)；X3.methyl.2.butenal (0.18)
F_malt_bread	acetaldehyde (0.26)；dehydroxy.cis.linalool.oxide (0.23)；caryophylla.4.12..8.13..dien.5.alpha.ol (0.15)
F_malt_cara	acetaldehyde (0.62)；ethyl.nicotinate (0.27)；isobutyl.hexanoate (0.23)
F_malt_burn	X2.acetylfuran (0.51)；methylpyrazine (0.57)；MeSH (0.25)
F_hops_all	cis.dihydro.beta.terpineol (0.70)；iron (0.70)；ethyl_isovalerate (0.40)
F_hops_citrus	geraniol (0.47)；citronellol (0.43)；cis.dihydro.beta.terpineol (0.41)
F_hops_tropical	X6.methyl.5.hepten.2.one (0.46)；DMS (0.24)；citronellol (0.27)
F_hops_noble	iron (0.68)；citronellol (0.23)；X14.hydroxy.alpha.humulene (0.30)

目标	Top-3 字段（条件入选频率）
F__hops__woody	manool (0.43)；cis.dihydro.beta.terpineol (0.28)；n.octanol (0.15)
F__esters__all	acetaldehyde (0.91)；ethyl__isovalerate (0.34)；n.hexanol (0.27)
F__esters__ethac	acetaldehyde (0.51)；ethyl__acetate (0.47)；n.tetradecanol (0.28)
F__esters__isoaa	n.hexanol (0.28)；ethyl__decanoate (0.14)；acetaldehyde (0.12)

目标	Top-3 字段（条件入选频率）
F__esters__flower	beta.phellandrene (0.30)；eucalyptol (0.29)；terpinolene (0.13)
F__esters__fruity	beta.phellandrene (0.49)；X.E..beta.ionone (0.38)；MeSH (0.17)
acidity	lactic__acid.mg.L. (1.00)；acetic__acid.g.L. (0.72)；X4.ethyl.guaiacol (0.12)
bitternes	iron (0.98)；cis.dihydro.beta.terpineol (0.63)；ethyl__octanoate (0.48)

目标	Top-3 字段（条件入选频率）
sweetness	ethyl__decanoate (0.81)；iron (0.52)；ethyl__octanoate (0.28)
X4vg	X4VG (1.00)；delta.terpinyl.acetate (0.26)；ethyl__isovalerate (0.22)
diacetyl	CS2 (0.21)；X.E..beta.ionone (0.09)；octyl__acetate (0.29)
dms	SO2.mg.L. (0.56)；caryophylla.4.12..8.13..dien.5.alpha.ol (0.18)；neryl.formate (0.17)

目标	Top-3 字段（条件入选频率）
metallic	iron (0.94)；ethyl__isovalerate (0.46)；neryl.formate (0.20)
stale__hops	X2.butyloctanoate (0.41)；iron (0.27)；X.E..ortho.methoxy.cinnamaldehyde (0.21)
t2n	germacrene.D (0.58)；isopentyl.hexanoate (0.46)；X.Z..dihydro.apofarnesal (0.18)
orange	X1.alpha.terpineol (0.73)；limonene (0.49)；linalool (0.38)

目标	Top-3 字段（条件入选频率）
coriander	x.methyl.1.octanol (0.57)；geraniol (0.55)；linalool (0.46)
clove	linalool (0.81)；limonene (0.67)；eucalyptol (0.48)
lactic	lactic__acid.mg.L. (1.00)；acetic__acid.g.L. (0.61)；ethyl.phenylacetate (0.28)
acetic	lactic__acid.mg.L. (1.00)；acetic__acid.g.L. (0.62)；DES (0.23)

目标	Top-3 字段（条件入选频率）
barnyard	X2.butyloctanoate (0.32)；gamma.eudesmol (0.27)；iron (0.23)
alcohol	acetaldehyde (0.83)；ethanol..v.v. (0.82)；X3.methyl.2.hexanol (0.47)
aftertaste	glycerol (0.68)；ethyl__isovalerate (0.62)；cis.dihydro.beta.terpineol (0.34)
body	glycerol (0.80)；acetaldehyde (0.68)；X3.methyl.2.hexanol (0.47)

目标	Top-3 字段（条件入选频率）
co2	ethyl__octanoate (0.62)；X3.methyl.2.hexanol (0.55)；ethyl__hexanoate (0.34)
overall	ethyl__acetate (0.49)；X2.butyloctanoate (0.34)；ethyl__hexanoate (0.14)

目录	保存内容
audit	对应关系、0-6 端点、人员重复、浓度例外、OAV 与候选公式
prediction	15 折均分预测、标定对照、区间、原始尺度普通岭敏感性
ordinal	真实七级分布、众数/中位数/均分取整、缺失压力与生成谱等级画像
raw__graph/q2	原始端点无图/真图/打乱图、跨拟合贡献、完整 Top-3

目录	保存内容
transfer	冻结候选复查、G/J 原始均分画像、浓度混合、LP 与候选锁
protocol	空白评分表与假设样本量；没有实际品评结果

命令：python scripts/fetch__stage4__sources.py；python run__stage4.py all –data-dir < 官方附件目录 >；python scripts/stage4__graph__comparison.py –data-dir < 官方附件目录 >；python -m pytest -q；python scripts/verify__stage4.py；python scripts/stage4__figures.py；python scripts/prepare__absolute__paper.py；使用 scripts/build__revised__pdf.py 按 paper__content.json 生成本稿。首次重建旧生成谱时须先运行 stage3；脚

本默认来源目录可由参数指定。

软件：Python 3.12 及 NumPy、pandas、SciPy、scikit-learn、NetworkX、Matplotlib、openpyxl；精确版本在 requirements-lock.txt 和各运行 manifest 中。PDF 排版另用 ReportLab、pypdf、pdfplumber 和 Pillow；无需声称外部依赖库源码属于本队。

B 完整源程序

src/___init___.py

```
"""Research support code. See docs/AI_USAGE.md before competition use."""
```

src/audit.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook

from .common import progress, save_csv, sha256, write_json
from .data import META


def run_audit(data, out, cfg):
    dest = Path(out) / "audit"
    dest.mkdir(parents=True, exist_ok=True)
    progress("Audit: checking source hashes, sample keys, scales and sensory templates")
    hashes, shapes = [], []
    for p in sorted(data.root.iterdir()):
        if p.is_file():
            hashes.append({"file": p.name, "bytes": p.stat().st_size, "sha256": sha256(p)})
            if p.suffix == ".csv":
                frame = pd.read_csv(p)
                shapes.append({"file": p.name, "rows": len(frame), "columns_including_metadata": len(frame.columns), "missing_cells": int(frame.isna().sum().sum())})
    save_csv(pd.DataFrame(hashes), dest / "source_manifest.csv")
    save_csv(pd.DataFrame(shapes), dest / "file_shapes.csv")
    save_csv(data.meta.groupby("tasting_category_fine").size().rename("n").reset_index(), dest / "style_counts.csv")
    save_csv(data.chemical_meta, dest / "chemical_dictionary.csv")
    names = data.sensory_names[data.sensory_names.dataset == "Trained panel"].drop_duplicates("R_name").set_index("R_name")
    targets = []
    for c in data.Y:
        alias = "X4vg" if c == "X4vg" else c
        targets.append({"field": c, "label": names.at[alias, "CleanName_PanelFeedback"] if alias in names.index else c,
                        "group": "Aroma" if c.startswith("A_") else "Flavour" if c.startswith("F_") else "Other sensory",
                        "observed_scale": "provided centered/standardized continuous values",
                        "absolute_0_6_calibration": "unresolved"})
    save_csv(pd.DataFrame(targets), dest / "sensory_dictionary.csv")
    records = []
    for col in data.X:
        a = data.X[col]
        raw = data.calibrated[col] if col in data.calibrated else None
        row = {"field": col, "n_missing": int(a.isna().sum()), "minimum": float(a.min()), "maximum": float(a.max()),
               "q01": float(a.quantile(.01)), "q99": float(a.quantile(.99)), "transform_evidence": "not_available",
               "raw_identity_fraction": np.nan, "raw_log10_fraction": np.nan, "n_raw_positive_pairs": 0}
        if raw is not None:
            mask = a.notna() & raw.notna() & (raw > 0)
            if mask.sum():
                identity = np.isclose(a[mask], raw[mask], atol=1e-7, rtol=1e-6)
                logarithm = np.isclose(a[mask], np.log10(raw[mask]), atol=1e-7, rtol=1e-6)
                fi, fl = float(identity.mean()), float(logarithm.mean())
                row.update(raw_identity_fraction=fi, raw_log10_fraction=fl, n_raw_positive_pairs=int(mask.sum()))
                row["transform_evidence"] = "verified_log10_positive_pairs" if fl >= .999 else "verified_identity_positive_pairs" if fi >= .999 else "mostly_identity_with_mismatches" if fi >= .9 else "mixed_or_unresolved"
        records.append(row)
    scales = pd.DataFrame(records)
    save_csv(scales, dest / "field_scales.csv")
    # Inspect actual input cells, not formula caches that evaluate blanks to zero.
    w = load_workbook(data.root / "3.xlsx", data_only=False)
    input_rows = [7, 20, 33, 45, 50, 54, 58, 62, 68, 72, 76, 82, 86, 90, 94]
    input_cols = [3, 5, 7, 9, 11, 13, 15, 19, 21, 23, 25, 27, 29, 31]
    template = []
    for sh in w:
        if not sh.title.startswith("beer_"):
            continue
        marks = [sh.cell(r, c).coordinate for r in input_rows for c in input_cols if sh.cell(r, c).value is not None]
        template.append({"sheet": sh.title, "nonempty_main_scoring_inputs": len(marks), "input_cells": ",".join(marks)})
    save_csv(pd.DataFrame(template), dest / "sensory_template_audit.csv")
    key_stats = []
    for n in [1, 4, 7]:
        d = pd.read_csv(data.root / f"{n}.csv")
        key_stats.append({"file": f"{n}.csv", "unique_beer": int(d.beer.nunique()), "unique_beer_id": int(d.beer_id.nunique())})
    save_csv(pd.DataFrame(key_stats), dest / "join_key_audit.csv")
    values = data.given_corr.to_numpy()
    evidence = {
        "pearson_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr().to_numpy() - values))),
        "spearman_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr(method="spearman").to_numpy() - values))),
        "n_samples": len(data.X), "n_features": data.X.shape[1], "n_targets": data.Y.shape[1],
        "n_styles": int(data.meta.tasting_category_fine.nunique()),
        "grade_calibration": "UNRESOLVED: no observed paired absolute 0-6 anchors confirmed",
        "template_summary_barnyard_formula": str(w["summary"]["AX2"].value),
        "template_barnyard_expected_cell": "beer_1!AG90 based on the form's Barnyard label; team review required",
        "scope": "source evidence verified by computation; model conclusions remain exploratory",
    }
    write_json(dest / "verified_facts.json", evidence)
```

src/common.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Team verification and modification are required; see docs/AI_USAGE.md.
"""
from __future__ import annotations

import hashlib
import json
import platform
from datetime import datetime
from importlib.metadata import version
from pathlib import Path
from zoneinfo import ZoneInfo

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


def write_json(path, value):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


def save_csv(frame, path):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.9g")


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def environment():
    return {
        "timestamp": datetime.now(ZoneInfo("Asia/Shanghai")).isoformat(),
```

```
        "python": platform.python_version(),
        "packages": {p: version(p) for p in ["numpy", "pandas", "scipy", "scikit-learn", "networkx", "matplotlib", "openpyxl"]},
    }


class TrainQuantileClipper(TransformerMixin, BaseEstimator):
    """Learn bounds on training samples only. Missing values remain missing."""

    def __init__(self, lower=0.01, upper=0.99):
        self.lower = lower
        self.upper = upper

    def fit(self, X, y=None):
        a = np.asarray(X, dtype=float)
        self.lower_ = np.nanquantile(a, self.lower, axis=0)
        self.upper_ = np.nanquantile(a, self.upper, axis=0)
        self.n_features_in_ = a.shape[1]
        return self

    def transform(self, X):
        a = np.asarray(X, dtype=float)
        if a.shape[1] != self.n_features_in_:
            raise ValueError("Feature count differs from training")
        return np.clip(a, self.lower_, self.upper_)


def preprocessing():
    return Pipeline([
        ("clip", TrainQuantileClipper()),
        ("impute", SimpleImputer(strategy="median", keep_empty_features=True)),
        ("scale", StandardScaler()),
    ])


def ecdf_grades(reference, values):
    """ASSUMPTION: seven empirical bins, NOT calibrated sensory intensity."""
    bounds = np.quantile(np.asarray(reference), np.arange(1, 7) / 7, axis=0)
    values = np.asarray(values)
    return (values[:, None, :] > bounds[None, :, :]).sum(axis=1).astype(int)


def progress(text):
    print(text, flush=True)
```

src/data.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
All source files are read-only. Never join on precision-damaged beer_id.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

META = ["beer", "beer_id", "tasting_category_fine"]
AGGREGATES = {"kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"}
NON_MOLECULES = {"color.EBC.", "betaglucan.mg.L.", "pH", "protein.g.L.", "bitterness", "GFS", "CO2.PSI.", "unfermentables"} | AGGREGATES
CHEMICAL_DEFECTS = ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"]


@dataclass
class BeerData:
    root: Path
    meta: pd.DataFrame
    X: pd.DataFrame
    Y: pd.DataFrame
    given_corr: pd.DataFrame
    thresholds: pd.DataFrame
    sensory_names: pd.DataFrame
    calibrated: pd.DataFrame
    chemical_meta: pd.DataFrame


def find_data(argument=None):
    if argument:
        root = Path(argument).resolve()
        if not (root / "1.csv").exists():
            raise FileNotFoundError(f"Expected 1.csv in {root}")
        return root
    repo = Path(__file__).resolve().parents[1]
    for root in [repo / "data/raw", repo.parent / "materials/A/附件"]:
        if (root / "1.csv").exists():
            return root
    raise FileNotFoundError("Supply --data-dir pointing to the original Attachments")


def align_by_beer(chemical, sensory):
    for label, frame in [("chemical", chemical), ("sensory", sensory)]:
        if frame["beer"].isna().any() or frame["beer"].duplicated().any():
            raise ValueError(f"{label}: beer must be non-null and unique")
    if set(chemical.beer) != set(sensory.beer):
        raise ValueError("Chemical and sensory sample sets differ")
    sensory = sensory.set_index("beer").loc[chemical.beer].reset_index()
    if not chemical.tasting_category_fine.reset_index(drop=True).equals(sensory.tasting_category_fine):
        raise ValueError("Style labels do not agree after beer alignment")
    return sensory


def load_data(root):
    root = Path(root)
    chemical = pd.read_csv(root / "1.csv")
    sensory = align_by_beer(chemical, pd.read_csv(root / "4.csv"))
    X = chemical.drop(columns=META).astype(float)
    Y = sensory.drop(columns=META).astype(float)
    if Y.isna().any().any():
        raise ValueError("Targets contain missing values; target handling needs explicit review")
    corr = pd.read_csv(root / "2.csv", index_col=0)
    if corr.shape != (X.shape[1], X.shape[1]):
        raise ValueError("Correlation matrix dimensions do not match chemical fields")
    if list(corr.index) != list(corr.columns):
        raise ValueError("Correlation row/column names differ")
    # The positional alias map is empirically checked over the full matrix.
    # Thus it is not inferred from just matching the number of columns.
    mismatch = np.nanmean(np.abs(X.corr(method="spearman").to_numpy() - corr.to_numpy()))
    if mismatch > 0.002:
        raise ValueError("Cannot verify ordered chemical aliases against sample correlations")
    book = root / "8.xlsx"
    thresholds = pd.read_excel(book, sheet_name="Sensory thresholds", na_values="NA")
    names = pd.read_excel(book, sheet_name="Sensory names", na_values="NA")
    calibrated = pd.read_excel(book, sheet_name="Calibrated chemical dataset", na_values="NA")
    if calibrated.beer.duplicated().any():
        raise ValueError("Calibrated reference has duplicate beer keys")
    calibrated = calibrated.set_index("beer").loc[chemical.beer].reset_index()
    cm = thresholds.dropna(subset=["field_name_R"]).drop_duplicates("field_name_R").set_index("field_name_R")
    rows = []
    for col, alias in zip(X.columns, corr.index):
        row = {"field": col, "matrix_alias": alias, "kind": "aggregate" if col in AGGREGATES else "physical_or_mixture" if col in NON_MOLECULES else "compound_or_element"}
        for name in ["chemical_name", "origin_paper", "unit", "off_flavors", "threshold_median", "threshold_mean"]:
            row[name] = cm.at[col, name] if col in cm.index else np.nan
        rows.append(row)
    return BeerData(root, chemical[META], X, Y, corr, thresholds, names, calibrated, pd.DataFrame(rows))
```

src/q1.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Nested preprocessing/tuning and paired out-of-fold prediction. No raw-score claim.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.cross_decomposition import PLSRegression
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold, RepeatedStratifiedKFold
from sklearn.pipeline import Pipeline

from .common import ecdf_grades, preprocessing, progress, save_csv, write_json
```

```

def model_specs(cfg):
    def pipe(model):
        return Pipeline([("prep", preprocessing()), ("model", model)])
    return {
        "mean": (pipe(DummyRegressor(strategy="mean")), {}),
        "ridge": (pipe(Ridge()), {"model__alpha": cfg["ridge_alphas"]}),
        "pls": (pipe(PLSRegression(scale=False)), {"model__n_components": cfg["pls_components"]}),
        "extra_trees": (pipe(ExtraTreesRegressor(n_estimators=cfg["extra_trees"], min_samples_leaf=3, max_features=.8, random_state=cfg["seed"], n_jobs=2)), {}),
    }


def fitted_model(name, X, y, cfg):
    estimator, params = model_specs(cfg)[name]
    if params:
        estimator = GridSearchCV(estimator, params, scoring="neg_mean_absolute_error", cv=KFold(cfg["inner_folds"], shuffle=True, random_state=cfg["seed"]), n_jobs=1, refit=True)
    estimator.fit(X, y)
    return estimator.best_estimator_ if params else estimator, estimator.best_params_ if params else {}


def metric_rows(y, pred, targets, model):
    return [{"model": model, "target": c,
             "mae": float(mean_absolute_error(y[:, j], pred[:, j])),
             "rmse": float(np.sqrt(mean_squared_error(y[:, j], pred[:, j]))),
             "r2": float(r2_score(y[:, j], pred[:, j]))}
            for j, c in enumerate(targets)]


def run_q1(data, out, cfg):
    dest = Path(out) / "q1"
    dest.mkdir(parents=True, exist_ok=True)
    (dest / "private").mkdir(exist_ok=True)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    targets = data.Y.columns.to_list()
    split = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"], random_state=cfg["seed"])
    sums = {m: np.zeros_like(Y) for m in model_specs(cfg)}
    count = np.zeros(len(X), dtype=int)
    assignments, selected, fold_metrics, robustness = [], [], [], []
    grades = np.zeros_like(Y)
    for fold, (train, test) in enumerate(split.split(X, data.meta.tasting_category_fine)):
        progress(f"Q1: outer fold {fold + 1}/{cfg['outer_folds'] * cfg['outer_repeats']}")
        count[test] += 1
        assignments.extend({"repeat": fold // cfg["outer_folds"], "fold": fold % cfg["outer_folds"], "beer": int(data.meta.iloc[i].beer), "role": "validation"} for i in test)
        for name in sums:
            est, params = fitted_model(name, X[train], Y[train], cfg)
            pred = est.predict(X[test])
            sums[name][test] += pred
            selected.append({"outer_fold": fold, "model": name, "parameters": str(params)})
            fold_metrics.append({"outer_fold": fold, "model": name, "mae": float(mean_absolute_error(Y[test], pred)), "macro_r2": float(r2_score(Y[test], pred)), "n_validation": len(test)})
            if name == "ridge":
                grades[test] += ecdf_grades(Y[train], pred)
                for rate in cfg["missing_rates"]:
                    corrupted = X[test].copy()
                    rng = np.random.default_rng(cfg["seed"] + fold)
                    artificial_missing = rng.random(corrupted.shape) < rate
                    corrupted[artificial_missing] = np.nan
                    pp = est.predict(corrupted)
                    robustness.append({"outer_fold": fold, "additional_missing_fraction": rate, "mae": float(mean_absolute_error(Y[test], pp)), "mask_seed": cfg["seed"] + fold})
    if not np.all(count == cfg["outer_repeats"]):
        raise AssertionError("Every sample must have exactly one held-out prediction per repeat")
    predictions = {name: value / count[:, None] for name, value in sums.items()}
    rows = []
    for name, pred in predictions.items():
        rows += metric_rows(Y, pred, targets, name)
        frame = pd.concat([data.meta[["beer", "tasting_category_fine"]], pd.DataFrame(pred, columns=targets)], axis=1)
        save_csv(frame, dest / f"oof_{name}_continuous.csv")
    metrics = pd.DataFrame(rows)
    save_csv(metrics, dest / "metrics_by_target.csv")
    save_csv(metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), dest / "metrics_summary.csv")
    save_csv(pd.DataFrame(fold_metrics), dest / "metrics_by_fold.csv")
    save_csv(pd.DataFrame(assignments), dest / "split_assignments.csv")
    save_csv(pd.DataFrame(selected), dest / "inner_selected_parameters.csv")
    save_csv(pd.DataFrame(robustness), dest / "missingness_robustness.csv")
    styles = []
    for style in sorted(data.meta.tasting_category_fine.unique()):
        ix = np.flatnonzero(data.meta.tasting_category_fine.to_numpy() == style)
        styles.append({"style": style, "n": len(ix), "ridge_mae": float(mean_absolute_error(Y[ix], predictions["ridge"][ix]))})
    save_csv(pd.DataFrame(styles), dest / "metrics_by_style.csv")
    # Predeclared ridge is used downstream; do not choose the winner using OOF scores.
    model, params = fitted_model("ridge", X, Y, cfg)
    joblib.dump({"model": model, "features": data.X.columns.tolist(), "targets": targets}, dest / "private/model.joblib")
    save_csv(pd.DataFrame(model.named_steps["model"].coef_, index=targets, columns=data.X.columns).reset_index(names="target"), dest / "final_ridge_coefficients.csv")
    if cfg["export_assumption_grades"]:
        gg = np.rint(grades / count[:, None]).astype(int)
        frame = pd.concat([data.meta[["beer"]], pd.DataFrame(gg, columns=targets)], axis=1)
        frame["status"] = "ASSUMPTION_ONLY_training_ECDF_bins_not_absolute_sensory_scores"
        save_csv(frame, dest / "ASSUMPTION_percentile_grades_0_6.csv")
    write_json(dest / "status.json", {
        "continuous_prediction": "computed_repeated_out_of_fold_exploratory_baselines",
        "absolute_0_6_prediction": "UNRESOLVED_missing_raw_score_calibration",
        "proxy_grades": "empirical equal-frequency bins, not a recovery of the original 0-6 intensity",
        "primary_model": "ridge selected before evaluation; inner tuning only",
        "final_inner_parameters": params,
        "external_generalization": "not established; supplied targets already preprocessed by source authors",
        "intervals": "not calibrated; no nominal coverage claim is made",
    })
```

src/q2.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Descriptive graph-regularized attribution, not causal identification.
"""
from pathlib import Path

import networkx as nx
import numpy as np
import pandas as pd
from scipy.linalg import solve
from scipy.stats import rankdata

from .common import preprocessing, progress, save_csv, write_json
from .data import NON_MOLECULES


def signed_laplacian(corr, threshold):
    W = np.asarray(corr, float).copy()
    W[~np.isfinite(W)] = 0
    W[np.abs(W) < threshold] = 0
    np.fill_diagonal(W, 0)
    W = (W + W.T) / 2
    return np.diag(np.abs(W).sum(axis=1)) - W


def graph_coefficients(Z, Y, corr, cfg, graph=True):
    L = signed_laplacian(corr, cfg["graph_threshold"]) if graph else np.zeros((Z.shape[1], Z.shape[1]))
    yc = Y - Y.mean(axis=0)
    return solve(Z.T @ Z + cfg["graph_ridge_alpha"] * np.eye(Z.shape[1]) + cfg["graph_ridge_lambda"] * L, Z.T @ yc, assume_a="pos")


def cross_spearman(X, Y):
    a, b = rankdata(X, axis=0), rankdata(Y, axis=0)
    a -= a.mean(axis=0)
    b -= b.mean(axis=0)
    denom = np.sqrt((a * a).sum(axis=0)[:, None] * (b * b).sum(axis=0)[None, :])
    return np.divide(a.T @ b, denom, out=np.zeros((a.shape[1], b.shape[1])), where=denom > 0)


def top_indices(values, eligible):
    return eligible[np.argsort(-np.abs(values[eligible]), axis=0, kind="stable")[:3]]


def run_q2(data, out, cfg):
    dest = Path(out) / "q2"
    dest.mkdir(parents=True, exist_ok=True)
```

```
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    eligible = np.array([j for j, c in enumerate(features) if c not in NON_MOLECULES])
    given = data.given_corr.to_numpy()
    graph = nx.Graph()
    graph.add_nodes_from(features)
    edge_rows = []
    for i in range(len(features)):
        for j in range(i + 1, len(features)):
            r = float(given[i, j])
            if abs(r) >= cfg["graph_threshold"]:
                graph.add_edge(features[i], features[j], weight=abs(r), correlation=r)
                edge_rows.append({"source": features[i], "target": features[j], "spearman_r": r, "absolute_weight": abs(r)})
    save_csv(pd.DataFrame(edge_rows), dest / "coexpression_edges.csv")
    communities = list(nx.community.greedy_modularity_communities(graph, weight="weight"))
    modules = {node: i for i, group in enumerate(communities) for node in group}
    save_csv(pd.DataFrame([{"field": c, "module": modules[c], "degree": graph.degree(c), "weighted_degree": graph.degree(c, weight="weight")} for c in features]), dest / "network_nodes.csv")
    graph_stats = []
    for threshold in [.6, .7, .8, .9]:
        gg = nx.Graph()
        gg.add_nodes_from(features)
        gg.add_edges_from((features[i], features[j]) for i in range(len(features)) for j in range(i + 1, len(features)) if abs(given[i, j]) >= threshold)
        graph_stats.append({"threshold": threshold, "edges": gg.number_of_edges(), "components": nx.number_connected_components(gg), "isolated_nodes": len(list(nx.isolates(gg)))})
    save_csv(pd.DataFrame(graph_stats), dest / "network_threshold_sensitivity.csv")
    prep = preprocessing()
    Z = prep.fit_transform(X)
    beta = graph_coefficients(Z, Y, given, cfg)
    ridge = graph_coefficients(Z, Y, given, cfg, graph=False)
    marginal = cross_spearman(Z, Y)
    frequencies = np.zeros_like(beta)
    rng = np.random.default_rng(cfg["seed"])
    for b in range(cfg["bootstrap_repeats"]):
        if b % 20 == 0:
            progress(f"Q2: bootstrap {b}/{cfg['bootstrap_repeats']}")
        ix = rng.integers(0, len(X), len(X))
        zb = preprocessing().fit_transform(X[ix])
        # Refit the network too. Do not merely resample previously computed ranks.
        corrb = np.corrcoef(rankdata(zb, axis=0), rowvar=False)
        bb = graph_coefficients(zb, Y[ix], corrb, cfg)
        for k in range(Y.shape[1]):
            frequencies[top_indices(bb[:, k], eligible), k] += 1
    frequencies /= cfg["bootstrap_repeats"]
    meta = data.chemical_meta.set_index("field")
    top_rows, agreement = [], []
    for k, target in enumerate(targets):
        chosen = top_indices(beta[:, k], eligible)
        other = top_indices(ridge[:, k], eligible)
        univariate = top_indices(marginal[:, k], eligible)
        def jac(a, b):
            return len(set(a) & set(b)) / len(set(a) | set(b))
        agreement.append({"target": target, "graph_vs_ridge_jaccard": jac(chosen, other), "graph_vs_spearman_jaccard": jac(chosen, univariate), "ridge_vs_spearman_jaccard": jac(other, univariate)})
        for rank, j in enumerate(chosen, 1):
            c = features[j]
            origin = meta.at[c, "origin_paper"]
            top_rows.append({"target": target, "rank": rank, "field": c, "chemical_name": meta.at[c, "matrix_alias"],
                             "graph_coefficient": beta[j, k], "association_spearman": marginal[j, k],
                             "bootstrap_top3_frequency": frequencies[j, k], "module": modules[c],
                             "origin": origin, "unit": meta.at[c, "unit"], "known_off_flavors": meta.at[c, "off_flavors"],
                             "stability_flag": "low_selection_frequency" if frequencies[j, k] < .5 else "frequently_selected_in_this_resampling",
                             "chemical_argument": f"Reference origin: {origin}. Verify target-specific mechanism and units; coefficient is association, not causation.",
                             "team_mechanism_review": "required"})
    save_csv(pd.DataFrame(top_rows), dest / "top3_by_sensory_dimension.csv")
    save_csv(pd.DataFrame(agreement), dest / "method_jaccard.csv")
    save_csv(pd.DataFrame(frequencies, index=features, columns=targets).reset_index(names="field"), dest / "bootstrap_selection_frequencies.csv")
    write_json(dest / "status.json", {
        "scope": "descriptive_exploratory_full_dataset_association",
        "nodes": graph.number_of_nodes(), "edges": graph.number_of_edges(), "modules": len(communities),
        "bootstrap_repeats": cfg["bootstrap_repeats"], "bootstrap_rebuilds_preprocessing_and_graph": True,
        "causal_status": "not_identified",
        "chemical_mechanism_status": "reference metadata attached; target-specific human literature review required",
        "regularization_parameters": "fixed research defaults, not optimized for causal recovery",
        "eligible_compounds": len(eligible),
    })
```

src/q3.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Small-sample conditional Gaussian reference and explicitly model-based transfers.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.covariance import LedoitWolf
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

from .common import ecdf_grades, preprocessing, progress, save_csv, write_json
from .data import CHEMICAL_DEFECTS


class ConditionalPCAGaussian:
    def __init__(self, components=10, seed=20261024):
        self.components, self.seed = components, seed

    def fit(self, X, labels):
        self.prep = preprocessing()
        z = self.prep.fit_transform(X)
        self.pca = PCA(n_components=min(self.components, len(z) - 1, z.shape[1]), random_state=self.seed)
        latent = self.pca.fit_transform(z)
        self.low = np.nanquantile(X, .01, axis=0)
        self.high = np.nanquantile(X, .99, axis=0)
        self.median = np.nanmedian(X, axis=0)
        pooled = LedoitWolf().fit(latent).covariance_
        self.means, self.covariances = {}, {}
        labels = np.asarray(labels)
        for style in np.unique(labels):
            a = latent[labels == style]
            self.means[style] = a.mean(axis=0)
            local = np.atleast_2d(np.cov(a, rowvar=False)) if len(a) > 1 else pooled
            weight = len(a) / (len(a) + 10)
            self.covariances[style] = weight * local + (1 - weight) * pooled + 1e-8 * np.eye(pooled.shape[0])
        return self

    def encode(self, X):
        return self.pca.transform(self.prep.transform(X))

    def decode(self, latent):
        z = self.pca.inverse_transform(latent)
        values = self.prep.named_steps["scale"].inverse_transform(z)
        return np.clip(values, self.low, self.high)

    def sample(self, style, n, rng):
        latent = rng.multivariate_normal(self.means[style], self.covariances[style], size=n)
        return self.decode(latent)


def chemical_nonincrease(source, candidate, positions, tolerance=1e-9):
    return np.all(np.asarray(candidate)[..., positions] <= np.asarray(source)[..., positions] + tolerance, axis=-1)


def biased_mmd(a, b):
    """RBF MMD squared; bandwidth from real reference only."""
    if len(a) < 2 or len(b) < 2:
        return np.nan
    dist = cdist(b, b, metric="sqeuclidean")
    positive = dist[dist > 0]
    bandwidth = np.median(positive) if len(positive) else 1.
    return float(np.exp(-cdist(a, a, "sqeuclidean") / (2 * bandwidth)).mean() + np.exp(-dist / (2 * bandwidth)).mean() - 2 * np.exp(-cdist(a, b, "sqeuclidean") / (2 * bandwidth)).mean())

```

```

def run_q3(data, out, cfg):
    dest = Path(out) / "q3"
    dest.mkdir(parents=True, exist_ok=True)
    path = Path(out) / "q1/private/model.joblib"
    if not path.exists():
        raise FileNotFoundError("Q3 needs the Q1 reference predictor; run q1 first")
    bundle = joblib.load(path)
    predictor = bundle["model"]
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    if bundle["features"] != features or bundle["targets"] != targets:
        raise ValueError("Q1 predictor schema mismatch")
    X = data.X.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy()
    train, test = train_test_split(np.arange(len(X)), test_size=cfg["generation_holdout_fraction"], stratify=labels, random_state=cfg["seed"])
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "holdout" for i in range(len(X))]}), dest / "generator_split.csv")
    progress("Q3: fitting conditional PCA Gaussian on generator training samples only")
    generator = ConditionalPCAGaussian(cfg["generation_components"], cfg["seed"]).fit(X[train], labels[train])
    classifier = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000, class_weight="balanced", random_state=cfg["seed"]))])
    classifier.fit(X[train], labels[train])
    class_index = {s: j for j, s in enumerate(classifier.classes_)}
    holdout_balanced = float(balanced_accuracy_score(labels[test], classifier.predict(X[test])))
    rng = np.random.default_rng(cfg["seed"])
    generated, metrics, portraits = [], [], []
    for style in cfg["generation_styles"]:
        values = generator.sample(style, cfg["generation_samples_per_style"], rng)
        ids = [f"{style}_{i + 1:03d}" for i in range(len(values))]
        frame = pd.DataFrame(values, columns=features)
        frame.insert(0, "style", style)
        frame.insert(0, "generated_id", ids)
        generated.append(frame)
        pred = predictor.predict(values)
        portrait = pd.DataFrame(pred, columns=targets)
        portrait.insert(0, "style", style)
        portrait.insert(0, "generated_id", ids)
        portraits.append(portrait)
        real = X[test[labels[test] == style]]
        zs = generator.prep.transform(values)
        zr = generator.prep.transform(real)
        ztrain = generator.prep.transform(X[train])
        distances = cdist(zs, ztrain)
        train_style = X[train[labels[train] == style]]
        bootstrap = train_style[rng.integers(0, len(train_style), len(values))]
        # Natural missingness in held-out measurements must not erase all KS diagnostics.
        ks_values = []
        for j in range(X.shape[1]):
            observed = real[:, j][np.isfinite(real[:, j])]
            if len(observed):
                ks_values.append(ks_2samp(values[:, j], observed, method="auto").statistic)
        # Both metrics use independent real holdout samples; small counts are explicit.
        metrics.append({"style": style, "n_generated": len(values), "n_training_style": len(train_style), "n_holdout_style": len(real),
                        "mean_marginal_KS_holdout": float(np.mean(ks_values)), "n_KS_fields_observed": len(ks_values),
                        "RBF_MMD2_holdout": biased_mmd(zs, zr),
                        "bootstrap_baseline_MMD2_holdout": biased_mmd(generator.prep.transform(bootstrap), zr),
                        "mean_target_style_probability": float(classifier.predict_proba(values)[:, class_index[style]].mean()),
                        "nearest_train_distance_mean": float(distances.min(axis=1).mean()),
                        "exact_training_copy_fraction": float((distances.min(axis=1) < 1e-8).mean()),
                        "heldout_reference_warning": "few independent style examples; descriptive distances only, no quality guarantee"})
    all_generated = pd.concat(generated, ignore_index=True)
    save_csv(all_generated, dest / "generated_compound_profiles.csv")
    save_csv(pd.concat(portraits, ignore_index=True), dest / "generated_sensory_continuous.csv")
    save_csv(pd.DataFrame(metrics), dest / "generation_diagnostics.csv")
    if cfg["export_assumption_grades"]:
        a = pd.concat(portraits, ignore_index=True)
        grade = ecdf_grades(data.Y.to_numpy()[train], a[targets].to_numpy())
        proxy = pd.concat([a[["generated_id", "style"]], pd.DataFrame(grade, columns=targets)], axis=1)
        proxy["status"] = "ASSUMPTION_ONLY_ECDF_not_absolute_sensory_scores"
        save_csv(proxy, dest / "ASSUMPTION_generated_percentile_grades.csv")
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target = cfg["transfer_target"]
    source = cfg["transfer_source"]
    transfers, diagnostics, checks, transfer_sensory = [], [], [], []
    for i in np.flatnonzero(labels == source):
        original = X[i]
        x = np.where(np.isfinite(original), original, generator.median)
        base_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, class_index[target]]
        cleaning_only = np.clip(x, generator.low, generator.high)
        cleaning_only[positions] = np.minimum(cleaning_only[positions], x[positions])
        cleaning_prob = classifier.predict_proba(cleaning_only[None])[0, class_index[target]]
        latent = generator.encode(x[None])[0]
        desired = generator.decode((latent + generator.means[target] - generator.means[source])[None])[0]
        candidates = []
        for t in np.linspace(.025, 1.25, 50):
            candidate = np.clip(x + t * (desired - x), generator.low, generator.high)
            candidate[positions] = np.minimum(candidate[positions], x[positions])
            candidates.append(candidate)
        candidates = np.asarray(candidates)
        candidate_pred = predictor.predict(candidates)
        chemical_ok = chemical_nonincrease(x, candidates, positions)
        stale_ok = candidate_pred[:, stale] <= base_pred[stale] + 1e-9
        target_prob = classifier.predict_proba(candidates)[:, class_index[target]]
        # Separate genuine style movement from the effects of clipping source outliers.
        feasible = chemical_ok & stale_ok & (target_prob > max(base_prob, cleaning_prob) + .01)
        if feasible.any():
            eligible = np.flatnonzero(feasible)
            chosen = eligible[np.argmax(target_prob[eligible])]
            value, pp, probability = candidates[chosen], candidate_pred[chosen], target_prob[chosen]
            status = "nontrivial_model_feasible_candidate"
        else:
            value, pp, probability = x.copy(), base_pred.copy(), base_prob
            status = "no_nontrivial_candidate_found_source_retained"
        beer = int(data.meta.iloc[i].beer)
        missing_defects = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(original[j])]
        for c, j in zip(CHEMICAL_DEFECTS, positions):
            checks.append({"beer": beer, "defect": c, "source_value": x[j], "candidate_value": value[j], "nonincrease": bool(value[j] <= x[j] + 1e-9), "scale": "supplied_feature_scale", "source_observed": bool(np.isfinite(original[j])), "absolute_threshold_certified": False})
        checks.append({"beer": beer, "defect": "stale_hops", "source_value": base_pred[stale], "candidate_value": pp[stale], "nonincrease": bool(pp[stale] <= base_pred[stale] + 1e-9), "scale": "Q1_model_predicted_continuous_sensory", "source_observed": False, "absolute_threshold_certified": False})
        transfers.append({"beer": beer, "source_style": source, "target_style": target, **dict(zip(features, value))})
        transfer_sensory.append({"beer": beer, **dict(zip(targets, pp))})
        diagnostics.append({"beer": beer, "generator_role": "holdout" if i in set(test) else "train", "status": status,
                            "source_target_probability": base_prob, "candidate_target_probability": probability,
                            "cleaning_only_target_probability": cleaning_prob,
                            "supplied_scale_change_l2": float(np.linalg.norm(value - x)),
                            "chemical_nonincrease": bool(chemical_nonincrease(x, value, positions)),
                            "predicted_stale_hops_nonincrease": bool(pp[stale] <= base_pred[stale] + 1e-9),
                            "missing_source_defects": ",".join(missing_defects),
                            "out_of_training_envelope_fields": int(((value < generator.low - 1e-9) | (value > generator.high + 1e-9)).sum()),
                            "guarantee_scope": "measurement nonincrease plus learned stale proxy; physical and sensory experiment not performed"})
    save_csv(pd.DataFrame(transfers), dest / "pils_to_tripel_profiles.csv")
    save_csv(pd.DataFrame(transfer_sensory), dest / "pils_to_tripel_sensory_continuous.csv")
    save_csv(pd.DataFrame(diagnostics), dest / "transfer_diagnostics.csv")
    save_csv(pd.DataFrame(checks), dest / "defect_nonincrease_checks.csv")
    write_json(dest / "status.json", {
        "model": "conditional_PCA_Gaussian_with_partial_covariance_pooling",
        "generator_train_n": len(train), "generator_holdout_n": len(test),
        "style_classifier_holdout_balanced_accuracy": holdout_balanced,
        "primary_feature_scale": "supplied_mixed_transforms; see audit/field_scales.csv",
        "marginal_bounds": "training 1%-99% descriptor range; not a thermodynamic or brewing-feasibility guarantee",
        "aggregate_consistency": "aggregate descriptors generated jointly; exact chemical sum identities not established",
        "sensory_portraits": "Q1 full-data fitted predictor, model-based only; not an independent tasting validation",
        "defect_thresholds": "absolute thresholds incomplete and some field
```

```
    scales unresolved; no complete OAV certification",
        "stale_hops_constraint": "prediction proxy only; actual sensory non-worsening unverified",
        "zero_change_transfers": "explicitly reported as no nontrivial feasible candidate, never counted as successful style transfer",
    })
```

src/reporting.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from .common import progress, sha256, write_json


def make_report(data, out, cfg):
    out = Path(out)
    figures = out / "figures"
    figures.mkdir(exist_ok=True)
    progress("Report: preparing experiment summaries and scientific figures")
    metrics = pd.read_csv(out / "q1/metrics_summary.csv").set_index("model")
    per_target = pd.read_csv(out / "q1/metrics_by_target.csv")
    robustness = pd.read_csv(out / "q1/missingness_robustness.csv")
    top = pd.read_csv(out / "q2/top3_by_sensory_dimension.csv")
    jaccard = pd.read_csv(out / "q2/method_jaccard.csv")
    generation = pd.read_csv(out / "q3/generation_diagnostics.csv")
    transfers = pd.read_csv(out / "q3/transfer_diagnostics.csv")
    checks = pd.read_csv(out / "q3/defect_nonincrease_checks.csv")
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False, "figure.dpi": 140, "savefig.facecolor": "white"})
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    order = ["mean", "ridge", "pls", "extra_trees"]
    ax[0].bar(order, metrics.loc[order, "mae"], color=["#9b9b9b", "#286eaa", "#549778", "#bd833a"])
    ax[0].set(ylabel="Macro MAE (provided continuous scale)", title="Repeated out-of-fold prediction")
    byrate = robustness.groupby("additional_missing_fraction").mae.agg(["mean", "std"])
    ax[1].errorbar(byrate.index * 100, byrate["mean"], yerr=byrate["std"], marker="o", capsize=4, color="#286eaa")
    ax[1].set(xlabel="Additional missing cells (%)", ylabel="Fold MAE", title="Ridge missingness sensitivity")
    ax[1].text(.02, .98, "Bars: fold SD, not a confidence interval", transform=ax[1].transAxes, va="top", fontsize=8)
    fig.savefig(figures / "q1_validation.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    ax[0].hist(top.bootstrap_top3_frequency, bins=np.linspace(0, 1, 11), color="#286eaa", edgecolor="white")
    ax[0].set(xlabel="Bootstrap Top-3 inclusion frequency", ylabel="Number of selected factors", title="Stability of 150 selected factors")
    cols = ["graph_vs_ridge_jaccard", "graph_vs_spearman_jaccard", "ridge_vs_spearman_jaccard"]
    ax[1].boxplot([jaccard[c] for c in cols], tick_labels=["Graph / ridge", "Graph / rank corr.", "Ridge / rank corr."])
    ax[1].set(ylabel="Top-3 Jaccard", ylim=(-.05, 1.05), title="Agreement across methods")
    fig.savefig(figures / "q2_stability.png", dpi=170)
    plt.close(fig)
    portraits = pd.read_csv(out / "q3/generated_sensory_continuous.csv")
    selected = ["A_malt_all", "A_hops_all", "A_esters_all", "F_malt_all", "F_hops_all", "F_esters_all", "acidity", "bitterness", "sweetness", "body", "diacetyl", "dms", "stale_hops"]
    values = portraits.groupby("style")[selected].mean().reindex(cfg["generation_styles"])
    fig, ax = plt.subplots(figsize=(12, 3.4), layout="constrained")
    cap = max(.5, float(np.max(np.abs(values.to_numpy()))))
    im = ax.imshow(values, aspect="auto", cmap="RdBu_r", vmin=-cap, vmax=cap)
    ax.set_xticks(range(len(selected)), selected, rotation=45, ha="right")
    ax.set_yticks(range(len(values)), values.index)
    ax.set_title("Generated sensory portraits: model predictions on supplied continuous scale")
    fig.colorbar(im, ax=ax, label="Predicted continuous intensity", shrink=.85)
    fig.savefig(figures / "q3_sensory_portraits.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    x = np.arange(len(transfers))
    ax[0].plot(x, transfers.source_target_probability, "o--", color="#999999", label="Source")
    ax[0].plot(x, transfers.candidate_target_probability, "o-", color="#286eaa", label="Candidate")
    ax[0].set_xticks(x, transfers.beer.astype(str), rotation=45)
    ax[0].set(xlabel="Source beer", ylabel="Tripel probability (model only)", title="Candidate style movement")
    ax[0].legend()
    groups = checks.groupby("defect").nonincrease.agg(["sum", "count"])
    ax[1].barh(groups.index, groups["sum"] / groups["count"], color="#549778")
    ax[1].set(xlim=(0, 1.05), xlabel="Nonincrease fraction in checked quantity", title="Chemical measurements and stale-hops proxy")
    ax[1].text(.02, -.2, "Unchanged failures are included; actual tasting unverified", transform=ax[1].transAxes, fontsize=8)
    fig.savefig(figures / "q3_transfer_checks.png", dpi=170)
    plt.close(fig)
    facts = json.loads((out / "audit/verified_facts.json").read_text(encoding="utf-8"))
    q3status = json.loads((out / "q3/status.json").read_text(encoding="utf-8"))
    successful = int((transfers.status == "nontrivial_model_feasible_candidate").sum())
    heldout = transfers[transfers.generator_role == "holdout"]
    heldout_success = int((heldout.status == "nontrivial_model_feasible_candidate").sum())
    baseline = float(metrics.loc["mean", "mae"])
    ridge_mae = float(metrics.loc["ridge", "mae"])
    improvement = (baseline - ridge_mae) / baseline
    low = int((top.bootstrap_top3_frequency < .5).sum())
    report = f"""# A 题阶段成果报告

本阶段为 AI 辅助研究基线，三问均有可运行实现和阶段产物。数值只对应当前配置与当前附件。原始 0–6 分标定、真实感官改善及化学因果仍未完成验证，不能将整份报告直接作为最终参赛结论。

## 已核实的数据事实

实际有 {facts['n_samples']} 个样本、{facts['n_features']} 个化学或理化数值字段、{facts['n_targets']} 个连续感官目标和 {facts['n_styles']} 种风格。以 `beer_唯一键` 一对一关联，不使用受损 `beer_id`。附件相关矩阵与重算 Spearman 的平均绝对差为 {facts['spearman_matrix_mean_absolute_difference']:.6g}，与 Pearson 为 {facts['pearson_matrix_mean_absolute_difference']:.6g}。逐字段变换证据、模板检查、哈希和字典保存在 `audit/`。这些是文件证据，关于合理性的解释仍需队内复核。

## 第一问 处理 方法与结果

输入为附件 1 的数值字段，目标为附件 4 的 50 个连续感官字段。样本编号、条码和风格标签均不作为化合物预测特征。训练折内拟合 1%—99% 截尾、缺失中位数与标准化。采用按风格分层的 {cfg['outer_folds']} 折、{cfg['outer_repeats']} 次重复外层验证，岭回归和 PLS 的超参数在外层训练数据内再做 {cfg['inner_folds']} 折选择。每个样本的重复留出预测取平均。预先指定岭回归作为下游参考，未根据外层成绩选择冠军再声称独立测试性能。

| 方法 | 宏平均 MAE | 宏平均 RMSE | 宏平均 $R^2$ |
|---|---|---|---|
"""
    for name in order:
        v = metrics.loc[name]
        report += f"| {name} | {v.mae:.6f} | {v.rmse:.6f} | {v.r2:.6f} |\n"
    report += f"""

岭回归相对于训练均值参照的宏平均 MAE 变化为 {improvement:.2%} 的下降。该结果衡量给定连续标签的插值预测，不能当作真实 0–6 分精度，也不证明对全新风格的迁移能力。逐维误差、各风格误差、折分配和附加缺失压力实验均已保存。图中的折间标准差不是置信区间。

`ASSUMPTION_percentile_grades_0_6.csv` 采用训练折目标的七等频经验分位边界，只是有明确假设的参考等级。它不能恢复“未感知”等绝对评分语义，集中于同一值的目标也会出现重复分位边界。绝对标定仍为 UNRESOLVED。最终模型的全数据拟合系数仅供复现与下游画像，不用于报告样本外精度。

![第一问验证](figures/q1_validation.png)

## 第二问 处理 方法与结果

按全矩阵秩相关一致性核对附件 1 与 2 的变量别名。网络边保留绝对相关大于等于 {cfg['graph_threshold']} 的关系，保存符号与绝对权重。用符号拉普拉斯约束岭回归，求解 $(Z^TZ + \alpha I + \lambda L)B = Z^T(Y - \text{均值})$。L 的非对角元素为负的有符号相关权，度数取绝对权和，因而为半正定。固定 $\alpha$={cfg['graph_ridge_alpha']}、$\lambda$={cfg['graph_ridge_lambda']}；这是探索参数，尚未证明最优。

Top-3 仅从明确的化合物或元素字段中选择，排除 pH、色度、综合总量及混合指标。每个目标给出 3 项，共 {len(top)} 条，附方向、秩相关、网络模块、来源元数据及人工机理核查字段。{cfg['bootstrap_repeats']} 次样本 bootstrap 中重新拟合预处理和网络，再计算入选频率。与无图岭回归、边际秩相关 Top-3 比较 Jaccard。

当前选定因素中，入选频率低于 0.5 的有 {low}/{len(top)} 条，提示单个因素排序存在明显不稳定性。Graph/ridge 的平均 Jaccard 为 {jaccard.graph_vs_ridge_jaccard.mean():.4f}，Graph/Spearman 为 {jaccard.graph_vs_spearman_jaccard.mean():.4f}。这些差异应保留，不宜只展示一致的维度。来源与缺陷描述来自附件元数据，每个目标的具体化学机理论证仍需队员查阅原始文献并复核。全数据关联分析不构成独立预测验证或因果识别。

较高频候选包括 4VG 感官目标与 X4VG、金属味与 iron、烘烤麦芽感官与 methylpyrazine；它们仍只是当前重采样设计下的候选。某些目标排名出现反直觉化合物，说明模型和化学论证需要继续改进，不能把完整 Top-3 表当作已经确定的机理清单。
"""
```

```
114
115 ![第二问稳定性](figures/q2_stability.png)
116
117 ## 第三问 处理 方法与结果
118
119 分层留出 20% 样本作为生成模型诊断参考。训练数据上拟合预处理和 {cfg['
    generation_components']} 维 PCA；各风格采用独立均值，协方差按 n/(n+10) 与全体
    训练协方差部分汇聚。采样后还原到附件特征尺度并限制在训练 1%—99% 范围，生成每
    种指定风格 {cfg['generation_samples_per_style']} 组化合物或理化描述符。全数据
    拟合的第一问参考模型输出感官画像，属于模型推断。
120
121 风格分类器在生成模型留出集上的平衡准确率为 {q3status['style_classifier_
    holdout_balanced_accuracy']:.4f}。该分类器与生成模型共享训练资料，并不是外部
    独立鉴定；其保留样本表现与生成物风格概率仅作模型诊断。Kriek、Lambic 等风格的
    真实留出样本很少，KS/MMD 指标无法证明完整分布已被可靠学会。KS 仅比较各字段实
    际非缺失的留出观测，字段数量另列；MMD 使用训练预处理后的表示。详见 generation
    _diagnostics.csv，并保留真实训练样本重采样的参照 MMD。
122
123 跨风格迁移先在潜空间施加风格均值差，再搜索连续插值候选。DMS、双乙酰、4VG 和
    trans-2-nonenal 的附件字段逐项不得增加；Stale hops 没有直接化合物观测，以第一
    问的预测值不增加作为单独代理约束。候选还需使固定分类器的 Tripel 概率相对于原
    始样本以及仅做截尾的对照均增加超过 0.01，避免把异常清理效果混为风格迁移。未找
    到满足条件的非零改变候选时，保留原样并标为未找到，不计成功。
124
125 当前共有 {len(transfers)} 个 Pils/Lager 来源样本，找到非平凡模型内可行候选 {
    successful} 个；其中生成模型留出来源样本为 {len(heldout)} 个，找到候选 {
    heldout_success} 个。任何缺陷源值缺失、越出训练经验边界或仅能依赖预测代理的情
    况均在 transfer_diagnostics.csv 标注。所有字段检查保存在 defect_nonincrease_
    checks.csv。即使这些检查全部通过，也只能说明被检查的测量量或模型代理满足条
    件，不能声称实际品评无恶化。
126
127 ![第三问感官画像](figures/q3_sensory_portraits.png)
128
129 ![第三问迁移检查](figures/q3_transfer_checks.png)
130
131 ## 保留结果与下一阶段
132
133 可保留：原件哈希和样本键审计、逐字段尺度证据、既定验证下的误差表、Top-3 与重
    采样频率、生成样本与诊断、逐项迁移约束核查。它们均绑定 config.json 与运行环
    境。
134
135 需要继续研究：真实评分标定锚点；未确定尺度与异常值的来源；目标相关的化学解
    释；网络与生成参数敏感性；新的独立验证；聚合指标的物理一致性；完整阈值和真实
    感官实验。原始研究或其他项目中的已发表成绩不属于本次实测结果。所有采纳的 AI
    辅助成果须依赛事要求披露并由队内独立验证。
136
137 方法详见 docs/，来源详见 docs/SOURCES.md，运行日志详见 docs/RESEARCH_LOG.md。
138 """
139     (out / "STAGE_REPORT.md").write_text(report, encoding="utf-8")
140     checksums = [{"file": str(p.relative_to(out)).replace("\\", "/"), "sha256
    ": sha256(p)} for p in sorted(out.rglob("*")) if p.is_file() and "private"
    not in p.parts and p.suffix in {".csv", ".json", ".md", ".png"} and p.name
    not in ["result_checksums.json", "run_manifest.json"]]
141     write_json(out / "result_checksums.json", checksums)
```

src/stage2.py

```
1  """AI-assisted extension: OpenAI Codex / GPT-6, 2026-10-24.
2  Scientific limitations and all failed experiments must remain visible.
3  """
4  from pathlib import Path
5
6  import joblib
7  import numpy as np
8  import pandas as pd
9  from scipy.spatial.distance import cdist
10 from scipy.stats import ks_2samp, norm
11 from sklearn.ensemble import GradientBoostingRegressor
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.metrics import balanced_accuracy_score, mean_absolute_error
14 from sklearn.model_selection import RepeatedStratifiedKFold, train_test_split
15 from sklearn.multioutput import MultiOutputRegressor
16 from sklearn.pipeline import Pipeline
17
18 from .audit import run_audit
19 from .common import ecdf_grades, preprocessing, progress, save_csv, sha256,
    write_json
20 from .data import CHEMICAL_DEFECTS
21 from .q1 import fitted_model, metric_rows
22 from .q3 import ConditionalPCAGaussian, biased_mmd, chemical_nonincrease
23
24
25 def conformal_quantile(errors, alpha):
26     n = len(errors)
27     rank = int(np.ceil((n + 1) * (1 - alpha)))
28     if rank > n:
29         raise ValueError("Calibration set too small for requested coverage")
30     return np.sort(np.asarray(errors), axis=0)[rank - 1]
31
32
33 def assumption_grades(reference, pred):
34     reference, pred = np.asarray(reference), np.asarray(pred)
35     empirical = ecdf_grades(reference, pred)
36     mu, sd = reference.mean(0), np.maximum(reference.std(0), 1e-8)
37     gaussian = np.minimum(6, np.floor(7 * norm.cdf((pred - mu) / sd))).astype
    (int)
38     low, high = np.quantile(reference, [.05, .95], axis=0)
39     linear = np.rint(6 * np.clip((pred - low) / np.maximum(high - low, 1e-8),
     0, 1)).astype(int)
40     return np.stack([empirical, gaussian, linear])
41
42
43 def local_convex_samples(real, n, rng):
44     """Conditional empirical local mixing; not a physical composition law."""
45     real = np.asarray(real)
46     scale = np.maximum(np.std(real, axis=0), 1e-8)
47     dist = cdist(real / scale, real / scale)
48     np.fill_diagonal(dist, np.inf)
49     neighbors = np.argsort(dist, axis=1)[:, :min(3, len(real) - 1)]
50     out = []
51     for _ in range(n):
52         anchor = rng.integers(len(real))
53         partner = rng.choice(neighbors[anchor]) if len(real) > 1 else anchor
54         weight = rng.uniform(.1, .9)
55         out.append(weight * real[anchor] + (1 - weight) * real[partner])
56     return np.asarray(out)
57
58
59 def run_extension(data, base, out, cfg, ext):
60     manifest = pd.read_csv(base / "audit/source_manifest.csv")
61     for row in manifest.itertuples():
62         if sha256(data.root / row.file) != row.sha256:
63             raise ValueError("Source version differs from stage one")
64     run_audit(data, out, cfg)
65     X, Y = data.X.to_numpy(), data.Y.to_numpy()
66     labels = data.meta.tasting_category_fine.to_numpy()
67     targets, features = data.Y.columns.tolist(), data.X.columns.tolist()
68     for sub in ["q1", "q2", "q3"]:
69         (out / sub).mkdir(exist_ok=True)
70     splits = list(RepeatedStratifiedKFold(n_splits=3, n_repeats=2, random_
    state=cfg["seed"]).split(X, labels))
71     sums = np.zeros_like(Y)
72     counts = np.zeros(len(Y))
73     permutation_sums = np.zeros((X.shape[1], Y.shape[1]))
74     positive_folds = np.zeros_like(permutation_sums)
75     grade_min, grade_max = np.full_like(Y, 6), np.zeros_like(Y)
76     grade_disagreement, assignments = [], []
77     for fold, (train, test) in enumerate(splits):
78         progress(f"Stage 2 Q1/Q2: held-out fold {fold + 1}/6")
79         model = Pipeline([("prep", preprocessing()), ("model",
    MultiOutputRegressor(GradientBoostingRegressor(n_estimators=ext["gbr_
    estimators"], learning_rate=ext["gbr_learning_rate"], max_depth=ext["gbr_
    depth"], min_samples_leaf=5, max_features=.7, random_state=cfg["seed"]), n_
    jobs=2))])
80         # Threads avoid process resource-tracker failures on Unicode Windows
    temp paths.
81         with joblib.parallel_backend("threading"):
82             model.fit(X[train], Y[train])
83             prediction = model.predict(X[test])
84         sums[test] += prediction
85         counts[test] += 1
86         assignments.extend({"repeat": fold // 3, "fold": fold % 3, "beer":
    int(data.meta.iloc[i].beer)} for i in test)
87         ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
88         ridge_pred = ridge.predict(X[test])
89         grades = assumption_grades(Y[train], ridge_pred)
90         grade_min[test] = np.minimum(grade_min[test], grades.min(axis=0))
91         grade_max[test] = np.maximum(grade_max[test], grades.max(axis=0))
92         grade_disagreement.append({"fold": fold, "any_rule_disagreement_
    fraction": float((grades.max(0) != grades.min(0)).mean()), "mean_grade_spread
    ": float((grades.max(0) - grades.min(0)).mean())})
93         # Exact linear predictor permutation. The trained preprocessing is
    frozen.
94         Z = ridge.named_steps["prep"].transform(X[test])
95         coefficients = ridge.named_steps["model"].coef_
96         baseline_error = np.abs(Y[test] - ridge_pred).mean(axis=0)
97         rng = np.random.default_rng(cfg["seed"] + fold)
98         deltas = np.zeros_like(permutation_sums)
99         for j in range(X.shape[1]):
100            for _ in range(ext["permutation_repeats"]):
101                permuted = Z[rng.permutation(len(Z)), j] - Z[:, j]
102                pp = ridge_pred + permuted[:, None] * coefficients[:, j]
103                deltas[j] += np.abs(Y[test] - pp).mean(axis=0) - baseline_
    error
104        deltas /= ext["permutation_repeats"]
105        permutation_sums += deltas
106        positive_folds += deltas > 0
107    gbr = sums / counts[:, None]
108    existing = pd.read_csv(base / "q1/metrics_by_target.csv")
109    additional = pd.DataFrame(metric_rows(Y, gbr, targets, "gbr_fixed"))
110    ridge_oof = pd.read_csv(base / "q1/oof_ridge_continuous.csv")[targets].to
    _numpy()
```

```
    tree_oof = pd.read_csv(base / "q1/oof_extra_trees_continuous.csv")[targets].to_numpy()
    blend = (ridge_oof + tree_oof) / 2
    additional = pd.concat([additional, pd.DataFrame(metric_rows(Y, blend, targets, "fixed_ridge_tree_blend"))], ignore_index=True)
    all_metrics = pd.concat([existing, additional], ignore_index=True)
    save_csv(all_metrics, out / "q1/metrics_by_target.csv")
    save_csv(all_metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), out / "q1/metrics_summary.csv")
    save_csv(pd.concat([data.meta[["beer"]], pd.DataFrame(gbr, columns=targets)], axis=1), out / "q1/oof_gbr_continuous.csv")
    save_csv(pd.DataFrame(assignments), out / "q1/split_assignments.csv")
    save_csv(pd.DataFrame(grade_disagreement), out / "q1/grade_rule_sensitivity.csv")
    grade_ranges = pd.concat([data.meta[["beer"]], pd.DataFrame(grade_min.astype(int), columns=[c + "__min" for c in targets]), pd.DataFrame(grade_max.astype(int), columns=[c + "__max" for c in targets])], axis=1)
    grade_ranges["status"] = "range_over_assumed_rules_and_CV_fits_not_a_confidence_interval"
    save_csv(grade_ranges, out / "q1/ASSUMPTION_grade_ranges.csv")

    # Proper train / calibration / test split for one demonstrative conformal experiment.
    train, rest = train_test_split(np.arange(len(X)), test_size=.4, random_state=cfg["seed"] + 71)
    calibration, test = train_test_split(rest, test_size=.5, random_state=cfg["seed"] + 72)
    ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
    q = conformal_quantile(np.abs(Y[calibration] - ridge.predict(X[calibration])), ext["conformal_alpha"])
    pp = ridge.predict(X[test])
    coverage = (np.abs(Y[test] - pp) <= q).mean(axis=0)
    save_csv(pd.DataFrame({"target": targets, "nominal_marginal_coverage": .9, "empirical_test_coverage": coverage, "interval_width": 2 * q, "calibration_n": len(calibration), "test_n": len(test)}), out / "q1/conformal_coverage.csv")
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "calibration" if i in set(calibration) else "test" for i in range(len(X))]}), out / "q1/conformal_split.csv")

    progress("Stage 2 Q2: assembling held-out attribution evidence")
    importances = permutation_sums / len(splits)
    rows = [{"target": target, "field": field, "heldout_permutation_MAE_increase": importances[j, k], "positive_fold_fraction": positive_folds[j, k] / len(splits)} for k, target in enumerate(targets) for j, field in enumerate(features)]
    importance_frame = pd.DataFrame(rows)
    save_csv(importance_frame, out / "q2/heldout_permutation_importance.csv")
    top = pd.read_csv(base / "q2/top3_by_sensory_dimension.csv").merge(importance_frame, on=["target", "field"], validate="one_to_one")
    top["evidence_status"] = np.where((top.bootstrap_top3_frequency >= .5) & (top.heldout_permutation_MAE_increase > 0) & (top.positive_fold_fraction >= 2 / 3), "comparatively_supported_association", "weak_or_unstable_association")
    top["causality"] = "not_established"
    save_csv(top, out / "q2/top3_evidence_review.csv")

    progress("Stage 2 Q3: comparing conditional generators across held-out splits")
    rng = np.random.default_rng(cfg["seed"])
    results = []
    final_generator, final_classifier, generated_final = None, None, []
    first_train = None
    for repeat in range(ext["generation_repeats"]):
        train, test = train_test_split(np.arange(len(X)), test_size=.2, stratify=labels, random_state=cfg["seed"] + repeat)
        generator = ConditionalPCAGaussian(10, cfg["seed"]).fit(X[train], labels[train])
        clf = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000, class_weight="balanced", random_state=cfg["seed"]))]).fit(X[train], labels[train])
        acc = balanced_accuracy_score(labels[test], clf.predict(X[test]))
        for style in ext["generation_styles"]:
            real = X[test[labels[test] == style]]
            real_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(X[train[labels[train] == style]]))
            samples = {"bootstrap": real_train[rng.integers(0, len(real_train), ext["generation_n"])], "local_convex": local_convex_samples(real_train, ext["generation_n"], rng), "PCA_Gaussian": generator.sample(style, ext["generation_n"], rng)}
            for method, values in samples.items():
                Z = generator.prep.transform(values)
                reference = generator.prep.transform(real)
                distances = cdist(Z, generator.prep.transform(X[train]))
                ks = [ks_2samp(values[:, j], real[:, j][np.isfinite(real[:, j])]).statistic for j in range(X.shape[1]) if np.isfinite(real[:, j]).any()]
                results.append({"repeat": repeat, "style": style, "method": method, "n_real_holdout": len(real), "KS_mean": np.mean(ks), "MMD2": biased_mmd(Z, reference), "exact_training_copy_fraction": float((distances.min(1) < 1e-8).mean()), "target_style_probability": clf.predict_proba(values)[:, list(clf.classes_).index(style)].mean(), "style_classifier_holdout_balanced_accuracy": acc})
                if repeat == 0 and method == "local_convex":
                    frame = pd.DataFrame(values, columns=features)
                    frame.insert(0, "style", style)
                    frame.insert(0, "generated_id", [f"LC_{style}_{j+1:03d}" for j in range(len(values))])
                    generated_final.append(frame)
        if repeat == 0:
            final_generator, final_classifier, first_train = generator, clf, train
    diagnostics = pd.DataFrame(results)
    save_csv(diagnostics, out / "q3/generator_comparison_by_split.csv")
    save_csv(diagnostics.groupby(["method", "style"])[["KS_mean", "MMD2", "exact_training_copy_fraction", "target_style_probability"]].mean().reset_index(), out / "q3/generator_comparison_mean.csv")
    generated = pd.concat(generated_final, ignore_index=True)
    save_csv(generated, out / "q3/local_convex_generated_profiles.csv")
    bundle = joblib.load(base / "q1/private/model.joblib")
    predictor = bundle["model"]
    portraits = pd.concat([generated[["generated_id", "style"]], pd.DataFrame(predictor.predict(generated[features].to_numpy()), columns=targets)], axis=1)
    save_csv(portraits, out / "q3/local_convex_sensory_continuous.csv")
    transfer_results = expanded_transfer(data, final_generator, final_classifier, first_train, predictor, ext)
    save_csv(transfer_results[0], out / "q3/expanded_transfer_diagnostics.csv")
    save_csv(transfer_results[1], out / "q3/expanded_transfer_profiles.csv")
    save_csv(transfer_results[2], out / "q3/expanded_transfer_defect_checks.csv")
    write_json(out / "status.json", {
        "absolute_score_calibration": "not_identifiable_from_provided_aggregated_per_taster_standardized_scores_without_additional_anchors",
        "grade_sensitivity_disagreement": float(pd.DataFrame(grade_disagreement).any_rule_disagreement_fraction.mean()),
        "conformal_nominal_marginal_coverage": .9, "conformal_empirical_average_coverage": float(coverage.mean()),
        "conformal_caveat": "single random split; marginal coverage on provided continuous labels, not simultaneous or original-scale coverage",
        "comparatively_supported_top3_associations": int((top.evidence_status == "comparatively_supported_association").sum()),
        "top3_total": len(top), "generation_repeated_splits": ext["generation_repeats"],
        "expanded_transfer_nontrivial": int((transfer_results[0].status == "nontrivial_model_feasible_candidate").sum()),
        "physical_or_tasting_validation": "not_performed", "research_stage": "exploratory_extension_after_stage_one_not_an_untouched_external_test",
    })


def expanded_transfer(data, generator, classifier, train, predictor, ext):
    X = data.X.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    labels = data.meta.tasting_category_fine.to_numpy()
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target_train = X[train[labels[train] == "Tripel"]]
    target_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(target_train))
    target_class = list(classifier.classes_).index("Tripel")
    diagnostics, profiles, checks = [], [], []
    for i in np.flatnonzero(labels == "Pils/Lager"):
        x = np.where(np.isfinite(X[i]), X[i], generator.median)
        source_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, target_class]
        cleaned = np.clip(x, generator.low, generator.high)
        cleaned[positions] = np.minimum(cleaned[positions], x[positions])
        clean_prob = classifier.predict_proba(cleaned[None])[0, target_class]
        candidates = []
        for target in target_train:
            for weight in np.linspace(.025, 1, ext["transfer_weights"]):
                value = np.clip(x + weight * (target - x), generator.low, generator.high)
                value[positions] = np.minimum(value[positions], x[positions])
                candidates.append(value)
        candidates = np.asarray(candidates)
        pp = predictor.predict(candidates)
        probabilities = classifier.predict_proba(candidates)[:, target_class]
        valid = chemical_nonincrease(x, candidates, positions) & (pp[:, stale] <= source_pred[stale] + 1e-9) & (probabilities > max(base_prob, clean_prob) + .01)
        if valid.any():
            ids = np.flatnonzero(valid)
            j = ids[np.argmax(probabilities[ids])]
            value, portrait, prob, status = candidates[j], pp[j], probabilities[j], "nontrivial_model_feasible_candidate"
        else:
            value, portrait, prob, status = x, source_pred, base_prob, "no_nontrivial_candidate_found_source_retained"
        beer = int(data.meta.iloc[i].beer)
        missing = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(X[i, j])]
        diagnostics.append({"beer": beer, "status": status, "generator_role": "train" if i in set(train) else "holdout", "source_trip_probability": base_prob, "candidate_trip_probability": prob, "cleaning_only_probability": clean_prob, "change_l2": np.linalg.norm(value - x), "missing_source_defects": ",".
```

```
    join(missing), "all_source_defects_observed": not missing, "stale_constraint
    ": "predicted_proxy_only"})
232         profiles.append({"beer": beer, **dict(zip(features, value))})
233         for name, j in zip(CHEMICAL_DEFECTS, positions):
234             checks.append({"beer": beer, "defect": name, "source": x[j], "
    candidate": value[j], "nonincrease": bool(value[j] <= x[j] + 1e-9), "source_
    observed": bool(np.isfinite(X[i, j]))), "scale": "provided_measurement"})
235         checks.append({"beer": beer, "defect": "stale_hops", "source": source
    _pred[stale], "candidate": portrait[stale], "nonincrease": bool(portrait[
    stale] <= source_pred[stale] + 1e-9), "source_observed": False, "scale": "
    learned_sensory_proxy"})
236     return pd.DataFrame(diagnostics), pd.DataFrame(profiles), pd.DataFrame(
    checks)
```

src/stage3_algorithms.py

```
1   """Stage-three algorithm prototypes; AI-assisted, OpenAI Codex, 2026-10-25.
2
3   These helpers validate mathematical properties. They are not evidence that
    the
4   proposed full study improves real prediction or chemical feasibility.
5   """
6   import numpy as np
7
8
9   def fixed_grade_boundaries(training_targets):
10      """Fit once on training labels; never refit with calibration/test labels
    ."""
11      y = np.asarray(training_targets, float)
12      if y.ndim != 2 or not np.isfinite(y).all() or len(y) < 2:
13          raise ValueError("Expected at least two finite training rows")
14      return np.quantile(y, np.arange(1, 7) / 7, axis=0)
15
16
17  def grade_with_fixed_boundaries(values, boundaries):
18      values, boundaries = np.asarray(values, float), np.asarray(boundaries,
    float)
19      if values.ndim != 2 or boundaries.shape != (6, values.shape[1]):
20          raise ValueError("Grade dimensions do not agree")
21      if not np.isfinite(values).all() or not np.isfinite(boundaries).all() or
    np.any(np.diff(boundaries, axis=0) < 0):
22          raise ValueError("Need finite values and ordered boundaries")
23      return (values[:, None, :] > boundaries[None, :, :]).sum(axis=1).astype(
    int)
24
25
26  def ordinal_interval_image(lower, upper, boundaries):
27      """Conservative image of a continuous interval under a fixed monotone map
    .
28
29      Coverage, if applicable, is for this operationally defined grade only.
30      Tied boundaries can make the interval hull include unreachable categories
    .
31      """
32      lower, upper = np.asarray(lower, float), np.asarray(upper, float)
33      if lower.shape != upper.shape or np.any(lower > upper):
34          raise ValueError("Invalid continuous intervals")
35      return grade_with_fixed_boundaries(lower, boundaries), grade_with_fixed_
    boundaries(upper, boundaries)
36
37
38  def shrunk_blend_weights(y_inner_oof, et_inner_oof, gbr_inner_oof, penalty):
39      """Fit using inner OOF predictions only; choose penalty within inner
    training.
40
41      w[j] multiplies GBR and (1-w[j]) multiplies ExtraTrees. The closed-form
42      objective is mean squared error plus penalty*(w[j]-global_weight)**2.
43      """
44      y, et, gbr = [np.asarray(a, float) for a in (y_inner_oof, et_inner_oof,
    gbr_inner_oof)]
45      if y.ndim != 2 or et.shape != y.shape or gbr.shape != y.shape or penalty
    < 0:
46          raise ValueError("Invalid blend inputs")
47      if not all(np.isfinite(a).all() for a in (y, et, gbr)):
48          raise ValueError("Blend arrays must be finite")
49      diff, residual = gbr - et, y - et
50      denom = np.mean(diff * diff)
51      global_weight = float(np.clip(np.mean(diff * residual) / denom, 0, 1)) if
     denom > 1e-14 else 0.5
52      local_denom = np.mean(diff * diff, axis=0) + penalty
53      numerator = np.mean(diff * residual, axis=0) + penalty * global_weight
54      weights = np.divide(numerator, local_denom, out=np.full(y.shape[1],
    global_weight), where=local_denom > 1e-14)
55      return np.clip(weights, 0, 1), global_weight
56
57
58  def paired_oof_bootstrap(y, pred_a, pred_b, strata, repeats=2000, seed
    =20261025):
59      """Stratified beer-level resampling of fixed predictions; NOT a
    retraining CI.
60
61      Inputs must have one row per unique beer, with repeats already averaged.
62      Negative differences favour A. Macro MAE is average over rows and targets
    .
63      """
64      y, a, b = [np.asarray(x, float) for x in (y, pred_a, pred_b)]
65      labels = np.asarray(strata)
66      if y.ndim != 2 or a.shape != y.shape or b.shape != y.shape or len(labels)
     != len(y):
67          raise ValueError("Paired dimensions differ")
68      if repeats < 1 or not all(np.isfinite(x).all() for x in (y, a, b)):
69          raise ValueError("Need finite paired predictions and positive repeats
    ")
70      per_beer = (np.abs(y-a)-np.abs(y-b)).mean(axis=1)
71      groups = [np.flatnonzero(labels == label) for label in np.unique(labels)]
72      rng = np.random.default_rng(seed)
73      draws = np.array([np.mean(per_beer[np.concatenate([rng.choice(g, len(g),
    replace=True) for g in groups])]) for _ in range(repeats)])
74      return {"difference_A_minus_B": float(per_beer.mean()), "conditional_
    percentile_95": np.quantile(draws, [.025, .975]).tolist(),
75              "scope": "conditional_on_fixed_OOF_predictions_does_not_include_
    model_retraining_or_selection_uncertainty",
76              "n_unique_beers": len(y), "resamples": repeats}
77
78
79  def pareto_minimize(objectives):
80      """Keep non-dominated finite candidates; no weighting can hide a
    violation."""
81      values = np.asarray(objectives, float)
82      if values.ndim != 2:
83          raise ValueError("Expected candidates by objectives")
84      keep = np.isfinite(values).all(axis=1)
85      for i in np.flatnonzero(keep):
86          dominated = np.all(values <= values[i], axis=1) & np.any(values <
    values[i], axis=1) & np.isfinite(values).all(axis=1)
87          if dominated.any():
88              keep[i] = False
89      return keep
90
91
92  def threshold_excess(concentration, threshold):
93      """Use only after unit and scale verification; invalid inputs stay
    unknown."""
94      concentration, threshold = np.broadcast_arrays(np.asarray(concentration,
    float), np.asarray(threshold, float))
95      valid = np.isfinite(concentration) & np.isfinite(threshold) & (
    concentration >= 0) & (threshold > 0)
96      out = np.full(concentration.shape, np.nan)
97      out[valid] = np.maximum(concentration[valid] / threshold[valid] - 1, 0)
98      return out
```

src/stage3_generation.py

```
1   """Disjoint-search/evaluation conditional generation and Pareto transfer.
2   AI-assisted reference implementation; no actual brewing or tasting validation
    .
3   """
4   from pathlib import Path
5   import hashlib
6   import time
7   import numpy as np
8   import pandas as pd
9   from scipy.spatial.distance import cdist
10  from scipy.stats import ks_2samp
11  from sklearn.ensemble import ExtraTreesClassifier, ExtraTreesRegressor
12  from sklearn.linear_model import LogisticRegression, Ridge
13  from sklearn.model_selection import train_test_split
14  from sklearn.metrics import balanced_accuracy_score
15  from sklearn.pipeline import Pipeline
16  from .common import preprocessing, save_csv, write_json, progress
17  from .data import AGGREGATES, CHEMICAL_DEFECTS
18  from .q3 import ConditionalPCAGaussian
19  from .stage2 import local_convex_samples
20  from .stage3_algorithms import pareto_minimize, threshold_excess
21
22
23  def mmd_fixed_bandwidth(a,b,bandwidth):
24      if len(a)<2 or len(b)<2: return np.nan
25      return float(np.exp(-cdist(a,a,'sqeuclidean')/(2*bandwidth)).mean()+np.
    exp(-cdist(b,b,'sqeuclidean')/(2*bandwidth)).mean()-2*np.exp(-cdist(a,b,'
    sqeuclidean')/(2*bandwidth)).mean())
26
27
28  def loo_support_radius(z):
29      if len(z)<2: return np.nan
30      distance=cdist(z,z); np.fill_diagonal(distance,np.inf)
31      return float(np.quantile(distance.min(1),.95))
```

```

def constrained_convex(real,generator,style_positions,n,rng):
    Z=generator.prep.transform(real); distance=cdist(Z,Z); np.fill_diagonal(distance,np.inf)
    neighbors=np.argsort(distance,axis=1)[:,:min(3,len(real)-1)]; radius=loo_support_radius(Z); result=[]; attempts=0
    if len(real)<2: return real[np.zeros(n,dtype=int)].copy(),{'attempts':n,'singleton_fallback':True,'support_radius':None}
    while len(result)<n and attempts<n*250:
        attempts+=1; a=rng.integers(len(real)); b=rng.choice(neighbors[a]); weight=rng.uniform(.1,.9)
        value=weight*real[a]+(1-weight)*real[b]
        for j in style_positions:
            value[j]=np.log10(weight*10**real[a,j]+(1-weight)*10**real[b,j])
        value=np.clip(value,generator.low,generator.high)
        if cdist(generator.prep.transform(value[None]),Z).min()<=radius+1e-9: result.append(value)
    return np.asarray(result).reshape((-1,real.shape[1])),{'attempts':attempts,'singleton_fallback':False,'support_radius':radius}


def locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg):
    positions=[features.index(t) for t in CHEMICAL_DEFECTS]; logpos=[features.index(t) for t in ['DMS','X4VG']]
    target_class=list(searcher.classes_).index('Tripel'); stale=targets.index('stale_hops')
    target_data=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]=='Tripel']]))
    target_Z=generator.prep.transform(target_data); support=loo_support_radius(target_Z)
    result=[]
    for i in np.flatnonzero(labels=='Pils/Lager'):
        source=np.where(np.isfinite(X[i]),X[i],generator.median); sourceZ=generator.prep.transform(source[None])
        source_p=searcher.predict_proba(source[None])[0,target_class]; stale_a=sensory_a.predict(source[None])[0,stale]; stale_c=consensus_a.predict(source[None])[0,stale]
        cleaned=np.clip(source,generator.low,generator.high); cleaned[positions]=np.minimum(cleaned[positions],source[positions]); clean_p=searcher.predict_proba(cleaned[None])[0,target_class]
        values=[]
        for anchor in target_data:
            for weight in np.linspace(.025,1,cfg['transfer_weights']):
                v=np.clip(source+weight*(anchor-source),generator.low,generator.high); v[positions]=np.minimum(v[positions],source[positions]); values.append(v)
        values=np.asarray(values); Z=generator.prep.transform(values); changes=np.linalg.norm(Z-sourceZ,axis=1)/np.sqrt(len(features)); support_distance=cdist(Z,target_Z).min(1)
        probabilities=searcher.predict_proba(values)[:,target_class]; pa=sensory_a.predict(values); pc=consensus_a.predict(values)
        bounds=((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)
        nonincrease=(values[:,positions]<=source[positions]+1e-9).all(1)
        known_proxy_risk=np.maximum(threshold_excess(10**values[:,logpos[0]],19.),threshold_excess(10**values[:,logpos[1]],.2))
        valid=bounds&nonincrease&(pa[:,stale]<=stale_a+1e-9)&(pc[:,stale]<=stale_c+1e-9)&(probabilities>=max(cfg['transfer_target_probability_floor'],source_p+.01,clean_p+.01))&(support_distance<=support+1e-9)&(changes>1e-8)
        for radius in cfg['trust_radii']:
            ids=np.flatnonzero(valid&(changes<=radius)); pareto_ids=[]
            if len(ids):
                nd=pareto_minimize(np.c_[changes[ids],support_distance[ids],known_proxy_risk[ids]]); pareto_ids=ids[nd]; j=int(pareto_ids[np.argmin(changes[pareto_ids])]); value=values[j]; status='model_feasible_nontrivial' if np.isfinite(X[i,positions]).all() else 'conditional_on_imputed_source_defect'
                p=float(probabilities[j]); change=float(changes[j]); target_distance=float(support_distance[j])
            else:
                value=source.copy(); status='no_feasible_nontrivial_candidate_source_retained'; p=float(source_p); change=0.; target_distance=float(cdist(sourceZ,target_Z).min())
            result.append({'row':int(i),'beer':int(beers[i]),'radius':radius,'source':source.copy(),'candidate':value.copy(),'status':status,'n_candidates':len(values),'n_valid':len(ids),'n_pareto':len(pareto_ids),
                'search_source_probability':float(source_p),'search_candidate_probability':p,'standardized_change':change,'target_support_distance':target_distance,
                'all_provided_source_defects_observed':bool(np.isfinite(X[i,positions]).all()),'missing_defects':','.join(features[j] for j in positions if not np.isfinite(X[i,j]))})
    return result


def run_generation(data,out,cfg):
    start=time.perf_counter(); dest=Path(out)/'q3'; dest.mkdir(parents=True,exist_ok=True)
    features=[f for f in data.X if f not in AGGREGATES]; targets=list(data.Y); X=data.X[features].to_numpy(); Y=data.Y.to_numpy(); labels=data.meta.tasting_category_fine.to_numpy(); beers=data.meta.beer.to_numpy()
    core=[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]; log_positions=[features.index(f) for f in ['DMS','X4VG']]; stale=targets.index('stale_hops'); positions=[features.index(f) for f in CHEMICAL_DEFECTS]
    metrics=[]; transfers=[]; checks=[]; roles=[]; evaluator_rows=[]; locked=[]; profiles=[]; portraits=[]; generated_profiles=[]
    for repeat in range(cfg['generation_repeats']):
        seed=cfg['seed']+repeat; rng=np.random.default_rng(seed); progress(f'S3 Q3 disjoint partition {repeat+1}/5')
        dev,T=train_test_split(np.arange(len(X)),test_size=50,stratify=labels,random_state=seed)
        G,J=train_test_split(dev,test_size=100,stratify=labels[dev],random_state=seed+999)
        assert len(G)==len(J)==100 and len(T)==50 and not(set(G)&set(J) or set(G)&set(T) or set(J)&set(T))
        roles.extend({'repeat':repeat,'beer':int(beers[i]),'role':'G_search_training' if i in set(G) else 'J_evaluator_training' if i in set(J) else 'T_real_holdout'} for i in range(len(X)))
        generator=ConditionalPCAGaussian(10,seed).fit(X[G],labels[G])
        searcher=Pipeline([('prep',preprocessing()),('model',LogisticRegression(C=1,max_iter=2000,class_weight='balanced',random_state=seed))]).fit(X[G],labels[G])
        sensory_a=Pipeline([('prep',preprocessing()),('model',Ridge(alpha=100))]).fit(X[G],Y[G])
        consensus_a=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed,n_jobs=2))]).fit(X[G],Y[G])
        generated=[]
        for style in cfg['generation_styles']:
            real=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]==style]]))
            constrained,info=constrained_convex(real,generator,log_positions,cfg['generation_n'],rng)
            samples={'bootstrap':real[rng.integers(0,len(real),cfg['generation_n'])],'local_convex':local_convex_samples(real,cfg['generation_n'],rng),
                'PCA_Gaussian':generator.sample(style,cfg['generation_n'],rng),'constrained_local_convex':constrained}
            for method,values in samples.items(): generated.append({'style':style,'method':method,'values':values,'training_n':len(real),'extra':info if method=='constrained_local_convex' else {}})
        selected=locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg)
        digest=hashlib.sha256()
        for group in generated: digest.update(group['values'].tobytes())
        for row in selected: digest.update(row['candidate'].tobytes())
        lock={'repeat':repeat,'candidate_sha256':digest.hexdigest(),'selection_used_J_or_T':False,'n_generated':sum(len(a['values']) for a in generated),'n_transfer_records':len(selected)}
        locked.append(lock); write_json(dest/f'locked_candidates_{repeat}.json',lock)
        # J models are fitted only after candidates and their order are fixed.
        evaluator=Pipeline([('prep',preprocessing()),('model',ExtraTreesClassifier(n_estimators=200,min_samples_leaf=2,max_features=.7,class_weight='balanced',random_state=seed+3000,n_jobs=2))]).fit(X[J],labels[J])
        sensory_b=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed+4000,n_jobs=2))]).fit(X[J],Y[J])
        acc_a=balanced_accuracy_score(labels[T],searcher.predict(X[T])); acc_b=balanced_accuracy_score(labels[T],evaluator.predict(X[T])); y_a=sensory_a.predict(X[T]); y_b=sensory_b.predict(X[T])
        evaluator_rows.append({'repeat':repeat,'searcher_holdout_balanced_accuracy':acc_a,'evaluator_holdout_balanced_accuracy':acc_b,'search_sensory_core30_mae':np.abs(Y[T][:,core]-y_a[:,core]).mean(),'evaluator_sensory_core30_mae':np.abs(Y[T][:,core]-y_b[:,core]).mean()})
        allG=generator.prep.transform(X[G]); distance=cdist(allG,allG,'sqeuclidean'); bandwidth=float(np.median(distance[distance>0]))
        for group in generated:
            style,method,values=group['style'],group['method'],group['values']; test=X[T[labels[T]==style]]; Z=generator.prep.transform(values) if len(values) else np.empty((0,len(features)))
            if not len(values):
                metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':0,'n_G_style':group['training_n'],'n_T_style':len(test),'status':'generation_budget_exhausted_no_sample'}); continue
            testZ=generator.prep.transform(test); nn=cdist(Z,allG).min(1)
            prob_a=searcher.predict_proba(values)[:,list(searcher.classes_).index(style)]; prob_b=evaluator.predict_proba(values)[:,list(evaluator.classes_).index(style)]
            jstyle=X[J[labels[J]==style]]; jz=evaluator.named_steps['prep'].transform(jstyle); threshold=loo_support_radius(jz); distances_b=cdist(evaluator.named_steps['prep'].transform(values),jz).min(1)
            ks=[ks_2samp(values[:,j],test[:,j][np.isfinite(test[:,j])]).statistic for j in range(X.shape[1]) if np.isfinite(test[:,j]).any()]
            metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':len(values),'n_G_style':group['training_n'],'n_J_style':len(jstyle),'n_T_style':len(test),
                'MMD2_fixed_training_bandwidth':mmd_fixed_bandwidth(Z,testZ,bandwidth),'KS_mean':np.mean(ks),'exact_copy_fraction':(nn<1e-8).mean(),
                'search_probability':prob_a.mean(),'independent_evaluator_probability':prob_b.mean(),'independent_evaluator_target_prediction_fraction':np.mean(evaluator.predict(values)==style),
```

```
                'J_support_fraction':np.mean(distances_b<=threshold) if np.isfinite(threshold) else np.nan,
                'provided_bounds_violation_fraction':np.mean(~((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)),
                'sensory_proxy_disagreement':np.abs(sensory_a.predict(values)-sensory_b.predict(values)).mean(),
                'bandwidth_from_G':bandwidth,'status':'statistical_profiles_not_actual_tasting','singleton_fallback':group['extra'].get('singleton_fallback',False)})
            if repeat==0:
                ids=[f'S3_{method}_{style}_{i+1:03d}' for i in range(len(values))]
                frame=pd.DataFrame(values,columns=features); frame.insert(0,'style',style); frame.insert(0,'method',method); frame.insert(0,'generated_id',ids); generated_profiles.append(frame)
                if method=='constrained_local_convex':
                    portrait=pd.DataFrame(sensory_a.predict(values),columns=targets); portrait.insert(0,'style',style); portrait.insert(0,'generated_id',ids); portraits.append(portrait)
        for row in selected:
            i=row['row']; source,candidate=row['source'],row['candidate']; pb0=evaluator.predict_proba(source[None])[0,list(evaluator.classes_).index('Tripel')]; pb1=evaluator.predict_proba(candidate[None])[0,list(evaluator.classes_).index('Tripel')]
            sb0=sensory_b.predict(source[None])[0,stale]; sb1=sensory_b.predict(candidate[None])[0,stale]
            nontrivial=row['status']!='no_feasible_nontrivial_candidate_source_retained'
            b_accept=nontrivial and pb1>pb0+.01 and evaluator.predict(candidate[None])[0]=='Tripel' and sb1<=sb0+1e-9
            record={k:v for k,v in row.items() if k not in ['row','source','candidate']}
            record.update(repeat=repeat,source_role='G_search_training' if i in set(G) else 'J_evaluator_training' if i in set(J) else 'T_real_holdout',
                evaluator_source_probability=pb0,evaluator_candidate_probability=pb1,evaluator_predicts_Tripel=bool(evaluator.predict(candidate[None])[0]=='Tripel'),
                evaluator_stale_source=sb0,evaluator_stale_candidate=sb1,evaluator_proxy_acceptance=bool(b_accept),absolute_chemical_certification=False,real_tasting_validated=False)
            transfers.append(record)
            profiles.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'status':row['status'],**dict(zip(features,candidate))})
            for field,j in zip(CHEMICAL_DEFECTS,positions):
                raw=float(data.calibrated.iloc[i][field]) if field in data.calibrated else np.nan
                supported=bool(np.isfinite(raw) and raw>0 and np.isclose(X[i,j],np.log10(raw) if field in ['DMS','X4VG'] else raw,rtol=1e-6,atol=1e-7))
                checks.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'field':field,'source_provided':X[i,j],'source_for_search':source[j],'candidate_provided':candidate[j],
                    'provided_nonincrease':bool(candidate[j]<=source[j]+1e-9),'source_provided_observed':bool(np.isfinite(X[i,j])),
                    'source_positive_calibrated_pair_matches':supported,'absolute_threshold_available_in_attachment':field!='X.E..2.nonenal',
                    'complete_absolute_certification':False,'caveat':'scale_and_detection_limit_conditions_not_fully_verified; candidate_has_no_new_measurement'})
        save_csv(pd.DataFrame(metrics),dest/'generation_comparison.csv'); save_csv(pd.DataFrame(transfers),dest/'transfer_independent_evaluation.csv')
    save_csv(pd.DataFrame(roles),dest/'disjoint_roles.csv'); save_csv(pd.DataFrame(evaluator_rows),dest/'real_holdout_diagnostics.csv'); save_csv(pd.DataFrame(checks),dest/'transfer_defect_checks.csv')
    save_csv(pd.DataFrame(profiles),dest/'transfer_profiles.csv'); save_csv(pd.concat(generated_profiles,ignore_index=True),dest/'generated_profiles_first_split.csv'); save_csv(pd.concat(portraits,ignore_index=True),dest/'generated_sensory_proxy_first_split.csv')
    summary=pd.DataFrame(metrics).groupby(['method','style'])[['MMD2_fixed_training_bandwidth','KS_mean','exact_copy_fraction','search_probability','independent_evaluator_probability','independent_evaluator_target_prediction_fraction','provided_bounds_violation_fraction']].mean().reset_index(); save_csv(summary,dest/'generation_summary.csv')
    write_json(dest/'status.json',{'complete':True,'repeats':5,'G_n':100,'J_n':100,'T_n':50,'basic_fields':len(features),'excluded_unverified_aggregates':sorted(AGGREGATES),
        'candidate_locks':locked,'independent_evaluation_scope':'J is disjoint within the current run; all original beers were seen in earlier research stages, not new external validation',
        'absolute_chemical_certification':False,'real_brewing_or_tasting':False,'elapsed_seconds':time.perf_counter()-start})
    progress('S3 Q3 complete')
```

src/stage3_graph.py

```
"""Train-only graph ablations and cross-fitted exploratory explanations.
AI-assisted, OpenAI Codex, 2026-10-25. Graph regularization is an existing method.
"""
from pathlib import Path
import time
import numpy as np
import pandas as pd
import networkx as nx
from scipy.linalg import solve
from scipy.stats import rankdata
from sklearn.model_selection import KFold, RepeatedStratifiedKFold
from .common import preprocessing, progress, save_csv, write_json
from .q2 import signed_laplacian, top_indices
from .data import NON_MOLECULES
from .stage3_algorithms import paired_oof_bootstrap


def train_correlation(Z):
    with np.errstate(invalid='ignore',divide='ignore'):
        corr=np.corrcoef(rankdata(Z,axis=0),rowvar=False)
    return np.nan_to_num(corr)


def solve_coeff(Z,Y,L,alpha,lam):
    return solve(Z.T@Z+alpha*np.eye(Z.shape[1])+lam*L,Z.T@(Y-Y.mean(0)),assume_a='pos',check_finite=False)


def candidates(cfg,kind):
    return [(alpha,lam,tau) for alpha in cfg['ridge_alphas'] for lam in ([0] if kind=='none' else cfg['graph_lambdas']) for tau in ([.7] if lam==0 else cfg['graph_thresholds'])]


def fit_tuned_graph(X,Y,cfg,kind,core,seed):
    choices=candidates(cfg,kind); scores=np.zeros(len(choices))
    for fold,(train,test) in enumerate(KFold(3,shuffle=True,random_state=seed).split(X)):
        prep=preprocessing(); Z=prep.fit_transform(X[train]); V=prep.transform(X[test]); corr=train_correlation(Z)
        lhs=Z.T@Z; rhs=Z.T@(Y[train]-Y[train].mean(0)); identity=np.eye(Z.shape[1])
        layers={tau:signed_laplacian(corr,tau) for tau in cfg['graph_thresholds']}
        perms=[np.random.default_rng(seed+1009*k+fold).permutation(Z.shape[1]) for k in range(cfg['graph_null_repeats'])] if kind=='null' else [None]
        for c,(alpha,lam,tau) in enumerate(choices):
            L=layers[tau] if lam else np.zeros_like(lhs)
            for perm in perms:
                LL=L[np.ix_(perm,perm)] if perm is not None else L
                beta=solve(lhs+alpha*identity+lam*LL,rhs,assume_a='pos',check_finite=False)
                pp=V@beta+Y[train].mean(0)
                scores[c]+=np.abs(Y[test][:,core]-pp[:,core]).sum()/(len(Y)*len(core)*len(perms))
    best=min(range(len(choices)),key=lambda i:(scores[i],choices[i][1],-choices[i][0]))
    alpha,lam,tau=choices[best]; prep=preprocessing(); Z=prep.fit_transform(X); corr=train_correlation(Z); L=signed_laplacian(corr,tau)
    if kind=='none': L=np.zeros_like(L)
    betas=[]
    for k in range(cfg['graph_null_repeats'] if kind=='null' else 1):
        perm=np.random.default_rng(seed+20000+k).permutation(Z.shape[1]) if kind=='null' else None
        LL=L[np.ix_(perm,perm)] if perm is not None else L
        betas.append(solve_coeff(Z,Y,LL,alpha,lam))
    return {'prep':prep,'Z':Z,'corr':corr,'beta':betas,'mean':Y.mean(0),'params':{'alpha':alpha,'lambda':lam,'tau':tau},'inner_core_mae':float(scores[best])}


def within_style_indices(labels,rng):
    labels=np.asarray(labels); indices=np.arange(len(labels))
    for label in np.unique(labels):
        ids=np.flatnonzero(labels==label); indices[ids]=rng.permutation(ids)
    return indices


def run_graph(data,out,cfg):
    started=time.perf_counter(); dest=Path(out)/'q2'; dest.mkdir(parents=True,exist_ok=True)
    X,Y=data.X.to_numpy(),data.Y.to_numpy(); features,targets=list(data.X),list(data.Y); labels=data.meta.tasting_category_fine.to_numpy()
    core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]); eligible=np.array([j for j,t in enumerate(features) if t not in NON_MOLECULES])
    counts=np.zeros(len(Y)); totals={kind:np.zeros_like(Y) for kind in ['none','real']}; rows=[]; chosen=[]; effects=[]; modules=[]; style_effects=[]; selected_sets=[]
    for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=3,n_repeats=cfg['outer_repeats'],random_state=cfg['seed']).split(X,labels)):
        progress(f'S3 Q2 graph fold {fold+1}/15, elapsed {(time.perf_counter()-started)/60:.1f} min')
        fits={kind:fit_tuned_graph(X[train],Y[train],cfg,kind,core,cfg['seed']+fold) for kind in ['none','real','null']}
        for kind,fitted in fits.items():
            V=fitted['prep'].transform(X[test])
```

```
            for k,beta in enumerate(fitted['beta']):
                pred=V@beta+fitted['mean']
                for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
                    rows.append({'fold':fold,'repeat':fold//3,'graph':'shuffled' if kind=='null' else kind,'null_rep':k if kind=='null' else -1,'scope':scope,'mae':np.abs(Y[test][:,ix]-pred[:,ix]).mean()})
                    if kind in totals: totals[kind][test]+=pred
            chosen.append({'fold':fold,'graph':'shuffled' if kind=='null' else kind,**fitted['params'],'inner_core_mae':fitted['inner_core_mae']})
        counts[test]+=1
        fit=fits['real']; Z=fit['Z']; V=fit['prep'].transform(X[test]); B=fit['beta'][0]; original=V@B+fit['mean']; base=np.abs(Y[test]-original).mean(0)
        rng=np.random.default_rng(cfg['seed']+fold)
        fold_chosen=np.stack([top_indices(B[:,k],eligible) for k in range(Y.shape[1])]); selected_sets.append(fold_chosen)
        for k,target in enumerate(targets):
            for rank,j in enumerate(fold_chosen[k],1):
                all_delta=[]; within_delta=[]
                for repeat in range(cfg['permutation_repeats']):
                    ix=rng.permutation(len(V)); ixw=within_style_indices(labels[test],rng)
                    all_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ix,j]-V[:,j])*B[j,k])).mean()-base[k])
                    within_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ixw,j]-V[:,j])*B[j,k])).mean()-base[k])
                effects.append({'fold':fold,'target':target,'rank':rank,'feature':features[j],'coefficient':B[j,k],
                    'heldout_delta_mae':np.mean(all_delta),'within_style_delta_mae':np.mean(within_delta),
                    'singleton_test_style_fraction':np.mean([np.sum(labels[test]==label)==1 for label in labels[test]])})
        # Joint module perturbations preserve internal feature relationships.
        graph=nx.Graph(); graph.add_nodes_from(range(X.shape[1])); W=np.abs(fit['corr'])>=fit['params']['tau']; np.fill_diagonal(W,False)
        graph.add_edges_from(zip(*np.where(np.triu(W,1))))
        interesting={frozenset(nx.node_connected_component(graph,int(j))) for j in np.unique(fold_chosen)}
        for group in interesting:
            ix=np.array(sorted(group)); keep=np.setdiff1d(np.arange(X.shape[1]),ix)
            L=signed_laplacian(fit['corr'][np.ix_(keep,keep)],fit['params']['tau'])
            reduced=solve_coeff(Z[:,keep],Y[train],L,fit['params']['alpha'],fit['params']['lambda'])
            drop=V[:,keep]@reduced+fit['mean']; joint=np.zeros(Y.shape[1])
            for repeat in range(cfg['permutation_repeats']):
                order=within_style_indices(labels[test],rng)
                pp=original+(V[np.ix_(order,ix)]-V[:,ix])@B[ix]
                joint+=np.abs(Y[test]-pp).mean(0)-base
            for k,t in enumerate(targets):
                if set(fold_chosen[k])&group:
                    modules.append({'fold':fold,'target':t,'module_fields':';'.join(features[j] for j in ix),'module_n':len(ix),'heldout_module_drop_mae':np.abs(Y[test,k]-drop[:,k]).mean()-base[k],'within_style_joint_permutation_mae':joint[k]/cfg['permutation_repeats']})
        # Style-centering is an explanatory diagnostic, not a Q1 model input.
        zr=Z.copy(); yr=Y[train].copy()
        for label in np.unique(labels[train]):
            ids=np.flatnonzero(labels[train]==label); zr[ids]-=Z[ids].mean(0); yr[ids]-=Y[train][ids].mean(0)
        adjusted=solve_coeff(zr,yr,signed_laplacian(train_correlation(zr),fit['params']['tau']),fit['params']['alpha'],fit['params']['lambda'])
        for k,t in enumerate(targets):
            adjusted_set=set(top_indices(adjusted[:,k],eligible))
            for j in fold_chosen[k]: style_effects.append({'fold':fold,'target':t,'feature':features[j],'still_top3_after_style_centering':j in adjusted_set,'style_adjusted_coefficient':adjusted[j,k]})
        save_csv(pd.DataFrame(rows),dest/'graph_fold_metrics.csv'); save_csv(pd.DataFrame(chosen),dest/'selected_parameters.csv')
    for kind,pred in totals.items(): save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred/counts[:,None],columns=targets)],axis=1),dest/f'oof_graph_{kind}.csv')
    summary=pd.DataFrame(rows).groupby(['graph','scope']).mae.agg(['mean','std']).reset_index(); save_csv(summary,dest/'graph_comparison.csv')
    save_csv(pd.DataFrame(effects),dest/'crossfit_top3_effects.csv'); save_csv(pd.DataFrame(modules),dest/'crossfit_module_effects.csv'); save_csv(pd.DataFrame(style_effects),dest/'style_centering_diagnostics.csv')
    paired=[]
    for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
        r=paired_oof_bootstrap(Y[:,ix],totals['real'][:,ix]/counts[:,None],totals['none'][:,ix]/counts[:,None],labels); r['target_scope']=scope; paired.append(r)
    write_json(dest/'paired_graph_vs_none.json',paired)
    progress('S3 Q2: full descriptive graph and 200 conditional bootstrap replicates')
    full=fit_tuned_graph(X,Y,cfg,'real',core,cfg['seed']+500); beta=full['beta'][0]; frequency=np.zeros_like(beta); rng=np.random.default_rng(cfg['seed']); p=full['params']
    for b in range(cfg['bootstrap_repeats']):
        ix=rng.integers(0,len(X),len(X)); z=preprocessing().fit_transform(X[ix]); L=signed_laplacian(train_correlation(z),p['tau']); bb=solve_coeff(z,Y[ix],L,p['alpha'],p['lambda'])
        for k in range(Y.shape[1]): frequency[top_indices(bb[:,k],eligible),k]+=1
    frequency/=cfg['bootstrap_repeats']; effect=pd.DataFrame(effects); adjusted=pd.DataFrame(style_effects); meta=data.chemical_meta.set_index('feature'); top=[]
    for k,t in enumerate(targets):
        for rank,j in enumerate(top_indices(beta[:,k],eligible),1):
            selected=effect[(effect.target==t)&(effect.feature==features[j])]; centered=adjusted[(adjusted.target==t)&(adjusted.feature==features[j])]
            within=float(selected.within_style_delta_mae.mean()) if len(selected) else None
            delta=float(selected.heldout_delta_mae.mean()) if len(selected) else None
            supported=frequency[j,k]>=.5 and len(selected)>=5 and delta is not None and delta>0 and within is not None and within>0
            top.append({'target':t,'rank':rank,'feature':features[j],'coefficient':beta[j,k],'bootstrap_frequency_conditional_on_full_selected_params':frequency[j,k],
                'crossfit_selection_count':len(selected),'heldout_delta_on_selected_folds':delta,'within_style_delta_on_selected_folds':within,
                'style_centering_retention_fraction':float(centered.still_top3_after_style_centering.mean()) if len(centered) else None,
                'origin':meta.at[features[j],'origin_paper'],'unit':meta.at[features[j],'unit'],'attachment_off_flavors':meta.at[features[j],'off_flavors'],
                'evidence_status':'comparatively_supported_exploratory_association' if supported else 'weak_or_incomplete_evidence',
                'chemical_review':'target_specific_literature_not_yet_complete','causal_status':'not_identified'})
    save_csv(pd.DataFrame(top),dest/'top3_with_evidence.csv'); save_csv(pd.DataFrame(frequency,index=features,columns=targets).reset_index(names='feature'),dest/'bootstrap_frequencies.csv')
    write_json(dest/'status.json',{'complete':True,'outer_folds':15,'null_graphs_per_fit':cfg['graph_null_repeats'],'full_graph_params':p,'bootstrap_repeats':cfg['bootstrap_repeats'],'bootstrap_params_reselected':False,
        'bootstrap_caveat':'conditional_on_hyperparameters_selected_on_full_data; descriptive_stability_not_independent_inference','supported_top3':sum(r['evidence_status'].startswith('comparatively') for r in top),'scope':'exploratory_association_not_causal','elapsed_seconds':time.perf_counter()-started})
    progress('S3 Q2 complete')
```

src/stage3_prediction.py

```
"""Nested shrinkage blending, ordinal sets and robustness; no novelty claim."""
from pathlib import Path
import time
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, RepeatedStratifiedKFold, train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from .common import preprocessing, TrainQuantileClipper, progress, save_csv, write_json
from .q1 import metric_rows
from .stage2 import conformal_quantile
from .stage3_algorithms import fixed_grade_boundaries, grade_with_fixed_boundaries, ordinal_interval_image, shrunk_blend_weights, paired_oof_bootstrap


def tree_model(cfg,kind='et',clip=.01):
    prep=preprocessing(); prep.set_params(clip=TrainQuantileClipper(clip,1-clip))
    if kind=='et':
        model=ExtraTreesRegressor(n_estimators=cfg['extra_trees'],min_samples_leaf=3,max_features=.8,random_state=cfg['seed'],n_jobs=2)
    else:
        model=MultiOutputRegressor(GradientBoostingRegressor(n_estimators=cfg['gbr_estimators'],learning_rate=.04,max_depth=2,min_samples_leaf=5,max_features=.7,random_state=cfg['seed']),n_jobs=2)
    return Pipeline([('prep',prep),('model',model)])


def fit_pair(X,Y,cfg):
    with joblib.parallel_backend('threading'):
        et=tree_model(cfg).fit(X,Y)
        gb=tree_model(cfg,'gbr').fit(X,Y)
    return et,gb


def predict_pair(pair,X):
    with joblib.parallel_backend('threading'):
        return pair[0].predict(X),pair[1].predict(X)
```

```

def learn_blend_nested(X,Y,cfg,core,fold_id):
    """No outer-test values enter this routine. V labels never enter S fits."""
    inner=list(KFold(3,shuffle=True,random_state=cfg['seed']+fold_id).split(X))
    pred_et,pred_gb=np.zeros_like(Y),np.zeros_like(Y)
    validation_predictions={eta:np.zeros_like(Y) for eta in cfg['blend_penalties']}
    for vfold,(s,v) in enumerate(inner):
        pe,pg=np.zeros_like(Y[s]),np.zeros_like(Y[s])
        for a,b in KFold(3,shuffle=True,random_state=cfg['seed']+vfold+99).split(s):
            fitted=fit_pair(X[s[a]],Y[s[a]],cfg)
            pe[b],pg[b]=predict_pair(fitted,X[s[b]])
        fitted=fit_pair(X[s],Y[s],cfg)
        ve,vg=predict_pair(fitted,X[v]); pred_et[v],pred_gb[v]=ve,vg
        for eta in cfg['blend_penalties']:
            w,_=shrunk_blend_weights(Y[s],pe,pg,eta)
            validation_predictions[eta][v]=ve*(1-w)+vg*w
    scores={eta:float(np.abs(Y[:,core]-pred[:,core]).mean()) for eta,pred in validation_predictions.items()}
    best=min(cfg['blend_penalties'],key=lambda eta:(scores[eta],-eta))
    w,glob=shrunk_blend_weights(Y,pred_et,pred_gb,best)
    unshrunk,_=shrunk_blend_weights(Y,pred_et,pred_gb,0)
    return w,unshrunk,glob,best,scores


def run_prediction(data,out,cfg):
    start=time.perf_counter(); dest=Path(out)/'q1'; dest.mkdir(parents=True,exist_ok=True)
    X,Y=data.X.to_numpy(),data.Y.to_numpy(); targets=list(data.Y)
    core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]); labels=data.meta.tasting_category_fine.to_numpy()
    names=['mean','ridge','extra_trees','gbr','fixed_half_blend','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']
    sums={m:np.zeros_like(Y) for m in names}; count=np.zeros(len(Y)); assignments=[]; fold_rows=[]; weight_rows=[]; stress=[]; clipping=[]
    grade_votes=np.zeros((len(Y),Y.shape[1],7),int)
    for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=cfg['outer_folds'],n_repeats=cfg['outer_repeats'],random_state=cfg['seed']).split(X,labels)):
        progress(f'S3 Q1 fold {fold+1}/15: nested inner weights; elapsed {(time.perf_counter()-start)/60:.1f} min')
        w,wu,glob,eta,eta_scores=learn_blend_nested(X[train],Y[train],cfg,core,fold)
        pair=fit_pair(X[train],Y[train],cfg); etp,gbp=predict_pair(pair,X[test])
        ridge_search=GridSearchCV(Pipeline([('prep',preprocessing()),('model',Ridge())]),{'model__alpha':cfg['ridge_alphas']},
            scoring=lambda est,xx,yy:-np.abs(yy[:,core]-est.predict(xx)[:,core]).mean(),cv=KFold(3,shuffle=True,random_state=cfg['seed']),n_jobs=1)
        ridge=ridge_search.fit(X[train],Y[train]).best_estimator_
        predictions={'mean':np.broadcast_to(Y[train].mean(0),Y[test].shape),'ridge':ridge.predict(X[test]),'extra_trees':etp,'gbr':gbp,
                     'fixed_half_blend':(etp+gbp)/2,'global_oof_blend':etp*(1-glob)+gbp*glob,'unshrunk_target_blend':etp*(1-wu)+gbp*wu,'shrunk_target_blend':etp*(1-w)+gbp*w}
        for name,pred in predictions.items():
            sums[name][test]+=pred
            for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
                fold_rows.append({'fold':fold,'repeat':fold//3,'model':name,'scope':scope,'mae':np.abs(Y[test][:,ix]-pred[:,ix]).mean(),'n_test':len(test)})
        count[test]+=1
        assignments.extend({'fold':fold,'repeat':fold//3,'beer':int(data.meta.iloc[i].beer),'role':'outer_validation'} for i in test)
        weight_rows.extend({'fold':fold,'target':target,'weight_GBR':w[j],'global_weight_GBR':glob,'eta':eta} for j,target in enumerate(targets))
        boundaries=fixed_grade_boundaries(Y[train]); grades=grade_with_fixed_boundaries(etp,boundaries)
        for j in range(Y.shape[1]): np.add.at(grade_votes[:,j],(test,grades[:,j]),1)
        rng=np.random.default_rng(cfg['seed']+fold); low=np.nanquantile(X[train],.2,axis=0)
        for scenario in ['MCAR','low_value_preferential']:
            for rate in [.0,.05,.1,.2]:
                eligible=np.flatnonzero(np.isfinite(X[test]).ravel()); n=int(rate*len(eligible)); a=X[test].copy()
                weights=np.where((a<=low).ravel()[eligible],4.,1.) if scenario!='MCAR' else np.ones(len(eligible))
                if n: a.ravel()[rng.choice(eligible,n,replace=False,p=weights/weights.sum())]=np.nan
                pe,pg=predict_pair(pair,a)
                for name,pred in [('extra_trees',pe),('shrunk_target_blend',pe*(1-w)+pg*w)]:
                    stress.append({'fold':fold,'scenario':scenario,'extra_missing_rate':rate,'model':name,'core30_mae':np.abs(Y[test][:,core]-pred[:,core]).mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        for lower in [0,.01,.025]:
            alternative=pair[0] if lower==.01 else tree_model(cfg,clip=lower).fit(X[train],Y[train])
            pred=alternative.predict(X[test]); clipping.append({'fold':fold,'lower_clip':lower,'core30_mae':np.abs(Y[test][:,core]-pred[:,core]).mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        # Save progress atomically per finished outer fold.
        save_csv(pd.DataFrame(fold_rows),dest/'fold_metrics.csv'); save_csv(pd.DataFrame(weight_rows),dest/'blend_weights.csv')
        write_json(dest/'progress.json',{'completed_outer_folds':fold+1,'total_outer_folds':15,'elapsed_seconds':time.perf_counter()-start})
    metric=[]
    for name in names:
        pred=sums[name]/count[:,None]
        save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred,columns=targets)],axis=1),dest/f'oof_{name}.csv')
        metric+=metric_rows(Y,pred,targets,name)
    bytarget=pd.DataFrame(metric); save_csv(bytarget,dest/'metrics_by_target.csv')
    scopes=[]
    for scope,fields in [('core30',[targets[j] for j in core]),('all50',targets)]:
        a=bytarget[bytarget.target.isin(fields)].groupby('model')[['mae','rmse','r2']].mean().reset_index(); a.insert(0,'scope',scope); scopes.append(a)
    summary=pd.concat(scopes,ignore_index=True); save_csv(summary,dest/'metrics_by_scope.csv')
    save_csv(pd.DataFrame(assignments),dest/'outer_roles.csv'); save_csv(pd.DataFrame(stress),dest/'missingness_stress.csv'); save_csv(pd.DataFrame(clipping),dest/'clipping_ablation.csv')
    save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(grade_votes.argmax(2),columns=targets)],axis=1),dest/'ASSUMPTION_oof_relative_grades.csv')
    paired=[]
    for name in ['gbr','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']:
        for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
            r=paired_oof_bootstrap(Y[:,ix],sums[name][:,ix]/count[:,None],sums['extra_trees'][:,ix]/count[:,None],labels)
            r.update(model_A=name,model_B='extra_trees',target_scope=scope); paired.append(r)
    write_json(dest/'paired_conditional_comparison.json',paired)
    intervals=[]; coverage_rows=[]
    for repeat in range(cfg['conformal_repeats']):
        train,rest=train_test_split(np.arange(len(X)),test_size=.4,random_state=cfg['seed']+71+repeat)
        calibration,test=train_test_split(rest,test_size=.5,random_state=cfg['seed']+72+repeat)
        model=tree_model(cfg).fit(X[train],Y[train]); pp=model.predict(X[test]); q=conformal_quantile(np.abs(Y[calibration]-model.predict(X[calibration])),cfg['conformal_alpha'])
        boundaries=fixed_grade_boundaries(Y[train]); lo,hi=ordinal_interval_image(pp-q,pp+q,boundaries); true=grade_with_fixed_boundaries(Y[test],boundaries); point=grade_with_fixed_boundaries(pp,boundaries)
        for j,t in enumerate(targets):
            coverage_rows.append({'repeat':repeat,'target':t,'continuous_coverage':((Y[test,j]>=pp[:,j]-q[j])&(Y[test,j]<=pp[:,j]+q[j])).mean(),'interval_width':2*q[j],
                'relative_grade_set_coverage':((true[:,j]>=lo[:,j])&(true[:,j]<=hi[:,j])).mean(),'mean_grade_set_size':(hi[:,j]-lo[:,j]+1).mean()})
        for i,row in enumerate(test):
            for j,t in enumerate(targets): intervals.append({'repeat':repeat,'beer':int(data.meta.iloc[row].beer),'target':t,'continuous_prediction':pp[i,j],'lower':pp[i,j]-q[j],'upper':pp[i,j]+q[j],'relative_grade':point[i,j],'relative_grade_min':lo[i,j],'relative_grade_max':hi[i,j],'status':'fixed_training_relative_bins_not_absolute_raw_scores'})
        save_csv(pd.DataFrame({'repeat':repeat,'beer':data.meta.beer,'role':['train' if i in set(train) else 'calibration' if i in set(calibration) else 'test' for i in range(len(X))]}),dest/f'conformal_roles_{repeat}.csv')
    save_csv(pd.DataFrame(intervals),dest/'ASSUMPTION_ordinal_sets.csv'); save_csv(pd.DataFrame(coverage_rows),dest/'conformal_coverage.csv')
    write_json(dest/'status.json',{'complete':True,'outer_folds':15,'unique_beers':len(X),'grade_semantics':'relative_training_quantile_not_original_0_6_intensity','absolute_anchors_observed':False,'elapsed_seconds':time.perf_counter()-start,'model_originality':'standard_estimators_and_regularized_stacking_variant_no_first_invention_claim'})
    progress('S3 Q1 complete')
```

src/stage4_audit.py

```
"""Source reconciliation and raw sensory endpoints. AI-assisted, 2026-10-25.
Identity matching is provenance analysis, never chemical-model fitting.
"""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment
from scipy.spatial.distance import cdist
from scipy.special import logsumexp
from .common import save_csv, write_json, sha256
from .data import META, CHEMICAL_DEFECTS
```

```python

ALIASES = {'A_ester_fruity': 'A_esters_fruity', 'F_ester_fruity': 'F_esters_fruity'}
RAW_NAME = 'Raw sensory data anonymized.csv'
BOOK_NAME = 'Supplemental Files and Figure source files.xlsx'


def aggregate_panel(raw, targets):
    """Repeated ratings from one person do not give that person extra weight."""
    person = raw.groupby(['beer', 'taster'])[targets].mean()
    return person.groupby('beer').mean(), raw.groupby('beer')[targets].mean(), person


def standardize_panel(raw, targets):
    group = raw.groupby('taster')[targets]
    sd = group.transform('std')
    return (raw[targets] - group.transform('mean')) / sd.replace(0, np.nan)


def panel_rating_distribution(raw, targets, beers):
    """Equal people, equal repeats per person; preserve actual integer categories."""
    encoded = np.eye(7)[raw[targets].to_numpy(int)].reshape(len(raw), -1)
    table = pd.DataFrame(encoded)
    per_person = table.groupby([raw.beer, raw.taster]).mean()
    return per_person.groupby(level=0).mean().reindex(beers).to_numpy().reshape(len(beers), len(targets), 7)


def infer_identity(standardized, raw_ids, published, discovery):
    means = standardized.groupby(raw_ids).mean().sort_index()
    distance = cdist(means[discovery], published[discovery]) / np.sqrt(len(discovery))
    i, j = linear_sum_assignment(distance)
    ordered = np.sort(distance, axis=1)
    return means, pd.DataFrame({'raw_beer': means.index.to_numpy()[i],
                                'beer': published.index.to_numpy()[j],
                                'discovery_rmse': distance[i, j],
                                'nearest_gap': ordered[:, 1] - ordered[:, 0],
                                'is_row_nearest': j == distance.argmin(1)})


def transform_candidates(raw):
    finite_positive = raw[np.isfinite(raw) & (raw > 0)]
    half_min = finite_positive.min() / 2 if len(finite_positive) else np.nan
    replacement = np.where(raw <= 0, half_min, raw)
    with np.errstate(invalid='ignore', divide='ignore'):
        return {'identity': raw, 'nonpositive_half_min': replacement,
                'log10_nonpositive_half_min': np.log10(replacement)}, half_min


def concentration_fields(calibrated, thresholds, units):
    metadata = thresholds.dropna(subset=['field_name_R']).drop_duplicates('field_name_R').set_index('field_name_R')
    fields = []
    for f in calibrated.columns:
        if f in META or f not in metadata.index or f == 'kcalperc':
            continue
        row = metadata.loc[f]
        if str(row.calibrated).startswith('yes') and row.unit in units:
            fields.append(f)
    return fields


def audit_aggregate_hypotheses(data, calibrated, out):
    """Reject unsupported aggregate identities on a disjoint row half.
    Fits are diagnostic hypotheses, not recovered author formulas.
    """
    x = data.X
    groups = {
        'esters_sum': [f for f in x if any(s in f for s in ['acetate', 'butyrate', 'hexanoate', 'octanoate', 'decanoate', 'propionate', 'isovalerate'])],
        'aroma_hops_sum': data.chemical_meta.loc[data.chemical_meta.origin_paper.astype(str).str.contains('Hops'), 'field'].tolist(),
        'acids_sum': ['acetic_acid.g.L.', 'lactic_acid.mg.L.'],
        'sulfur_sum': ['CS2', 'DES', 'DMS', 'H2S', 'MeSH', 'SO2.mg.L.']}
    rows = []
    for target, fields in groups.items():
        a = x[fields].to_numpy(float)
        probes = {'mean_processed': np.nanmean(a, axis=1), 'sum_processed': np.nansum(a, axis=1),
                  'log10_sum_exponentiated_processed': logsumexp(np.where(np.isfinite(a), a, -np.inf) * np.log(10), axis=1) / np.log(10)}
        y = x[target].to_numpy()
        for name, value in probes.items():
            valid = np.isfinite(value) & np.isfinite(y); train = valid & (np.arange(len(y)) % 2 == 0); test = valid & ~train
            slope, intercept = np.linalg.lstsq(np.c_[value[train], np.ones(train.sum())], y[train], rcond=None)[0]
            error = np.abs(value[test] * slope + intercept - y[test])
            rows.append({'aggregate': target, 'hypothesis': name, 'component_count': len(fields), 'fitted_slope_on_discovery': slope,
                         'fitted_intercept_on_discovery': intercept, 'validation_mae': error.mean(), 'validation_max_error': error.max(),
                         'identity_verified': bool(error.max() < 1e-7), 'scope': 'exploratory_formula_hypothesis_not_author_documentation'})
    features = ['ethanol..v.v.', 'GFS', 'unfermentables', 'protein.g.L.', 'glycerol']
    a = calibrated[features].to_numpy(float); y = calibrated.kcalperc.to_numpy(float)
    valid = np.isfinite(a).all(1) & np.isfinite(y); train = valid & (np.arange(len(y)) % 2 == 0); test = valid & ~train
    coef = np.linalg.lstsq(np.c_[a[train], np.ones(train.sum())], y[train], rcond=None)[0]
    error = np.abs(np.c_[a[test], np.ones(test.sum())] @ coef - y[test])
    rows.append({'aggregate': 'kcalperc', 'hypothesis': 'five_raw_nutrient_affine_fit', 'component_count': len(features),
                 'validation_mae': error.mean(), 'validation_max_error': error.max(), 'identity_verified': bool(error.max() < 1e-7),
                 'scope': 'exploratory_formula_hypothesis_not_author_documentation'})
    save_csv(pd.DataFrame(rows), Path(out) / 'aggregate_formula_hypotheses.csv')


def reconcile(data, external, out, cfg):
    external, out = Path(external), Path(out)
    out.mkdir(parents=True, exist_ok=True)
    for name, key in [(RAW_NAME, 'raw_md5'), (BOOK_NAME, 'workbook_md5')]:
        if hashlib.md5((external / name).read_bytes()).hexdigest() != cfg[key]:
            raise ValueError('External source checksum differs: ' + name)
    raw = pd.read_csv(external / RAW_NAME, encoding='cp1252').rename(columns=ALIASES)
    published = pd.read_excel(external / BOOK_NAME, sheet_name='Supplementary File S4')
    targets = list(data.Y)
    if set(raw.beer_name) != set(range(1, 251)) or len(published) != 250:
        raise ValueError('Unexpected source numbering; no positional fallback permitted')
    if set(raw.columns[5:-1]) != set(targets) or not raw[targets].isin(range(7)).all().all():
        raise ValueError('Raw target domain mismatch')
    reference = published.set_index('beer')[targets]
    if not np.allclose(reference.reindex(data.meta.beer), data.Y, atol=1e-7, rtol=0):
        raise ValueError('Published S4 differs from contest labels')
    standardized = standardize_panel(raw, targets)
    discovery = sorted(targets)[:25]
    validation = sorted(targets)[25:]
    means, mapping = infer_identity(standardized, raw.beer_name, reference, discovery)
    positional = dict(zip(range(1, 251), published.beer))
    if not (mapping.beer == mapping.raw_beer.map(positional)).all() or not mapping.is_row_nearest.all():
        raise ValueError('Source order and independent profile matching do not agree')
    # A second, disjoint attribute group must recover the same identity permutation.
    _, reverse_mapping = infer_identity(standardized, raw.beer_name, reference, validation)
    if not mapping.set_index('raw_beer').beer.equals(reverse_mapping.set_index('raw_beer').beer):
        raise ValueError('Disjoint-attribute identity validation failed')
    comparison = reference.reindex(mapping.beer).to_numpy()
    reconstructed = means.reindex(mapping.raw_beer).to_numpy()
    no_constant = ~standardized.isna().any().to_numpy()
    exact_error = np.abs(reconstructed[:, no_constant] - comparison[:, no_constant])
    if exact_error.max() > cfg['mapping_tolerance']:
        raise ValueError('Nonconstant attributes do not reproduce published scores')
    mapping['published_position'] = mapping.raw_beer
    mapping['identity_status'] = 'source_order_reconstructed_and_disjoint_attribute_verified_not_author_mapping_table'
    save_csv(mapping, out / 'identity_mapping.csv')
    # Reverse engineering only: fit unknown constant replacement on odd source IDs,
    # validate on even IDs. Do not call this the documented original preprocessing.
    counts = raw.groupby('beer_name').size().reindex(mapping.raw_beer).to_numpy()
    source_ids = mapping.raw_beer.to_numpy()
    replacement_rows, error_rows = [], []
    for j, target in enumerate(targets):
        sums = standardized[target].fillna(0).groupby(raw.beer_name).sum().reindex(source_ids).to_numpy()
        missing = standardized[target].isna().groupby(raw.beer_name).sum().reindex(source_ids).to_numpy()
        if missing.sum():
            fit = (source_ids % 2 == 1) & (missing > 0)
            constant = float(np.dot(missing[fit], comparison[fit, j] * counts
```

```
    [fit] - sums[fit]) / np.dot(missing[fit], missing[fit]))
            rebuilt = (sums + missing * constant) / counts
            held = (source_ids % 2 == 0) & (missing > 0)
            replacement_rows.append({'target': target, 'replacement': constant, 'fit_beers': int(fit.sum()),
                                     'validation_beers': int(held.sum()),
                                     'validation_max_error': float(np.abs(rebuilt[held] - comparison[held, j]).max()),
                                     'status': 'empirically_reconstructed_not_source_documented'})
        else:
            rebuilt = reconstructed[:, j]
        error_rows.append({'target': target, 'constant_taster_attribute': bool(missing.sum()),
                           'max_error_without_fitted_replacement': float(np.abs(reconstructed[:, j] - comparison[:, j]).max()),
                           'max_error_after_conditional_reconstruction': float(np.abs(rebuilt - comparison[:, j]).max())})
    save_csv(pd.DataFrame(replacement_rows), out / 'constant_attribute_reconstruction.csv')
    save_csv(pd.DataFrame(error_rows), out / 'sensory_reconstruction_errors.csv')
    raw['beer'] = raw.beer_name.map(mapping.set_index('raw_beer').beer)
    balanced, records, person = aggregate_panel(raw, targets)
    balanced, records = balanced.reindex(data.meta.beer), records.reindex(data.meta.beer)
    save_csv(balanced.reset_index(), out / 'observed_panel_means_0_6.csv')
    save_csv(records.reset_index(), out / 'record_weighted_means_sensitivity.csv')
    n = person.groupby('beer').size().reindex(data.meta.beer)
    save_csv(pd.DataFrame({'beer': data.meta.beer.to_numpy(), 'distinct_panelists': n.to_numpy(),
                           'rating_records': raw.groupby('beer').size().reindex(data.meta.beer).to_numpy()}), out / 'panel_sample_counts.csv')
    rng = np.random.default_rng(cfg['seed'])
    precision, repeats = [], []
    for beer, group in person.groupby(level='beer'):
        a = group.to_numpy()
        draws = a[rng.integers(len(a), size=(cfg['panel_bootstrap_repeats'], len(a)))].mean(1)
        low, high = np.quantile(draws, [.025, .975], axis=0)
        for j, target in enumerate(targets):
            precision.append({'beer': int(beer), 'target': target, 'observed_mean': a[:, j].mean(),
                              'panel_bootstrap_low': low[j], 'panel_bootstrap_high': high[j], 'panelists': len(a)})
    for (beer, taster), group in raw.groupby(['beer', 'taster']):
        if group.date.nunique() < 2:
            continue
        first, last = group.sort_values('date').iloc[[0, -1]][targets].to_numpy(float)
        repeats.extend({'beer': int(beer), 'target': target, 'absolute_first_last_difference': abs(last[j] - first[j])} for j, target in enumerate(targets))
    save_csv(pd.DataFrame(precision), out / 'panel_sampling_precision.csv')
    save_csv(pd.DataFrame(repeats).groupby('target', as_index=False).agg(mean_absolute_first_last_difference=('absolute_first_last_difference', 'mean'), repeated_person_beer_pairs=('beer', 'size')), out / 'within_person_repeatability.csv')
    calibrated = pd.read_excel(external / BOOK_NAME, sheet_name='Calibrated chemical dataset').set_index('beer').reindex(data.meta.beer)
    fields = list(calibrated.columns[2:])
    chemistry, mismatches = [], []
    for field in fields:
        a = calibrated[field].to_numpy(float)
        supplied = data.X[field].to_numpy(float)
        candidates, half = transform_candidates(a)
        valid = np.isfinite(a) & np.isfinite(supplied)
        rates = {name: float(np.isclose(pred[valid], supplied[valid], atol=1e-7, rtol=1e-7).mean()) for name, pred in candidates.items()}
        best = max(rates, key=rates.get)
        good = np.isclose(candidates[best], supplied, atol=1e-7, rtol=1e-7)
        chemistry.append({'field': field, 'best_transform': best, 'match_fraction': rates[best],
                          'comparable_pairs': int(valid.sum()), 'nonpositive_raw': int((a <= 0).sum()),
                          'missing_raw': int(np.isnan(a).sum()), 'half_min_positive': half,
                          'status': 'all_comparable_pairs_reconciled' if rates[best] == 1 else 'exceptions_retained'})
        mismatches.extend({'beer': int(data.meta.iloc[i].beer), 'field': field,
                           'absolute_difference': abs(candidates[best][i] - supplied[i]),
                           'status': 'unexplained_source_difference_not_repaired'} for i in np.flatnonzero(valid & ~good))
    save_csv(pd.DataFrame(chemistry), out / 'chemical_scale_reconciliation.csv')
    save_csv(pd.DataFrame(mismatches), out / 'chemical_source_exceptions.csv')
    selected = concentration_fields(calibrated, data.thresholds, cfg['concentration_units'])
    meta = data.thresholds.dropna(subset=['field_name_R']).drop_duplicates('field_name_R').set_index('field_name_R')
    save_csv(pd.DataFrame([{'field': f, 'unit': meta.at[f, 'unit'], 'calibrated_metadata': meta.at[f, 'calibrated'],
                            'negative_raw': int((calibrated[f] < 0).sum()), 'zero_raw': int((calibrated[f] == 0).sum()),
                            'missing_raw': int(calibrated[f].isna().sum())} for f in selected]), out / 'concentration_feature_registry.csv')
    thresholds = {'DMS': (19., 'ug/L', 'attachment_median'), 'X2.3.butanedione': (.09, 'mg/L', 'attachment_median'),
                  'X4VG': (.2, 'mg/L', 'attachment_median'),
                  'X.E..2.nonenal': (cfg['literature_nonenal_threshold_mg_L'], 'mg/L', 'Meilgaard_1993_Santos_2003_external_reference_not_batch_calibration')}
    oav = []
    for field, (threshold, unit, source) in thresholds.items():
        for factor in cfg['threshold_sensitivity_multipliers']:
            for i, value in enumerate(calibrated[field]):
                oav.append({'beer': int(data.meta.iloc[i].beer), 'field': field, 'unit': unit,
                            'threshold_source': source, 'reference_threshold': threshold, 'threshold_multiplier': factor,
                            'reference_oav': value / (factor * threshold) if pd.notna(value) and value >= 0 else np.nan,
                            'measurement_status': 'positive_reported_concentration_LOD_unknown' if pd.notna(value) and value > 0 else 'zero_or_missing_not_certified_absent',
                            'sensory_target': {'DMS': 'dms', 'X2.3.butanedione': 'diacetyl', 'X4VG': 'X4vg', 'X.E..2.nonenal': 't2n'}[field]})
    save_csv(pd.DataFrame(oav), out / 'reference_oav_sensitivity.csv')
    audit_aggregate_hypotheses(data, calibrated, out)
    summary = {'raw_rows': len(raw), 'beers': len(balanced), 'panelists': int(raw.taster.nunique()), 'targets': len(targets),
               'duplicate_person_beer_records': int(raw.duplicated(['beer', 'taster']).sum()),
               'identity_matches_both_attribute_groups': len(mapping), 'nonconstant_targets_reproduced': int(no_constant.sum()),
               'nonconstant_max_error': float(exact_error.max()), 'replacement_rules_empirically_reconstructed': len(replacement_rows),
               'panelists_per_beer_min': int(n.min()), 'panelists_per_beer_max': int(n.max()),
               'mean_abs_record_vs_person_aggregation_difference': float(np.abs(balanced - records).to_numpy().mean()),
               'calibrated_reference_fields': len(fields), 'concentration_model_fields': len(selected),
               'fully_reconciled_fields': sum(row['match_fraction'] == 1 for row in chemistry),
               'unexplained_chemical_pairs': len(mismatches),
               'source_sha256': {RAW_NAME: sha256(external / RAW_NAME), BOOK_NAME: sha256(external / BOOK_NAME)},
               'mapping_scope': 'reconstructed_source_identity_not_independent_new_external_samples',
               'absolute_target_scope': cfg['primary_absolute_target'], 'physical_validation_performed': False}
    write_json(out / 'audit_summary.json', summary)
    return raw, balanced, records, calibrated[selected].where(calibrated[selected] >= 0), selected
```

src/stage4_models.py

```
"""Absolute panel-mean prediction, factor checks, and physical-scale diagnostics.
AI-assisted research implementation, 2026-10-25; no original algorithm claim.
"""
from pathlib import Path
import hashlib
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.isotonic import IsotonicRegression
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import GridSearchCV, KFold, RepeatedStratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from .common import preprocessing, save_csv, write_json, progress
from .data import NON_MOLECULES, CHEMICAL_DEFECTS, AGGREGATES
from .q1 import metric_rows
from .stage2 import conformal_quantile
from .stage3_prediction import fit_pair, predict_pair, tree_model
from .stage4_audit import panel_rating_distribution


def calibrated_from_training(z_train, raw_train, z_pred):
    return np.column_stack([IsotonicRegression(out_of_bounds='clip', y_min=0, y_max=6).fit(z_train[:, j], raw_train[:, j]).predict(z_pred[:, j])
                            for j in range(raw_train.shape[1])])

```

```

def ridge_search(X, Y, core, cfg):
    return GridSearchCV(Pipeline([('prep', preprocessing()), ('model', Ridge())]),
                        {'model__alpha': cfg['ridge_alphas']}, cv=KFold(3, shuffle=True, random_state=cfg['seed']),
                        scoring=lambda est, xx, yy: -np.abs(yy[:, core] - est.predict(xx)[:, core]).mean(), n_jobs=1).fit(X, Y).best_estimator_


def raw_top3(model, fields, targets):
    candidates = np.array([j for j, f in enumerate(fields) if f not in NON_MOLECULES])
    coef = model.named_steps['model'].coef_
    return [(t, int(j), float(coef[k, j]), rank + 1) for k, t in enumerate(targets)
            for rank, j in enumerate(candidates[np.argsort(-np.abs(coef[k, candidates]), kind='stable')[:3]])]


def run_absolute(data, panel, records, concentration, out, cfg):
    dest = Path(out) / 'prediction'; dest.mkdir(parents=True, exist_ok=True)
    X, Y, C = data.X.to_numpy(), panel.to_numpy(), concentration.to_numpy()
    targets, fields = list(data.Y), list(data.X)
    core = np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))])
    labels, beers = data.meta.tasting_category_fine.to_numpy(), data.meta.beer.to_numpy()
    names = ['mean', 'ridge', 'extra_trees', 'gbr', 'fixed_half_blend', 'concentration_ET', 'standardized_ET_train_isotonic']
    sums = {m: np.zeros_like(Y) for m in names}; count = np.zeros(len(Y)); folds, factors, sensitivity, roles = [], [], [], []
    start = time.perf_counter()
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=3, n_repeats=5, random_state=cfg['seed']).split(X, labels)):
        progress(f'S4 absolute fold {fold+1}/15; {(time.perf_counter()-start):.0f}s')
        et, gb = fit_pair(X[train], Y[train], cfg)
        pe, pg = predict_pair((et, gb), X[test])
        ridge = ridge_search(X[train], Y[train], core, cfg)
        cp = tree_model(cfg).fit(C[train], Y[train]).predict(C[test])
        old = tree_model(cfg).fit(X[train], data.Y.to_numpy()[train]).predict(X[test])
        iso = calibrated_from_training(data.Y.to_numpy()[train], Y[train], old)
        predictions = {'mean': np.broadcast_to(Y[train].mean(0), Y[test].shape), 'ridge': ridge.predict(X[test]),
                       'extra_trees': pe, 'gbr': pg, 'fixed_half_blend': (pe + pg) / 2, 'concentration_ET': cp,
                       'standardized_ET_train_isotonic': iso}
        for name, pred in predictions.items():
            pred = np.clip(pred, 0, 6); sums[name][test] += pred
            for scope, indices in [('core30', core), ('all50', np.arange(50))]:
                folds.append({'fold': fold, 'repeat': fold // 3, 'model': name, 'scope': scope,
                              'mae_0_6': float(np.abs(Y[test][:, indices] - pred[:, indices]).mean()), 'n_test': len(test)})
        count[test] += 1
        record_model = tree_model(cfg).fit(X[train], records.to_numpy()[train])
        record_pred = record_model.predict(X[test])
        sensitivity.append({'fold': fold, 'person_balanced_ET_mae_own_endpoint': float(np.abs(Y[test] - pe).mean()),
                            'record_weighted_ET_mae_own_endpoint': float(np.abs(records.to_numpy()[test] - record_pred).mean()),
                            'prediction_aggregation_difference': float(np.abs(record_pred - pe).mean())})
        roles.extend({'fold': fold, 'repeat': fold // 3, 'beer': int(beers[i]), 'role': 'outer_test'} for i in test)
        # Linear model permits exact input-coordinate permutation changes without refits.
        zp = ridge.named_steps['prep'].transform(X[test]); rp = ridge.predict(X[test]); rng = np.random.default_rng(cfg['seed'] + fold)
        for target, j, coefficient, rank in raw_top3(ridge, fields, targets):
            k = targets.index(target); baseline = np.abs(Y[test, k] - rp[:, k]).mean(); ordinary, within = [], []
            for _ in range(20):
                permutation = rng.permutation(len(test))
                ordinary.append(np.abs(Y[test, k] - (rp[:, k] + coefficient * (zp[permutation, j] - zp[:, j]))).mean() - baseline)
                permutation = np.arange(len(test))
                for style in np.unique(labels[test]):
                    ids = np.flatnonzero(labels[test] == style); permutation[ids] = rng.permutation(ids)
                within.append(np.abs(Y[test, k] - (rp[:, k] + coefficient * (zp[permutation, j] - zp[:, j]))).mean() - baseline)
            factors.append({'fold': fold, 'target': target, 'field': fields[j], 'rank': rank, 'coefficient': coefficient,
                            'ordinary_delta_mae': np.mean(ordinary), 'within_style_delta_mae': np.mean(within)})
        save_csv(pd.DataFrame(folds), dest / 'fold_metrics.csv')
    metric = []
    for name in names:
        pred = sums[name] / count[:, None]
        save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(pred, columns=targets)], axis=1), dest / f'oof_{name}_0_6.csv')
        metric += metric_rows(Y, pred, targets, name)
    detail = pd.DataFrame(metric); save_csv(detail, dest / 'metrics_by_target.csv')
    summaries = []
    for scope, selected in [('core30', [targets[j] for j in core]), ('all50', targets)]:
        f = detail[detail.target.isin(selected)].groupby('model')[['mae', 'rmse', 'r2']].mean().reset_index(); f.insert(0, 'scope', scope); summaries.append(f)
    save_csv(pd.concat(summaries), dest / 'metrics_by_scope.csv')
    save_csv(pd.DataFrame(roles), dest / 'outer_roles.csv')
    save_csv(pd.DataFrame(sensitivity), dest / 'aggregation_sensitivity.csv')
    effect = pd.DataFrame(factors); save_csv(effect, dest / 'raw_factor_crossfit.csv')
    # Prespecified ET is used for uncertainty; evaluation does not select the model.
    intervals, coverage, interval_roles = [], [], []
    for repeat in range(cfg['conformal_repeats']):
        train, rest = train_test_split(np.arange(len(X)), test_size=.4, random_state=cfg['seed'] + 71 + repeat)
        calibration, test = train_test_split(rest, test_size=.5, random_state=cfg['seed'] + 72 + repeat)
        model = tree_model(cfg).fit(X[train], Y[train]); pred = model.predict(X[test])
        q = conformal_quantile(np.abs(Y[calibration] - model.predict(X[calibration])), .1)
        lo, hi = np.maximum(0, pred - q), np.minimum(6, pred + q)
        for j, target in enumerate(targets):
            coverage.append({'repeat': repeat, 'target': target, 'coverage': float(((Y[test, j] >= lo[:, j]) & (Y[test, j] <= hi[:, j])).mean()),
                             'mean_width_0_6': float((hi[:, j] - lo[:, j]).mean())})
        intervals.extend({'repeat': repeat, 'beer': int(beers[i]), 'target': target, 'prediction_0_6': pred[k, j],
                          'lower_0_6': lo[k, j], 'upper_0_6': hi[k, j], 'observed_panel_mean': Y[i, j]}
                         for k, i in enumerate(test) for j, target in enumerate(targets))
        interval_roles.extend({'repeat': repeat, 'beer': int(beers[i]), 'role': 'fit' if i in train else 'calibration' if i in calibration else 'test'} for i in range(len(X)))
    save_csv(pd.DataFrame(intervals), dest / 'absolute_panel_mean_intervals.csv')
    save_csv(pd.DataFrame(coverage), dest / 'conformal_coverage.csv')
    save_csv(pd.DataFrame(interval_roles), dest / 'conformal_roles.csv')
    final_ridge = ridge_search(X, Y, core, cfg); alpha = float(final_ridge.named_steps['model'].alpha)
    final = raw_top3(final_ridge, fields, targets); frequencies = {}
    rng = np.random.default_rng(cfg['seed'] + 333)
    for _ in range(200):
        ii = rng.integers(len(X), size=len(X))
        m = Pipeline([('prep', preprocessing()), ('model', Ridge(alpha=alpha))]).fit(X[ii], Y[ii])
        for target, j, _, _ in raw_top3(m, fields, targets): frequencies[(target, fields[j])] = frequencies.get((target, fields[j]), 0) + 1
    grouped = effect.groupby(['target', 'field']).agg(selected_folds=('fold', 'size'), ordinary_delta_mae=('ordinary_delta_mae', 'mean'), within_style_delta_mae=('within_style_delta_mae', 'mean'))
    metadata = data.chemical_meta.set_index('field'); top_rows = []
    for target, j, coefficient, rank in final:
        field = fields[j]; key = (target, field); f = frequencies.get(key, 0) / 200
        evidence = grouped.loc[key] if key in grouped.index else pd.Series({'selected_folds': 0, 'ordinary_delta_mae': np.nan, 'within_style_delta_mae': np.nan})
        top_rows.append({'target': target, 'rank': rank, 'field': field, 'coefficient_raw_score_per_standardized_input': coefficient,
                         'bootstrap_frequency_fixed_alpha': f, **evidence.to_dict(),
                         'descriptive_support': bool(f >= .5 and evidence.selected_folds >= 5 and evidence.ordinary_delta_mae > 0 and evidence.within_style_delta_mae > 0),
                         'unit': metadata.at[field, 'unit'], 'origin': metadata.at[field, 'origin_paper'], 'causal_validated': False})
    save_csv(pd.DataFrame(top_rows), dest / 'absolute_raw_top3.csv')
    write_json(dest / 'status.json', {'complete': True, 'elapsed_seconds': time.perf_counter() - start, 'outer_fits': 15,
                                      'selected_full_ridge_alpha': alpha, 'absolute_endpoint': cfg['primary_absolute_target'],
                                      'source_identity_reconstructed': True, 'new_external_beers': 0})


def mix_concentrations(source, anchor, weights):
    return source[None, :] + np.asarray(weights)[:, None] * (anchor[None, :] - source[None, :])


def exact_nonincrease(values, source, positions):
```

```
    if not np.isfinite(source[positions]).all():
        return np.zeros(len(values), dtype=bool)
    return np.isfinite(values[:, positions]).all(1) & (values[:, positions] <= source[positions] + 1e-12).all(1)


def ordinal_summary(observed, probabilities, grades):
    axis = np.arange(7)
    individual_mae = (observed * np.abs(axis - grades[..., None])).sum(-1)
    individual_accuracy = np.take_along_axis(observed, grades[..., None], axis=-1)[..., 0]
    modal_accuracy = grades == observed.argmax(-1)
    result = {'expected_individual_grade_MAE': float(individual_mae.mean()),
              'expected_individual_exact_accuracy': float(individual_accuracy.mean()),
              'observed_panel_modal_grade_accuracy': float(modal_accuracy.mean()),
              'predicted_zero_fraction': float((grades == 0).mean()),
              'observed_panel_zero_mode_fraction': float((observed.argmax(-1) == 0).mean())}
    nonzero = observed.argmax(-1) > 0
    result['nonzero_panel_mode_accuracy'] = float(modal_accuracy[nonzero].mean()) if nonzero.any() else np.nan
    if probabilities is not None:
        result.update(RPS=float(((probabilities.cumsum(-1)[..., :-1] - observed.cumsum(-1)[..., :-1]) ** 2).mean()),
                      Brier=float(((probabilities - observed) ** 2).sum(-1).mean()),
                      expected_panel_mean_MAE=float(np.abs((probabilities * axis).sum(-1) - (observed * axis).sum(-1)).mean()))
    return result


def run_ordinal(data, raw, panel, out, cfg, previous):
    dest = Path(out) / 'ordinal'; dest.mkdir(parents=True, exist_ok=True)
    targets = list(data.Y); X = data.X.to_numpy(); Y = panel.to_numpy(); beers = data.meta.beer.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy(); observed = panel_rating_distribution(raw, targets, beers)
    core = np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))]); scopes = [('core30', core), ('all50', np.arange(50))]
    sums = np.zeros_like(observed); base_sum = np.zeros_like(observed); point_sum = np.zeros_like(Y); counts = np.zeros(len(X)); folds, stress = [], []
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=3, n_repeats=5, random_state=cfg['seed']).split(X, labels)):
        progress(f'S4 ordinal distribution fold {fold+1}/15')
        model = tree_model(cfg).fit(X[train], observed[train].reshape(len(train), -1))
        p = model.predict(X[test]).reshape(len(test), 50, 7); p /= p.sum(-1, keepdims=True)
        base = np.broadcast_to(observed[train].mean(0), p.shape)
        point = tree_model(cfg).fit(X[train], Y[train]).predict(X[test])
        sums[test] += p; base_sum[test] += base; point_sum[test] += point; counts[test] += 1
        for name, prob, grades in [('ET_rating_distribution', p, p.argmax(-1)),
                                   ('ET_rating_distribution_median', p, (p.cumsum(-1) >= .5).argmax(-1)),
                                   ('training_population_distribution', base, base.argmax(-1)),
                                   ('ET_mean_round_half_up', None, np.floor(point + .5).clip(0, 6).astype(int))]:
            for scope, selected in scopes:
                folds.append({'fold': fold, 'scope': scope, 'method': name,
                              **ordinal_summary(observed[test][:, selected], None if prob is None else prob[:, selected], grades[:, selected])})
        rng = np.random.default_rng(cfg['seed'] + fold)
        for rate in [0, .05, .1, .2]:
            masked = X[test].copy(); eligible = np.flatnonzero(np.isfinite(masked).ravel()); n = int(rate * len(eligible))
            if n: masked.ravel()[rng.choice(eligible, n, replace=False)] = np.nan
            changed = model.predict(masked).reshape(len(test), 50, 7)
            for scope, selected in scopes:
                stress.append({'fold': fold, 'scope': scope, 'additional_MCAR_rate': rate,
                               'modal_grade_change_fraction': float((changed.argmax(-1)[:, selected] != p.argmax(-1)[:, selected]).mean()),
                               **ordinal_summary(observed[test][:, selected], changed[:, selected], changed.argmax(-1)[:, selected])})
    probability = sums / counts[:, None, None]; baseline = base_sum / counts[:, None, None]; point = point_sum / counts[:, None]
    results = []
    for method, prob, grades in [('ET_rating_distribution', probability, probability.argmax(-1)),
                                 ('ET_rating_distribution_median', probability, (probability.cumsum(-1) >= .5).argmax(-1)),
                                 ('training_population_distribution', baseline, baseline.argmax(-1)),
                                 ('ET_mean_round_half_up', None, np.floor(point + .5).clip(0, 6).astype(int))]:
        for scope, selected in scopes:
            results.append({'scope': scope, 'method': method, **ordinal_summary(observed[:, selected], None if prob is None else prob[:, selected], grades[:, selected])})
    save_csv(pd.DataFrame(results), dest / 'metrics_by_scope.csv')
    save_csv(pd.DataFrame(folds), dest / 'fold_metrics.csv'); save_csv(pd.DataFrame(stress), dest / 'missingness_stress.csv')
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(probability.argmax(-1), columns=targets)], axis=1), dest / 'oof_modal_grades_0_6.csv')
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame((probability.cumsum(-1) >= .5).argmax(-1), columns=targets)], axis=1), dest / 'oof_median_grades_0_6.csv')
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(np.floor(point + .5).clip(0, 6).astype(int), columns=targets)], axis=1), dest / 'oof_rounded_panel_mean_grades_0_6.csv')
    rows = []
    for i, beer in enumerate(beers):
        for j, target in enumerate(targets):
            rows.append({'beer': int(beer), 'target': target, 'predicted_modal_grade': int(probability[i, j].argmax()),
                         'observed_panel_modal_grade': int(observed[i, j].argmax()),
                         **{f'predicted_p{k}': probability[i, j, k] for k in range(7)},
                         **{f'observed_panel_p{k}': observed[i, j, k] for k in range(7)}})
    save_csv(pd.DataFrame(rows), dest / 'oof_rating_distributions.csv')
    roles = pd.read_csv(Path(previous) / 'q3/disjoint_roles.csv'); generated = pd.read_csv(Path(previous) / 'q3/generated_profiles_first_split.csv')
    features = [f for f in data.X if f not in AGGREGATES]; repeat0 = roles[roles.repeat == 0].set_index('beer').role.reindex(beers)
    for name, role in [('G', 'G_search_training'), ('J', 'J_evaluator_training')]:
        subset = np.flatnonzero(repeat0.to_numpy() == role)
        model = tree_model(cfg).fit(data.X[features].to_numpy()[subset], observed[subset].reshape(len(subset), -1))
        p = model.predict(generated[features].to_numpy()).reshape(len(generated), 50, 7)
        save_csv(pd.concat([generated[['generated_id', 'method', 'style']], pd.DataFrame(p.argmax(-1), columns=targets)], axis=1),
                 dest / f'generated_modal_grade_proxy_{name}_first_split.csv')
    write_json(dest / 'status.json', {'complete': True, 'beer_grouped_outer_fits': 15, 'grade_definition': 'modal_prediction_of_actual_0_6_panelist_rating_distribution',
                                      'zero_definition': 'most_probable_panelist_rating_zero_not_chemical_absence',
                                      'equal_panelist_and_within_person_repeat_weights': True, 'probabilities_claimed_calibrated': False,
                                      'new_physical_measurements': False})


def run_transfer_diagnostics(data, panel, concentration, out, cfg, previous):
    dest = Path(out) / 'transfer'; dest.mkdir(parents=True, exist_ok=True)
    previous = Path(previous); roles = pd.read_csv(previous / 'q3/disjoint_roles.csv')
    profiles = pd.read_csv(previous / 'q3/transfer_profiles.csv'); old_eval = pd.read_csv(previous / 'q3/transfer_independent_evaluation.csv')
    targets = list(data.Y); stale = targets.index('stale_hops'); beers = data.meta.beer.to_numpy(); labels = data.meta.tasting_category_fine.to_numpy()
    nonaggregate = [f for f in data.X if f not in AGGREGATES]; X = data.X[nonaggregate].to_numpy(); Y = panel.to_numpy(); C = concentration.to_numpy(); features = list(concentration)
    positions = [features.index(f) for f in CHEMICAL_DEFECTS]
    frozen_rows, mixture_rows, mixture_profiles, locks, diagnostics, lp_rows = [], [], [], [], [], []
    for repeat in range(5):
        progress(f'S4 frozen-candidate and concentration mixture evaluation {repeat+1}/5')
        role = roles[roles.repeat == repeat].set_index('beer').role.reindex(beers).to_numpy()
        G = np.flatnonzero(role == 'G_search_training'); J = np.flatnonzero(role == 'J_evaluator_training'); T = np.flatnonzero(role == 'T_real_holdout')
        assert len(G) == len(J) == 100 and len(T) == 50 and not set(G) & set(J)
        # Old candidates are already frozen; no raw-scale J feedback changes them.
        graw = tree_model(cfg).fit(X[G], Y[G]); jraw = tree_model(cfg).fit(X[J], Y[J])
        if repeat == 0:
            generated = pd.read_csv(previous / 'q3/generated_profiles_first_split.csv')
            for name, predictor in [('G', graw), ('J', jraw)]:
                portrait = pd.DataFrame(predictor.predict(generated[nonaggregate].to_numpy()), columns=targets)
                save_csv(pd.concat([generated[['generated_id', 'method', 'style']].reset_index(drop=True), portrait], axis=1),
                         dest / f'generated_raw_score_proxy_{name}_first_split.csv')
        old = profiles[profiles.repeat == repeat].copy(); candidate = old[nonaggregate].to_numpy(); ids = pd.Index(beers).get_indexer(old.beer)
        sp, cp = jraw.predict(X[ids])[:, stale], jraw.predict(candidate)[:, stale]
        ga, gc = graw.predict(X[ids])[:, stale], graw.predict(candidate)[:, stale]
```

```
        prior = old.merge(old_eval[['repeat', 'beer', 'radius', 'evaluator_proxy_acceptance']], on=['repeat', 'beer', 'radius'], validate='one_to_one')
        for k, row in old.reset_index(drop=True).iterrows():
            feasible = not row.status.startswith('no_feasible')
            frozen_rows.append({'repeat': repeat, 'beer': int(row.beer), 'radius': row.radius, 'source_role': role[ids[k]],
                                'original_search_feasible': feasible, 'raw_G_stale_source': ga[k], 'raw_G_stale_candidate': gc[k],
                                'raw_J_stale_source': sp[k], 'raw_J_stale_candidate': cp[k],
                                'raw_G_nonincrease': bool(gc[k] <= ga[k] + 1e-9), 'raw_J_nonincrease': bool(cp[k] <= sp[k] + 1e-9),
                                'original_proxy_acceptance': bool(prior.iloc[k].evaluator_proxy_acceptance),
                                'original_and_raw_proxy_support': bool(feasible and prior.iloc[k].evaluator_proxy_acceptance and gc[k] <= ga[k] + 1e-9 and cp[k] <= sp[k] + 1e-9),
                                'physical_validation': False})
        # New mixtures use only documented concentration-unit fields, not pH/pressure/A.U.
        generator_prep = preprocessing().fit(C[G]); fill = generator_prep.named_steps['impute'].statistics_
        source_data = np.where(np.isfinite(C), C, fill)
        search = Pipeline([('prep', preprocessing()), ('model', LogisticRegression(C=1, max_iter=3000, random_state=cfg['seed']))]).fit(C[G], labels[G])
        sensory = tree_model(cfg).fit(C[G], Y[G])
        consensus = Pipeline([('prep', preprocessing()), ('model', Ridge(alpha=1000))]).fit(C[G], Y[G])
        target_position = list(search.classes_).index('Tripel'); anchors = G[labels[G] == 'Tripel']; locked = []
        for i in np.flatnonzero(labels == 'Pils/Lager'):
            source = source_data[i]; srcp = search.predict_proba(source[None])[0, target_position]
            candidates = np.concatenate([mix_concentrations(source, source_data[a], cfg['mixture_weights']) for a in anchors])
            candidate_recipes = [(int(beers[a]), weight, {int(beers[a]): weight}) for a in anchors for weight in cfg['mixture_weights']]
            # An exploratory LP extension checks combinations of multiple target beers.
            # Source/anchor missing defect values cannot support a chemical certificate.
            known_anchors = anchors[np.isfinite(C[anchors][:, positions]).all(1)]
            lp_fraction = None
            if np.isfinite(C[i, positions]).all() and len(known_anchors):
                difference = (C[known_anchors][:, positions] - C[i, positions]).T
                lp = linprog(-np.ones(len(known_anchors)), A_ub=np.vstack([difference, np.ones(len(known_anchors))]),
                             b_ub=np.r_[np.zeros(len(positions)), .75], bounds=(0, None), method='highs')
                if lp.success:
                    lp_fraction = float(lp.x.sum())
                    if lp_fraction > 1e-8:
                        for shrink in np.linspace(.05, 1, 20):
                            weights = shrink * lp.x
                            value = source * (1 - weights.sum()) + weights @ source_data[known_anchors]
                            candidates = np.vstack([candidates, value])
                            recipe = {int(beers[a]): float(w) for a, w in zip(known_anchors, weights) if w > 1e-10}
                            candidate_recipes.append((None, float(weights.sum()), recipe))
            lp_rows.append({'repeat': repeat, 'beer': int(beers[i]), 'source_role': role[i],
                            'max_target_mixture_fraction': lp_fraction, 'known_target_anchors': len(known_anchors),
                            'scope': 'reported_concentration_four_linear_constraints_only_no_LOD_or_reaction_certification'})
            known = np.isfinite(C[i, positions]).all()
            chem_ok = exact_nonincrease(candidates, C[i], positions)
            probabilities = search.predict_proba(candidates)[:, target_position]
            gs = sensory.predict(source[None])[0, stale]; rs = consensus.predict(source[None])[0, stale]
            gpred = sensory.predict(candidates)[:, stale]; rpred = consensus.predict(candidates)[:, stale]
            valid = chem_ok & (probabilities >= max(.5, srcp + .01)) & (gpred <= gs + 1e-9) & (rpred <= rs + 1e-9)
            change = np.linalg.norm(generator_prep.transform(candidates) - generator_prep.transform(source[None]), axis=1) / np.sqrt(C.shape[1])
            valid &= change > 1e-8
            good = np.flatnonzero(valid); k = int(good[np.argmin(change[good])]) if len(good) else None
            value = candidates[k].copy() if k is not None else source.copy()
            locked.append({'row': i, 'candidate': value, 'source': source, 'candidate_index': k,
                           'anchor_beer': candidate_recipes[k][0] if k is not None else None,
                           'weight': candidate_recipes[k][1] if k is not None else 0,
                           'recipe': candidate_recipes[k][2] if k is not None else {},
                           'source_prob': srcp, 'candidate_prob': probabilities[k] if k is not None else srcp,
                           'candidates': len(candidates), 'chemical_feasible': int(chem_ok.sum()), 'valid': len(good),
                           'known_defects': bool(known), 'positive_defects': bool(known and (C[i, positions] > 0).all()),
                           'conditional_missing_any_source_input': bool(np.isnan(C[i]).any()),
                           'source_role': role[i], 'status': 'concentration_mixture_proxy_feasible' if k is not None else 'no_feasible_mixture_source_retained'})
        # Lock exact numeric candidates BEFORE fitting independent J classifiers/sensory.
        digest = hashlib.sha256(np.asarray([v['candidate'] for v in locked], dtype='<f8').tobytes()).hexdigest()
        write_json(dest / f'concentration_candidate_lock_{repeat}.json', {'repeat': repeat, 'sha256_float64_candidates': digest,
                                                                          'features': features, 'selection_completed_before_J_fitting': True})
        evaluator = Pipeline([('prep', preprocessing()), ('model', ExtraTreesClassifier(n_estimators=200, min_samples_leaf=3, max_features=.7, random_state=cfg['seed'], n_jobs=2))]).fit(C[J], labels[J])
        judge = tree_model(cfg).fit(C[J], Y[J]); target_j = list(evaluator.classes_).index('Tripel')
        diagnostics.append({'repeat': repeat, 'concentration_search_balanced_accuracy_on_T': balanced_accuracy_score(labels[T], search.predict(C[T])),
                            'concentration_J_balanced_accuracy_on_T': balanced_accuracy_score(labels[T], evaluator.predict(C[T])),
                            'raw_stale_J_mae_on_T': float(np.abs(judge.predict(C[T])[:, stale] - Y[T, stale]).mean())})
        for v in locked:
            i = v['row']; source, value = v['source'], v['candidate']; jp = evaluator.predict_proba(np.stack([source, value]))[:, target_j]
            js = judge.predict(np.stack([source, value]))[:, stale]; predicted = evaluator.predict(value[None])[0]
            feasible = v['candidate_index'] is not None
            accepted = feasible and predicted == 'Tripel' and jp[1] > jp[0] + .01 and js[1] <= js[0] + 1e-9
            mixture_rows.append({'repeat': repeat, 'beer': int(beers[i]), 'source_role': v['source_role'], 'status': v['status'],
                                 'anchor_beer': v['anchor_beer'], 'anchor_volume_fraction': v['weight'], 'n_candidates': v['candidates'],
                                 'target_component_fractions': str(v['recipe']),
                                 'n_chemical_feasible': v['chemical_feasible'], 'n_search_feasible': v['valid'],
                                 'four_source_defects_observed': v['known_defects'], 'four_source_defects_positive': v['positive_defects'],
                                 'conditional_on_other_source_input_imputation': v['conditional_missing_any_source_input'],
                                 'G_source_Tripel_probability': v['source_prob'], 'G_candidate_Tripel_probability': v['candidate_prob'],
                                 'J_source_Tripel_probability': jp[0], 'J_candidate_Tripel_probability': jp[1],
                                 'J_stale_source_raw_0_6': js[0], 'J_stale_candidate_raw_0_6': js[1],
                                 'J_predicts_Tripel': predicted == 'Tripel', 'independent_proxy_support': bool(accepted),
                                 'real_tasting_validated': False, 'LOD_certification': False})
            mixture_profiles.append({'repeat': repeat, 'beer': int(beers[i]), 'status': v['status'], **dict(zip(features, value))})
    save_csv(pd.DataFrame(frozen_rows), dest / 'frozen_stage3_candidates_raw_score_check.csv')
    save_csv(pd.DataFrame(mixture_rows), dest / 'concentration_mixture_evaluation.csv')
    save_csv(pd.DataFrame(mixture_profiles), dest / 'concentration_mixture_profiles.csv')
    save_csv(pd.DataFrame(diagnostics), dest / 'concentration_holdout_diagnostics.csv')
    save_csv(pd.DataFrame(lp_rows), dest / 'mixture_lp_feasibility.csv')
    write_json(dest / 'status.json', {'complete': True, 'concentration_features': features, 'frozen_candidates_evaluated': len(frozen_rows),
                                      'mixture_records': len(mixture_rows), 'physical_experiments_performed': False})
```

src/stage4_trial.py

```
"""Power planning and analysis for a future paired, blinded physical experiment.
No human observations are generated here. AI-assisted, 2026-10-25.
"""
import numpy as np
import pandas as pd
from scipy.stats import t, nct

DEFECT_ENDPOINTS = ['dms', 'diacetyl', 'X4vg', 't2n', 'stale_hops']

```

```

def planned_noninferiority_power(n, sigma, margin=.5, alpha=.01):
    critical = t.ppf(1 - alpha, n - 1)
    return float(nct.cdf(-critical, n - 1, -margin * np.sqrt(n) / sigma))


def analyze_paired_trial(frame, margin=.5):
    endpoints = DEFECT_ENDPOINTS + ['style_Tripel_fit']
    required = ['panelist', 'batch', 'condition'] + endpoints
    if not set(required).issubset(frame) or frame.empty:
        raise ValueError('No observed paired data; a blank template is not an experiment')
    if frame[required].isna().any().any() or frame.duplicated(['panelist', 'batch', 'condition']).any():
        raise ValueError('Missing values or duplicated participant-batch-condition records require review')
    if set(frame.condition) != {'control', 'candidate'} or not frame[endpoints].ge(0).all().all() or not frame[endpoints].le(6).all().all():
        raise ValueError('Two conditions and valid 0-6 ratings are required')
    control = frame[frame.condition == 'control'].set_index(['panelist', 'batch'])[endpoints]
    candidate = frame[frame.condition == 'candidate'].set_index(['panelist', 'batch'])[endpoints]
    if set(control.index) != set(candidate.index):
        raise ValueError('Unpaired batches cannot silently enter paired analysis')
    difference = (candidate - control).groupby(level='panelist').mean()
    if len(difference) < 3:
        raise ValueError('At least three independent panelists are required for the diagnostic')
    rows = []
    for endpoint in endpoints:
        d = difference[endpoint].to_numpy(); n = len(d); mean = d.mean(); se = d.std(ddof=1) / np.sqrt(n)
        alpha = .01 if endpoint in DEFECT_ENDPOINTS else .05
        upper = mean + t.ppf(1 - alpha, n - 1) * se
        lower = mean - t.ppf(1 - alpha, n - 1) * se
        rows.append({'endpoint': endpoint, 'independent_panelists': n, 'candidate_minus_control': mean,
                     'one_sided_alpha': alpha, 'lower_confidence_limit': lower, 'upper_confidence_limit': upper,
                     'criterion_passed': bool(upper < margin if endpoint in DEFECT_ENDPOINTS else lower > 0),
                     'criterion': 'upper_limit_below_0.5_noninferiority_margin' if endpoint in DEFECT_ENDPOINTS else 'lower_limit_above_zero_style_improvement'})
    return pd.DataFrame(rows)
```

scripts/analyze__trial.py

```
"""Analyze real future blinded paired ratings; refuse empty/simulated templates."""
from pathlib import Path
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import argparse
import pandas as pd
from src.stage4_trial import analyze_paired_trial
from src.common import save_csv
parser = argparse.ArgumentParser()
parser.add_argument('--ratings', required=True)
parser.add_argument('--out', required=True)
args = parser.parse_args()
result = analyze_paired_trial(pd.read_csv(args.ratings))
save_csv(result, args.out)
print('Analysis uses independent participant means; repeated batches do not increase the participant count.')
```

scripts/build__paper__pdf.py

```
"""AI-assisted PDF layout: OpenAI Codex / GPT-6, 2026-10-24.
Run with the bundled document runtime, not the numerical research environment.
"""
from pathlib import Path
from xml.sax.saxutils import escape
import json

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY, TA_LEFT
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import cm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, Frame, PageTemplate, Paragraph, Spacer, PageBreak, Image, LongTable, TableStyle, KeepTogether
from reportlab.platypus.tableofcontents import TableOfContents
from PIL import Image as PILImage

ROOT = Path(__file__).resolve().parents[1]
CONTENT = json.loads((ROOT / "paper/paper_content.json").read_text(encoding="utf-8"))
pdfmetrics.registerFont(TTFont("Song", "C:/Windows/Fonts/simsun.ttc", subfontIndex=0))
pdfmetrics.registerFont(TTFont("Hei", "C:/Windows/Fonts/simhei.ttf"))
pdfmetrics.registerFontFamily("Song", normal="Song", bold="Hei", italic="Song", boldItalic="Hei")

styles = {
    "Title": ParagraphStyle("Title", fontName="Hei", fontSize=17, leading=25, alignment=TA_CENTER, spaceAfter=15, textColor=colors.black),
    "Front": ParagraphStyle("Front", fontName="Hei", fontSize=13, leading=20, alignment=TA_CENTER, spaceBefore=6, spaceAfter=9),
    "Body": ParagraphStyle("Body", fontName="Song", fontSize=10.5, leading=17, firstLineIndent=21, alignment=TA_JUSTIFY, wordWrap="CJK", spaceAfter=6),
    "Heading1": ParagraphStyle("Heading1", fontName="Hei", fontSize=13, leading=20, spaceBefore=12, spaceAfter=8, keepWithNext=True),
    "Heading2": ParagraphStyle("Heading2", fontName="Hei", fontSize=11.5, leading=18, spaceBefore=8, spaceAfter=5, keepWithNext=True),
    "Equation": ParagraphStyle("Equation", fontName="Song", fontSize=10, leading=17, alignment=TA_CENTER, wordWrap="CJK", spaceBefore=5, spaceAfter=8),
    "Caption": ParagraphStyle("Caption", fontName="Song", fontSize=9, leading=14, alignment=TA_CENTER, wordWrap="CJK", spaceBefore=5, spaceAfter=9),
    "Cell": ParagraphStyle("Cell", fontName="Song", fontSize=8.5, leading=12, wordWrap="CJK", alignment=TA_LEFT),
    "CellHeader": ParagraphStyle("CellHeader", fontName="Hei", fontSize=9, leading=13, wordWrap="CJK", alignment=TA_CENTER),
    "Reference": ParagraphStyle("Reference", fontName="Song", fontSize=9, leading=14, wordWrap="CJK", spaceAfter=6),
}


def text(s):
    return escape(str(s)).replace("B̂", "B_hat").replace("D̂", "D_hat").replace("\u0302", "_hat").replace("−", "-").replace("ŷ", "y_hat").replace("ᵀ", "<super>T</super>").replace("²", "<super>2</super>")


class PaperDoc(BaseDocTemplate):
    def afterFlowable(self, flowable):
        if isinstance(flowable, Paragraph) and flowable.style.name in ["Heading1", "Heading2"]:
            level = 0 if flowable.style.name == "Heading1" else 1
            key = flowable.bookmark_key
            self.canv.bookmarkPage(key)
            self.canv.addOutlineEntry(flowable.getPlainText(), key, level=level, closed=False)
            if level == 0:
                self.notify("TOCEntry", (level, flowable.getPlainText(), self.page - 2, key))


def footer(canvas, doc):
    canvas.setAuthor("")
    canvas.setTitle(CONTENT["title"])
    canvas.setSubject("啤酒化合物感官预测与风格迁移参考研究")
    if doc.page >= 3:
        canvas.setFont("Song", 9)
        canvas.drawCentredString(A4[0] / 2, 1.2 * cm, str(doc.page - 2))


output = ROOT / "paper/论文参考稿.pdf"
doc = PaperDoc(str(output), pagesize=A4, leftMargin=2.5*cm, rightMargin=2.5*cm, topMargin=2*cm, bottomMargin=2*cm, pageCompression=1)
frame = Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)
doc.addPageTemplates([PageTemplate(id="paper", frames=[frame], onPage=footer)])
story = [Spacer(1, .4 * cm), Paragraph(text(CONTENT["title"]), styles["Title"]), Paragraph("摘 要", styles["Front"]), Paragraph(text(CONTENT["abstract"]), styles["Body"]), Spacer(1,.3*cm), Paragraph("<b>关键词：</b>" + text(CONTENT["keywords"]), styles["Body"]), PageBreak(), Paragraph("目 录", styles["Front"])]
toc = TableOfContents()
toc.levelStyles = [ParagraphStyle("TOC1", fontName="Song", fontSize=10.5, leading=16, leftIndent=0, firstLineIndent=0, spaceBefore=3), ParagraphStyle("TOC2", fontName="Song", fontSize=9.5, leading=14, leftIndent=18, firstLineIndent=0, spaceBefore=1)]
story += [toc, PageBreak()]
in_references = False
for i, block in enumerate(CONTENT["blocks"]):
    kind = block["type"]
    if kind == "heading":
        in_references = block["text"] == "参考文献"
        para = Paragraph(text(block["text"]), styles["Heading1" if block["
```

```
level"] == 1 else "Heading2"])
        para.bookmark_key = "heading_%d" % i
        story.append(para)
    elif kind == "paragraph":
        story.append(Paragraph(text(block["text"]), styles["Reference" if in_references else "Body"]))
    elif kind == "equation":
        equation_path = ROOT / "paper/equations" / ("eq_%02d.png" % int(block["number"]))
        with PILImage.open(equation_path) as im:
            width, height = im.size
        ratio = min(72 / 250, (doc.width - 44) / width)
        equation_image = Image(str(equation_path), width=width*ratio, height=height*ratio)
        equation_image.hAlign = "CENTER"
        equation_table = LongTable([[equation_image, Paragraph("（%s）" % block["number"], styles["Equation"])]], colWidths=[doc.width-38,38])
        equation_table.setStyle(TableStyle([("VALIGN",(0,0),(-1,-1),"MIDDLE"),("ALIGN",(0,0),(0,0),"CENTER"),("TOPPADDING",(0,0),(-1,-1),5),("BOTTOMPADDING",(0,0),(-1,-1),8)]))
        story.append(equation_table)
    elif kind == "table":
        rows = [[Paragraph(text(c), styles["CellHeader"]) for c in block["headers"]]]
        rows += [[Paragraph(text(c), styles["Cell"]) for c in row] for row in block["rows"]]
        ratios = block["widths"] or [1/len(rows[0])] * len(rows[0])
        table = LongTable(rows, colWidths=[doc.width*x for x in ratios], repeatRows=1, hAlign="CENTER")
        table.setStyle(TableStyle([("GRID",(0,0),(-1,-1),.45,colors.HexColor("#D9D9D9")),("BACKGROUND",(0,0),(-1,0),colors.HexColor("#E8EEF2")),("VALIGN",(0,0),(-1,-1),"MIDDLE"),("LEFTPADDING",(0,0),(-1,-1),6),("RIGHTPADDING",(0,0),(-1,-1),6),("TOPPADDING",(0,0),(-1,-1),6),("BOTTOMPADDING",(0,0),(-1,-1),6)]))
        story.extend([Spacer(1,5),table,Spacer(1,9)])
    elif kind == "figure":
        path = ROOT / block["path"]
        with PILImage.open(path) as im: width,height=im.size
        max_height = (10.5 if "style_counts" in str(path) else 7) * cm
        ratio=min(doc.width/width,max_height/height)
        image = Image(str(path),width=width*ratio,height=height*ratio)
        image.hAlign="CENTER"
        story.append(KeepTogether([Spacer(1,4),image,Paragraph(text(block["caption"]), styles["Caption"])]))
doc.multiBuild(story)
print(json.dumps({"pdf":str(output),"bytes":output.stat().st_size},ensure_ascii=False))
```

scripts/build_revised_pdf.py

```
"""Anonymous PDF with math figures, bookmarks and complete source appendix.
AI-assisted layout, OpenAI Codex, 2026-10-25. Run with document runtime.
"""
from pathlib import Path
from xml.sax.saxutils import escape
import argparse
import hashlib
import json
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import cm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, Frame, PageTemplate, Paragraph, Spacer, PageBreak, Image, LongTable, TableStyle, KeepTogether
from reportlab.platypus.tableofcontents import TableOfContents
from PIL import Image as PILImage

ROOT=Path(__file__).resolve().parents[1]
parser=argparse.ArgumentParser(); parser.add_argument('--content',required=True); parser.add_argument('--output',required=True); args=parser.parse_args()
content=json.loads(Path(args.content).read_text(encoding='utf-8')); output=Path(args.output); output.parent.mkdir(parents=True,exist_ok=True)
for name,path in [('Song','C:/Windows/Fonts/simsun.ttc'),('Hei','C:/Windows/Fonts/simhei.ttf'),('Code','C:/Windows/Fonts/consola.ttf')]:
    pdfmetrics.registerFont(TTFont(name,path,subfontIndex=0))
pdfmetrics.registerFontFamily('Song',normal='Song',bold='Hei',italic='Song',boldItalic='Hei')
styles={
 'Title':ParagraphStyle('Title',fontName='Hei',fontSize=17,leading=25,alignment=TA_CENTER,spaceAfter=15),
 'Front':ParagraphStyle('Front',fontName='Hei',fontSize=13,leading=20,alignment=TA_CENTER,spaceAfter=12),
 'Body':ParagraphStyle('Body',fontName='Song',fontSize=10.5,leading=17,firstLineIndent=21,alignment=TA_JUSTIFY,wordWrap='CJK',spaceAfter=6),
 'Heading1':ParagraphStyle('Heading1',fontName='Hei',fontSize=13,leading=20,spaceBefore=12,spaceAfter=8,keepWithNext=True),
 'Heading2':ParagraphStyle('Heading2',fontName='Hei',fontSize=11.5,leading=18,spaceBefore=8,spaceAfter=5,keepWithNext=True),
 'Caption':ParagraphStyle('Caption',fontName='Song',fontSize=9,leading=14,alignment=TA_CENTER,wordWrap='CJK',spaceAfter=9),
 'Cell':ParagraphStyle('Cell',fontName='Song',fontSize=8.2,leading=12,wordWrap='CJK'),
 'Header':ParagraphStyle('Header',fontName='Hei',fontSize=8.5,leading=12,wordWrap='CJK',alignment=TA_CENTER),
 'Reference':ParagraphStyle('Reference',fontName='Song',fontSize=9,leading=14,wordWrap='CJK',spaceAfter=5),
 'Code':ParagraphStyle('Code',fontName='Code',fontSize=6.8,leading=8.8,wordWrap='CJK',spaceAfter=0),
}
def txt(s):
    s=str(s).replace('−','-').replace('^ ','_hat').replace('ŷ','y_hat').replace('ᵀ','^T').replace('–','-').replace('—','-')
    return escape(s).replace('²','<super>2</super>')

class Doc(BaseDocTemplate):
    def afterFlowable(self,flowable):
        if isinstance(flowable,Paragraph) and flowable.style.name in ['Heading1','Heading2']:
            level=0 if flowable.style.name=='Heading1' else 1; key=flowable.bookmark_key
            self.canv.bookmarkPage(key); self.canv.addOutlineEntry(flowable.getPlainText(),key,level=level,closed=False)
            if level==0: self.notify('TOCEntry',(0,flowable.getPlainText(),self.page-2,key))

def footer(canvas,doc):
    canvas.setAuthor(''); canvas.setTitle(content['title']); canvas.setSubject(content.get('subject','数学建模参考研究'))
    if doc.page>=3: canvas.setFont('Song',9); canvas.drawCentredString(A4[0]/2,1.2*cm,str(doc.page-2))

doc=Doc(str(output),pagesize=A4,leftMargin=2.5*cm,rightMargin=2.5*cm,topMargin=2*cm,bottomMargin=2*cm,pageCompression=1)
frame=Frame(doc.leftMargin,doc.bottomMargin,doc.width,doc.height,leftPadding=0,rightPadding=0,topPadding=0,bottomPadding=0)
doc.addPageTemplates([PageTemplate(id='paper',frames=[frame],onPage=footer)])
story=[Spacer(1,.4*cm),Paragraph(txt(content['title']),styles['Title']),Paragraph(content.get('front_label','摘 要'),styles['Front']),Paragraph(txt(content['abstract']),styles['Body']),Spacer(1,10),Paragraph('<b>'+txt(content.get('keyword_label','关键词'))+': </b>'+txt(content['keywords']),styles['Body']),PageBreak(),Paragraph('目 录',styles['Front'])]
toc=TableOfContents(); toc.levelStyles=[ParagraphStyle('TOC',fontName='Song',fontSize=10,leading=15,spaceBefore=2)]; story.extend([toc,PageBreak()])

def heading(s,level,key):
    p=Paragraph(txt(s),styles['Heading1' if level==1 else 'Heading2']); p.bookmark_key=key; return p

def code_markup(s):
    # Switch fonts only to preserve characters; no source text is redacted.
    runs=[]; old=None; run=''
    for char in s:
        font='Code' if ord(char) in pdfmetrics.getFont('Code').face.charToGlyph else 'Song'
        if ord(char) not in pdfmetrics.getFont(font).face.charToGlyph:
            char='\\u%04x'%ord(char); font='Code'
        if old is not None and font!=old:
            runs.append(f'<font name="{old}">'+escape(run).replace(' ',' ')+'</font>'); run=''
        old=font; run+=char
    if run: runs.append(f'<font name="{old}">'+escape(run).replace(' ',' ')+'</font>')
    return ''.join(runs)

in_refs=False; source_index=[]
for index,b in enumerate(content['blocks']):
    kind=b['type']
    if kind=='heading':
        in_refs=b['text']=='参考文献'; story.append(heading(b['text'],b['level'],f'h_{index}'))
    elif kind=='paragraph': story.append(Paragraph(txt(b['text']),styles['Reference' if in_refs else 'Body']))
    elif kind=='equation':
        path=ROOT/content.get('equation_dir','paper/equations')/f"eq_{b.get('asset_number',b['number']):02d}.png"
        with PILImage.open(path) as im: w,h=im.size
        scale=min(72/250,(doc.width-44)/w); img=Image(str(path),width=w*scale,height=h*scale)
        table=LongTable([[img,Paragraph(f"（{b['number']}）",styles['Caption'])]],colWidths=[doc.width-38,38]); table.setStyle(TableStyle([('VALIGN',(0,0),(-1,-1),'MIDDLE'),('ALIGN',(0,0),(0,0),'CENTER'),('TOPPADDING',(0,0),(-1,-1),5),('BOTTOMPADDING',(0,0),(-1,-1),8)])); story.append(table)
    elif kind=='table':
        rows=[[Paragraph(txt(c),styles['Header']) for c in b['headers']]]+[[Paragraph(txt(c),styles['Cell']) for c in row] for row in b['rows']]
        widths=b.get('widths') or [1/len(rows[0])]*len(rows[0]); table=LongTable(rows,colWidths=[doc.width*w for w in widths],repeatRows=1)
```

```
        table.setStyle(TableStyle([('GRID',(0,0),(-1,-1),.4,colors.HexColor('#D5DCE1')),('BACKGROUND',(0,0),(-1,0),colors.HexColor('#E8EEF2')),('VALIGN',(0,0),(-1,-1),'MIDDLE'),('NOSPLIT',(0,0),(-1,min(2,len(rows)-1))),('LEFTPADDING',(0,0),(-1,-1),5),('RIGHTPADDING',(0,0),(-1,-1),5),('TOPPADDING',(0,0),(-1,-1),5),('BOTTOMPADDING',(0,0),(-1,-1),5)])); story.extend([Spacer(1,4),table,Spacer(1,8)])
    elif kind=='figure':
        path=ROOT/b['path']
        with PILImage.open(path) as im: w,h=im.size
        scale=min(doc.width/w,b.get('height_cm',7)*cm/h); img=Image(str(path),width=w*scale,height=h*scale)
        story.append(KeepTogether([img,Paragraph(txt(b['caption']),styles['Caption'])]))
    elif kind=='pagebreak': story.append(PageBreak())
    elif kind=='source_files':
        for number,relative in enumerate(b['paths'],1):
            path=ROOT/relative; raw=path.read_bytes(); source=raw.decode('utf-8-sig'); source_index.append({'file':relative,'sha256':hashlib.sha256(raw).hexdigest(),'lines':len(source.splitlines())})
            story.append(heading(f'源程序 {number}  {relative}',2,f'src_{number}'))
            for lineno,line in enumerate(source.splitlines(),1):
                source_line=line.expandtabs(4)
                story.append(Paragraph(f'<font color="#777777">{lineno:04d} </font>'+code_markup(source_line or ' '),styles['Code']))
            story.append(Spacer(1,10))
doc.multiBuild(story)
if source_index: (output.parent/'source_appendix_manifest.json').write_text(json.dumps(source_index,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'output':str(output),'bytes':output.stat().st_size,'source_files':len(source_index)},ensure_ascii=False))
```

scripts/fetch__stage4__sources.py

```
"""Fetch two public author sources with pinned MD5; keep raw files outside repo."""
from pathlib import Path
from urllib.request import urlopen
from urllib.parse import quote
import argparse
import hashlib
import json

ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser()
parser.add_argument('--out', default=str(ROOT.parent / 'external_sources/zenodo_v4_23624915'))
args = parser.parse_args(); out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8'))
for filename, key in [('Raw sensory data anonymized.csv', 'raw_md5'), ('Supplemental Files and Figure source files.xlsx', 'workbook_md5')]:
    path = out / filename
    if path.exists() and hashlib.md5(path.read_bytes()).hexdigest() == cfg[key]:
        print('Verified existing source: ' + filename); continue
    url = cfg['external_record'] + '/files/' + quote(filename) + '?download=1'
    raw = urlopen(url, timeout=60).read()
    if hashlib.md5(raw).hexdigest() != cfg[key]:
        raise ValueError('Publisher checksum mismatch: ' + filename)
    path.write_bytes(raw)
    print('Downloaded and verified: ' + filename)
```

scripts/package__reference.py

```
"""Package only authorized research artifacts; preserve exact bytes for publication."""
from pathlib import Path
import argparse
import base64
import hashlib
import json
import zipfile

ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser()
parser.add_argument("--out", required=True)
parser.add_argument("--payload")
args = parser.parse_args()
root_names = {"README.md", ".gitignore", "config.json", "config_stage2.json", "config_stage3_design.json", "config_stage3.json", "config_stage4.json", "run.py", "run_stage2.py", "run_stage3.py", "run_stage4.py", "requirements.txt", "requirements-lock.txt", "requirements-paper.txt"}
directories = {"src", "tests", "docs", "outputs", "paper", "scripts"}
suffixes = {".md", ".py", ".txt", ".json", ".csv", ".png", ".pdf"}
files = []
for path in sorted(ROOT.rglob("*")):
    if not path.is_file():
        continue
    rel = path.relative_to(ROOT)
    if "private" in rel.parts or "__pycache__" in rel.parts:
        continue
    if rel.as_posix() in root_names or (rel.parts[0] in directories and path.suffix.lower() in suffixes):
        assert path.suffix.lower() not in {".xlsx", ".joblib", ".xml"}
        files.append(path)
output = Path(args.out)
output.parent.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(output, "w", zipfile.ZIP_DEFLATED) as z:
    for path in files:
        z.write(path, path.relative_to(ROOT).as_posix())
with zipfile.ZipFile(output) as z:
    assert z.testzip() is None and len(z.namelist()) == len(files)
if args.payload:
    entries = []
    for path in files:
        raw = path.read_bytes()
        binary = path.suffix.lower() in {".png", ".pdf"}
        entries.append({"path": path.relative_to(ROOT).as_posix(), "encoding": "base64" if binary else "utf-8",
                        "content": base64.b64encode(raw).decode("ascii") if binary else raw.decode("utf-8"),
                        "blob_sha": hashlib.sha1(b"blob " + str(len(raw)).encode() + b"\0" + raw).hexdigest()})
    Path(args.payload).write_text(json.dumps(entries, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
print(json.dumps({"zip": str(output), "files": len(files), "bytes": output.stat().st_size,
                  "sha256": hashlib.sha256(output.read_bytes()).hexdigest(),
                  "payload_characters": len(Path(args.payload).read_text(encoding="utf-8")) if args.payload else None}, ensure_ascii=False))
```

scripts/prepare__absolute__paper.py

```
"""Compose the revised paper from executed raw-score and chemical-audit results.
Earlier manuscripts remain unchanged; all bibliography numbers follow appearance.
"""
from pathlib import Path
import copy
import json
import re
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]; OUT = ROOT / 'outputs/stage4'; PAPER = ROOT / 'paper/revised_absolute_20261025'; PAPER.mkdir(parents=True, exist_ok=True)
old = json.loads((ROOT / 'paper/revised_20261025/paper_content.json').read_text(encoding='utf-8'))
keys = ['source', 'et', 'gbr', 'stack', 'graph', 'cp', 'sklearn', 'cv', '4vg', 'iron', 'coriander', 'mmd', 'ai']
references = {}
in_refs = False
for b in old['blocks']:
    if b['type'] == 'heading': in_refs = b['text'] == '参考文献'
    if in_refs and b['type'] == 'paragraph':
        match = re.match(r'\[(\d+)\]\s*(.*)', b['text'])
        if match: references[keys[int(match.group(1)) - 1]] = match.group(2)
references.update({
    'raw': 'Schreurs M, et al. Data from: Predicting and improving complex beer flavor through machine learning, Version v4. Zenodo, 2026-10-24. https://zenodo.org/records/23624915, 访问日期: 2026-10-25. DOI:10.5281/zenodo.23624915.',
    'nonenal': 'Santos J R, Carneiro J R, Guido L F, et al. Determination of E-2-nonenal by high-performance liquid chromatography with UV detection assay for the evaluation of beer ageing. Journal of Chromatography A, 985(1-2):395-402, 2003. DOI:10.1016/S0021-9673(02)01396-1.',
    'threshold': 'Meilgaard M C. Individual differences in sensory threshold for aroma chemicals added to beer. Food Quality and Preference, 4(3):153-167, 1993. DOI:10.1016/0950-3293(93)90158-3.',
})
order, blocks = [], []
def C(key):
    if key not in order: order.append(key)
    return '[' + str(order.index(key) + 1) + ']'
def H(text, level=1): blocks.append({'type': 'heading', 'level': level, 'text': text})
def P(text): blocks.append({'type': 'paragraph', 'text': text})
def T(headers, rows, widths=None): blocks.append({'type': 'table', 'headers': headers, 'rows': rows, 'widths': widths})
```

```
def E(text, asset): blocks.append({'type': 'equation', 'text': text, 'asset_number': asset})
def F(name, caption, height=6.1, stage='stage4'): blocks.append({'type': 'figure', 'path': f'outputs/{stage}/figures/{name}', 'caption': caption, 'height_cm': height})

audit = json.loads((OUT / 'audit/audit_summary.json').read_text(encoding='utf-8'))
metrics = pd.read_csv(OUT / 'prediction/metrics_by_scope.csv'); ordinal = pd.read_csv(OUT / 'ordinal/metrics_by_scope.csv'); cover = pd.read_csv(OUT / 'prediction/conformal_coverage.csv')
graph = pd.read_csv(OUT / 'raw_graph/q2/graph_comparison.csv'); graph_status = json.loads((OUT / 'raw_graph/q2/status.json').read_text())
top = pd.read_csv(OUT / 'raw_graph/q2/top3_with_evidence.csv'); factors_supported = int(top.evidence_status.str.startswith('comparatively').sum())
frozen = pd.read_csv(OUT / 'transfer/frozen_stage3_candidates_raw_score_check.csv'); radius1 = frozen[frozen.radius == 1]; held = radius1[radius1.source_role == 'T_real_holdout']
gen = pd.read_csv(ROOT / 'outputs/stage3/q3/generation_summary.csv'); mixture = pd.read_csv(OUT / 'transfer/concentration_mixture_evaluation.csv'); lp = pd.read_csv(OUT / 'transfer/mixture_lp_feasibility.csv')
checks = json.loads((OUT / 'validation_summary.json').read_text(encoding='utf-8')); tests = checks['tests']['tests']
def M(model, scope): return float(metrics[(metrics.model == model) & (metrics.scope == scope)].mae.iloc[0])
def O(method, scope): return ordinal[(ordinal.method == method) & (ordinal.scope == scope)].iloc[0]

H('1 问题重述与研究目标')
P(f'本题要求从化合物数据给出Aroma/Flavour的0-6分级评分，建立化合物网络并给出逐维Top-3、稳定性和化学解释，最后按风格生成化合物谱及缺陷不恶化的迁移。附件与Schreurs等的啤酒研究高度吻合{C("source")}。本研究补查作者新版公开原始评分{C("raw")}，将此前不能识别的绝对评分问题改为对实际逐人评分的直接建模。作者已有模型和加标实验不计作本队新增发现。')
P('附件实际为250款酒、231个数值输入和50个感官标签。A_与F_前缀各15项，合计30个核心维度；其他20项也完整输出，另标明评分卡位置。这个范围选择不代表官方只要求30项；题面“约40”与实际目标数量不一致处明确列出，不补造缺失的监督列。')
P('本文的任务交付为：真实0-6评分分布及整数等级、0-6面板均分与不确定性；基于相关图对照的逐维因素表；统计条件生成谱、原始评分尺度的预测画像和迁移诊断。统计画像、来源表记载的浓度和新批次化学/真实品评证据分别记录。')

H('2 已有方法、贡献定位与假设')
P(f'ExtraTrees{C("et")}、梯度提升{C("gbr")}、交叉验证组合权重{C("stack")}、图约束回归{C("graph")}和分割保形预测{C("cp")}均为已有方法。整数等级的众数和中位数决策也属于标准统计决策。本文贡献在于来源对应核查、适合该任务的验证与约束、尺度改变后的复查及失败结果记录，不主张发明新的基础算法。')
P('假设一，比赛附件内部按beer对齐有效；新增匿名原始评分的编号不直接等于beer，而由公开补充表行序重建，并以两组分开的属性交叉核验。假设二，历史面板的均分和评分分布是本任务可用观测端点，但不等于所有消费者的唯一真实感受。假设三，插补、截尾和模型只提供统计预测，不能确认异常值或缺失机制。假设四，相关图表示共变，因素不是因果通路。假设五，历史浓度单位与条件阈值尚不足以保证新批次酿造和品评结论。')
T(['符号', '定义'], [['X、Z', '231维附件输入及仅在训练内拟合的预处理结果'], ['s_btrj、q_bjk', '啤酒b、人员t、重复r的原始0-6评分；逐人员等权的七级分布'], ['s_bar、g', '面板均分；评分分布上的整数决策'], ['L、B', '符号图拉普拉斯及多目标回归系数'], ['G、J、T', '100款搜索训练、100款独立评价训练、50款真实留出'], ['c_s、C_A、w', '来源浓度、Tripel锚点浓度矩阵及混合份额']], [.24, .76])

H('3 数据来源、评分重建与化学尺度核查')
H('3.1 原附件、相关矩阵与评分卡', 2)
P('1.csv为250×234（含3列元信息），4.csv为250×53；程序按唯一beer对齐并核对风格，精度受损的beer_id不作主键。原附件未改动并保存SHA-256。2.csv与重算Spearman的平均绝对差约1.58×10^-16，与Pearson约差0.0873，因此按秩相关解释，不沿用题面中未得到数据支持的Pearson描述。')
P('评分卡中的A/F_ester_fruity与监督表A/F_esters_fruity、4vg与X4vg作显式别名映射。乙醛的Aroma/Flavour位置没有独立监督列；12处Barnyard汇总引用AG76而标签在AG90，列为疑似源公式问题，未擅自修改，也不据此断言4.csv必然错误。空白输入的公式缓存零不作原始评分。')
P(f'预处理为训练分位截尾、中位数插补与标准化，以Pipeline管理{C("sklearn")}；验证和测试只应用已拟合变换。原始表中负值不能一律当错误或负浓度。')
E('z_ij=[clip(x_ij;a_j,b_j)-m_j]/s_j', 1)
H('3.2 新版原始评分与编号对应', 2)
P(f'2026-10-24发布的作者v4仓库新增逐人员原始评分{C("raw")}。文件含2197条记录、16名匿名人员、250款酒和50个0-6整数目标，评分无缺失；存在187条同酒同人的额外重复记录。新增文件与补充工作簿均通过作者MD5校验，原件在仓库外只读保存，公开支撑材料记录来源与校验，不重新分发原始记录。')
P('直接把raw beer_name与附件beer同号连接会错配。重建规则是：原始编号1-250对应公开Supplementary File S4的第1-250行。该规则不是作者另行公布的映射表，因此另以按人员、各属性的样本标准差标准化后汇总的轮廓检验；按属性名字排序分成互不重叠的25项发现组与25项验证组，分别作一对一最小距离匹配，两组和行序规则对全部250款酒一致。化合物特征、预测成绩和迁移结果均未参与匹配。')
P(f'38个不存在人员内零方差的属性重现公开连续标签，最大差{audit["nonconstant_max_error"]:.2g}。其余12项的差异只发生在部分人员从未使用该属性、导致标准差为零的位置：用奇数源编号估计每属性的固定替代常数，在偶数源编号核验，误差也低于10^-7。该常数是经验重建，并非已取得原作者完整处理程序；不把它包装成已证明的源算法。对应关系的核查使用了既有标签，只作数据身份追踪，不称为新增外部验证。')
F('source_reconciliation.png', '图1 全50项的来源复核；12项零方差替代常数为条件重建，不是新观测')
H('3.3 原始端点与人员重复的处理', 2)
P('对同酒同人的重复评分先平均，再在不同人员间等权平均。每款有3-14名不同人员；全部记录直接平均仅作为敏感性，二者全表平均绝对差约0.00322分。该端点是本批面板的观测均分，可直接落在0-6范围，不再依赖经验分位反演。')
E('s_bar_bj=(1/lT_bl) sum_t [(1/n_bt) sum_r s_btrj]', 18)

P('为了满足整数分级要求，另对每人各次真实整数评分构造七级频率，先在人内等权、再在人间等权，得到每款酒每属性的观测评分分布q。没有把人员均分先四舍五入成伪个人标签，也没有把2197条评分当2197款独立啤酒。')
E('q_bjk=(1/lT_bl) sum_t (1/n_bt) sum_r I(s_btrj=k)', 19)
P('1000次酒内人员重采样给出面板抽样敏感性，平均95%百分位区间宽约1.287分。跨日期的同人同酒首末评分平均绝对差约0.798分。这些量反映历史面板噪声，不是模型误差、一般消费者可靠度或新批次验证。人员bootstrap不纳入新批次漂移。')
H('3.4 浓度变换、异常对与综合量', 2)
P('公开Calibrated chemical dataset提供70个数值参考字段。逐列比较恒等、非正数替换为该列最小正值的一半，以及替换后log10三种候选规则；38列在全部可比对中一致，另外32列合计保留80处数值例外。DMS和4VG的正值与零替代变换完全对应；半最小正值是可重现的数据处理规则，不能当作仪器LOD，也不能把零记录认定为真实绝对零。')
P('在来源列具备calibrated标记且单位为mg/L、g/L、ug/L或v/v的条件下，选出61个报告浓度字段作次级模型；A.U.、无单位、pH、压力、色度及热量等不进入该浓度混合模型。这里核实的是来源声明和数值一致性，没有获得全部标准曲线、回收率或新批次测量。非正信号与缺失均保留标记，不能从元数据yes推导完整实验认证。')
P('对五个综合量进行了13种探索性组分/伪射候选公式的交替行留出核查，没有一个达到10^-7重现精度。该失败只否定已检验候选，不能证明真实公式不存在。生成主模型继续去掉五个综合量并重训226个非聚合输入；不补造总量或用任意插补恢复物理一致性。')

H('4 第一问：真实0-6分级、均分预测与缺失鲁棒性')
H('4.1 整数等级的概率与决策定义', 2)
P('用多目标ET回归250×(50×7)的观测频率，输出非负、和为1的七级分布。树叶训练分布的加权平均使输出保持概率单纯形；这些概率尚未证明经过概率校准。众数规则用于最大化精确等级命中，中位数规则用于最小化绝对等级损失，二者均提供250×50的整数结果，平局取较低等级。')
E('g_mode=argmax_k q_hat_k; g_med=min{k:sum_h<=k q_hat_h>=0.5}', 20)
P('模型给出0分表示预测的个人评分众数或中位数为0，不证明化合物不存在；连续面板均分则是另一种端点，不与最常见个人等级混为一谈。旧稿的经验分位等级保留为历史ASSUMPTION结果，不作为本稿的真实等级交付。')
P('评分分布用按6个累计边界归一化的RPS和七类Brier评价。精确个人命中率按该酒实测评分频率加权；“面板众数一致率”是预测等级与实测面板众数的一致率，二者不同。另报非零面板众数的表现，避免大量0分掩盖问题。')
E('RPS=(1/6nq) sum_bj sum_k=0..5 (CDF_hat_bjk-CDF_bjk)^2', 21)
ord_rows = []
for method, label in [('training_population_distribution', '训练总体分布'), ('ET_rating_distribution', 'ET众数规则'), ('ET_rating_distribution_median', 'ET中位数规则'), ('ET_mean_round_half_up', 'ET均分四舍五入')]:
    r = O(method, 'all50'); ord_rows.append([label, f'{r.expected_individual_grade_MAE:.4f}', f'{r.observed_panel_modal_grade_accuracy:.2%}', f'{r.nonzero_panel_mode_accuracy:.2%}', '-' if pd.isna(r.RPS) else f'{r.RPS:.5f}'])
T(['方法（全50项）', '预期个人等级MAE', '面板众数一致率', '非零众数一致率', 'RPS'], ord_rows, [.29, .19, .18, .18, .16])
oc, ob = O('ET_rating_distribution', 'core30'), O('training_population_distribution', 'core30')
P(f'核心30项的ET众数一致率为{oc.observed_panel_modal_grade_accuracy:.2%}，总体分布基线也有{ob.observed_panel_modal_grade_accuracy:.2%}；实测众数为0的比例为{oc.observed_panel_zero_mode_fraction:.2%}。非零众数准确率仅{oc.nonzero_panel_mode_accuracy:.2%}，所以不能把约80%一致率称为强的精细分级能力。RPS由{ob.RPS:.5f}降至{oc.RPS:.5f}，表明概率分布有增益；众数和中位数在各自损失下的取舍同时保留。')
F('ordinal_distribution.png', '图2 等级分布优于总体基线，但非零等级识别仍弱；较高总体命中受0分占比影响')
H('4.2 连续面板均分模型与比较', 2)
P(f'均分模型比较均值、岭回归、ET、GBR、固定等权融合、61浓度字段ET，以及训练内单调标定的旧尺度ET。ET为200棵树、叶节点最少3样本、特征比例0.8；GBR每目标100棵深度2的树、学习率0.04、叶节点最少5样本、特征比例0.7。岭α在1、10、100、1000中以内层三折核心MAE选择。单调标定仅使用外层训练酒的标准化标签与原始均分配对，外层测试标签不参与。标准方法依据见{C("et")}{C("gbr")}{C("sklearn")}。')
E('B_hat=argmin_B ||Yc-ZB||^2_F+alpha||B||^2_F', 3)
P(f'外层按风格三折、五次重复，共15个划分，同酒所有人员记录聚合后作为一个样本。所有方法共享角色。报告每酒五次留出预测平均后的聚合OOF误差，单折误差另存。重复折共享样本，不能作为独立样本进行简单t检验{C("cv")}。所有模型输出限制在已知0-6范围。')
E('MAE=(1/nq) sum_i sum_j |y_ij-y_hat_ij|', 4)
model_labels = [('mean', '训练均值'), ('ridge', '岭回归'), ('extra_trees', 'ET'), ('gbr', 'GBR'), ('fixed_half_blend', '固定等权融合'), ('concentration_ET', '61浓度字段ET'), ('standardized_ET_train_isotonic', '旧尺度ET+训练内单调标定')]
T(['方法', '核心30项MAE/分', '全50项MAE/分'], [[label, f'{M(model,"core30"):.5f}', f'{M(model,"all50"):.5f}'] for model, label in model_labels], [.48, .26, .26])
P(f'固定等权融合的核心MAE为{M("fixed_half_blend","core30"):.5f}，全维为{M("fixed_half_blend","all50"):.5f}；相对均值基线核心误差下降{1-M("fixed_half_blend","core30")/M("mean","core30"):.2%}。这些数字在原始0-6面板均分尺度上计算，不能和旧标准化尺度的0.34648直接比较。原始尺度ET略优于训练内单调标定；新原始记录让直接学习真实端点成为可能，并非仅凭4.csv反演丢失信息。')
F('absolute_prediction.png', '图3 同样本划分下原始0-6面板均分预测；61字段模型与231字段模型分别报告')
H('4.3 区间、重复评分与缺失压力', 2)
P('另以预先固定的ET作5次150训练/50校准/50测试的分割保形示例。逐维校准残差使用第46顺序统计量，区间与已知[0,6]相交。它针对历史协议下的观测面板均分，不能解释为某一个人的评分区间或所有50项同时覆盖保证。')
E('r_ij=|y_ij-f_j(x_i)|; q_j=calibration_order_statistic_46', 5)
E('C_j(x)=[f_j(x)-q_j,f_j(x)+q_j] intersect [0,6]', 6)
P(f'五次分割逐维平均覆盖{cover.coverage.mean():.2%}，平均宽度{cover.mean_width_0_6.mean():.4f}分。每次测试50款酒，重复测试不产生新的外部样本。使用标准分割保形原理{C("cp")}，但原数据筛选、面板构成和交换性假设限制推广。')
P('对整数分布ET在外层测试上额外模拟0%、5%、10%、20%的MCAR化合物缺失，使用同一已拟合预处理，不重估测试中位数，保留RPS、预期等级误差与等级变化率。原有真实缺失仍存在；MCAR压力不证明真实缺失随机，也不覆盖任意未知缺失机制。记录加权与人
```

```
    员等权的ET分别训练，敏感性结果另存，不混合两种端点成绩。')
106
107 H('5 第二问：原始评分下的图对照与关键因素')
108 H('5.1 描述图与训练内图回归', 2)
109 P('给定231节点秩相关图在|r|≥0.7时有109条边。预测图只由当前训练折重算，不能直
    接把含测试样本的全图用于模型。采用符号拉普拉斯和岭项，使负相关边也有明确的二
    次惩罚。')
110 E('W_ij=r_ij  I(|r_ij|>=tau);  L=diag(sum_j|W_ij|)-W', 7)
111 E('min_B ||Yc-ZB||^2_F+alpha||B||^2_F+lambda tr(B^T L B)', 8)
112 P(f'在新原始均分上重新执行三折五次完整图实验：α∈{{1,10,100,1000}}、λ
    ∈{{0,1,5,20}}、τ∈{{0.6,0.7,0.8}}，均由内层核心MAE选择。负对照为训练图节点标签
    打乱，每次10个图，汇总误差而非平均预测形成额外集成；使用已有图约束方法{C("
    graph")}。')
113 rows = []
114 for kind, label in [('none', '无图'), ('real', '真实图'), ('shuffled', '打乱
    图')]:
115     rows.append([label] + [f'{graph[(graph.graph==kind)&(graph.scope==scope)
    ]["mean"].iloc[0]:.6f}' for scope in ['core30', 'all150']])
116 T(['图条件', '核心30项折均MAE', '全50项折均MAE'], rows, [.4, .3, .3])
117 P('原始尺度真实图仍未改善无图结果，全数据内层选择α=1000、λ=0。此表为单折均
    值，与第一问五次预测平均后的OOF指标定义不同。两个评分尺度的负结果一致支持保留
    简单岭基线，但不证明所有网络方法在所有数据上都无用。')
118 H('5.2 Top-3、稳定性与风格控制', 2)
119 P('按最终描述模型标准化输入系数绝对值，逐目标选择三个化合物或元素，排除pH、理
    化混合量与综合量。λ=0时最终模型就是普通岭，不能把Top-3归功于图惩罚。200次
    bootstrap每次重建预处理和图，固定全数据选定参数；留出普通与风格内置换各20次，
    并保留模块联合置换和删除模块重拟合诊断。')
120 E('pi_jk=bootstrap_Top3_selection_count/bootstrap_repeats', 9)
121 E('Delta_jk=MAE_k(permuted_holdout)-MAE_k(original_holdout)', 10)
122 P(f'描述性支持条件是频率≥0.5、至少5个训练折入选、所选折平均普通和风格内置换贡
    献均为正；新原始端点下有{factors_supported}/150条满足。独立普通岭实现得到相同
    150条Top-3，但换一组重采样与置换种子后有37条满足条件，说明二分阈值和蒙特卡洛
    波动会影响支持数。它们不是独立显著性检验或因果结论，完整连续频率、贡献和弱项
    全部保留。')
123 F('raw_factor_support.png', '图4 原始评分Top-3的条件bootstrap稳定性与风格内留
    出贡献；弱项不删除')
124 H('5.3 化学合理性与单位核验', 2)
125 P(f'X4vg与X4VG的关联有4VG陈化及丁香相关香气研究支持{C("4vg")}；iron与metallic
    有口腔氧化实验支持{C("iron")}，但铁价态和金属溶液基质不能直接替代总铁在啤酒中
    的阈值。coriander与linalool可参照芫荽籽添加低醇啤酒研究{C("coriander")}；原始
    评分模型中linalool的bootstrap频率未达0.5，文献合理性不覆盖统计稳定性不足。')
126 P('乳酸/醋酸与酸度、乙醇与酒精感等可核查测量相容性。其余未核实候选只记为共变
    或预测关联，不为150条名单补造气味机理。每条保留单位、来源、系数、频率、跨折入
    选、风格内贡献与因果未识别状态。附录给出全部50×3结果。')
127 P(f'缺陷参考阈值采用DMS=19 ug/L、双乙酰=0.09 mg/L、4VG=0.2 mg/L（附件中位
    值）；nonenal补入原始研究中的约0.1 ug/L参考值{C("threshold")}{C("nonenal")}，
    换算为0.0001 mg/L。全部四项均完成0.5、1、2倍敏感性。该外部值未在本批啤酒重新
    校准；参考OAV较高并不等于本批面板必然感知缺陷。')
128 P('来源记录中的nonenal以mg/L标注，参考OAV中位数约120.4，部分测量尾部很大；这
    与个体阈值、基质、测量及数据处理口径共同相关。数值差异和单位问题尚未完全解
    决，因此这里只作条件风险筛查。Stale hops单独按感官终点处理，不给它伪造一个通
    用化合物阈值。')
129 F('reference_oav.png', '图5 仅对正浓度记录绘制参考OAV；零与缺失不当作已认证的
    缺陷不存在')
130
131 H('6 第三问：条件生成、原始评分画像与迁移边界')
132 H('6.1 风格生成与独立评价', 2)
133 P('保留已执行的五次分层G100/J100/T50角色隔离，比较风格内重采样、局部凸组合、
    十维条件PCA高斯及受约束局部凸组合。每种方法、每风格、每划分生成40个样本；全部
    使用226个非综合字段。字段含理化描述量，不能当作226种可任意称量的纯化合物。')
134 E('h|c~N(mu_c,w_c Sigma_c+(1-w_c)Sigma_pool)', 11)
135 E('x_new=u x_a+(1-u)x_b', 12)
136 P(f'统计合理性使用共同训练表示与带宽的MMD{C("mmd")}、KS、复制比例、范围和独立
    J类别判断。候选先锁定再拟合J，T仅作诊断。每次角色不重叠，但同一批250款酒已用
    于研究开发，因此称内部角色隔离，不称新增外部验证。稀有风格留出通常很少，不能
    把小距离或不显著当作分布一致。')
137 T([['风格', '方法', '平均MMD平方', 'J判断为目标风格'], [[r['style'], r['method
    '], f'{r["MMD2_fixed_training_bandwidth"]:.4f}', f'{r["independent_evaluator_
    target_prediction_fraction"]:.1%}'] for _, r in gen.iterrows()], [.17, .37,
    .2, .26])
138 P('受约束生成未精确复制训练点且已检查的训练范围违反为零，但MMD未全面优于简单
    重采样。新原始评分到位后，在原G和J各100款酒上重新拟合原始均分与七级分布ET，给
    冻结的首划分480个生成谱补齐50项0-6均分及整数预测画像。画像来自模型，不是新制
    备酒的实测感官标签。')
139 H('6.2 冻结统计候选的原始尺度复核', 2)
140 P('原迁移先对范围、目标支持、四项字段不增加、两个G感官代理的stale_hops不增加
    以及Tripel概率增益作筛选，再按改变量、目标距离和已核实阈值代理作Pareto选择。
    保留原半径0.25、0.5、1、2及所有失败，源字段缺失依赖插补时单独标明。')
141 E('x_candidate,d<=x_source,d; d in {DMS,diacetyl,4VG,nonenal}', 13)
142 E('f_stale(candidate)<=f_stale(source)', 14)
143 E('||Z_candidate-Z_source||_2/sqrt(p)<=r', 17)
144 P('新增复核不改变此前候选或排序：在既定G/J角色内分别重训0-6均分ET，检查同一候
    选的原始尺度stale_hops是否不增加。新终点信息没有用于重新选择J支持的候选。原分
    类器的独立风格判断保持原定义，另报新增G和J感官条件。')
145 T([['半径1的同一批记录', '全角色55条', 'T来源10条'], [['原G搜索可行', str(int(
    radius1.original_search_feasible.sum())), str(int(held.original_search_
    feasible.sum()))], ['原J代理支持', str(int(radius1.original_proxy_acceptance.
    sum())), str(int(held.original_proxy_acceptance.sum()))], ['再同时满足原始尺
    度G/J陈旧酒花约束', str(int(radius1.original_and_raw_proxy_support.sum())),
    str(int(held.original_and_raw_proxy_support.sum()))]], [.62, .19, .19])
146 P(f'T来源10条记录涉及7款不同酒，新增条件后仅{int(held.original_and_raw_proxy_
    support.sum())}条支持。它既不是10款新独立酒，也不是真实品评通过率；原先2条支
    持不能继续作为新版结论。原始评分到位后，独立代理分歧增加本身也是重要负结果。
    ')
147 F('raw_transfer_recheck.png', '图6 同一冻结候选加入原始评分尺度检查后的支持
    数；两幅图分母分别为55和10')
148 H('6.3 报告浓度域的混合与线性规划', 2)
149 P('另建61报告浓度字段的次级分析，排除未标定A.U.及不能线性混合的pH/压力等描述
    量。以Pils来源和G中的Tripel为锚点，在浓度域按份额混合，绝不把两个log浓度作物
    理线性相加。四项缺陷来源缺失时不以中位数证明硬上界；零记录仍受未知检测限限
    制。')
150 E('c_mix=(1-sum_a w_a)c_source+C_anchors w', 23)
151 P('先检验各单锚点0.025-0.75份额，再在失败后增加探索性的多锚点线性规划，最大化
    总Tripel份额，要求四项报告浓度均不增加、份额非负且总和不超过0.75。多锚点扩展
    是看过先导失败后的探索分析，未伪称预注册。')
152 E('maximize sum(w);  (C_A-c_source)_D w<=0;  sum(w)<=0.75;  w>=0', 22)
153 P(f'55条来源/划分记录的线性规划最大非平凡目标份额均为0，按同样规则的搜索与J复
    核也均无通过。该数值结论仅覆盖本组锚点、报告浓度和约束，不能证明所有酿造工艺
    都不可能实现迁移。它说明严格的来源保持型混配无法解决当前任务，后续需有独立去
    除/转化工艺、合适原料及实际测量，不能把统计剪裁浓度当作可制备配方。')
154 P('原统计迁移仍提供条件候选，61字段混合用于检查其现实解释边界。两套变量、模型
    和可行域不同，结果不混算。所有源保留失败、未知检测限和实际品评未验证的状态继
    续输出。')
155
156 H('7 实物验证方案、样本量与复现')
157 P('真实品评未执行。已完成实际可用的方案、空白采集表、样本量敏感性和分析程序：
    先由实验室把候选转化为有批号的可制备小样，测定四项化合物并提供LOD/LOQ、重复、
    单位和回收；再对3个独立批次组织随机编码、平衡顺序的配对盲评。正式随机映射由管
    理员私有保存，不能把公开历史编号当现有实物批次。')
158 P('示例计划为30名训练人员、3次批次配对。按人平均批次差，不能当90名独立人员。
    五个缺陷终点预设单侧α=0.01、非劣界0.5分，全部通过后再检验Tripel符合度是否提
    高；0.5分非劣不等于数学上严格差值≤0。该符合度是新定义终点，需要参考酒训练，不
    能冒充原评分表已有变量。')
159 E('U_j=mean(d_j)+t_(1-alpha,n-1)sd(d_j)/sqrt(n);  U_j<Delta', 24)
160 P('非中心t规划中，假设人员平均差标准差0.8分、真实差0、界0.5、α=0.01，单个终点
    80%功效至少需29人，30人约82.5%；标准差1.0则需43人。若界0.25、标准差0.8则需106
    人。这是单终点假设功效，不保证五项共同通过80%。方案不能替代真实参与者、制备或
    测量，空白表不含虚构观测。')
161 P(f'当前{tests}项测试通过，核对重复人员权重、匿名对应、非正变换、训练标定、浓
    度约束、未配对/空白品评拒绝及历史实现性质。结果另核查12500个均分和每规则12500
    个整数等级、概率和、角色、候选锁与原件。源码、配置、版本、运行环境及哈希全部
    保留。测试通过不等于科学假设已全部成立。')
162 P('复现顺序：按来源MD5取得作者v4文件；运行run_stage4.py all；运行scripts/
    stage4_graph_comparison.py；运行pytest与scripts/verify_stage4.py；然后生成图
    表和论文。原始公开文件通过scripts/fetch_stage4_sources.py下载到仓库外；官方附
    件仍由参赛者自行取得，不上传签名资料或原始赛题数据。精确命令和输出索引见附
    录。')
163
164 H('8 结论、适用范围与剩余问题')
165 P(f'通过新版逐人原始评分与来源对应核查，本研究已完成历史面板真实0-6端点的构
    造、整数分级和连续预测。固定等权融合的核心MAE为{M("fixed_half_blend","core30
    "):.4f}分；整数分布模型相对总体基线的概率评分有改进，但非零等级识别仍弱。图约
    束在原始尺度也未见额外收益，完整Top-3及不稳定项如实给出。')
166 P(f'统计生成提供风格条件谱和原始评分预测画像。更严格检查后T来源10条重复迁移记
    录只有{int(held.original_and_raw_proxy_support.sum())}条获原有及新增代理共同
    支持；报告浓度的锚点混配未找到满足四项不增加的非平凡解。模型内候选、化学计量
    可行和真实品评是不同层面的结论，论文不把失败掩盖成成功。')
167 P('仍需补充的是：来源数据中80处化学差异、未公开的检测限/校准和五个综合量精确
    公式，多数因素的目标特异性实验证据、稀有风格的小样本外推，以及当前新批次制备
    与真实品评。旧稿“没有取得原始评分”的限制已被本次公开数据核查实质推进；其余
    信息未因文字修订被宣称完成。队员仍需独立理解、复跑和记录实际采纳与修改。')
168 P(f'本稿代码、分析和文字由OpenAI Codex辅助{C("ai")}；精确公开模型版本与队员真
    实贡献须据实际记录补充，未虚构已完成人工推导或独立验证。')
169
170 H('参考文献')
171 for i, key in enumerate(order, 1): P(f'[{i}] {references[key]}')
172 H('AI工具使用与研究贡献声明')
173 P('AI参与来源检索、审计、实现、实验、图表和写作。新增逐人原始评分核查改变了研
    究证据范围，前稿结论均按版本保留。本稿未采用新实物品评，不代填队员签名、学校
    或队号；关键交互、实际执行与未完成人工工作随附更新说明。')
174 H('附录A 全部50项Top-3、结果索引与运行命令')
175 P('以下为原始面板均分的最终描述模型Top-3，括号为固定全数据参数条件下的200次
    bootstrap入选频率。λ=0，实际为普通岭回归；不是因果概率、独立检验p值或新化学试
    验。')
176 rows = []
177 for target in pd.read_csv(OUT / 'audit/observed_panel_means_0_6.csv', nrows
    =0).columns[1:]:
        a = top[top.target == target].sort_values('rank')
        rows.append([target, '; '.join(f'{r.field} ({r.bootstrap_frequency_
    conditional_on_full_selected_params:.2f})' for _, r in a.iterrows())])
178 T(['目标', 'Top-3字段（条件入选频率）'], rows, [.25, .75])
179 T(['目录', '保存内容'], [['audit', '对应关系、0-6端点、人员重复、浓度例外、
    OAV与候选公式'], ['prediction', '15折均分预测、标定对照、区间、原始尺度普通岭
    敏感性'], ['ordinal', '真实七级分布、众数/中位数/均分取整、缺失压力与生成谱等
    级画像'], ['raw_graph/q2', '原始端点无图/真图/打乱图、跨拟合贡献、完整Top
    -3'], ['transfer', '冻结候选复查、G/J原始均分画像、浓度混合、LP与候选锁'], ['
    protocol', '空白评分表与假设样本量；没有实际品评结果']], [.23, .77])
180 P('命令：python scripts/fetch_stage4_sources.py；python run_stage4.py all --
    data-dir <官方附件目录>；python scripts/stage4_graph_comparison.py --data-dir
     <官方附件目录>；python -m pytest -q；python scripts/verify_stage4.py；python
     scripts/stage4_figures.py；python scripts/prepare_absolute_paper.py；使用
    scripts/build_revised_pdf.py按paper_content.json生成本稿。首次重建旧生成谱时
    须先运行stage3；脚本默认来源目录可由参数指定。')
181 P('软件：Python 3.12及NumPy、pandas、SciPy、scikit-learn、NetworkX、
    Matplotlib、openpyxl；精确版本在requirements-lock.txt和各运行manifest中。PDF
    排版另用ReportLab、pypdf、pdfplumber和Pillow；无需声称外部依赖库源码属于本
```

```
队。')
blocks.append({'type': 'pagebreak'})
H('附录B 本项目完整源程序与执行配置')
P('以下为本项目计算、检查、图表、排版、打包、测试程序和配置。长行自动折行，行号对应原文件；第三方依赖源码不重复印入。历史程序保留以复核各阶段，最新入口为run_stage4.py。可执行原文件及逐文件哈希随支撑包提供。')
paths = [p.relative_to(ROOT).as_posix() for folder in ['src', 'scripts', 'tests'] for p in sorted((ROOT / folder).glob('*.py'))] + [p.name for p in sorted(ROOT.glob('run*.py'))] + [p.name for p in sorted(ROOT.glob('config*.json'))]
blocks.append({'type': 'source_files', 'paths': paths})
for number, b in enumerate([b for b in blocks if b['type'] == 'equation'], 1): b['number'] = number
title = '原始评分重建与独立约束检验下的啤酒感官预测及风格迁移'
abstract = (f'针对啤酒0-6感官分级、关键因素和风格生成任务，补查作者新版2197条逐人评分并重建250款酒的匿名对应，以两组分离属性核验，明确重复人员权重。由实际七级分布输出整数等级，同时预测原始面板均分。三折五次留出中，固定等权融合核心30项MAE为{M("fixed_half_blend","core30"):.4f}分，全50项为{M("fixed_half_blend","all50"):.4f}分；等级分布核心RPS为{oc.RPS:.4f}，优于总体基线，但大量0分掩盖了较弱的非零等级识别。原始端点的真实图仍未优于无图，全数据选择图惩罚为0；150条因素中{factors_supported}条满足描述性支持条件。核查70个化学参考字段，38列可比对完全对应，保留80处例外并完成四项阈值敏感性。冻结生成谱补齐原始尺度画像；严格复查后，真实留出来源的10条重复迁移记录仅{int(held.original_and_raw_proxy_support.sum())}条获全部代理支持，浓度锚点混配无非平凡解。研究保留失败和完整复现材料，实物制备与真实品评另给执行方案，未冒充已完成。')
content = {'title': title, 'abstract': abstract, 'keywords': '原始感官评分；七级分布；图对照；化学尺度；条件生成；迁移诊断', 'equation_dir': 'paper/revised_absolute_20261025/equations', 'blocks': blocks}
(PAPER / 'paper_content.json').write_text(json.dumps(content, ensure_ascii=False, indent=2), encoding='utf-8')
md = ['# ' + title, '', '## 摘要', abstract, '', '关键词：' + content['keywords'], '']
for b in blocks:
    if b['type'] == 'heading': md += ['#' * (b['level'] + 1) + ' ' + b['text'], '']
    elif b['type'] == 'paragraph': md += [b['text'], '']
    elif b['type'] == 'equation': md += [f'公式（{b["number"]}）：{b["text"]}', '']
    elif b['type'] == 'table': md += ['| ' + ' | '.join(b['headers']) + ' |', '|' + '---|' * len(b['headers'])] + ['| ' + ' | '.join(map(str, row)) + ' |' for row in b['rows']] + ['']
    elif b['type'] == 'figure': md += [f'![{b["caption"]}](../../{b["path"]})', '']
    elif b['type'] == 'source_files': md += ['PDF附录印入完整源程序，原文件列表：' + '、'.join(b['paths']), '']
(PAPER / '论文完善稿.md').write_text('\n'.join(md), encoding='utf-8')

ai = copy.deepcopy(json.loads((ROOT / 'paper/revised_20261025/ai_usage_content.json').read_text(encoding='utf-8')))
ai['title'] = 'AI工具使用详情说明（原始评分完善阶段）'
ai['abstract'] = '本说明保留此前重要交互及辅助范围，新增2026-10-25原始评分对应核查、真实0-6分级和均分实验、原始端点图对照、浓度和迁移复核。所有计算与负结果按实际执行记录；真实实物试验及队员独立贡献尚未发生的部分继续保留未完成状态。'
ai['blocks'].insert(2, {'type': 'paragraph', 'text': '第2至5节按原样保留上一阶段的重要交互与当时完成状态；其中“40项检查”和“尚未取得原始评分”仅指前一阶段，不代表本轮结论。本轮新增执行与更新状态见第6节。'})
ai['blocks'].append({'type': 'heading', 'level': 1, 'text': '6 新版原始评分与本轮实际改进'})
for text in [
    '用户：目前仍缺原始绝对评分锚点、完整化学标定和真实品评验证这些东西你是做不了了吗还是怎么样。',
    '关键回复/执行摘要：发现原作者v4新增逐人原始评分，承认前轮未跟踪最新版；首次直接按同号核查失败，未擅自把不对应记录当真值。',
    '用户：那你把你现在能做的都做了，并对论文进行修改完善。用户补充：继续。',
    '实际完成：来源行序重建与两组属性交叉核验；38属性重现、12个零方差常数条件重建；等人员均分和七级分布；三折五次均分/整数模型对照、五次区间、缺失压力、原始端点图对照；61报告浓度字段、四阈值敏感性、13种总量公式候选失败记录；冻结迁移与原始尺度复查及多锚点线性规划；未来试验方案、空表和功效分析。',
    f'已有{tests}项检查通过。关键负结果：非零等级识别弱；原始尺度图无增益；严格复查缩减迁移支持；报告浓度锚点混配无非平凡解。此前“原始评分未取得”已由新数据核查推进，旧稿按版本保留，不能照抄其未更新状态。',
    '未完成：新的实物啤酒、仪器LOD/LOQ和回收率、全部总量公式、真实新批次品评、队员独立复跑和最终采纳。没有生成虚构人员评分，没有将原作者已有试验写成本队实测。',
    '以上是重要提示词与关键回复/执行摘要，不是完整聊天逐字稿。正式使用仍需实际核验可公开模型版本、队员真实贡献和重要交互，不代填签名或承诺书。']:
    ai['blocks'].append({'type': 'paragraph', 'text': text})
(PAPER / 'ai_usage_content.json').write_text(json.dumps(ai, ensure_ascii=False, indent=2), encoding='utf-8')
print(json.dumps({'blocks': len(blocks), 'equations': sum(b['type']=='equation' for b in blocks), 'source_files': len(paths), 'references': len(order), 'tests': tests}, ensure_ascii=False))
```

scripts/prepare_paper.py

```
"""AI-assisted manuscript preparation: OpenAI Codex / GPT-6, 2026-10-24.
Every reported result is read from saved experiment tables.
"""
from pathlib import Path
import json
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
PAPER = ROOT / "paper"
PAPER.mkdir(exist_ok=True)
status = json.loads((OUT / "status.json").read_text(encoding="utf-8"))
metrics = pd.read_csv(OUT / "q1/metrics_summary.csv").set_index("model")
top = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
gen = pd.read_csv(OUT / "q3/generator_comparison_mean.csv")
transfer = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
facts = json.loads((OUT / "audit/verified_facts.json").read_text(encoding="utf-8"))
coverage = pd.read_csv(OUT / "q1/conformal_coverage.csv")
blocks = []
def H(text, level=1): blocks.append({"type":"heading", "level":level, "text":text})
def P(text): blocks.append({"type":"paragraph", "text":text})
def E(text, number): blocks.append({"type":"equation", "text":text, "number":number})
def T(headers, rows, widths=None): blocks.append({"type":"table", "headers":headers, "rows":rows, "widths":widths})
def F(name, caption, height=6): blocks.append({"type":"figure", "path":"outputs/stage2/figures/"+name, "caption":caption, "height_cm":height})

baseline = float(metrics.loc["mean", "mae"])
gbr = float(metrics.loc["gbr_fixed", "mae"])
reduction = (baseline-gbr)/baseline
abstract = f"针对啤酒化合物到感官画像的预测、关键因素分析和风格生成问题，构建数据审计、多输出预测、网络关联解释及约束迁移的统一研究流程。依据实际附件，以 beer 为唯一键关联 250 个样本、231 个数值字段和 50 个连续感官目标，逐列核查对数尺度与缺失处理。原始研究采用按品评人员标准化的评分，缺少原始参数时绝对 0—6 分不可唯一恢复，故将连续预测与分级假设分开验证。在训练折内预处理和调参的三折两次重复验证中，固定配置 GBR 的宏平均 MAE 为 {gbr:.4f}，相对均值参照下降 {reduction:.2%}；独立训练、校准和测试分割下，名义 90% 的逐维边际区间获得 {status['conformal_empirical_average_coverage']:.2%} 的平均测试覆盖率。三类分级假设在 {status['grade_sensitivity_disagreement']:.2%} 的预测位置存在分歧。第二问以相关网络和图约束岭回归产生 Top-3 候选，结合 bootstrap 与留出置换贡献，仅 {status['comparatively_supported_top3_associations']}/150 条通过当前描述性证据条件。第三问比较重采样、局部凸组合和 PCA 高斯生成；局部凸组合保留非复制样本优势，但分布距离未全面优于重采样。扩展迁移搜索为 11 个源样本找到模型内可行候选，其中 2 个含缺失缺陷源值。结果支持相对感官预测与受约束的统计设计，尚不支持绝对评分恢复、化学因果或真实酿造效果的结论。"

H("1 问题重述与研究范围")
P("啤酒风味由原料、发酵过程和多种化合物相互作用共同形成。原始研究已将实测化学资料与训练品评联系起来，说明此类数据具有建模价值[1]。本题进一步要求处理缺失、分级评分、稳定因素识别和跨风格生成，因此研究重点不仅是拟合精度，还包括目标定义是否清楚、解释是否稳健，以及生成谱满足什么层次的约束。")
P("第一问以化合物和理化测量为输入，估计品评表各感官维度；第二问建立化合物关系网络并对每维给出三个候选因素；第三问按 tasting_category_fine 生成风格条件样本，并将 Pils/Lager 向 Tripel 调整，要求指定缺陷项不恶化。本文对实际附件的 50 个连续目标完整输出，Aroma 与 Flavour 维度单独标识；最终提交维度应依据官方明确口径核对。")
P("本文研究范围为对给定附件的统计建模与可复现验证。原始 0—6 分锚点、部分浓度尺度、聚合量公式和真实感官试验均存在信息缺口。对这些缺口采用明确假设、敏感性和失败记录处理，保留可核查结果，不以预测代理替代实测。")

H("2 模型假设与符号说明")
H("2.1 基本假设", 2)
P("假设一，beer 能稳定标识同一款啤酒，不同文件按该字段正确对应。该假设经唯一性、集合和风格一致性检查支持；受损的 beer_id 不用于合并。假设二，在相同数据获取机制下，现有样本可用于研究新样本的相对感官预测；此假设不保证未见风格或不同酿造工艺下的外推。")
P("假设三，训练分布的分位截尾与中位数插补可作为噪声、极端值与缺失的研究基线。它们不证明被处理值是错误观测，后续需要针对检测限与非随机缺失改进。假设四，相关网络可作为参数正则化信息，但不具有先验因果含义。假设五，生成与迁移的经验范围和代理模型只定义统计可行性，不构成物理、化学或生产可行性的充分条件。")
H("2.2 主要符号", 2)
T(["符号", "含义"], [["n、p、q", "样本数 250、输入字段数 231、感官目标数 50"], ["X、Y", "附件尺度输入矩阵与连续感官目标矩阵"], ["Z", "仅由训练数据拟合预处理得到的标准化输入"], ["B、α、λ", "回归系数、岭惩罚和网络惩罚强度"], ["W、L", "截断有符号相关权与对应拉普拉斯矩阵"], ["f、g", "连续感官预测器与风格诊断分类器"], ["c、h", "目标风格条件与低维潜变量"], ["D、D̃", "缺陷化学字段集合与学习得到的感官缺陷代理"]], [.23,.77])

H("3 数据审计与预处理")
H("3.1 样本与字段核查", 2)
P("实际附件中 1.csv 有 250 行、234 列，前 3 列为元信息，其余 231 列包括化合物、理化和综合字段；4.csv 含 50 个连续感官目标。化合物条码在感官文件中存在精度损失，直接用条码合并可能产生多对多错配。程序要求 beer 非空、唯一、两表集合相同，并在对齐后检查风格标签一致。")
T(["资料", "实际结构", "研究用途"], [["1.csv", "250x234，含 3 列元信息", "化合物及理化输入"], ["2.csv", "去行名后 231x231", "描述性相关网络"], ["3.xlsx", "12 个 beer 页及 summary", "感官定义与空模板审计"], ["4.csv", "250x53，含 50 个目标", "连续感官监督信息"], ["6.csv", "去行名后 44x50", "跨域关联参考，不作新样本"], ["8.xlsx", "阈值、名称、校准等工作表", "尺度和术语证据"]], [.16,.38,.46])
P("完整原件以 SHA-256 绑定版本。字典、字段尺度、缺失统计和样本键检查随支撑文件保存。题面与文件的数量差异按实际证据记录，不补造缺失行列。8.xlsx 中已有 SHAP 与重要度属于既有研究结果，未作为本文重新训练得到的结论，也不用于第一问的
```

外层变量筛选。")

F("style_counts.png", "图1 风格样本数分布 显示小样本与类别不平衡", 8)

H("3.2 量纲与缺失处理", 2)

P("对可比的原始正值配对，逐字段比较恒等关系与 log10 关系。检查支持 37 个字段在这些配对上符合 log10，包括题面列为可能保留原量纲的部分理化指标；另有字段只在多数配对一致，或缺少可比原值。正值配对一致性并不能识别零替代值与检测限。故保留附件尺度建模，不对所有字段统一逆变换。")

P(f"给定相关矩阵与重算 Spearman 的平均绝对差为 {facts['spearman_matrix_mean_absolute_difference']:.6g}，与 Pearson 为 {facts['pearson_matrix_mean_absolute_difference']:.6g}，更支持采用秩相关解释。矩阵的完整结构一致性也用于核对字段别名次序。自然缺失由训练中位数填补，训练 1% 和 99% 分位定义截尾范围，再作训练均值与标准差标准化。所有操作置于 Pipeline 中[2]。")

E("z_ij = [clip(x_ij; a_j, b_j) − m_j] / s_j", 1)

P("式（1）中的边界、插补值、均值和标准差均只在当前训练数据估计。验证与测试样本仅执行变换，不参与参数学习。对极端值的处理属于可检验的建模假设，而非原始观测纠错。")

H("4 第一问 感官预测与分级不确定性")

H("4.1 评分尺度的可识别性", 2)

P("原始研究明确说明按品评人员分别中心化和标准化，并非仅对汇总后的每个属性作一次统一线性变换[1]。设品评人员 t 的原始评分为 s_it，标准化后为 z_it，若其均值 μ_t 与标准差 σ_t 未知，则不能从 z_it 唯一得到 s_it；不同品评人员的出席与汇总权重进一步增加信息需求。")

E("z_it = (s_it − μ_t) / σ_t, s_it = μ_t + σ_t z_it", 2)

P("对于缺少锚点的数据，多组满足原始评分范围的 μ_t、σ_t 都可能产生相同标准化值。空模板公式返回的零也不等于真实打分。因此本文把可验证的连续预测和有假设的相对分级分开，未把简单拉伸、裁剪或经验分位声称为原始评分恢复。")

H("4.2 多输出预测模型与求解", 2)

P("设第 k 个感官目标的预测为 f_k(x)。比较训练均值、岭回归、PLS、ExtraTrees[3]、固定配置梯度提升回归[4]以及固定比例的岭回归与 ExtraTrees 融合。岭回归通过 L2 惩罚控制高维共线，PLS 学习与目标相关的低维方向，树模型捕捉非线性。输入中不包含编号、条码、风格标签或样本的感官评论信息。")

E("B̂ = argmin_B {‖Yc − ZB‖²_F + α‖B‖²_F}", 3)

P("岭回归 α 在 {1,10,100,1000} 中由内层三折选择；PLS 成分数在 {3,8} 中选择。ExtraTrees 使用 200 棵树、最小叶样本 3、特征比例 0.8。GBR 对每个目标分别拟合 100 棵深度为 2 的树，学习率 0.04，最小叶样本 5，特征比例 0.7。GBR 参数为第二阶段固定探索配置，未根据本阶段外层目标逐折调整。")

H("4.3 验证设计与性能", 2)

P("采用按风格分层的三折、两次重复外层验证。每个样本每次重复仅被留出一次，预处理和调参在外层训练数据内完成。重复留出预测取均值后计算逐维指标，并对 50 维等权宏平均。最少风格有 3 个样本，选用三折避免强行采用超过该样本数的分层折数。第二阶段是在第一阶段观察之后的探索改进，不能视为未接触过数据的外部验证。")

E("MAE = (1/nq) Σ_i Σ_k |y_ik − ŷ_ik|", 4)

T(["方法", "宏平均 MAE", "宏平均 RMSE", "宏平均 R²"], [[name, f"{metrics.loc[key,'mae']:.4f}", f"{metrics.loc[key,'rmse']:.4f}", f"{metrics.loc[key,'r2']:.4f}"] for key,name in [("mean","均值参照"),("ridge","岭回归"),("pls","PLS"),("extra_trees","ExtraTrees"),("gbr_fixed","固定 GBR"),("fixed_ridge_tree_blend","固定比例融合")]], [.28,.24,.24,.24])

P(f"GBR 的 MAE 为 {gbr:.4f}，相对均值参照下降 {reduction:.2%}。它与 ExtraTrees 的差距较小，不能仅据单个平均值断言普遍优越。宏平均 R² 约 {metrics.loc['gbr_fixed','r2']:.3f}，说明仍有较大未解释变异；逐维性能可能差别明显。上述单位均为附件的连续目标尺度，不能改写成 0—6 分的平均误差。")

F("model_comparison.png", "图2 六类方法在相同重复留出划分下的连续目标误差", 6)

H("4.4 分级敏感性与预测区间", 2)

P("在训练目标上分别构造七等频经验分位、正态分布累计概率和以 5%—95% 分位为端点的线性映射三类等级规则。它们均输出 0 至 6，但语义不同；重复分位边界也会影响离散分级。对每个外层留出预测比较三者，完整保存差异与跨规则范围。")

P(f"三种规则在 {status['grade_sensitivity_disagreement']:.2%} 的预测位置给出不同等级，显示绝对标定对假设高度敏感。跨规则的最小值和最大值仅表示假设范围，不是统计置信区间。若获得参考酒原始评分或逐品评人员转换参数，应重新标定和验证，再向原始 0—6 分作实质性解释。")

P("另用 150 个训练、50 个校准、50 个测试样本展示分割保形预测[5]。用训练模型在校准集的绝对残差定义逐维分位，再为测试预测添加对称区间。名义水平为 90%，目标是单个感官维度的边际覆盖，并非所有 50 维同时覆盖。")

E("r_ik = |y_ik − f_k(x_i)|; r*_k = 第 ⌈(n+1)(1−α)⌉ 个校准残差", 5)

E("C_k(x) = [f_k(x) − r*_k, f_k(x) + r*_k]", 6)

P(f"本次平均测试覆盖率为 {status['conformal_empirical_average_coverage']:.2%}，平均区间宽度为 {coverage.interval_width.mean():.4f}。该结果只对应一次分割和当前连续标签；交换性假设、样本筛选与原作者预处理都会限制其外推。不能把边际覆盖率解释为每种风格或每款啤酒都达到同一保障，更不能据此推导原始等级的覆盖率。")

F("calibration_uncertainty.png", "图3 分级规则分歧与逐维连续预测区间的测试覆盖分布", 6)

H("5 第二问 化合物网络与解释证据")

H("5.1 共表达图及图约束回归", 2)

P("以 231 个数值字段为节点，给定相关矩阵中 |r|≥0.7 时建立边，同时保存符号与绝对权重。当前有 109 条边；0.6、0.7、0.8、0.9 阈值的网络变化另存。模块包含孤立节点，不能直接当作生化通路。对每个目标最终只从化合物或元素字段中选择候选，pH、色度、混合和聚合指标不冒充单一化合物。")

E("W_ij = r_ij I(|r_ij|≥τ), L = diag(Σ_j W_ij) − W", 7)

E("B̂ = argmin_B {‖Yc−ZB‖²_F + α‖B‖²_F + λ tr(BᵀLB)}", 8)

P("符号拉普拉斯的二次型为相关边上系数差异或相反关系的加权惩罚，矩阵为半正定。固定 α=20、λ=5 后，求解 (ZᵀZ+αI+λL)B=ZᵀYc。该模型提供一种利用关系结构的正则化基线，尚未证明固定参数最优。每个目标按标准化特征的系数绝对值取前三项，方向和相关性均保留。")

H("5.2 重采样稳定性与留出置换贡献", 2)

P("采用 100 次样本有放回重采样，每次重新估计截尾、插补、标准化和秩相关网络，重新拟合并计数 Top-3。另比较图约束回归、无图岭回归和边际 Spearman 的名单 Jaccard。相关因素可能相互替代，所以单个因素频率低与模块关联存在并不矛盾。")

E("π_jk = bootstrap 中化合物 j 进入目标 k 的 Top-3 次数 / 重采样次数", 9)

P("第二阶段对每个外层训练拟合的岭回归，在其留出样本上单独置换某一特征，观察逐维 MAE 增量，重复 5 次并汇总六个外层划分。对线性预测利用固定预处理后的变量差与系数精确计算，避免反复拟合同一模型。置换贡献取决于模型与相关结构，不是化合物的内在因果效应[6]。")

E("Δ_jk = MAE_k(置换第 j 个变量后的留出预测) − MAE_k(原留出预测)", 10)

P("定义探索性证据条件：入选频率至少 0.5，平均留出 MAE 增量大于零，且至少三分之二外层划分增量为正。阈值为本研究的描述性判据，不是显著性检验或官方评分标准。由于候选名单包含全数据探索信息，筛选后的支持数仍存在选择适应性，不能作未经选择的独立假设检验。")

P(f"150 条候选中，{status['comparatively_supported_top3_associations']} 条同时满足当前条件。第一阶段有 137 条入选频率低于 0.5，说明多数 Top-3 还不适合写成确定因素。完整 150 条及所有变量的贡献保存在支撑表中，弱证据项和反直觉项不删除。")

selected = top[top.evidence_status == "comparatively_supported_association"]

T(["感官目标", "候选字段", "入选频率", "留出 MAE 增量"], [[r.target,r.field,f"{r.bootstrap_top3_frequency:.2f}",f"{r.heldout_permutation_MAE_increase:.5f}"] for r in selected.itertuples()], [.23,.43,.16,.18])

F("factor_evidence.png", "图4 Top-3 候选的重采样频率与留出置换贡献 通过描述条件仍不表示因果", 6)

H("5.3 化学合理性及反例", 2)

P("4VG 感官目标与 X4VG 的联系具有较强的统计一致性。原始化学研究指出 4VG 与特种啤酒的丁香气味相关，并研究了其陈化降解[7]，可作为该关联的外部合理性依据。金属味与 iron、烘烤麦芽与 methylpyrazine 等候选也可结合附件的名称、来源和缺陷描述继续论证，但本文未开展加标、盲评或反应机理实验。")

P("表中也存在不宜直接解释为气味来源的关联，例如苦味与 iron、乳酸相关描述与某酯类字段。它们可能反映风格共变、共线替代或模型不足。源于同一附件的元数据只能提供术语与来源线索，不能独立验证每个目标的机制。对这些结果应保留待证状态，不能人为换成期望化合物来美化模型。")

H("6 第三问 风格生成与约束迁移")

H("6.1 小样本条件生成与对照", 2)

P("按风格分层留出 20% 样本，训练预处理和生成模型仅使用训练部分。比较三类方法：风格内 bootstrap、同风格近邻之间的局部凸组合，以及十维 PCA 中的条件高斯采样。PCA 模型以风格均值为中心，协方差按 n_c/(n_c+10) 将风格内估计与汇聚估计混合。")

E("h|c ~ N(μ_c, w_c Σ_c + (1−w_c)Σ_pool), w_c = n_c/(n_c+10)", 11)

P("局部凸组合在相同风格中选择一个训练样本和其近邻，以 0.1—0.9 的随机权重插值。该方法是一种条件经验生成模型，生成的新点不等同于化合物可直接混配的物理过程，尤其在对数尺度上只代表统计插值。")

E("x_new = u x_a + (1−u)x_b, u ~ Uniform(0.1, 0.9), a、b 属于同一风格", 12)

H("6.2 生成验证与结果", 2)

P("用五种随机划分重复比较，每次为 Tripel、Kriek、Lambic 各生成 40 个谱。指标包括对真实非缺失留出测量的平均 KS、训练表示上的有偏 RBF MMD²、目标风格分类概率及精确复制比例。分类器与生成器共享训练集，留出表现用于说明诊断模型能力；生成概率并不是外部独立风格鉴定。五次划分也会重复使用同一批酒。")

T(["风格", "方法", "平均 MMD²", "精确复制比例"], [[r.style,r.method,f"{r.MMD2:.4f}",f"{r.exact_training_copy_fraction:.2f}"] for r in gen.itertuples()], [.18,.38,.24,.20])

P("局部凸组合没有精确复制训练样本，目标风格分类概率较高；但其 MMD 未全面优于直接重采样。PCA 高斯的复杂性也没有带来分布距离优势。因每风格只有极少留出样本，尤其 Kriek 和 Lambic 每次通常只有两款，距离指标不能证明整个分布已被学会。生成模型的评价应同时看接近程度、复制风险与可解释约束，而不能只追求分类器高分。")

F("generator_comparison.png", "图5 五次划分下生成方法的分布距离与内部风格概率误差线是划分标准差", 6)

P("第一种划分的局部凸组合样本及连续感官画像完整保存。画像来自第一问预先指定的全数据岭回归代理，因此不能用作独立的真实感官验证。五个综合指标随谱联合生成，原作者聚合公式尚未完整恢复，精确代数一致性也未被证明。")

H("6.3 逐项缺陷约束与迁移搜索", 2)

P("第一阶段采用 PCA 风格均值方向搜索，11 个源样本中只有 7 个找到非平凡候选。第二阶段扩大为目标风格条件训练锚点搜索：对每个源样本，沿每个 Tripel 训练锚点方向取 30 个权重，再作经验范围投影与四项缺陷字段的逐项上界投影。四项分别为 DMS、双乙酰、4VG 和 trans-2-nonenal。相比单一潜方向，该搜索保留更多目标风格变化方向。")

E("x_candidate,d ⩽ x_source,d, d∈{DMS、Diacetyl、4VG、trans-2-nonenal}", 13)

E("f_stale(x_candidate) ⩽ f_stale(x_source)", 14)

P("同时要求固定分类器的 Tripel 概率超过原始样本和仅做截尾对照至少 0.01。该阈值是操作性的非平凡变化标准，并非真实风格成功阈值。候选选择按模型概率进行，因此会适应诊断分类器；高概率不能作为优化之后仍独立的验证证据。未找到候选时保留原样并记失败，原样不算成功迁移。")

P(f"扩大搜索后，11 个源样本都找到满足当前模型条件的候选，其中 {int(transfer.all_source_defects_observed.sum())} 个具有四项缺陷源字段实测，另 2 个缺少双乙酰值，只能相对插补值检查；Stale hops 对全部样本都是学习得到的代理。源样本中 2 个属于生成器留出集，其余为训练来源。增加搜索范围带来更多模型可行候选，但没有增加真实品评证据。")

F("transfer_comparison.png", "图6 扩展搜索与单一 PCA 方向搜索的内部分类概率比较 缺失源值另作标注", 6)

P("DMS 与其他缺陷的单位并不统一，部分尺度和 trans-2-nonenal 阈值也未确定。本文因此不声称全部绝对 OAV 阈值已通过。测量字段逐项不增加是较保守的模型条件，但若来源值有检测限替代、缺失或尺度歧义，仍需逐项补证。下一步应恢复基本成分的浓度和聚合公式，加入发酵过程与酿造可行性，再以盲评检验感官效应。")

H("7 稳健性 可复现性与模型评价")

H("7.1 已完成的稳健性检查", 2)

P("本流程分别保存了分层重复留出、训练折调参、额外随机缺失、网络阈值变化、bootstrap 网络重建、跨方法排名、留出置换、分级规则变化和生成划分变化。第一阶段与第二阶段分目录存储，保留方法变化前后的成绩和失败样本。23 项程序检查覆盖主键对齐、原件未改、预处理隔离、符号拉普拉斯性质、区间残差排序、留出角色、逐项缺陷和失败标记；这些检查证明实现满足指定性质，不证明全部科学假设成立。")

H("7.2 优点与适用范围", 2)

P("流程的主要价值是把尺度审计、预测验证、关联证据和生成约束分开评估。简单参照使性能提升可比较，弱因素与失败案例保留使解释不过度确定，模型内约束也明确限定在可观测字段或代理。对类似小样本、高维、混合量纲的风味研究，这些步骤比单纯增加模型复杂度更有实际参考价值。")

```
H("7.3 局限与改进方向", 2)
P("其一，原始逐人评分与绝对锚点缺失使真实 0—6 分不可恢复；分级规则高度分歧，不能把假设等级包装成真值。其二，噪声、检测限、品评差异和有限样本限制预测能力，且源数据预处理范围未完全明确。其三，Top-3 多数不稳定，反直觉项缺少因果证据；图约束参数与解释阈值也需要更系统敏感性。")
P("其四，生成验证的稀有风格留出过少，分类器共享训练信息且会被迁移搜索适应；重复分割不能代替新数据。其五，聚合量公式、物理约束、缺陷阈值和感官实验未完成。其六，第二阶段已经基于第一阶段经验设计，缺少完全封存的最终外部测试。因此本文定位为可复核的参考研究，后续应以新增测量、品评锚点和独立实验补充关键证据。")

H("8 结论与建议")
P(f"在当前附件尺度上，化合物对感官画像具有一定预测信息：固定配置 GBR 的连续宏平均 MAE 为 {gbr:.4f}，优于均值参照；但原始 0—6 分缺少可识别条件，不应声称已恢复。加入训练、校准和测试隔离的区间示例，可补充连续预测的不确定性表达，仍需遵守边际覆盖及交换性限制。")
P(f"化合物网络可组织共变关系并提供正则化信息，但 150 条 Top-3 仅 {status['comparatively_supported_top3_associations']} 条满足当前相对支持判据，说明稳定解释和化学机制是后续重点。生成比较显示局部凸组合在非复制与风格诊断之间有实用折中，复杂高斯模型尚未表现出全面优势。扩展搜索扩大了模型内可行迁移范围，却不能替代真实浓度阈值、酿造过程或品评验证。")
P("后续研究优先顺序为：获取原始评分或参考酒锚点；复核尺度、检测限与异常；对关键因素开展浓度和文献核查；用封存的新样本验证生成与迁移；最后据实更新论文与 AI 使用记录。")

H("参考文献")
references = [
"[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.",
"[2] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.",
"[3] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3-42, 2006. DOI:10.1007/s10994-006-6226-1.",
"[4] Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189-1232, 2001. DOI:10.1214/aos/1013203451.",
"[5] Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094-1111, 2018. DOI:10.1080/01621459.2017.1307116.",
"[6] Scikit-learn developers. Permutation Importance with Multicollinear or Correlated Features. https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance_multicollinear.html, 访问日期：2026-10-24.",
"[7] Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983-11988, 2008. DOI:10.1021/jf8019453.",
"[8] OpenAI Codex, GPT-6 家族（精确可公开版本待核验），OpenAI，使用日期：2026-10-24."
]
for reference in references: P(reference)
H("AI 工具使用声明")
P("本参考研究使用 OpenAI Codex 辅助进行资料检索、代码实现、数据审计、实验记录、图表和稿件编写[8]。正文所述计算为 AI 辅助代码的实际运行结果，不能据此声明核心工作已由参赛队员独立完成。队员后续应独立理解、验证、修改并记录实际贡献；采纳内容须按赛事要求在相应位置标注。模型家族信息可由会话核验，精确版本应按实际界面补充，不臆造版本号。重要交互与采纳修改记录见随附 AI 使用阶段记录，最终提交前须补全规定的 PDF 说明。")
H("附录 复现与结果索引")
P("第一阶段运行 run.py all，第二阶段运行 run_stage2.py，配置分别为 config.json 和 config_stage2.json。原始附件由使用者自行取得并经 source_manifest.csv 核对哈希。Python 与依赖版本锁定在 requirements-lock.txt；生成论文的脚本位于 scripts/，原始数据未随公开仓库分发。")
T(["材料", "路径与内容"], [["数据证据", "outputs/stage2/audit/：字典、原件哈希、尺度和模板检查"], ["第一问", "outputs/stage2/q1/：连续误差、GBR 留出预测、分级敏感性和区间覆盖"], ["第二问", "outputs/stage2/q2/：完整置换贡献与 150 条候选证据；原始网络见 stage1/q2"], ["第三问", "outputs/stage2/q3/：生成对照、局部样本、画像、迁移与缺陷检查"], ["研究源代码", "src/、run.py、run_stage2.py；算法与完整运行代码随支撑包提供"], ["入门教程", "docs/从数据到论文的解题教程.md；按数据、模型、验证与论文顺序阅读"]], [.20,.80])
P("本稿未逐页排入全部源代码，也未填写队号、承诺书和队员独立贡献。若转为正式参赛作品，应依据官方要求把全部实际使用源程序纳入附录，并完成匿名、页数、引用与 AI 披露检查。参考稿中的条件、失败结果和未证实事项不得在修改时无依据删除。")

title = "基于尺度审计与稳定关联的啤酒感官预测及风格约束迁移"
payload = {"title":title, "abstract":abstract, "keywords":"感官预测；数据尺度；图约束回归；重采样稳定性；条件生成；约束迁移", "blocks":blocks}
(PAPER / "paper_content.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
md = ["# "+title,"","## 摘要",abstract,"","关键词："+payload["keywords"],"","## 目录","见 PDF 的自动生成目录。正文以下各节与实际结果文件对应。","",]
for b in blocks:
    if b["type"]=="heading": md += ["#"*(b["level"]+1)+" "+b["text"],""]
    elif b["type"]=="paragraph": md += [b["text"],""]
    elif b["type"]=="equation": md += [f"公式（{b['number']}）：{b['text']}",""]
    elif b["type"]=="table":
        md += ["| "+" | ".join(b["headers"])+" |","|"+"---|"*len(b["headers"])]
        md += ["| "+" | ".join(map(str,row))+" |" for row in b["rows"]]
        md += [""]
    elif b["type"]=="figure": md += [f"![{b['caption']}]({../{b['path']}})",""]
(PAPER / "论文参考稿.md").write_text("\n".join(md), encoding="utf-8")
print(json.dumps({"blocks":len(blocks),"characters":len('\n'.join(md)),"paper":str(PAPER/'论文参考稿.md')},ensure_ascii=False))
```

scripts/prepare__revised__paper.py

```
"""Build a result-backed revision and transparent AI-use stage record.
AI-assisted, OpenAI Codex, 2026-10-25. No planned result is written as observed.
"""
from pathlib import Path
import hashlib
import json
import pandas as pd

ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'; PAPER=ROOT/'paper/revised_20261025'; PAPER.mkdir(parents=True,exist_ok=True)
for part in ['q1','q2','q3']:
    if not json.loads((OUT/part/'status.json').read_text())['complete']: raise ValueError('Incomplete experiment '+part)
metrics=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); graph=pd.read_csv(OUT/'q2/graph_comparison.csv'); top=pd.read_csv(OUT/'q2/top3_with_evidence.csv')
gen=pd.read_csv(OUT/'q3/generation_summary.csv'); transfers=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); coverage=pd.read_csv(OUT/'q1/conformal_coverage.csv')
gstatus=json.loads((OUT/'q2/status.json').read_text()); roles=pd.read_csv(OUT/'q3/disjoint_roles.csv'); diagnostics=pd.read_csv(OUT/'q3/real_holdout_diagnostics.csv')
checks=json.loads((OUT/'validation_summary.json').read_text()); tests=checks['tests']['tests']
def mae(model,scope): return float(metrics[(metrics.model==model)&(metrics.scope==scope)].mae.iloc[0])
etcore=mae('extra_trees','core30'); etall=mae('extra_trees','all50'); blendcore=mae('shrunk_target_blend','core30'); blendall=mae('shrunk_target_blend','all50')
r1=transfers[transfers.radius==1]; ht=r1[r1.source_role=='T_real_holdout']; success=int(ht.evaluator_proxy_acceptance.sum()); supported=int(top.evidence_status.str.startswith('comparatively').sum())
blocks=[]
def H(s,level=1): blocks.append({'type':'heading','level':level,'text':s})
def P(s): blocks.append({'type':'paragraph','text':s})
def E(s,n): blocks.append({'type':'equation','text':s,'number':sum(b['type']=='equation' for b in blocks)+1,'asset_number':n})
def T(headers,rows,widths=None): blocks.append({'type':'table','headers':headers,'rows':rows,'widths':widths})
def F(name,caption,height=7): blocks.append({'type':'figure','path':'outputs/stage3/figures/'+name,'caption':caption,'height_cm':height})
references={
 'source':'Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.',
 'et':'Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3-42, 2006. DOI:10.1007/s10994-006-6226-1.',
 'gbr':'Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189-1232, 2001. DOI:10.1214/aos/1013203451.',
 'stack':'Breiman L. Stacked Regressions. Machine Learning, 24:49-64, 1996. DOI:10.1007/BF00117832.',
 'graph':'Li C, Li H. Variable selection and regression analysis for graph-structured covariates with an application to genomics. The Annals of Applied Statistics, 4(3):1498-1516, 2010. DOI:10.1214/10-AOAS332.',
 'cp':"Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094-1111, 2018. DOI:10.1080/01621459.2017.1307116.",
 'sklearn':'Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.',
 'cv':'Bengio Y, Grandvalet Y. No Unbiased Estimator of the Variance of K-Fold Cross-Validation. Journal of Machine Learning Research, 5:1089-1105, 2004.',
 '4vg':'Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983-11988, 2008. DOI:10.1021/jf8019453.',
 'iron':'Omur-Ozbek P, Dietrich A M, Duncan S E, Lee Y. Role of lipid oxidation, chelating agents, and antioxidants in metallic flavor development in the oral cavity. Journal of Agricultural and Food Chemistry, 60(9):2274-2280, 2012. DOI:10.1021/jf204277v.',
 'coriander':'Pater A, Satora P, Januszek M. Effect of Coriander Seed Addition at Different Stages of Brewing on Selected Parameters of Low-Alcohol Wheat Beers. Molecules, 29(4):844, 2024. DOI:10.3390/molecules29040844.',
 'mmd':'Gretton A, Borgwardt K M, Rasch M J, et al. A Kernel Two-Sample Test. Journal of Machine Learning Research, 13:723-773, 2012.',
 'ai':'OpenAI. Codex（系统提示模型家族 GPT-6，精确可公开版本待实际界面核验），使用日期：2026-10-24 至 2026-10-25. 重要交互、采纳与未完成的人工作业见随附 AI 工具使用详情说明阶段版。',
}
order=[]
def C(key):
    if key not in order: order.append(key)
```

```
    return '['+str(order.index(key)+1)+']'

H('1 问题重述与研究目标')
P(f'本题要求由化合物测量给出感官评分、逐维 Top-3 及稳定性和化学解释，并按风格生成化合物谱和完成缺陷不恶化的迁移。与附件高度吻合的原始研究已用化学资料预测感官并开展加标试验{C("source")}；其数据和已有发现是研究背景，不计作本研究新增贡献。')
P('本文以实际 250 款啤酒、231 个数值输入和 50 个监督目标为基础。保留全部目标输出，另把 A_ 与 F_ 前缀的 30 个核心维度作为预先明确的主要比较范围，其他 20 项按评分卡章节补充报告。这个范围选择不代表官方只要求 30 项，也不把题面“约 40”强行解释为一个与附件不符的固定数量。')
P('三项研究目标分别为：明确连续标签与 0-6 等级的可解释范围；检验网络和候选因素的预测信息是否稳定；把统计可行的风格调整与独立模型支持、真实化学和品评证据区分。无法获得的新测量不以模型输出补造。')

H('2 相关工作、贡献边界与模型假设')
P(f'ExtraTrees{C("et")}、梯度提升回归{C("gbr")}及利用交叉验证预测学习组合权重的 stacking{C("stack")}均为成熟方法。图拉普拉斯约束回归已有系统研究{C("graph")}，分割保形预测也已有理论和方法{C("cp")}。本文采用这些方法并作任务化组合，不主张首次发明基础算法、收缩融合、Pareto 优化或新的保形理论。')
P('本文的可核查贡献定位于应用建模和证据评估：逐字段尺度及评分位置核查；在明确目标范围内对标准模型及收缩融合公平比较；加入无图和打乱图对照，检验网络的额外价值；分离生成搜索资料与评价资料，并如实报告代理判断分歧和失败。贡献是否成立由具体实验决定，不要求所有复杂设计获胜。')
P('假设一，beer 唯一标识相同酒，跨表正确对应，经集合、唯一性和风格一致性支持。假设二，训练内插补、截尾和标准化提供可检验的统计处理，但不证明极端值错误或缺失随机。假设三，相关网络只提供共变结构，系数不是因果效应。假设四，风格内统计插值与经验支持约束不充分保证酿造可行。假设五，覆盖解释依赖相应交换性与测量条件，不能自动推广到自适应搜索后的分布外样本。')
T([['符号','定义'],[['X、Y','231 个输入与 50 个连续目标；生成主模型使用 226 个非聚合字段'],['Z、B、L','训练预处理后的输入、回归系数及有符号图拉普拉斯'],['w_j、η','目标 j 的 GBR 权重及向全局权重收缩的惩罚'],['g、[L_y,U_y]','固定训练分位等级映射与连续预测区间'],['G、J、T','生成/搜索训练组、独立评价训练组、真实留出组'],['r、D','标准化改动半径和四项指定化学缺陷字段集合']], [.23,.77])

H('3 数据审计与处理')
H('3.1 对齐、字段与评分卡核查',2)
P('实际 1.csv 为 250×234，含 3 列元信息；4.csv 含 50 个目标。beer_id 在感官文件精度受损，不作为主键。程序按 beer 对齐并核对风格；原件以 SHA-256 绑定版本，未修改附件。2.csv 与重算 Spearman 的平均绝对差约 1.58×10^-16，而与 Pearson 约差 0.0873，因此按秩相关解释。')
P('评分卡 summary 与监督目标的三个显式别名为 A_ester_fruity→A_esters_fruity、F_ester_fruity→F_esters_fruity、4vg→X4vg。50 个目标均已对应。评分卡另有 Aroma、Flavour 乙醛位置而没有对应的独立监督列，保留缺标签说明，未补零。12 处 Barnyard 汇总引用 AG76，表单标签却对应 AG90，列为疑似源公式问题；这不自动证明 4.csv 继承该问题。')
T([['资料或范围','数量/用途','处理决定'],[['核心 Aroma / Flavour','15+15 个前缀目标','主比较，全部保留输出'],['其他感官目标','20 个','口感、缺陷、香料、Lambic 与总体等另报'],['输入字段','231 个','预测阶段保持附件尺度'],['生成主输入','226 个非聚合描述字段','五个未核实综合量不独立生成'],['3.xlsx 原评分卡','12 张 beer 页及 summary','输入空白和公式缓存零不作为评分真值']], [.25,.29,.46])
H('3.2 尺度、缺失和异常处理',2)
P('逐列比对可比的原始正值配对，有 37 个字段支持 log10 关系；其他字段存在多数恒等、少数不匹配或缺参考。正值匹配不证明零替代值、检测限或全部量纲均已恢复，因此不把全表统一反对数。DMS、4VG 的正值变换证据与双乙酰、trans-2-nonenal 的不匹配另列。')
P(f'所有预测预处理只在当前训练部分估计：1%/99% 截尾、中位数插补、均值和标准差标准化，使用 Pipeline 管理{C("sklearn")}。验证和测试只执行已拟合变换。去截尾、2.5% 截尾和模拟缺失作为消融，不能把被截值称为错误测量。')
E('z_ij = [clip(x_ij;a_j,b_j)-m_j]/s_j',1)

H('4 第一问：连续预测、相对分级与不确定性')
H('4.1 绝对评分的信息缺口与交付定义',2)
P('原始研究按品评人员中心化和标准化。若逐人均值、标准差、出席情况及汇总权重缺失，同一聚合连续值可能对应多组原始评分，无法唯一反演真实 0-6 分。空白评分卡不能提供锚点。本文不把经验映射包装成真实品评等级。')
E('z_it=(s_it-μ_t)/σ_t, s_it=μ_t+σ_t z_it',2)
P('实际交付包括全 50 维连续留出预测和带 ASSUMPTION 标记的相对 0-6 等级。相对 0 是训练参考的最低一档，不等于真实未检出；相对 6 也不等于真实压倒性。若题目要求真实绝对等级，仍需原始锚点或转换参数；本文诚实保留这一未解决的测量信息条件。')
H('4.2 模型与严格训练内融合',2)
P('比较均值、岭回归、ET、GBR、固定半数 ET/GBR、全局权重、无收缩逐维权重和收缩逐维权重。ET 200 棵树、最小叶样本 3、特征比例 0.8；GBR 每目标 100 棵深度 2 的树，学习率 0.04、最小叶样本 5、特征比例 0.7。岭回归 α 在 {1,10,100,1000} 中以内层核心 MAE 选择；元信息和风格不作为第一问预测输入。')
E('B_hat=argmin_B ||Yc-ZB||²_F+α||B||²_F',3)
P('融合权重的具体目标是内层留出预测的均方误差加向全局权重的二次惩罚，最终 η 由训练内验证的核心 MAE 选择。设 d=p_GBR-p_ET、e=y-p_ET，得到受限闭式权重：')
E('w_j=clip{[mean_i(d_ij e_ij)+ηw_0]/[mean_i(d_ij²)+η],0,1}',15)
P('η∈{0,0.01,0.1,1,10}。对每个外层训练集，再分 S/V；只在 S 内三折产生基础预测并拟合权重，重训 S 后预测 V、选择 η。V 标签不进入 S 内拟合。选定 η 后在全部外层训练内重新产生 OOF、学习权重，重训基础模型后预测外层测试。该嵌套结构避免直接在外层测试上拼接最优维度。')
H('4.3 同范围验证与实际结果',2)
P(f'按风格三折、五次重复，共 15 个外层划分；内层使用三折 KFold。每个样本每次恰被留出一次，所有方法共享外层角色。报告先对每个 beer 的五次预测平均、再计算逐维宏指标的聚合 OOF 成绩；它不同于未来单次拟合模型的误差，单折成绩另存。重复折共享资料，不能当独立样本简单作 t 检验{C("cv")}。')
E('MAE=(1/nq) Σ_i Σ_j |y_ij-y_hat_ij|',4)
display=[('mean','均值'),('ridge','岭回归'),('extra_trees','ET'),('gbr','GBR'),('fixed_half_blend','固定半数融合'),('global_oof_blend','训练内全局融合'),('unshrunk_target_blend','无收缩逐维融合'),('shrunk_target_blend','收缩逐维融合')]
T([['方法','核心30维 MAE','全50维 MAE'],[[name,f'{mae(key,"core30"):.5f}',f'{mae(key,"all50"):.5f}'] for key,name in display],[.44,.28,.28])
P(f'ET 的核心 MAE 为 {etcore:.5f}，全维为 {etall:.5f}；收缩融合分别为 {blendcore:.5f}、{blendall:.5f}。核心相对 ET 的变化为 {(blendcore/etcore-1):+.2%}。固定半数融合核心 MAE 为 {mae("fixed_half_blend","core30"):.5f}，略优于收缩方案；当前没有证明更复杂的权重学习必要。本文不把全维平均最优推导为核心维度最优。主交付的相对等级与区间采用事先固定的 ET 基线，复杂融合仅在其实际收益和代价范围内评价。')
F('prediction_by_scope.png','图1 相同外层划分的聚合留出误差：核心30维与全50维分开报告',7)
P('配对比较按 beer 重采样，保持同一酒两模型的误差配对及风格构成。2000 次固定 OOF 条件重采样范围只反映保存预测的样本敏感性，不包含训练和选模不确定性。完整差值、范围、逐维 $R^2$ 及失败目标随结果表提供，未把条件范围当作总体算法优势的严格证明。')
F('blend_weights.png','图2 权重完全在外层训练内部学习，反映目标异质性而非外层成绩事后挑选',5.8)
H('4.4 固定相对分级与预测集合',2)
P('每个训练参考的六个边界取 k/7 分位。固定边界定义 g(y)，对留出连续预测给出相对等级。全 250 款酒的交付表使用五次留出等级众数；各折参考稍有差别，因此该表是相对画像，不是共同绝对强度量表。')
E('g(y)=Σ_k I(y>t_k), G(x)={g(L_y(x)),…,g(U_y(x))}',16)
P('另作五次 150 训练、50 校准、50 测试的分割保形示例。ET 和等级边界只在 150 个训练样本拟合；50 个校准残差的第 ceil[51×0.9]=46 顺序统计量给出逐维连续区间，再映射为相对等级集合包络。重复边界可能让包络包含不可达的中间等级，但不缩窄包络。')
E('r_ij=|y_ij-f_j(x_i)|; q_j=第 ceil[(m+1)(1-α)] 个校准残差',5)
E('C_j(x)=[f_j(x)-q_j, f_j(x)+q_j]',6)
P(f'五次分割、50 个目标的平均连续测试覆盖为 {coverage.continuous_coverage.mean():.2%}，平均区间宽 {coverage.interval_width.mean():.4f}；固定规则下的相对等级集合覆盖为 {coverage.relative_grade_set_coverage.mean():.2%}，平均集合大小 {coverage.mean_grade_set_size.mean():.2f}。这不是原始等级准确率或 50 维同时覆盖保证；同一批酒重复出现也不构成五套外部数据。')
P('平均集合接近全部七档，说明目前很难在可靠性要求下细分感官等级；较高集合覆盖伴随宽集合，不能解读为七级预测很精确。细化等级需要更好的测量锚点、更多样本或更小的连续预测误差。')
F('ordinal_uncertainty.png','图3 五次分割的逐维覆盖与相对等级集合大小；真实绝对评分仍未标定',6.2)
grades=pd.read_csv(OUT/'q1/ASSUMPTION_oof_relative_grades.csv').head(3)
fields=['A_malt_all','A_hops_all','A_esters_all','F_malt_all','F_hops_all','F_esters_all']
T([['beer','麦芽香','酒花香','酯香','麦芽味','酒花味','酯味']，[[str(int(r.beer))]+[str(int(r[f])) for f in fields] for _,r in grades.iterrows()],[.13]+[.145]*6)
P('上表只展示全量输出中的三个样本和六个整体属性，数值均为相对档位；完整 250×50 等级、连续预测和区间角色保存在支撑文件。真实评分锚点获取路径未执行，不虚报绝对等级准确率。')
H('4.5 截尾、缺失压力与失败分析',2)
stress=pd.read_csv(OUT/'q1/missingness_stress.csv'); clip=pd.read_csv(OUT/'q1/clipping_ablation.csv')
st=stress[stress.model=='extra_trees'].groupby(['scenario','extra_missing_rate']).core30_mae.mean().reset_index()
T(['模拟缺失情境','额外比例','ET核心 MAE'],[[r.scenario,f'{r.extra_missing_rate:.0%}',f'{r.core30_mae:.5f}'] for r in st.itertuples()],[.52,.18,.30])
P('低值优先遮蔽使训练 20% 分位以下的观测被遮蔽权重为其他值的四倍；它是检测限相关缺失的压力情境，不是对真实缺失机制的确认。每种情境中 ET 与融合使用完全相同的遮蔽位置。截尾对照使用 ET，未把它冒充全部方法都完成的消融。')
P('截尾比例 0、1%、2.5% 的 ET 单折核心 MAE 均值依次为 '+ '、'.join(f'{v:.5f}' for v in clip.groupby('lower_clip').core30_mae.mean())+'。额外缺失表为逐折误差均值，与前述重复预测平均后的误差不是同一种汇总，不能混作同一成绩提升。')

H('5 第二问：网络消融、稳定因素与化学证据')
H('5.1 描述图与训练折图',2)
P('按给定秩相关矩阵建立 231 节点的描述图，|r|≥0.7 时有 109 条边。预测验证中的网络必须由当前训练数据重算，不能直接使用含外层样本信息的全数据图。符号拉普拉斯及图约束回归定义如下；图用于正则化，不是反应通路。')
E('W_ij=r_ij I(|r_ij|≥τ), L=diag(Σ_j W_ij)-W',7)
E('B_hat=argmin_B {||Yc-ZB||²_F+α||B||²_F+λ tr(B^T L B)}',8)
P('α∈{1,10,100,1000}，λ∈{0,1,5,20}，非零 λ 时 τ∈{0.6,0.7,0.8}，全部在内层以核心 MAE 选择。无图基线使用相同 α 候选；负对照将训练图节点标签随机置换，保留无标签图的边权、符号和度分布。每个拟合使用 10 个随机图，汇总各图的误差而非平均预测形成额外集成。')
H('5.2 网络的额外收益检验',2)
T([['模型','核心30维单折 MAE','全50维单折 MAE'],[[{'none':'无图','real':'真实图','shuffled':'打乱图'}[k],f'{graph[(graph.graph==k)&(graph.scope=="core30")]["mean"].iloc[0]:.6f}',f'{graph[(graph.graph==k)&(graph.scope=="all50")]["mean"].iloc[0]:.6f}'] for k in ['none','real','shuffled']],[.38,.31,.31])
P(f'真实图与无图误差几乎相同，全数据内层选择 α={gstatus["full_graph_params"]["alpha"]}、λ={gstatus["full_graph_params"]["lambda"]}。因此当前数据和网格不支持“图正则化改善预测”的结论。网络仍可组织共变关系，但最终参数退化到普通岭回归时，Top-3 不能归功于图惩罚。该表是单折均值，不能与第一问聚合 OOF 数值直接比较来判断模型优劣。')
F('graph_ablation.png','图4 有图、无图和打乱图的对照及内层选择：真实图未显示实用预测收益',6.8)
H('5.3 跨拟合贡献、风格控制与模块替代',2)
P('每个外层训练内选择每目标三个化合物/元素候选，排除 pH、色度及聚合字段。留出贡献用冻结模型的特征置换 MAE 增量评价，重复 20 次；另在相同风格内置换，以及联合置换整个相关模块、删去模块重训。风格内置换不是完整条件于全部化合物的严格条件置换，模块与单变量贡献也不是因果效应。')
E('π_jk=bootstrap 中入选 Top-3 次数 / 重采样次数',9)
E('Δ_jk=MAE_k(置换后的留出预测)-MAE_k(原留出预测)',10)
P(f'全数据描述模型的 200 次 bootstrap 每次重建预处理与图，但固定全数据选定的
```

```
超参数，所以频率是该参数条件下的稳定性。当前描述性支持条件为频率≥0.5、至少 5 个外层训练入选、这些入选折的平均普通与风格内贡献均为正，共 {supported}/150 条。它与前阶段 11 条采用的模型和判据不同，不能把数量变化解释为直接的可靠性提升；筛后汇总仍有选择适应性，不作独立显著性检验。')
F('factor_crossfit.png','图5 150 条最终候选的条件重采样稳定性及风格内留出贡献：正贡献不代表因果',6.4)
H('5.4 化学合理性、证据层级与反例',2)
P(f'X4vg 与 X4VG 为较一致的候选。已有啤酒陈化研究观察 4VG 降低并讨论丁香向香草印象变化{C("4vg")}，可支持气味合理性，但不是本数据因果验证。iron 与 metallic 的关系有铁离子诱发口腔脂质氧化的实验依据{C("iron")}，该研究使用金属溶液且不同价态反应不同，因此不能直接给本附件总 iron 套用统一啤酒阈值。')
P(f'coriander 与 linalool 的关联可结合低醇小麦啤酒添加芫荽籽的研究核查：该研究的 GC-O 结果指出芫荽籽中 linalool 等组分有较高气味活性{C("coriander")}。这只支持机制相容性，不能证明本附件 coriander 标签由 linalool 单独造成。酸度与乳酸、醋酸以及酒精感与乙醇可作测量相容性的检查，其余未知联系不补写机理。')
T(['关联示例','可支持的解释','仍缺什么'],[['X4vg / X4VG','啤酒中丁香相关香气有原始研究支持','本批次加标或独立测量'],['metallic / iron','金属离子可诱发口腔金属味机制','价态、基质、浓度和本附件因果'],['coriander / linalool','芫荽籽与啤酒研究的香气依据','标签特异性与浓度验证'],['bitternes / iron','统计贡献可能反映风格或共变','不能据此称铁是苦味的直接来源'],['clove / linalool','可重复关联但不等于典型丁香来源','共线替代与特异性证据']], [.26,.40,.34])
P('完整 150 条结果附方向、频率、入选折数、风格内贡献、单位和附件气味描述。独立文献只支持少量明确案例，其余化学解释逐条标为未充分核实；不以统计支持状态冒充机制通过。论文附录逐维给出 Top-3，弱项和反例均保留。')

H('6 第三问：受约束生成与独立迁移诊断')
H('6.1 三组隔离与226字段模型',2)
P('每个种子将资料按风格分层为 G=100、J=100、T=50。G 拟合生成器、Logistic 风格搜索器、岭感官代理和 ET 感官一致性代理；J 拟合不参与搜索的 ET 风格评价器及另一感官代理；T 只评价真实样本诊断能力。每次 G/J/T 不重叠，但所有酒在前两阶段研究中均已出现，因此只是本次拟合角色隔离，不是新增外部验证。')
P('五个未核实综合量不独立生成，生成器、搜索器与评价器全部重训为相同 226 个非聚合字段。字段仍含理化描述量，不能当作 226 种可任意混合的纯物质；未核实综合量输出为待核实，而不是补造数值。候选及顺序先固定并计算哈希，再拟合 J 评价模型，J/T 结果不用于重新挑候选。')
P(f'五次真实 T 诊断中，搜索器与独立评价器的平均平衡准确率分别为 {diagnostics.searcher_holdout_balanced_accuracy.mean():.3f}、{diagnostics.evaluator_holdout_balanced_accuracy.mean():.3f}。因此独立评价者本身也并非完美风格鉴定。两类模型的概率未经共同校准，不能仅以概率数值高低比较其可靠性；另报目标类别预测比例和代理约束。')
H('6.2 生成方法与相同核带宽比较',2)
P('比较风格内 bootstrap、局部凸组合、十维条件 PCA 高斯及受约束局部凸组合，每种风格每种方法每划分生成 40 个样本。PCA 协方差按 n_c/(n_c+10) 部分汇聚。局部凸组合避免直接复制，但不等于物理混配。')
E('h|c~N(μ_c,w_cΣ_c+(1-w_c)Σ_pool)，w_c=n_c/(n_c+10)',11)
E('x_new=u x_a+(1-u)x_b，u~Uniform(0.1,0.9)',12)
P('受约束版本在同风格最近三个邻居间取权重；DMS 和 4VG 按已支持的正值 log10 关系先回到正值域插值再返回，其他字段仍在附件尺度统计插值。检测限替代值的解释不完整，因此前者也只是有证据条件的建模变换。投影训练范围后，以同风格训练近邻距离的95%分位限制联合支持；每组最多尝试 40×250 次，失败不伪装为新样本。')
P(f'MMD²{C("mmd")} 的表示和核带宽只用 G 训练资料确定，所有方法共用；真实参考为 T 的非缺失测量，另报 KS、复制率、J 模型目标类别判断及范围违反。风格留出常只有两款，不能以小距离或不显著证明分布一致。')
T(['风格','方法','MMD²','J目标类别比例'],[[r.style,{'bootstrap':'重采样','local_convex':'局部凸组合','PCA_Gaussian':'PCA高斯','constrained_local_convex':'受约束凸组合'}[r.method],f'{r.MMD2_fixed_training_bandwidth:.4f}',f'{r.independent_evaluator_target_prediction_fraction:.1%}'] for r in gen.itertuples()],[.15,.35,.23,.27])
P('受约束方法在已检查的训练范围内违反率为零，未精确复制训练点；但 MMD 未全面优于重采样，且与普通局部凸组合各有取舍。J 的类别判断也不能作为生成酒真实标签。完整生成谱和感官代理画像保存于支撑文件，未将其称为重新测得的感官结果。')
F('generation_independent.png','图6 固定训练核带宽下的分布比较及未参与搜索的J模型判断，五次划分平均',7)
H('6.3 信赖域、Pareto选择与逐项缺陷',2)
P('迁移沿源样本与 Tripel 的 G 训练锚点各取30个权重。候选投影范围后逐项限制 DMS、双乙酰、4VG、trans-2-nonenal 不增加，并同时要求岭与 ET 两个 G 感官代理的 stale_hops 不增加。搜索 Tripel 概率至少0.5，且超过原样和仅截尾对照至少0.01；同时满足目标经验支持与指定改动半径。')
E('x_candidate,d≤x_source,d，d∈D',13)
E('f_stale(x_candidate)≤f_stale(x_source)',14)
E('‖Z_candidate-Z_source‖₂/√p≤r，r∈{0.25,0.5,1,2}',17)
P('排除违反硬约束的候选后，对标准化改变量、目标训练支持距离、DMS/4VG正值变换下的阈值代理超额作 Pareto 筛选，再从非支配候选选择最小改动者。19 ug/L 与0.2 mg/L 来自附件中位阈值，仅作为这两项正值代理，不替代逐样本浓度和检测限核验。trans-2-nonenal 阈值缺失；双乙酰等配对不匹配未完全解释，所以没有任何候选获“完整绝对化学认证”。')
P('源缺陷缺失时仅以训练中位数参与条件搜索并单独标记，不能作真实源上界。源值非缺失也不表示已验证浓度、基质或感官阈值。找不到非平凡候选时保留原样且记失败；原样满足不增加不计作迁移成功。')
rows=[]
for radius,a in transfers.groupby('radius'):
    held=a[a.source_role=='T_real_holdout']; rows.append([str(radius),f'{(a.status!="no_feasible_nontrivial_candidate_source_retained").sum()}/{len(a)}',f'{a.evaluator_proxy_acceptance.sum()}/{len(a)}',f'{held.evaluator_proxy_acceptance.sum()}/{len(held)}'])
T(['改动半径','搜索非平凡可行','全部记录J代理支持','T源记录J代理支持'],rows,[.18,.25,.29,.28])
P(f'半径1的搜索可行记录中，{int((r1.status=="conditional_on_imputed_source_defect").sum())}条依赖缺失源值插补条件，{int((r1.status=="model_feasible_nontrivial").sum())}条具有四项附件源字段观测。后者仍不等于原浓度、全部阈值或真实缺陷已认证。')
P(f'半径1时，T源样本的 {len(ht)} 条重复划分记录对应 {ht.beer.nunique()} 款不同酒，只有 {success} 条同时满足 J 的 Tripel 概率改善、目标类别判断及 stale_hops 代理不增加。G/J训练源记录另报，不与T源合并冒充独立结果。这个比例是跨模型代理一致性，不是真实品评成功率；不同半径重复使用同一源酒，也不能累加为独立样本。')
F('transfer_independent.png','图7 搜索可行率与独立代理支持率的差异；分母为重复划分记录而非新增酒款',6.7)
P('与前阶段只追求单个分类器高分相比，本阶段增加了最小改动、双代理、联合支持和J评价。这些更严格条件带来更低的通过率，不能与旧稿11/11直接当同定义成绩比较。若某候选利用模型弱点，其失败本身是重要结果。自适应优化后的候选也不能继承普通逐点保形区间的无条件覆盖解释。')

H('7 结果评价、贡献与限制')
P('预测对照揭示目标范围和维度异质性，收缩融合的收益必须与实际误差及复杂度一起判断；网络消融没有支持额外预测优势，因此不把网络图形复杂性当作贡献。独立迁移诊断揭示搜索成绩与跨模型支持之间的明显差距。主要贡献是形成可核查、不过度宣称的任务化证据，而不是创造新的基础算法。')
P(f'当前 {tests} 项程序检查通过，涵盖主键、原件、训练内预处理、嵌套预测角色、图零惩罚等价、风格置换、固定核MMD、生成范围、G/J/T隔离、缺失源标记、失败迁移与校准角色。检查验证实现性质，不能证明全部科学假设成立。配置、源程序与原件哈希、划分和结果均保存。')
P('剩余限制包括：缺少真实绝对评分锚点；原作者预处理和检测限未完全恢复；大部分化学机理尚无目标特异性文献或加标验证；稀有风格留出过少；生成的理化量未形成完整酿造方程；独立评价仍是统计代理且无新外部酒或真实品评。拟议0.5/1/2倍阈值敏感性、所有聚合公式恢复和新增真实锚点并未完成，不能写成已验证。')

H('8 结论与建议')
P(f'在当前附件连续尺度及明确核心范围下，ET 的聚合留出 MAE 为 {etcore:.5f}，收缩融合为 {blendcore:.5f}。相对分级与区间集合给出可复核画像，但不能恢复真实原始0-6分。图约束未提供实用预测增益，因素解释应结合风格内贡献、模块替代和化学证据。')
P(f'受约束局部生成实现了无精确复制和已检范围可行，但分布拟合没有全面超越简单重采样。半径1的T源迁移记录仅 {success}/{len(ht)} 获得独立代理条件共同支持，尚无完整绝对化学或真实品评认证。后续优先获取评分锚点和明确检测限/阈值，补充目标特异性化学证据，再以新酒或盲评确认效果；不建议仅继续增加模型名称。')

ai_number=C('ai')
H('参考文献')
for i,key in enumerate(order,1): P(f'[{i}] '+references[key])
H('AI工具使用与研究贡献声明')
P(f'本参考研究由 OpenAI Codex 辅助资料核对、代码实现、实验运行、图表与稿件编写{ai_number}。所述实验确已运行，但尚无证据表明队员已独立完成或复核核心工作，不能据此作相反声明。精确公开模型版本、队员实际贡献及最终采纳修改需据实补充；随附阶段版说明保留重要提示词、关键回复和实际执行范围。')
H('附录A 完整Top-3、复现与结果索引')
P('以下为全部50个目标的描述性Top-3。顺序按全数据描述模型系数绝对值，括号为固定全数据所选参数条件下的bootstrap入选频率；不等于致因概率。逐折选择与贡献、化学证据层级、全部连续预测和250×50相对等级另存，原始赛题数据不随公开支撑包分发。')
rows=[]
for target,a in top.groupby('target',sort=False): rows.append([target,'; '.join(f'{r.field} ({r.bootstrap_frequency_conditional_on_full_selected_params:.2f})' for r in a.sort_values('rank').itertuples())])
T(['目标','Top-3 字段及条件入选频率'],rows,[.25,.75])
P('复现顺序：使用官方A附件并核对哈希；安装 requirements-lock.txt；运行 run_stage3.py q1、q2、q3；运行 pytest；然后生成图表与本稿。全部角色和参数位于 outputs/stage3/，执行版配置为 config_stage3.json。软件为Python 3.12、NumPy、pandas、SciPy、scikit-learn、NetworkX、Matplotlib、openpyxl；精确运行版本见各问run_manifest。文档另用ReportLab、pypdf和Pillow。')
T(['材料','内容与相对路径'],[['评分核查','outputs/stage3_design/scoring_field_map.json 等，只读核查'],['第一问','outputs/stage3/q1：分范围成绩、嵌套权重、相对评分、区间、缺失与截尾'],['第二问','outputs/stage3/q2：图消融、跨拟合Top-3、模块与风格诊断'],['第三问','outputs/stage3/q3：G/J/T角色、候选锁、生成谱、画像、迁移及未知项'],['验证与来源','outputs/stage3/validation_summary.json；docs/SOURCES.md'],['历史记录','stage1、stage2及旧稿保留，不混作同定义新成绩']], [.24,.76])
blocks.append({'type':'pagebreak'})
H('附录B 本项目完整源程序')
P('以下列出本项目自编计算、核查、图表、排版、打包和测试源程序，以及执行配置。外部依赖库不逐行重印，名称版本见锁定文件和环境记录。长行会为版面自动折行；极少数字体不支持字符以Unicode转义显示。可执行原文件随支撑包保留，文件哈希列于附录索引。')
paths=[p.relative_to(ROOT).as_posix() for folder in ['src','scripts','tests'] for p in sorted((ROOT/folder).glob('*.py'))]+[p.name for p in sorted(ROOT.glob('run*.py'))]+[p.name for p in sorted(ROOT.glob('config*.json'))]
blocks.append({'type':'source_files','paths':paths})
title='尺度核查与独立评价下的啤酒感官预测及约束风格迁移'
abstract=f'针对啤酒感官评分、关键因素与风格生成问题，构建尺度审计、分范围预测和独立代理评价流程。以250款酒的231个输入和50个目标建模，明确30个Aroma/Flavour核心维度并保留全表。三折五次重复留出中，ET核心宏平均MAE为{etcore:.4f}，训练内收缩融合为{blendcore:.4f}；五次独立角色分割的平均逐维连续覆盖为{coverage.continuous_coverage.mean():.2%}，但相对等级不等于原始0-6分。加入无图及打乱图对照后，真实图与无图误差几乎相同，全数据内层选择图惩罚为0，不支持额外预测收益。150条因素候选中{supported}条满足当前描述条件，化学证据分级保留。生成阶段去除五个未核实综合量，以100款搜索、100款评价、50款真实留出隔离，比较四类生成方法；非复制优势未转化为全面分布距离优势。半径1下，真实留出源酒的{len(ht)}条重复迁移记录仅{success}条获独立风格及陈旧酒花代理共同支持。研究提供任务化建模与证据边界，不主张新的基础算法、绝对评分恢复或真实品评改善。'
payload={'title':title,'abstract':abstract,'keywords':'感官预测；相对分级；嵌套验证；网络消融；条件生成；独立评价','equation_dir':'paper/revised_20261025/equations','blocks':blocks}
(PAPER/'paper_content.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
md=['# '+title,'','## 摘要',abstract,'','关键词：'+payload['keywords'],'']
for b in blocks:
```

```
    if b['type']=='heading': md += ['#'*(b['level']+1)+' '+b['text'],'']
    elif b['type']=='paragraph': md += [b['text'],'']
    elif b['type']=='equation': md += [f"公式（{b['number']}）：{b['text']}",'']
    elif b['type']=='table':
        md += ['| '+' | '.join(b['headers'])+' |','|'+'---|'*len(b['headers'])]+['| '+' | '.join(map(str,row))+' |' for row in b['rows']]+['']
    elif b['type']=='figure': md += [f"![{b['caption']}](../../{b['path']})",'']
    elif b['type']=='source_files': md += ['可执行完整源程序已排入PDF附录，并随支撑包提供。文件：'+'、'.join(b['paths']),'']
(PAPER/'论文修订稿.md').write_text('\n'.join(md),encoding='utf-8')

ai_blocks=[]
def AH(s): ai_blocks.append({'type':'heading','level':1,'text':s})
def AP(s): ai_blocks.append({'type':'paragraph','text':s})
AH('1 工具、日期与范围')
AP('工具OpenAI Codex，开发机构OpenAI；系统提示模型家族GPT-6，精确可公开版本待队员以实际界面或配置核验，未臆造。使用日期2026-10-24至2026-10-25。公开阶段记录不含队员身份、签名或私密凭据。')
AP('用途包括公开资料检索、附件结构与尺度审计、参考模型代码、实际实验运行、结果核查、图表、论文和支撑包归档。本记录由AI辅助整理，尚无队员独立复核证据，不能声明核心工作已经由队员独立完成。')
AH('2 重要提示词与关键回复')
interactions=[
('用户：你帮我们来做一下A题的三问，至少要吧大致的框架做出来，没疑问的结果要保留，每一问的分析，以及处理过程方法等都要记录。','关键执行内容：先检索公开GitHub与原作者资料，再建立三问可运行基线，保存数据证据、假设分级、探索关联、模型内约束及失败结果。此项为回复/执行摘要，不伪称逐字全文。'),
('用户：你帮我完善一下完整的解题过程，需要你推进到写论文的地方，我会把你的解题过程，模型等作为我后续进行解题的一个参考，因为我是第一次参赛。','关键回复节选：已经把 A 题三问推进到完整论文参考稿。每问都记录了问题分析、模型假设、数学公式、求解过程、验证方法、结果与局限。'),
('用户：现在你产出的这些东西还缺少什么针对于一篇完整的论文或者说研究。','关键回复节选：目前已经有完整的论文结构和可运行的研究流程，但还缺少足够扎实的证据，把它变成最终参赛论文。'),
('用户：你根据现有的这六个缺口，设计相关的实验或者算法来解决他。用户补充：注意同步GitHub。','关键回复节选：目前完成的是设计、算法原型和先导检查，完整第三阶段训练尚未运行。'),
('用户：这些算法是创新设计的还是已经有人设计好的，现在这篇文章还有什么问题吗。','关键回复节选：基础算法大多是已有成熟方法；我做的是针对本题的组合、改造和实验设计，目前不能称为已经证实的原创算法。'),
('用户：针对你刚才说的这些来改进这篇论文。','关键回复节选：我会先补跑能用现有数据验证的关键实验，重点检查评分范围、网络约束的作用，以及迁移结果能否得到独立评价的支持，再据此修改论文。原始评分锚点等数据缺口会保留条件性结论。')]
for question,reply in interactions: AP(question); AP(reply)
AP('以上保留重要提示词节选和关键回复，未冒充全部对话。正式采纳时应结合原会话补齐队伍实际使用的完整关键交互，尤其是模型选择、修改与人工验证过程；不得将本阶段摘录称为全部聊天记录。')
AH('3 实际完成的辅助工作')
AP(f'已执行三折五次的预测与图对照、五次分割保形与相对等级集合、五次G/J/T生成及迁移独立诊断；已有{tests}项检查通过。核对字段别名和疑似源公式引用，原附件未改。实现采用成熟估计器、正则化组合和标准优化思想，不声明发明全新基础算法。')
AP('实际负结果保留：图约束未见预测收益；生成未全面优于重采样；搜索可行率高于独立代理支持；原始绝对评分、完整浓度阈值、检测限、酿造与真实品评尚未认证。所有数字对应保存结果，而不是事先设计的预期效果。')
AH('4 采纳、人工修改与责任记录')
AP('本阶段代码、计算、图表和参考论文由AI辅助完成。队员独立推导、重跑、化学核对、实际修改和最终采纳尚需真实填写。未发生的验证保持未完成，不填“已人工确认”。')
ai_blocks.append({'type':'table','headers':['内容','AI辅助范围','队员实际验证/修改'],'rows':[['评分与预测','处理、代码、验证与文字','尚未记录'],['网络与化学解释','网络对照、候选与来源核对','尚未记录'],['生成与迁移','约束、搜索、独立代理对照','尚未记录'],['论文与附录','结果整合、图表与排版','尚未记录']],'widths':[.23,.44,.33]})
AH('5 版本、复现与最终提交前补充')
AP('以config_stage3.json及原件/代码哈希、各问run_manifest和结果校验表识别本阶段。正式提交前需核验精确工具版本、队员真实贡献与独立复跑，保留重要原始交互，并按实际采纳在正文、图表和源程序相应位置披露。承诺书、队号命名和签名不由本记录代填。')
ai={'title':'AI工具使用详情说明（阶段版）','front_label':'记录范围','keyword_label':'记录要素','abstract':'本说明记录参考研究中OpenAI Codex的实际辅助范围、重要交互节选、已运行实验和未完成的人工作业。它是可核查的阶段记录，尚未替队员完成精确公开模型版本核验、独立贡献确认和全部关键对话的最终整理。','keywords':'工具信息；重要交互；采纳与修改；实际验证；版本记录','blocks':ai_blocks}
(PAPER/'ai_usage_content.json').write_text(json.dumps(ai,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'blocks':len(blocks),'markdown_characters':len('\n'.join(md)),'source_files':len(paths),'references':len(order),'tests':tests},ensure_ascii=False))
```

scripts/prepare__trial__protocol.py

```
"""Create a future-trial data header and explicitly assumed power sensitivity."""
from pathlib import Path
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import pandas as pd
from src.common import save_csv
from src.stage4_trial import planned_noninferiority_power
ROOT = Path(__file__).resolve().parents[1]
out = ROOT / 'outputs/stage4/protocol'; out.mkdir(parents=True, exist_ok=True)
targets = pd.read_csv(ROOT / 'outputs/stage4/audit/observed_panel_means_0_6.csv', nrows=0).columns[1:].tolist()
save_csv(pd.DataFrame(columns=['panelist', 'batch', 'condition', 'blind_code', 'session_date', 'style_Tripel_fit'] + targets), out / 'blank_panel_rating_template.csv')
rows = []
for sigma in [.4, .6, .8, 1., 1.2]:
    for margin in [.25, .5]:
        n = next((n for n in range(3, 501) if planned_noninferiority_power(n, sigma, margin) >= .8), None)
        rows.append({'assumed_SD_of_person_mean_paired_difference': sigma, 'noninferiority_margin': margin,
                     'one_sided_alpha_per_defect': .01, 'target_power': .8, 'minimum_independent_panelists': n,
                     'power_at_30': planned_noninferiority_power(30, sigma, margin), 'status': 'planning_assumptions_not_observed_trial_data'})
save_csv(pd.DataFrame(rows), out / 'sample_size_sensitivity.csv')
print('Blank template and assumed power sensitivity saved; no actual tasting data generated.')
```

scripts/render__equations.py

```
"""Typeset reference-paper equations; AI-assisted, OpenAI Codex, 2026-10-24."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
from matplotlib.mathtext import math_to_image

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "paper/equations"
OUT.mkdir(parents=True, exist_ok=True)
EQUATIONS = [
    r"z_{ij} = \frac{\operatorname{clip}(x_{ij};a_j,b_j)-m_j}{s_j}",
    r"z_{it}=\frac{s_{it}-\mu_t}{\sigma_t},\qquad s_{it}=\mu_t+\sigma_t z_{it}",
    r"\widehat{B}=\arg\min_B\left\{\Vert Y_c-ZB\Vert_F^2+\alpha\Vert B\Vert_F^2\right\}",
    r"\operatorname{MAE}=\frac{1}{nq}\sum_{i=1}^{n}\sum_{k=1}^{q}|y_{ik}-\widehat{y}_{ik}|",
    r"r_{ik}=|y_{ik}-f_k(x_i)|,\quad r_k^*=r_{(\lceil(m+1)(1-\alpha)\rceil),k}",
    r"C_k(x)=\left[f_k(x)-r_k^*,\ f_k(x)+r_k^*\right]",
    r"W_{ij}=r_{ij}\,\mathbf{1}(|r_{ij}|\geq\tau),\quad L=\operatorname{diag}\left(\sum_j |W_{ij}|\right)-W",
    r"\widehat{B}=\arg\min_B\left\{\Vert Y_c-ZB\Vert_F^2+\alpha\Vert B\Vert_F^2+\lambda\operatorname{tr}(B^\top LB)\right\}",
    r"\pi_{jk}=\frac{1}{R}\sum_{b=1}^{R}\mathbf{1}\!\left(j\in\operatorname{Top3}_k^{(b)}\right)",
    r"\Delta_{jk}=\operatorname{MAE}_k(\widehat{Y}^{\operatorname{perm}(j)})-\operatorname{MAE}_k(\widehat{Y})",
    r"h\mid c\sim\mathcal{N}\!\left(\mu_c,w_c\Sigma_c+(1-w_c)\Sigma_{\mathrm{pool}}\right),\quad w_c=\frac{n_c}{n_c+10}",
    r"x_{\mathrm{new}}=u x_a+(1-u)x_b,\quad u\sim\operatorname{Uniform}(0.1,0.9),\quad a,b\in c",
    r"x_{\mathrm{candidate},d}\leq x_{\mathrm{source},d},\qquad \forall d\in D",
    r"f_{\mathrm{stale}}(x_{\mathrm{candidate}})\leq f_{\mathrm{stale}}(x_{\mathrm{source}})",
]
for number, expression in enumerate(EQUATIONS, 1):
    math_to_image("$" + expression + "$", OUT / f"eq_{number:02d}.png", dpi=250, format="png", color="black")
print(f"Typeset {len(EQUATIONS)} equations")
```

scripts/revised__equations.py

```
"""Typeset new equations and retain the verified historical equations."""
from pathlib import Path
import shutil
import matplotlib
matplotlib.use('Agg')
from matplotlib.mathtext import math_to_image
ROOT=Path(__file__).resolve().parents[1]; out=ROOT/'paper/revised_20261025/equations'; out.mkdir(parents=True,exist_ok=True)
for p in (ROOT/'paper/equations').glob('*.png'): shutil.copyfile(p,out/p.name)
expressions={15:r'w_j=\operatorname{clip}\!\left(\frac{\operatorname{mean}_i(d_{ij}e_{ij})+\eta w_0}{\operatorname{mean}_i(d_{ij}^2)+\eta},0,1\right)',
```

```
16:r'g(y)=\sum_{k=1}^{6}\mathbf{1}(y>t_k),\quad\mathcal{G}(x)=\{g(L_y(x)),\ldots,g(U_y(x))\}',
17:r'\frac{\Vert Z_{\mathrm{candidate}}-Z_{\mathrm{source}}\Vert_2}{\sqrt{p}}\leq r,\quad r\in\{0.25,0.5,1,2\}'}
for number,expression in expressions.items(): math_to_image('$'+expression+'$',out/f'eq_{number:02d}.png',dpi=250,format='png',color='black')
print('Prepared 17 math figures')
```

scripts/stage2__figures.py

```
"""AI-assisted scientific plots: OpenAI Codex / GPT-6, 2026-10-24."""
from pathlib import Path
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
FIG = OUT / "figures"
FIG.mkdir(exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False, "savefig.facecolor": "white"})

def finish(fig, name):
    fig.savefig(FIG / name, dpi=180, bbox_inches="tight")
    plt.close(fig)

metrics = pd.read_csv(OUT / "q1/metrics_summary.csv").set_index("model")
names = ["mean", "ridge", "pls", "extra_trees", "gbr_fixed", "fixed_ridge_tree_blend"]
fig, ax = plt.subplots(figsize=(9.5, 4), layout="constrained")
ax.bar(np.arange(6), metrics.loc[names, "mae"], color=["#999999", "#3974a6", "#549978", "#bb8439", "#a45f87", "#728c99"])
ax.set_xticks(range(6), ["Mean", "Ridge", "PLS", "ExtraTrees", "GBR", "Fixed blend"])
ax.set(ylabel="Macro MAE on supplied continuous targets", title="Identical repeated held-out splits")
finish(fig, "model_comparison.png")

g = pd.read_csv(OUT / "q1/grade_rule_sensitivity.csv")
c = pd.read_csv(OUT / "q1/conformal_coverage.csv")
fig, ax = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
ax[0].bar(np.arange(6) + 1, 100 * g.any_rule_disagreement_fraction, color="#3974a6")
ax[0].set(xlabel="Held-out fold", ylabel="Disagreement across grading assumptions (%)", ylim=(0, 100))
ax[1].hist(c.empirical_test_coverage, bins=np.linspace(.6, 1, 9), color="#549978", edgecolor="white")
ax[1].axvline(.9, ls="--", color="black", label="Nominal marginal coverage")
ax[1].set(xlabel="Empirical coverage (50 test beers)", ylabel="Number of sensory targets")
ax[1].legend(fontsize=8)
finish(fig, "calibration_uncertainty.png")

t = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
good = t.evidence_status == "comparatively_supported_association"
fig, ax = plt.subplots(figsize=(9.5, 4), layout="constrained")
ax.scatter(t.loc[~good, "bootstrap_top3_frequency"], t.loc[~good, "heldout_permutation_MAE_increase"], s=23, alpha=.65, label="Weak or unstable")
ax.scatter(t.loc[good, "bootstrap_top3_frequency"], t.loc[good, "heldout_permutation_MAE_increase"], s=45, color="#b26044", label="Passed current descriptive criteria")
ax.axhline(0, color="grey", lw=.8)
ax.axvline(.5, color="grey", lw=.8, ls="--")
ax.set(xlabel="Bootstrap Top-3 frequency", ylabel="Held-out MAE increase after permutation", title="Association evidence for 150 selected factors")
ax.legend(fontsize=8)
finish(fig, "factor_evidence.png")

d = pd.read_csv(OUT / "q3/generator_comparison_by_split.csv")
fig, ax = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
styles = ["Tripel", "Kriek", "Lambic"]
methods = ["bootstrap", "local_convex", "PCA_Gaussian"]
for j, method in enumerate(methods):
    group = d[d.method == method].groupby("style").agg(mmd=("MMD2", "mean"), sd=("MMD2", "std"), prob=("target_style_probability", "mean")).reindex(styles)
    xx = np.arange(3) + (j - 1) * .23
    ax[0].bar(xx, group.mmd, width=.22, yerr=group.sd, capsize=3, label=method)
    ax[1].bar(xx, group.prob, width=.22, label=method)
for a in ax:
    a.set_xticks(range(3), styles)
ax[0].set(ylabel="MMD squared (mean and split SD)", title="Held-out distribution distance")
ax[1].set(ylabel="Target-style model probability", title="Internal classifier diagnostic", ylim=(0, 1.05))
ax[0].legend(fontsize=7)
finish(fig, "generator_comparison.png")

a = pd.read_csv(ROOT / "outputs/stage1/q3/transfer_diagnostics.csv").set_index("beer")
b = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
fig, ax = plt.subplots(figsize=(10, 4), layout="constrained")
xx = np.arange(len(b))
ax.plot(xx, b.source_trip_probability, "o--", color="#999999", label="Source")
ax.plot(xx, a.loc[b.beer, "candidate_target_probability"], "s-", color="#3974a6", label="PCA direction search")
ax.plot(xx, b.candidate_trip_probability, "^-", color="#b26044", label="Expanded conditional-anchor search")
for i, row in b.iterrows():
    if not row.all_source_defects_observed:
        ax.annotate("missing source diacetyl", (i, row.candidate_trip_probability), xytext=(i-.8, .72-.06*(i%2)), fontsize=7, arrowprops={"arrowstyle":"-", "lw":.5})
ax.set_xticks(xx, b.beer.astype(str), rotation=45)
ax.set(xlabel="Source Pils/Lager beer", ylabel="Tripel classifier probability", ylim=(0, 1.09), title="Model-based transfer; actual sensory response remains unverified")
ax.legend(fontsize=8, loc="center right")
finish(fig, "transfer_comparison.png")

counts = pd.read_csv(OUT / "audit/style_counts.csv").sort_values("n")
fig, ax = plt.subplots(figsize=(8, 6), layout="constrained")
ax.barh(counts.tasting_category_fine, counts.n, color="#3974a6")
ax.set(xlabel="Observed beers", title="Small and unbalanced style samples")
finish(fig, "style_counts.png")
print("Created six stage-two figures")
```

scripts/stage3__figures.py

```
"""Publication figures from actual stage-three outputs; AI-assisted, 2026-10-25."""
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'; FIG=OUT/'figures'; FIG.mkdir(exist_ok=True)
plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False,'savefig.dpi':200})
def save(fig,name): fig.savefig(FIG/name,bbox_inches='tight'); plt.close(fig)

metrics=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); names=['mean','ridge','extra_trees','gbr','fixed_half_blend','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']
labels=['Mean','Ridge','ET','GBR','Half blend','Global blend','Target blend','Shrunk blend']
fig,axes=plt.subplots(1,2,figsize=(11,3.5))
for ax,scope in zip(axes,['core30','all50']):
    a=metrics[metrics.scope==scope].set_index('model').loc[names]; ax.bar(np.arange(len(a)),a.mae,color=['#999999','#779ab2','#3978a8','#b36b73','#8ca98c','#87a7b7','#b79a75','#477e69']); ax.set_xticks(np.arange(len(a)),labels,rotation=40,ha='right'); ax.set_ylabel('MAE on supplied continuous targets'); ax.set_title('Core Aroma/Flavour: 30 targets' if scope=='core30' else 'All 50 targets')
save(fig,'prediction_by_scope.png')

coverage=pd.read_csv(OUT/'q1/conformal_coverage.csv'); fig,axes=plt.subplots(1,2,figsize=(10,3.2))
axes[0].hist(coverage.continuous_coverage,bins=np.linspace(.5,1,11),color='#3978a8'); axes[0].axvline(.9,color='black',ls='--'); axes[0].set_xlabel('Observed marginal continuous coverage'); axes[0].set_ylabel('Target-split combinations')
axes[1].hist(coverage.mean_grade_set_size,bins=10,color='#477e69'); axes[1].set_xlabel('Mean size of relative ordinal interval hull'); axes[1].set_ylabel('Target-split combinations'); fig.suptitle('Five train/calibration/test splits; relative grades are not raw intensity')
save(fig,'ordinal_uncertainty.png')

graph=pd.read_csv(OUT/'q2/graph_comparison.csv'); params=pd.read_csv(OUT/'q2/selected_parameters.csv'); fig,axes=plt.subplots(1,2,figsize=(10,3.3))
a=graph[graph.scope=='core30'].set_index('graph').loc[['none','real','shuffled']]; axes[0].bar(['No graph','Real graph','Shuffled graph'],a['mean'],color=['#999999','#3978a8','#b36b73']); axes[0].set_ylim(0,.45); axes[0].set_ylabel('Mean outer-fold core MAE')
for j,v in enumerate(a['mean']): axes[0].text(j,v+.006,f'{v:.6f}',ha='center',fontsize=9)
p=params[params.graph=='real']['lambda'].value_counts().sort_index(); axes[1].bar(p.index.astype(str),p.values,color='#477e69'); axes[1].set_xlabel('
```

```
Graph penalty selected inside training'); axes[1].set_ylabel('Outer folds (15 total)'); fig.suptitle('Graph regularization ablation; no meaningful predictive gain observed')
save(fig,'graph_ablation.png')

top=pd.read_csv(OUT/'q2/top3_with_evidence.csv'); fig,ax=plt.subplots(figsize=(7,3.6)); good=top.evidence_status.str.startswith('comparatively'); ax.scatter(top.loc[~good,'bootstrap_frequency_conditional_on_full_selected_params'],top.loc[~good,'within_style_delta_on_selected_folds'],s=16,color='#84a9c2',label='Weak or incomplete'); ax.scatter(top.loc[good,'bootstrap_frequency_conditional_on_full_selected_params'],top.loc[good,'within_style_delta_on_selected_folds'],s=24,color='#b85e43',label='Current descriptive criteria'); ax.axhline(0,color='gray',lw=.7); ax.axvline(.5,color='gray',ls='--',lw=.7); ax.set_xlabel('Conditional bootstrap Top-3 frequency'); ax.set_ylabel('Within-style held-out permutation MAE change'); ax.legend(fontsize=8); save(fig,'factor_crossfit.png')

gen=pd.read_csv(OUT/'q3/generation_comparison.csv'); methods=['bootstrap','local_convex','constrained_local_convex','PCA_Gaussian']; styles=['Tripel','Kriek','Lambic']; colors=['#999999','#3978a8','#477e69','#b38a58']; fig,axes=plt.subplots(1,2,figsize=(11,3.5)); width=.19
for k,(method,color) in enumerate(zip(methods,colors)):
    a=gen[gen.method==method].groupby('style').mean(numeric_only=True).loc[styles]; x=np.arange(3)+(k-1.5)*width
    axes[0].bar(x,a.MMD2_fixed_training_bandwidth,width,color=color,label=method); axes[1].bar(x,a.independent_evaluator_target_prediction_fraction,width,color=color)
for ax in axes: ax.set_xticks(np.arange(3),styles)
axes[0].set_ylabel('MMD squared (shared training bandwidth)'); axes[1].set_ylabel('Independent evaluator predicts target style'); axes[1].set_ylim(0,1.05); axes[0].legend(fontsize=7); save(fig,'generation_independent.png')

transfer=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); fig,ax=plt.subplots(figsize=(7.5,3.6)); radii=sorted(transfer.radius.unique()); values=[]
for r in radii:
    a=transfer[transfer.radius==r]; t=a[a.source_role=='T_real_holdout']; values.append([(a.status!='no_feasible_nontrivial_candidate_source_retained').mean(),a.evaluator_proxy_acceptance.mean(),t.evaluator_proxy_acceptance.mean()])
values=np.array(values)
for j,(label,color) in enumerate([('Search model feasible (55 records)','#3978a8'),('Independent proxy acceptance (55 records)','#b36b73'),('Real-holdout source proxy acceptance (10 records)','#477e69')]): ax.plot(radii,values[:,j],marker='o',label=label,color=color)
ax.set_xlabel('Standardized change radius'); ax.set_ylabel('Fraction of records (repeated beers across splits)'); ax.set_ylim(-.03,1.03); ax.legend(fontsize=8,loc='upper left'); save(fig,'transfer_independent.png')

weights=pd.read_csv(OUT/'q1/blend_weights.csv'); fig,ax=plt.subplots(figsize=(7,3)); core=weights.target.str.startswith(('A_','F_')); ax.hist([weights.loc[core,'weight_GBR'],weights.loc[~core,'weight_GBR']],bins=np.linspace(0,1,11),label=['Core 30','Additional 20'],color=['#3978a8','#b38a58'],alpha=.8); ax.set_xlabel('GBR weight learned inside outer training only'); ax.set_ylabel('Target-fold combinations'); ax.legend(); save(fig,'blend_weights.png')
print('Saved seven stage-three figures')
```

scripts/stage3_preflight.py

```
"""Read-only source audit and conditional analysis of existing OOF predictions.

Run from the bundled document runtime (openpyxl + numpy). No workbook is edited.
This is a protocol preflight, not the full stage-three experiment runner.
"""
from pathlib import Path
import argparse
import csv
import hashlib
import json
import re
import sys
import xml.etree.ElementTree as ET
import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.stage3_algorithms import paired_oof_bootstrap

parser = argparse.ArgumentParser()
parser.add_argument("--data-dir", type=Path, default=ROOT.parent / "materials/A/附件")
parser.add_argument("--out", type=Path, default=ROOT / "outputs/stage3_design")
args = parser.parse_args()
DATA, OUT = args.data_dir, args.out
OUT.mkdir(parents=True, exist_ok=True)


def read_csv(path):
    with path.open(encoding="utf-8-sig", newline="") as stream:
        return list(csv.DictReader(stream))


def write(name, value):
    (OUT / name).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


original_hashes = {name: hashlib.sha256((DATA / name).read_bytes()).hexdigest() for name in ["1.csv", "3.xlsx", "4.csv", "8.xlsx"]}
previous_hashes = {r['file']:r['sha256'] for r in read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')}
if any(previous_hashes.get(name) != sha for name,sha in original_hashes.items()):
    raise ValueError("Source version differs from the existing stage-two predictions/evidence")
target_rows = read_csv(DATA / "4.csv")
targets = [c for c in target_rows[0] if c not in ["beer", "beer_id", "tasting_category_fine"]]
aliases = {"A_ester_fruity": "A_esters_fruity", "F_ester_fruity": "F_esters_fruity", "4vg": "X4vg"}
workbook = load_workbook(DATA / "3.xlsx", data_only=False, read_only=True)
summary = workbook["summary"]
mapping, anomalies = [], []
for col in range(2, summary.max_column+1):
    original = summary.cell(1,col).value
    target = aliases.get(original,original)
    if target not in targets:
        continue
    formula = str(summary.cell(2,col).value)
    match = re.fullmatch(r"=beer_1!\$([A-Z]+)\$(\d+)",formula)
    if not match:
        raise ValueError(f"Unexpected score formula for {target}: {formula}")
    column, row = match.group(1), int(match.group(2))
    expected_cell = "AG90" if target == "barnyard" else f"{column}{row}"
    section = ("Aroma_core" if target.startswith("A_") else "Flavour_core" if target.startswith("F_") else
               "Faults" if target in ["dms","metallic","X4vg","t2n","diacetyl","stale_hops"] else
               "Spiced_beers" if target in ["orange","coriander","clove"] else
               "Lambic_Geuze" if target in ["lactic","acetic","barnyard"] else
               "Overall" if target == "overall" else "Palate_or_basic_taste")
    mapping.append({"target":target,"summary_header":original,"summary_column":col,"original_formula":formula,
                    "expected_score_cell":expected_cell,"section":section,
                    "formula_status":"suspected_wrong_reference_derived_map_corrects_only" if target=='barnyard' else "consistent_with_form_position",
                    "official_submission_scope":"requires_explicit_reporting_core_30_and_full_50_not_inferred_from_count"})
    if target == "barnyard":
        assert "Barnyard" in str(workbook["beer_1"]["R89"].value)
        assert str(workbook["beer_1"]["AG90"].value).startswith("=")
        for i in range(2,14):
            actual=str(summary.cell(i,col).value)
            expected=f"=beer_{i-1}!$AG$90"
            if actual != expected:
                anomalies.append({"summary_cell":summary.cell(i,col).coordinate,"target":target,"actual_formula":actual,"form_position_expected_formula":expected})
assert len(mapping) == len(targets) == 50 and {r['target'] for r in mapping} == set(targets)
write("scoring_field_map.json",mapping)
write("scoring_scope_audit.json",{"mapped_targets":50,"core_Aroma":15,"core_Flavour":15,"additional_targets":20,
    "aliases":aliases,"suspected_formula_mismatches":anomalies,
    "card_items_without_separate_supervised_targets":[{"card_label":"Acetaldehyde under Aroma","score_cell":"Q45"},{"card_label":"Acetaldehyde under Flavour","score_cell":"AG45"}],
    "original_files_edited":False,"absolute_score_anchors_available":False,
    "scope_caveat":"30 is the prefixed core comparison scope, not a ruling that only 30 answers are required."})

scales = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/field_scales.csv")}
chem = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/chemical_dictionary.csv")}
registry=[]
for field in ["DMS","X2.3.butanedione","X4VG","X.E..2.nonenal"]:
    scale,meta=scales[field],chem[field]
    try: threshold=float(meta['threshold_median'])
    except (TypeError,ValueError): threshold=None
    if threshold is not None and not np.isfinite(threshold): threshold=None
    supported=scale['transform_evidence']=='verified_log10_positive_pairs'
    registry.append({"field":field,"unit_in_attachment":meta['unit'],"threshold_median_in_attachment":threshold,
        "transformation_evidence":scale['transform_evidence'],"positive_pair_
```

```
    log10_supported":supported,
        "concentration_rule_if_eligible":"10**x" if supported else "unresolved_per_sample_check_needed",
        "zero_LOD_or_substitution_provenance":"not_fully_verified",
        "absolute_defect_certification":"not_complete_even_if_positive_pairs_match",
        "next_action":"audit_per_sample_calibrated_pairs_and_detection_limits" if supported else "resolve_mismatching_pairs_and_scale_before_concentration_claim"})
write("defect_registry_preflight.json",registry)

beer_ids=[int(r['beer']) for r in target_rows]
truth=np.array([[float(r[t]) for t in targets] for r in target_rows])
def predictions(path):
    rows=read_csv(path)
    bybeer={int(r['beer']):r for r in rows}
    if len(bybeer)!=len(rows) or set(bybeer)!=set(beer_ids): raise ValueError("OOF keys do not align uniquely")
    return np.array([[float(bybeer[b][t]) for t in targets] for b in beer_ids])
gbr=predictions(ROOT/'outputs/stage2/q1/oof_gbr_continuous.csv')
et=predictions(ROOT/'outputs/stage1/q1/oof_extra_trees_continuous.csv')
labels=[r['tasting_category_fine'] for r in target_rows]
reports=[]
for scope,indices in [('all_50',list(range(50))),('prefixed_core_30',[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))])]:
    result=paired_oof_bootstrap(truth[:,indices],gbr[:,indices],et[:,indices],labels)
    result.update(scope=scope,comparison='fixed_stage2_GBR_minus_stage1_ExtraTrees',metric='macro_MAE',
                  uncertainty_scope='beer-resampling_conditional_on_existing_predictions_not_fresh_external_validation')
    reports.append(result)
write('existing_oof_paired_pilot.json',reports)

test_report=ROOT/'tmp/stage3_tests.xml'
suite=ET.parse(test_report).getroot().find('testsuite')
assert all(int(suite.attrib[k])==0 for k in ['failures','errors','skipped'])
for name,old in original_hashes.items():
    assert hashlib.sha256((DATA/name).read_bytes()).hexdigest()==old
manifest={'date':'2026-10-25','stage':'design_and_preflight','complete_stage3_training_run':False,
    'source_sha256':original_hashes,'protocol_sha256':hashlib.sha256((ROOT/'config_stage3_design.json').read_bytes()).hexdigest(),
    'algorithm_tests':{k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']},
    'no_human_validation_claimed':True,'no_original_workbook_edited':True,
    'full_experiments_status':'planned_not_executed; only source mapping and conditional existing-OOF pilot executed'}
write('preflight_manifest.json',manifest)
print(json.dumps({'mapped_targets':len(mapping),'formula_mismatches':len(anomalies),'paired_pilot':reports,'test_summary':manifest['algorithm_tests']},ensure_ascii=False))
```

scripts/stage4__figures.py

```
"""Scientific figures for the observed-score revision; no synthetic observations."""
from pathlib import Path
import shutil
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]; OUT = ROOT / 'outputs/stage4'; DEST = OUT / 'figures'; DEST.mkdir(parents=True, exist_ok=True)
plt.rcParams.update({'font.size': 9, 'axes.spines.top': False, 'axes.spines.right': False, 'savefig.dpi': 210})

def finish(fig, name):
    fig.tight_layout(); fig.savefig(DEST / name, bbox_inches='tight'); plt.close(fig)

metrics = pd.read_csv(OUT / 'prediction/metrics_by_scope.csv')
names = ['mean', 'ridge', 'extra_trees', 'gbr', 'fixed_half_blend', 'concentration_ET', 'standardized_ET_train_isotonic']
labels = ['Mean', 'Ridge', 'ET', 'GBR', 'Half blend', '61-field ET', 'ET + isotonic']
fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
for ax, scope in zip(axes, ['core30', 'all50']):
    a = metrics[metrics.scope == scope].set_index('model').reindex(names)
    ax.bar(np.arange(len(a)), a.mae, color=['#888888', '#7C9BB8', '#4B8BB5', '#BF7277', '#4F8B70', '#B5976F', '#907AAB'])
    ax.set_xticks(np.arange(len(a)), labels, rotation=32, ha='right'); ax.set_ylabel('MAE on observed 0-6 panel means'); ax.set_title(scope)
finish(fig, 'absolute_prediction.png')

ordinal = pd.read_csv(OUT / 'ordinal/metrics_by_scope.csv')
fig, axes = plt.subplots(1, 2, figsize=(10, 3.2))
for k, scope in enumerate(['core30', 'all50']):
    a = ordinal[ordinal.scope == scope].set_index('method')
    for offset, (method, name, color) in enumerate([('training_population_distribution', 'Population baseline', '#999999'), ('ET_rating_distribution', 'ET rating distribution', '#4B8BB5')]):
        axes[0].bar(k + (offset - .5) * .28, a.loc[method, 'RPS'], width=.27, label=name if k == 0 else None, color=color)
        axes[1].bar(k + (offset - .5) * .28, a.loc[method, 'nonzero_panel_mode_accuracy'], width=.27, label=name if k == 0 else None, color=color)
axes[0].set_ylabel('Ranked probability score / 6 (lower is better)'); axes[1].set_ylabel('Accuracy among nonzero observed modes')
for ax in axes: ax.set_xticks([0, 1], ['core30', 'all50']); ax.legend(fontsize=8)
finish(fig, 'ordinal_distribution.png')

errors = pd.read_csv(OUT / 'audit/sensory_reconstruction_errors.csv')
fig, ax = plt.subplots(figsize=(10, 2.7))
ax.plot(np.arange(1, 51), np.maximum(errors.max_error_without_fitted_replacement, 1e-12), '.', label='Before constant-cell reconstruction', color='#B56766')
ax.plot(np.arange(1, 51), np.maximum(errors.max_error_after_conditional_reconstruction, 1e-12), '.', label='After conditional reconstruction', color='#4F8B70')
ax.set_yscale('log'); ax.set_xlabel('Sensory attribute index (all 50 retained)'); ax.set_ylabel('Maximum absolute discrepancy'); ax.legend(fontsize=8)
finish(fig, 'source_reconciliation.png')

top = pd.read_csv(OUT / 'raw_graph/q2/top3_with_evidence.csv').rename(columns={
    'bootstrap_frequency_conditional_on_full_selected_params': 'bootstrap_frequency_fixed_alpha',
    'within_style_delta_on_selected_folds': 'within_style_delta_mae'})
top['descriptive_support'] = top.evidence_status.str.startswith('comparatively')
fig, ax = plt.subplots(figsize=(7.8, 3.1))
for flag, name, color in [(False, 'Remaining candidates', '#BBBBBB'), (True, 'Descriptive support', '#4F8B70')]:
    a = top[top.descriptive_support == flag]
    ax.scatter(a.bootstrap_frequency_fixed_alpha, a.within_style_delta_mae, color=color, s=15, alpha=.8, label=name)
ax.axhline(0, color='#777777', linewidth=.7); ax.axvline(.5, color='#777777', linewidth=.7, linestyle='--'); ax.set_xlabel('Bootstrap Top-3 frequency (fixed selected alpha)'); ax.set_ylabel('Held-out within-style permutation delta MAE'); ax.legend(fontsize=8)
finish(fig, 'raw_factor_support.png')

oav = pd.read_csv(OUT / 'audit/reference_oav_sensitivity.csv'); oav = oav[oav.threshold_multiplier == 1]
fig, ax = plt.subplots(figsize=(8.7, 3.1)); fs = ['DMS', 'X2.3.butanedione', 'X4VG', 'X.E..2.nonenal']
arrays = [np.log10(a[a > 0]) for f in fs for a in [oav[oav.field == f].reference_oav.dropna().to_numpy()]]
ax.boxplot(arrays, tick_labels=['DMS', 'Diacetyl', '4VG', 'E-2-nonenal'], showfliers=True, flierprops={'markersize': 2})
ax.axhline(0, color='#B56766', linestyle='--', linewidth=1); ax.set_ylabel('log10(reference OAV), positive reports only')
ax.set_title('Reference thresholds are conditional; assay LOD and matrix are unresolved', fontsize=9)
finish(fig, 'reference_oav.png')

transfer = pd.read_csv(OUT / 'transfer/frozen_stage3_candidates_raw_score_check.csv'); transfer = transfer[transfer.radius == 1]
fig, axes = plt.subplots(1, 2, figsize=(9.5, 3))
for ax, data, title in [(axes[0], transfer, 'All source roles: 55 repeated records'), (axes[1], transfer[transfer.source_role == 'T_real_holdout'], 'T sources: 10 records from 7 distinct beers')]:
    values = [len(data), int(data.original_search_feasible.sum()), int(data.original_proxy_acceptance.sum()), int(data.original_and_raw_proxy_support.sum())]
    ax.bar(np.arange(4), values, color=['#999999', '#7C9BB8', '#B5976F', '#4F8B70']); ax.set_xticks(np.arange(4), ['Source', 'G feasible', 'Original J', '+ raw G/J'], rotation=20)
    ax.set_ylim(0, len(data) * 1.18); ax.set_title(title, fontsize=9)
    for i, v in enumerate(values): ax.text(i, v + len(data) * .025, str(v), ha='center')
finish(fig, 'raw_transfer_recheck.png')

paper = ROOT / 'paper/revised_absolute_20261025'; eq = paper / 'equations'; eq.mkdir(parents=True, exist_ok=True)
for p in (ROOT / 'paper/revised_20261025/equations').glob('*.png'): shutil.copyfile(p, eq / p.name)
formulas = {
    6: r'$C_j(x)=[f_j(x)-q_j,\;f_j(x)+q_j]\cap[0,6]$',
    18: r'$\bar{s}_{bj}=\frac{1}{|T_b|}\sum_{t\in T_b}\left(\frac{1}{n_{bt}}\sum_{r=1}^{n_{bt}}s_{btrj}\right)$',
    19: r'$q_{bjk}=\frac{1}{|T_b|}\sum_{t\in T_b}\frac{1}{n_{bt}}\sum_{r=1}^{n_{bt}}\mathbf{1}(s_{btrj}=k),\quad k=0,\ldots,6$',
    20: r'$\hat{g}^{\rm mode}_{bj}=\arg\max_k\hat{q}_{bjk},\qquad\hat{g}^{\rm med}_{bj}=\min\{k:\sum_{h=0}^{k}\hat{q}_{bjh}\geq0.5\}$',
    21: r'$\mathrm{RPS}=\frac{1}{6nq}\sum_{b,j}\sum_{k=0}^{5}\left(\sum_{h=0}^{k}\hat{q}_{bjh}-\sum_{h=0}^{k}q_{bjh}\right)^2$',
```

```
    22: r'$\max_{w\geq0}\;\mathbf{1}^{T}w\quad\mathrm{s.t.}\quad(C_A-c_s\
mathbf{1}^{T})_Dw\leq0,\;\mathbf{1}^{T}w\leq0.75$',
    23: r'$c_{\rm mix}=(1-\mathbf{1}^{T}w)c_s+C_Aw$',
    24: r'$U_j=\bar d_j+t_{1-\alpha_j,n-1}\frac{s_{d_j}}{\sqrt{n}},\qquad U_j
<\Delta$',
}
for number, formula in formulas.items():
    fig = plt.figure(figsize=(11, 1.1)); fig.text(.5, .5, formula, fontsize
=16, ha='center', va='center'); fig.savefig(eq / f'eq_{number:02d}.png', dpi
=220, bbox_inches='tight', transparent=False); plt.close(fig)
print('Six result figures and eight updated equation assets generated.')
```

scripts/stage4__graph__comparison.py

```
"""Repeat the full graph control experiment on the newly observed raw
endpoint."""
from pathlib import Path
import os
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import json
import argparse
from dataclasses import replace
import pandas as pd
from src.common import environment, write_json, sha256
from src.data import load_data, find_data
from src.stage3_graph import run_graph
ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser(); parser.add_argument('--data-dir'); args =
 parser.parse_args()
cfg = json.loads((ROOT / 'config_stage3.json').read_text(encoding='utf-8'))
cfg.update(json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf
-8')))
data = load_data(find_data(args.data_dir))
panel = pd.read_csv(ROOT / 'outputs/stage4/audit/observed_panel_means_0_6.csv
').set_index('beer').reindex(data.meta.beer)
raw_data = replace(data, Y=pd.DataFrame(panel.to_numpy(), columns=data.Y.
columns))
destination = ROOT / 'outputs/stage4/raw_graph'
run_graph(raw_data, destination, cfg)
write_json(destination / 'run_manifest.json', {**environment(), 'config': cfg
, 'target': 'observed_raw_equal_panelist_mean_0_6',
                                               'code_sha256': {p.relative_to(
ROOT).as_posix(): sha256(p) for p in [Path(__file__), ROOT / 'src/stage3_
graph.py']}})
```

scripts/verify__absolute__paper.py

```
"""Saved-PDF structure and source appendix verification for the raw-score
revision."""
from pathlib import Path
import hashlib
import json
import re
from pypdf import PdfReader
import pdfplumber
ROOT = Path(__file__).resolve().parents[1]; PAPER = ROOT / 'paper/revised_
absolute_20261025'
reports = []
for name in ['论文完善稿.pdf', 'AI工具使用详情说明_更新版.pdf']:
    path = PAPER / name; reader = PdfReader(path); texts = [p.extract_text()
for p in reader.pages]
    assert reader.metadata.get('/Author', '') == '' and '目 录' in texts[1]
and texts[2].lstrip().startswith('1\n1 ')
    assert all('\x00' not in t for t in texts) and not any(re.search(r'[A-Za-
z0-9_.+-]+@qq\.com', t) for t in texts)
    overflow = []
    with pdfplumber.open(path) as document:
        for i, page in enumerate(document.pages):
            bad = [c for c in page.chars if c.get('text', '').strip() and (c
['x0'] < 67 or c['x1'] > float(page.width) - (60 if c.get('text') in '
。，、；： ' else 67) or c['top'] < 43 or c['bottom'] > float(page.height) -
25)]
            if bad: overflow.append({'page': i + 1, 'sample': ''.join(c['text
'] for c in bad[:40])})
    assert not overflow, overflow
    destinations = [p for p in reader.outline if not isinstance(p, list)]
    appendix = next((reader.get_destination_page_number(p) + 1 for p in
destinations if p.title.startswith('附录A')), None)
    if appendix: assert appendix - 3 <= 30
    reports.append({'file': name, 'physical_pages': len(reader.pages), '
appendix_A_starts_physical_page': appendix,
                    'missing_glyphs': 0, 'overflow_pages': 0, 'sha256':
hashlib.sha256(path.read_bytes()).hexdigest()})
content = json.loads((PAPER / 'paper_content.json').read_text(encoding='utf
-8'))
numbers = [b['number'] for b in content['blocks'] if b['type'] == 'equation
']; assert numbers == list(range(1, len(numbers) + 1))
manifest = json.loads((PAPER / 'source_appendix_manifest.json').read_text(
encoding='utf-8'))
for row in manifest: assert hashlib.sha256((ROOT / row['file']).read_bytes())
.hexdigest() == row['sha256']
report = {'documents': reports, 'numbered_equations': len(numbers), 'source_
files': len(manifest), 'source_lines': sum(row['lines'] for row in manifest),
          'source_hashes_match': True, 'visual_review': 'pending_rendered_
page_inspection'}
(PAPER / 'pdf_validation.json').write_text(json.dumps(report, ensure_ascii=
False, indent=2), encoding='utf-8')
print(json.dumps(report, ensure_ascii=False))
```

scripts/verify__reference.py

```
"""Validate the actual paper and saved evidence, without publishing host
identifiers."""
from pathlib import Path
import csv
import hashlib
import json
import xml.etree.ElementTree as ET
from pypdf import PdfReader

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
reader = PdfReader(ROOT / "paper/论文参考稿.pdf")
texts = [page.extract_text() for page in reader.pages]
assert len(reader.pages) == 13
assert "摘 要" in texts[0] and "目 录" in texts[1]
assert "1 问题重述与研究范围" in texts[2]
assert all("\x00" not in text for text in texts)
assert reader.metadata.get("/Author", "") == ""
assert all("github.com" not in text and "@qq.com" not in text for text in
texts)
assert all(abs(float(p.mediabox.width) - 595.2756) < 0.1 for p in reader.
pages)
content = json.loads((ROOT / "paper/paper_content.json").read_text(encoding="
utf-8"))
for block in content["blocks"]:
    if block["type"] == "heading" and block["level"] == 1:
        title = block["text"]
        expected = next(i-1 for i, page in enumerate(texts[2:], 2) if title
in page)
        destination = next(item for item in reader.outline if not isinstance(
item, list) and item.title == title)
        assert reader.get_destination_page_number(destination)-1 == expected
with (OUT / "q1/metrics_summary.csv").open(encoding="utf-8-sig", newline="")
as f:
    metrics = list(csv.DictReader(f))
gbr = next(row for row in metrics if row["model"] == "gbr_fixed")
assert abs(float(gbr["mae"]) - 0.339759) < 0.000001
suite = ET.parse(OUT / "tests.xml").getroot().find("testsuite")
assert int(suite.attrib["tests"]) == 23
assert all(int(suite.attrib[k]) == 0 for k in ["errors", "failures", "skipped
"])
summary = {
    "date": "2026-10-24",
    "tests": {k: int(suite.attrib[k]) for k in ["tests", "errors", "failures
", "skipped"]},
    "test_names": [item.attrib["name"] for item in suite.findall("testcase")
],
    "paper": {"physical_pages": len(reader.pages), "abstract_page": 1, "
contents_page": 2,
              "body_starts_physical_page": 3, "body_starts_printed_page": 1,
              "equations": sum(b["type"] == "equation" for b in content["
blocks"]),
              "figures": sum(b["type"] == "figure" for b in content["blocks
"]),
              "missing_glyph_nulls": 0, "author_metadata_empty": True,
              "visual_review": "all 13 rendered pages inspected; final
amendment on defect names reviewed",
              "sha256": hashlib.sha256((ROOT / "paper/论文参考稿.pdf").read_
bytes()).hexdigest()},
    "scientific_scope": "Exploratory AI-assisted reference; no absolute-score
 recovery, causal or tasting validation claimed."
}
(OUT / "validation_summary.json").write_text(json.dumps(summary, ensure_ascii
=False, indent=2), encoding="utf-8")
files = [p for p in OUT.rglob("*") if p.is_file() and p.name not in ["tests.
xml", "result_checksums.json"] and "private" not in p.parts]
checksums = [{"file": p.relative_to(OUT).as_posix(), "sha256": hashlib.sha256
(p.read_bytes()).hexdigest()} for p in sorted(files)]
```

```
(OUT / "result_checksums.json").write_text(json.dumps(checksums, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps({"pages": len(reader.pages), "tests_passed": 23, "hashed_results": len(checksums)}, ensure_ascii=False))
```

scripts/verify__revised__paper.py

```
"""Check actual revised PDFs, anonymous metadata, pagination and source appendix."""
from pathlib import Path
import hashlib
import json
import re
from pypdf import PdfReader
import pdfplumber
ROOT=Path(__file__).resolve().parents[1]; PAPER=ROOT/'paper/revised_20261025'
reports=[]
for name in ['论文修订稿.pdf','AI工具使用详情说明_阶段版.pdf']:
    path=PAPER/name; reader=PdfReader(path); texts=[p.extract_text() for p in reader.pages]
    assert reader.metadata.get('/Author','')=='' and '目 录' in texts[1]
    assert texts[2].lstrip().startswith('1\n1 ')
    assert all('\x00' not in t for t in texts)
    assert not any(re.search(r'[A-Za-z0-9_.+-]+@qq\.com',t) for t in texts)
    assert all(abs(float(p.mediabox.width)-595.2756)<.1 for p in reader.pages)
    headings=[p for p in reader.outline if not isinstance(p,list)]
    appendix=next((reader.get_destination_page_number(p)+1 for p in headings if p.title.startswith('附录A')),None)
    overflow=[]
    with pdfplumber.open(path) as pdf:
        for i,page in enumerate(pdf.pages):
            # CJK hanging punctuation includes a wide side-bearing in its advance box.
            bad=[c for c in page.chars if c.get('text','').strip() and (c['x0'] < 67 or c['x1'] > float(page.width)-(60 if c.get('text') in '。，、；： ' else 67) or c['top']<43 or c['bottom']>float(page.height)-25)]
            if bad: overflow.append({'page':i+1,'count':len(bad),'sample':''.join(c['text'] for c in bad[:60])})
    assert not overflow,overflow
    reports.append({'file':name,'physical_pages':len(reader.pages),'body_starts_page':3,'appendix_A_starts_physical_page':appendix,'missing_glyph_nulls':0,'text_overflow_pages':0,
                    'sha256':hashlib.sha256(path.read_bytes()).hexdigest()})
content=json.loads((PAPER/'paper_content.json').read_text(encoding='utf-8')); numbers=[b['number'] for b in content['blocks'] if b['type']=='equation']
assert numbers==list(range(1,len(numbers)+1))
manifest=json.loads((PAPER/'source_appendix_manifest.json').read_text(encoding='utf-8'))
paths=next(b['paths'] for b in content['blocks'] if b['type']=='source_files')
assert {p['file'] for p in manifest}==set(paths)
for row in manifest: assert hashlib.sha256((ROOT/row['file']).read_bytes()).hexdigest()==row['sha256']
report={'client_date':'2026-10-25','documents':reports,'numbered_equations':len(numbers),'source_files':len(manifest),'source_lines':sum(r['lines'] for r in manifest),
        'source_appendix_matches_current_files':True,'visual_review':'rendered_pages_must_be_inspected_before_delivery'}
(PAPER/'pdf_validation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(report,ensure_ascii=False))
```

scripts/verify__stage3.py

```
"""Cross-check saved scientific artifacts and record exact executed scope."""
from pathlib import Path
import hashlib
import json
import xml.etree.ElementTree as ET
import numpy as np
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'
suite=ET.parse(ROOT/'tmp/stage3_full_tests.xml').getroot().find('testsuite')
tests={k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']}
assert tests['tests']>=40 and tests['failures']==tests['errors']==tests['skipped']==0
status={part:json.loads((OUT/part/'status.json').read_text()) for part in ['q1','q2','q3']}
assert all(a['complete'] for a in status.values())
roles=pd.read_csv(OUT/'q1/outer_roles.csv')
assert len(roles)==1250 and roles['repeat'].nunique()==5
assert (roles.groupby(['repeat','beer']).size()==1).all()
grades=pd.read_csv(OUT/'q1/ASSUMPTION_oof_relative_grades.csv'); a=grades.drop(columns='beer').to_numpy()
assert a.shape==(250,50) and np.isfinite(a).all() and np.all((a>=0)&(a<=6)&(a==a.astype(int)))
weights=pd.read_csv(OUT/'q1/blend_weights.csv'); assert len(weights)==750 and weights.weight_GBR.between(0,1).all()
metadata=pd.read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')
from sys import path
path.insert(0,str(ROOT))
from src.data import find_data,load_data
original=find_data(); data=load_data(original)
for row in metadata.itertuples(): assert hashlib.sha256((original/row.file).read_bytes()).hexdigest()==row.sha256
scope=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); core=[c for c in data.Y if c.startswith(('A_','F_'))]
for model in scope.model.unique():
    pred=pd.read_csv(OUT/f'q1/oof_{model}.csv').set_index('beer').loc[data.meta.beer]
    for name,fields in [('core30',core),('all50',list(data.Y))]:
        expected=float(np.abs(data.Y[fields].to_numpy()-pred[fields].to_numpy()).mean())
        actual=float(scope[(scope.model==model)&(scope.scope==name)].mae.iloc[0]); assert abs(expected-actual)<2e-8
graph=pd.read_csv(OUT/'q2/graph_fold_metrics.csv'); assert graph.graph.notna().all() and len(graph[graph.graph=='shuffled'])==300
gen=pd.read_csv(OUT/'q3/generation_comparison.csv'); assert len(gen)==60 and (gen.n_generated==40).all()
assert (gen.provided_bounds_violation_fraction==0).all()
transfer=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); assert len(transfer)==220
assert not transfer.absolute_chemical_certification.any() and not transfer.real_tasting_validated.any()
defects=pd.read_csv(OUT/'q3/transfer_defect_checks.csv'); assert defects.provided_nonincrease.all()
top=pd.read_csv(OUT/'q2/top3_with_evidence.csv'); assert len(top)==150 and (top.groupby('target').size()==3).all()
literature=[]
for r in top.itertuples():
    row={'target':r.target,'field':r.field,'statistical_status':r.evidence_status,'causal_status':'not_identified','independent_reference':'','chemical_evidence':'attachment_metadata_only_target_specific_mechanism_unverified','limitations':'direction_and_style_confounding_require_review'}
    if r.target=='X4vg' and r.field=='X4VG': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/19053838/',chemical_evidence='beer_research_supports_clove_related_aroma_not_current_dataset_causality',limitations='aging_and_matrix_conditions_not_a_new_spiking_test')
    elif r.target=='metallic' and r.field=='iron': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/22304665/',chemical_evidence='metal_solution_and_salivary_lipid_oxidation_mechanism',limitations='iron_speciation_and_beer_matrix_not_resolved_no_beer_threshold_inferred')
    elif r.target=='coriander' and r.field=='linalool': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/38398596/',chemical_evidence='coriander_seed_low_alcohol_beer_primary_research_context',limitations='not_specific_causality_for_this_panel_or_this_concentration')
    elif (r.target=='acidity' and r.field in ['lactic_acid.mg.L.','acetic_acid.g.L.']) or (r.target=='lactic' and r.field=='lactic_acid.mg.L.') or (r.target=='alcohol' and r.field=='ethanol..v.v.'):
        row.update(chemical_evidence='measurement_compatible_relation_not_independent_causal_validation',limitations='matrix_mixture_and_source_preprocessing_remain')
    literature.append(row)
pd.DataFrame(literature).to_csv(OUT/'q2/chemical_evidence_ledger.csv',index=False,encoding='utf-8-sig')
summary={'client_date':'2026-10-25','tests':tests,'original_sources_unchanged':True,'prediction_outer_folds':15,'unique_beers':250,
 'graph_null_fits_per_outer':10,'generation_disjoint_partitions':5,'generated_records_per_group':40,
 'core_targets':30,'all_targets':50,'relative_grade_cells':12500,
 'complete_absolute_score_calibration':False,'complete_chemical_certification':False,'real_tasting_validated':False,
 'scientific_scope':'exploratory_internal_validation_after_earlier_stages_not_new_external_validation',
 'executed':['nested_prediction_and_shrinkage','fixed_relative_grade_sets','missingness_and_clipping','train_only_graph_ablations','crossfit_within_style_module_effects','disjoint_generation_and_Pareto_transfer','failure_and_unknown_item_preservation'],
 'still_unresolved':['real_absolute_score_anchors','detection_limits_and_all_units','all_aggregate_formulas','full_target_specific_chemical_mechanisms','new_external_beers_and_tasting','human_independent_review'],
 'source_hashes':[{'file':r.file,'sha256':r.sha256} for r in metadata.itertuples()],
 'code_sha256':{p.relative_to(ROOT).as_posix():hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted((ROOT/'src').glob('stage3*.py'))}}
(OUT/'validation_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
checks=[{'file':p.relative_to(OUT).as_posix(),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in sorted(OUT.rglob('*')) if p.is_file() and p.name!='result_checksums.json' and 'private' not in p.parts]
(OUT/'result_checksums.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'tests':tests,'files_hashed':len(checks),'relative_grade_cells':12500,'all_three_experiments_complete':True},ensure_ascii=False))
```

scripts/verify__stage4.py

```python
"""Validate source identity, absolute outputs, grading and frozen-candidate roles."""
from pathlib import Path
import json
import hashlib
import xml.etree.ElementTree as ET
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]; OUT = ROOT / 'outputs/stage4'
audit = json.loads((OUT / 'audit/audit_summary.json').read_text(encoding='utf-8'))
assert audit['identity_matches_both_attribute_groups'] == 250 and audit['nonconstant_targets_reproduced'] == 38
assert audit['nonconstant_max_error'] < 1e-7 and audit['concentration_model_fields'] == 61
mapping = pd.read_csv(OUT / 'audit/identity_mapping.csv'); assert mapping.beer.nunique() == 250 and mapping.raw_beer.nunique() == 250 and mapping.is_row_nearest.all()
reconstruction = pd.read_csv(OUT / 'audit/constant_attribute_reconstruction.csv'); assert len(reconstruction) == 12 and reconstruction.validation_max_error.max() < 1e-7
panel = pd.read_csv(OUT / 'audit/observed_panel_means_0_6.csv').set_index('beer'); assert panel.shape == (250, 50) and panel.ge(0).all().all() and panel.le(6).all().all()
for part in ['prediction', 'ordinal', 'transfer']: assert json.loads((OUT / part / 'status.json').read_text())['complete']
graph_status = json.loads((OUT / 'raw_graph/q2/status.json').read_text()); assert graph_status['complete'] and graph_status['outer_folds'] == 15
graph_folds = pd.read_csv(OUT / 'raw_graph/q2/graph_fold_metrics.csv')
assert len(graph_folds[graph_folds.graph == 'shuffled']) == 300
roles = pd.read_csv(OUT / 'prediction/outer_roles.csv'); assert len(roles) == 1250 and (roles.groupby(['repeat', 'beer']).size() == 1).all()
metrics = pd.read_csv(OUT / 'prediction/metrics_by_scope.csv')
core = [f for f in panel if f.startswith(('A_', 'F_'))]
for model in metrics.model.unique():
    p = pd.read_csv(OUT / f'prediction/oof_{model}_0_6.csv').set_index('beer').reindex(panel.index)
    assert p.ge(0).all().all() and p.le(6).all().all()
    for scope, fields in [('core30', core), ('all50', list(panel))]:
        value = np.abs(panel[fields] - p[fields]).to_numpy().mean()
        assert abs(value - metrics[(metrics.model == model) & (metrics.scope == scope)].mae.iloc[0]) < 1e-7
for name in ['oof_modal_grades_0_6.csv', 'oof_median_grades_0_6.csv', 'oof_rounded_panel_mean_grades_0_6.csv']:
    grades = pd.read_csv(OUT / 'ordinal' / name).iloc[:, 1:].to_numpy(); assert grades.shape == (250, 50) and np.isin(grades, np.arange(7)).all()
distributions = pd.read_csv(OUT / 'ordinal/oof_rating_distributions.csv'); assert len(distributions) == 12500
prob = distributions[[f'predicted_p{k}' for k in range(7)]].to_numpy(); observed = distributions[[f'observed_panel_p{k}' for k in range(7)]].to_numpy()
assert (prob >= 0).all() and np.allclose(prob.sum(1), 1, atol=1e-8) and np.allclose(observed.sum(1), 1, atol=1e-8)
assert np.array_equal(prob.argmax(1), distributions.predicted_modal_grade)
assert abs(np.abs((prob * np.arange(7)).sum(1) - (observed * np.arange(7)).sum(1)).mean() - pd.read_csv(OUT / 'ordinal/metrics_by_scope.csv').query("scope == 'all50' and method == 'ET_rating_distribution'").expected_panel_mean_MAE.iloc[0]) < 1e-7
intervals = pd.read_csv(OUT / 'prediction/absolute_panel_mean_intervals.csv'); assert len(intervals) == 12500 and (intervals.lower_0_6 <= intervals.upper_0_6).all() and intervals.lower_0_6.ge(0).all() and intervals.upper_0_6.le(6).all()
top = pd.read_csv(OUT / 'prediction/absolute_raw_top3.csv'); assert len(top) == 150 and (top.groupby('target').size() == 3).all() and not top.causal_validated.any()
frozen = pd.read_csv(OUT / 'transfer/frozen_stage3_candidates_raw_score_check.csv'); assert len(frozen) == 220 and not frozen.physical_validation.any()
assert not (frozen.original_and_raw_proxy_support & ~frozen.original_proxy_acceptance).any()
mixture = pd.read_csv(OUT / 'transfer/concentration_mixture_evaluation.csv'); assert len(mixture) == 55 and not mixture.real_tasting_validated.any() and not mixture.LOD_certification.any()
assert all(json.loads((OUT / f'transfer/concentration_candidate_lock_{i}.json').read_text())['selection_completed_before_J_fitting'] for i in range(5))
assert pd.read_csv(OUT / 'protocol/blank_panel_rating_template.csv').empty
tests = ET.parse(ROOT / 'tmp/stage4_tests.xml').getroot(); suites = list(tests.iter('testsuite'))
test_summary = {key: sum(int(s.attrib.get(key, 0)) for s in suites) for key in ['tests', 'failures', 'errors', 'skipped']}
assert test_summary['tests'] == 48 and not any(test_summary[k] for k in ['failures', 'errors', 'skipped'])
summary = {'all_four_stage4_components_complete': True, 'tests': test_summary, 'identity_mapping_verified': True,
           'raw_endpoint_graph_controls_complete': True,
           'absolute_panel_mean_cells': 12500, 'integer_grade_cells_per_rule': 12500, 'ordinal_probability_rows': 12500,
           'frozen_transfer_records': len(frozen), 'concentration_mixture_records': len(mixture), 'physical_trial_observations': 0}
(OUT / 'validation_summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
hashes = {p.relative_to(OUT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(OUT.rglob('*')) if p.is_file() and p.name != 'result_checksums.json'}
(OUT / 'result_checksums.json').write_text(json.dumps(hashes, ensure_ascii=False, indent=2), encoding='utf-8')
print(json.dumps({**summary, 'hashed_files': len(hashes)}, ensure_ascii=False))
```

tests/test__invariants.py

```python
"""AI-assisted verification: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest
from sklearn.model_selection import RepeatedStratifiedKFold

from src.common import TrainQuantileClipper, ecdf_grades, sha256
from src.data import CHEMICAL_DEFECTS, NON_MOLECULES, align_by_beer, find_data, load_data
from src.q2 import signed_laplacian
from src.q3 import ConditionalPCAGaussian, chemical_nonincrease


ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs/stage1"


def test_join_uses_unique_beer_and_recovers_permuted_rows():
    x = pd.DataFrame({"beer": [1, 2], "beer_id": [100, 101], "tasting_category_fine": ["a", "b"]})
    y = pd.DataFrame({"beer": [2, 1], "beer_id": [100, 100], "tasting_category_fine": ["b", "a"], "score": [9, 4]})
    assert align_by_beer(x, y).score.tolist() == [4, 9]


def test_duplicate_key_is_rejected_before_join():
    a = pd.DataFrame({"beer": [1, 1], "tasting_category_fine": ["a", "a"]})
    with pytest.raises(ValueError, match="unique"):
        align_by_beer(a, a)


def test_preprocessing_bounds_do_not_learn_from_validation_extreme():
    train = np.array([[0.], [1.], [2.], [3.]])
    clip = TrainQuantileClipper(lower=0, upper=1).fit(train)
    assert clip.transform([[1e12]])[0, 0] == 3
    np.testing.assert_array_equal(clip.upper_, [3.])


def test_signed_laplacian_is_psd_with_negative_edges():
    L = signed_laplacian([[1, -.8, .7], [-.8, 1, .1], [.7, .1, 1]], .6)
    assert np.linalg.eigvalsh(L).min() > -1e-10


def test_ecdf_grades_are_bounded_and_monotone():
    ref = np.arange(100.)[:, None]
    g = ecdf_grades(ref, np.array([[-1.], [50.], [1000.]]))[:, 0]
    assert g[0] == 0 and g[-1] == 6 and np.all(np.diff(g) >= 0)


def test_each_defect_is_checked_separately():
    # A falling total must not hide a rise in one defect.
    source, candidate = np.array([2., 2.]), np.array([0., 3.])
    assert candidate.sum() < source.sum()
    assert not chemical_nonincrease(source, candidate, [0, 1])


def test_conditional_generator_is_finite_bounded_and_reproducible():
    rng = np.random.default_rng(4)
    x = rng.normal(size=(30, 7))
    x[0, 0] = np.nan
    gen = ConditionalPCAGaussian(3).fit(x, np.array(["a"] * 15 + ["b"] * 15))
    a = gen.sample("a", 10, np.random.default_rng(1))
    b = gen.sample("a", 10, np.random.default_rng(1))
    np.testing.assert_allclose(a, b)
    assert np.isfinite(a).all()
    assert np.all(a >= gen.low - 1e-10) and np.all(a <= gen.high + 1e-10)


@pytest.fixture(scope="module")
def real_data():
    try:
        root = find_data()
    except FileNotFoundError:
        pytest.skip("Original competition data not available")
    return load_data(root)
```

```

def need_results():
    if not (OUTPUT / "q3/status.json").exists():
        pytest.skip("Run all stages before artifact validation")


def test_original_sources_unchanged(real_data):
    need_results()
    manifest = pd.read_csv(OUTPUT / "audit/source_manifest.csv")
    for row in manifest.itertuples():
        assert sha256(real_data.root / row.file) == row.sha256


def test_oof_mean_matches_actual_training_only_mean(real_data):
    need_results()
    cfg = json.loads((ROOT / "config.json").read_text())
    Y = real_data.Y.to_numpy()
    expected = np.zeros_like(Y)
    counts = np.zeros(len(Y))
    splitter = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"], random_state=cfg["seed"])
    for train, test in splitter.split(real_data.X, real_data.meta.tasting_category_fine):
        expected[test] += Y[train].mean(axis=0)
        counts[test] += 1
    saved = pd.read_csv(OUTPUT / "q1/oof_mean_continuous.csv")[real_data.Y.columns].to_numpy()
    np.testing.assert_allclose(saved, expected / counts[:, None], atol=1e-7)


def test_each_sample_held_out_once_per_repeat(real_data):
    need_results()
    s = pd.read_csv(OUTPUT / "q1/split_assignments.csv")
    assert not s.duplicated(["repeat", "beer"]).any()
    for _, part in s.groupby("repeat"):
        assert set(part.beer) == set(real_data.meta.beer)


def test_top3_has_unique_eligible_factors_per_target(real_data):
    need_results()
    top = pd.read_csv(OUTPUT / "q2/top3_by_sensory_dimension.csv")
    assert len(top) == 3 * real_data.Y.shape[1]
    assert set(top.target) == set(real_data.Y.columns)
    assert not top.duplicated(["target", "field"]).any()
    assert not set(top.field) & NON_MOLECULES
    assert top.bootstrap_top3_frequency.between(0, 1).all()


def test_transfer_failure_is_not_hidden_as_success():
    need_results()
    t = pd.read_csv(OUTPUT / "q3/transfer_diagnostics.csv")
    failed = t[t.status == "no_nontrivial_candidate_found_source_retained"]
    assert np.allclose(failed.supplied_scale_change_l2, 0)
    success = t[t.status == "nontrivial_model_feasible_candidate"]
    assert (success.candidate_target_probability > success.source_target_probability + .01).all()
    assert (success.candidate_target_probability > success.cleaning_only_target_probability + .01).all()
    assert t.chemical_nonincrease.all() and t.predicted_stale_hops_nonincrease.all()
    checks = pd.read_csv(OUTPUT / "q3/defect_nonincrease_checks.csv")
    assert checks.nonincrease.all()
    assert set(checks.defect) == set(CHEMICAL_DEFECTS) | {"stale_hops"}


def test_calibration_remains_explicitly_unresolved():
    need_results()
    status = json.loads((OUTPUT / "q1/status.json").read_text())
    assert status["absolute_0_6_prediction"].startswith("UNRESOLVED")
    grades = pd.read_csv(OUTPUT / "q1/ASSUMPTION_percentile_grades_0_6.csv")
    assert grades.status.str.startswith("ASSUMPTION_ONLY").all()


def test_generator_holdout_does_not_overlap_training(real_data):
    need_results()
    split = pd.read_csv(OUTPUT / "q3/generator_split.csv")
    train = set(split[split.role == "train"].beer)
    heldout = set(split[split.role == "holdout"].beer)
    assert train.isdisjoint(heldout)
    assert train | heldout == set(real_data.meta.beer)


def test_generation_diagnostics_do_not_hide_missing_reference_values(real_data):
    need_results()
    diagnostics = pd.read_csv(OUTPUT / "q3/generation_diagnostics.csv")
    assert np.isfinite(diagnostics.mean_marginal_KS_holdout).all()
    assert diagnostics.mean_marginal_KS_holdout.between(0, 1).all()
    assert (diagnostics.n_KS_fields_observed > 0).all()
```

tests/test_stage2.py

```

"""AI-assisted validation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest

from src.stage2 import assumption_grades, conformal_quantile, local_convex_samples

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"


def test_conformal_quantile_uses_finite_sample_rank():
    errors = np.arange(1, 51)[:, None]
    assert conformal_quantile(errors, .1)[0] == 46
    with pytest.raises(ValueError):
        conformal_quantile(errors[:2], .01)


def test_three_grade_assumptions_remain_bounded():
    rng = np.random.default_rng(3)
    ref = rng.normal(size=(50, 4))
    grades = assumption_grades(ref, np.array([[-100.] * 4, [100.] * 4]))
    assert grades.shape == (3, 2, 4)
    assert np.all((grades >= 0) & (grades <= 6))
    assert np.all(grades[:, 0] <= grades[:, 1])


def test_local_convex_samples_stay_inside_training_hull_box():
    real = np.array([[0., 1.], [1., 2.], [2., 3.]])
    generated = local_convex_samples(real, 50, np.random.default_rng(4))
    assert np.all(generated >= real.min(0)) and np.all(generated <= real.max(0))
    assert np.allclose(generated[:, 1] - generated[:, 0], 1.)
    assert not np.any(np.all(np.isclose(generated[:, None], real[None]), axis=2))


def need_stage2():
    if not (OUT / "status.json").exists():
        pytest.skip("Stage two has not been run")


def test_conformal_roles_are_disjoint():
    need_stage2()
    frame = pd.read_csv(OUT / "q1/conformal_split.csv")
    assert len(frame) == frame.beer.nunique() == 250
    assert frame.role.value_counts().to_dict() == {"train": 150, "calibration": 50, "test": 50}


def test_attribution_flags_do_not_claim_causality():
    need_stage2()
    top = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
    assert len(top) == 150
    selected = top[top.evidence_status == "comparatively_supported_association"]
    assert (selected.bootstrap_top3_frequency >= .5).all()
    assert (selected.heldout_permutation_MAE_increase > 0).all()
    assert (selected.positive_fold_fraction >= 2 / 3).all()
    assert (top.causality == "not_established").all()


def test_repeated_generator_comparison_has_all_methods_and_splits():
    need_stage2()
    d = pd.read_csv(OUT / "q3/generator_comparison_by_split.csv")
    assert len(d) == 5 * 3 * 3
    assert set(d.method) == {"bootstrap", "local_convex", "PCA_Gaussian"}
    assert np.isfinite(d[["KS_mean", "MMD2", "target_style_probability"]]).all().all()


def test_extended_transfer_preserves_failure_and_each_defect():
    need_stage2()
    d = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
    good = d[d.status == "nontrivial_model_feasible_candidate"]
    assert (good.candidate_trip_probability > good.source_trip_probability + .01).all()
    assert (good.candidate_trip_probability > good.cleaning_only_probability + .01).all()
    bad = d[d.status != "nontrivial_model_feasible_candidate"]
    assert np.allclose(bad.change_l2, 0)
    assert pd.read_csv(OUT / "q3/expanded_transfer_defect_checks.csv").nonincrease.all()


def test_stage2_keeps_absolute_calibration_unidentified():
```

```
    need_stage2()
    assert json.loads((OUT / "status.json").read_text())["absolute_score_calibration"].startswith("not_identifiable")
```

tests/test_stage3_algorithms.py

```
"""Meaningful property tests for the new mathematical helpers."""
import numpy as np
import pytest
from src.stage3_algorithms import (fixed_grade_boundaries, grade_with_fixed_boundaries,
    ordinal_interval_image, shrunk_blend_weights, paired_oof_bootstrap, pareto_minimize, threshold_excess)


def test_fixed_ordinal_map_preserves_contained_outcome_with_tied_boundaries():
    boundaries = fixed_grade_boundaries(np.array([[0,0],[0,1],[0,2],[1,3],[1,4],[1,5],[2,6]], float))
    lower, upper = np.array([[-1,1.5],[0,2]]), np.array([[2,4],[0,2]])
    lo, hi = ordinal_interval_image(lower, upper, boundaries)
    for weight in np.linspace(0,1,21):
        inside = grade_with_fixed_boundaries(lower+weight*(upper-lower), boundaries)
        assert np.all((lo <= inside) & (inside <= hi))


def test_invalid_interval_is_rejected():
    with pytest.raises(ValueError):
        ordinal_interval_image(np.ones((1,2)), np.zeros((1,2)), np.zeros((6,2)))


def test_blend_recovers_opposite_best_models_and_shrinks_to_global():
    y = np.zeros((20,2)); et = np.tile([0,1.],(20,1)); gbr=np.tile([1.,0],(20,1))
    w,_ = shrunk_blend_weights(y,et,gbr,0)
    np.testing.assert_allclose(w,[0,1])
    w,global_w = shrunk_blend_weights(y,et,gbr,1e8)
    np.testing.assert_allclose(w,global_w,atol=1e-7)


def test_fixed_prediction_resampling_uses_paired_errors_and_declares_scope():
    y=np.zeros((12,2)); a=np.zeros_like(y); b=np.ones_like(y)
    r=paired_oof_bootstrap(y,a,b,np.repeat(['A','B'],6),100)
    assert r['difference_A_minus_B'] == -1
    assert r['conditional_percentile_95'] == [-1,-1]
    assert 'does_not_include_model_retraining' in r['scope']


def test_pareto_retains_tradeoffs_and_rejects_nonfinite_candidate():
    keep=pareto_minimize([[0,2],[1,1],[2,0],[2,2],[np.nan,0],[1,1]])
    np.testing.assert_array_equal(keep,[True,True,True,False,False,True])


def test_unknown_threshold_and_source_values_never_become_safe_zero():
    result=threshold_excess([.5,2,2,np.nan,-1],[1,1,np.nan,1,1])
    np.testing.assert_allclose(result[:2],[0,1])
    assert np.isnan(result[2:]).all()
```

tests/test_stage3_research.py

```
"""Check actual isolation and mathematical invariants of the executed study."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import pytest
from src.stage3_prediction import learn_blend_nested
from src.stage3_graph import fit_tuned_graph, within_style_indices
from src.stage3_generation import mmd_fixed_bandwidth, constrained_convex
from src.q3 import ConditionalPCAGaussian

ROOT=Path(__file__).resolve().parents[1]


def test_nested_blend_base_predictions_never_predict_their_own_training_rows(monkeypatch):
    import src.stage3_prediction as module
    calls=[]
    def fit(X,Y,cfg): return set(X[:,0].astype(int))
    def predict(seen,X):
        assert not seen.intersection(X[:,0].astype(int)); calls.append((seen, set(X[:,0].astype(int))))
        return np.zeros((len(X),2)),np.ones((len(X),2))
    monkeypatch.setattr(module,'fit_pair',fit); monkeypatch.setattr(module,'predict_pair',predict)
    X=np.c_[np.arange(12),np.arange(12)**2]; Y=np.c_[np.arange(12)/12,np.arange(12)/24]
    w,_,_,eta,_=learn_blend_nested(X,Y,{'seed':1,'blend_penalties':[0,.1]},np.array([0]),0)
    assert len(calls)==12 and np.all((w>=0)&(w<=1)) and eta in [0,.1]


def test_zero_penalty_graph_and_permuted_graph_reduce_to_same_ridge():
    rng=np.random.default_rng(5); X=rng.normal(size=(18,4)); Y=rng.normal(size=(18,2))
    cfg={'ridge_alphas':[10],'graph_lambdas':[0],'graph_thresholds':[.7],'graph_null_repeats':2}
    fits=[fit_tuned_graph(X,Y,cfg,k,np.array([0]),17) for k in ['none','real','null']]
    for f in fits:
        for b in f['beta']: np.testing.assert_allclose(b,fits[0]['beta'][0],atol=1e-12)


def test_style_permutation_preserves_each_style_and_singleton():
    labels=np.array(['A','A','B','B','B','C']); order=within_style_indices(labels,np.random.default_rng(9))
    np.testing.assert_array_equal(labels[order],labels); assert sorted(order)==list(range(6)) and order[-1]==5


def test_fixed_bandwidth_mmd_is_symmetric_and_zero_for_identical_samples():
    rng=np.random.default_rng(8); a=rng.normal(size=(12,4)); b=rng.normal(size=(7,4))
    assert abs(mmd_fixed_bandwidth(a,a,2))<1e-12
    assert np.isclose(mmd_fixed_bandwidth(a,b,2),mmd_fixed_bandwidth(b,a,2))


def test_constrained_convex_respects_fitted_bounds_and_budget():
    rng=np.random.default_rng(3); X=rng.normal(size=(12,4)); g=ConditionalPCAGaussian(3,2).fit(X,np.repeat('A',12))
    real=g.prep.named_steps['scale'].inverse_transform(g.prep.transform(X))
    a,info=constrained_convex(real,g,[0],20,rng)
    assert len(a)==20 and info['attempts']<=5000
    assert np.all(a>=g.low-1e-9) and np.all(a<=g.high+1e-9)


def test_saved_graph_comparison_contains_all_15_folds_and_ten_null_replicates():
    path=ROOT/'outputs/stage3/q2/graph_fold_metrics.csv'
    if not path.exists(): pytest.skip('Third-stage graph study not present')
    d=pd.read_csv(path); assert d.fold.nunique()==15
    shuffled=d[d.graph=='shuffled']; assert len(shuffled)==300
    assert (shuffled.groupby(['fold','scope']).size()==10).all()


def test_saved_search_evaluator_and_test_roles_are_disjoint():
    path=ROOT/'outputs/stage3/q3/disjoint_roles.csv'
    if not path.exists(): pytest.skip('Third-stage generation study not present')
    d=pd.read_csv(path)
    for _,a in d.groupby('repeat'):
        assert len(a)==a.beer.nunique()==250
        assert sorted(a.groupby('role').size())==[50,100,100]


def test_saved_failed_transfers_are_not_counted_as_improvements():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path); failed=d.status=='no_feasible_nontrivial_candidate_source_retained'
    assert not d.loc[failed,'evaluator_proxy_acceptance'].any()
    assert (d.loc[failed,'standardized_change']==0).all()


def test_missing_sources_are_never_reported_as_unconditional_feasible():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path)
    assert not ((~d.all_provided_source_defects_observed)&(d.status=='model_feasible_nontrivial')).any()
    assert not d.absolute_chemical_certification.any() and not d.real_tasting_validated.any()


def test_saved_ordinal_calibration_roles_are_disjoint():
    paths=list((ROOT/'outputs/stage3/q1').glob('conformal_roles_*.csv'))
    if len(paths)!=5: pytest.skip('Third-stage conformal experiments not complete')
    for path in paths:
        d=pd.read_csv(path); assert len(d)==d.beer.nunique()==250
        assert d.groupby('role').size().to_dict()=={'calibration':50,'test':50,'train':150}
```

```
94
95
96  def test_saved_prediction_study_is_complete_and_absolute_scores_remain_
    unidentified():
97      path=ROOT/'outputs/stage3/q1/status.json'
98      if not path.exists(): pytest.skip('Third-stage prediction study not
    complete')
99      d=json.loads(path.read_text()); assert d['complete'] and d['outer_folds
    ']==15
100     assert not d['absolute_anchors_observed']
```

tests/test_stage4.py

```
1   """Check identity, repeated-rater weighting and physical constraint
    boundaries."""
2   import numpy as np
3   import pandas as pd
4   from src.stage4_audit import aggregate_panel, infer_identity, transform_
    candidates, concentration_fields, panel_rating_distribution
5   from src.stage4_models import calibrated_from_training, mix_concentrations,
    exact_nonincrease
6   from src.stage4_trial import analyze_paired_trial, planned_noninferiority_
    power
7   import pytest
8
9
10  def test_repeat_ratings_do_not_overweight_one_panelist():
11      raw = pd.DataFrame({'beer': [1, 1, 1], 'taster': [1, 1, 2], 'rating': [0,
     2, 6]})
12      balanced, records, _ = aggregate_panel(raw, ['rating'])
13      assert balanced.loc[1, 'rating'] == 3.5
14      assert records.loc[1, 'rating'] == 8 / 3
15      distribution = panel_rating_distribution(raw, ['rating'], [1])
16      assert np.allclose(distribution[0, 0, [0, 2, 6]], [.25, .25, .5])
17      assert np.allclose(distribution.sum(-1), 1)
18
19
20  def test_identity_uses_attributes_despite_shuffled_published_rows():
21      z = pd.DataFrame({'a': [-2., 0., 3.], 'b': [3., -1., 0.]})
22      raw_id = pd.Series([1, 2, 3])
23      published = pd.DataFrame({'a': [3., -2., 0.], 'b': [0., 3., -1.]}, index
    =[40, 90, 7])
24      _, mapping = infer_identity(z, raw_id, published, ['a', 'b'])
25      assert mapping.set_index('raw_beer').beer.to_dict() == {1: 90, 2: 7, 3:
    40}
26
27
28  def test_negative_blank_signal_is_not_logged_as_negative_concentration():
29      values, half = transform_candidates(np.array([-2., 0., 2.5, 10., np.nan])
    )
30      assert half == 1.25
31      assert np.allclose(values['log10_nonpositive_half_min'][:2], np.log10
    (1.25))
32      assert np.isnan(values['log10_nonpositive_half_min'][-1])
33
34
35  def test_unit_and_calibration_registry_excludes_relative_area_and_pressure():
36      calibrated = pd.DataFrame(columns=['beer_id', 'tasting_category_fine', 'a
    ', 'b', 'c'])
37      registry = pd.DataFrame({'field_name_R': ['a', 'b', 'c'], 'calibrated':
    ['yes', np.nan, 'yes'], 'unit': ['mg/L', 'A.U.', 'PSI']})
38      assert concentration_fields(calibrated, registry, ['mg/L']) == ['a']
39
40
41  def test_isotonic_calibration_respects_only_supplied_training_anchor_range():
42      result = calibrated_from_training(np.array([[0.], [1.], [2.]]), np.array
    ([[1.], [2.], [3.]]), np.array([[-10.], [10.]]))
43      assert np.array_equal(result[:, 0], [1., 3.])
44
45
46  def test_mixture_is_linear_in_concentration_and_missing_source_cannot_certify
    ():
47      source, anchor = np.array([1., 10.]), np.array([3., 2.])
48      values = mix_concentrations(source, anchor, [.25, .5])
49      assert np.allclose(values, [[1.5, 8.], [2., 6.]])
50      assert not exact_nonincrease(values, source, [0, 1]).any()
51      assert exact_nonincrease(values, source, [1]).all()
52      assert not exact_nonincrease(values, np.array([1., np.nan]), [1]).any()
53
54
55  def test_trial_repeated_batches_do_not_inflate_independent_participant_count
    ():
56      rows = []
57      for person in range(4):
58          for batch in range(3):
59              for condition in ['control', 'candidate']:
60                  values = dict.fromkeys(['dms', 'diacetyl', 'X4vg', 't2n', '
    stale_hops'], 1.)
61                  rows.append({'panelist': person, 'batch': batch, 'condition':
     condition,
62                               'style_Tripel_fit': 1. if condition == 'control'
     else 3., **values})
63      result = analyze_paired_trial(pd.DataFrame(rows))
64      assert (result.independent_panelists == 4).all() and result.criterion_
    passed.all()
65
66
67  def test_no_real_trial_claim_from_an_empty_or_unpaired_template():
68      with pytest.raises(ValueError):
69          analyze_paired_trial(pd.DataFrame())
70      assert planned_noninferiority_power(30, .8) > planned_noninferiority_
    power(10, .8)
```

run.py

```
1   """AI-assisted orchestration: OpenAI Codex / GPT-6, 2026-10-24."""
2   import argparse
3   import json
4   import os
5   from pathlib import Path
6
7   os.environ.setdefault("MPLBACKEND", "Agg")
8   os.environ.setdefault("OMP_NUM_THREADS", "2")
9   os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")
10
11  from src.common import environment, progress, write_json
12  from src.data import find_data, load_data
13
14
15  def main():
16      parser = argparse.ArgumentParser(description="Reproducible A-track
    research baselines")
17      parser.add_argument("stage", choices=["audit", "q1", "q2", "q3", "report
    ", "all"])
18      parser.add_argument("--data-dir")
19      parser.add_argument("--out", default="outputs/stage1")
20      parser.add_argument("--config", default="config.json")
21      args = parser.parse_args()
22      cfg = json.loads(Path(args.config).read_text(encoding="utf-8"))
23      out = Path(args.out)
24      out.mkdir(parents=True, exist_ok=True)
25      data = load_data(find_data(args.data_dir))
26      if args.stage in ["all", "audit"]:
27          from src.audit import run_audit
28          run_audit(data, out, cfg)
29      if args.stage in ["all", "q1"]:
30          from src.q1 import run_q1
31          run_q1(data, out, cfg)
32      if args.stage in ["all", "q2"]:
33          from src.q2 import run_q2
34          run_q2(data, out, cfg)
35      if args.stage in ["all", "q3"]:
36          from src.q3 import run_q3
37          run_q3(data, out, cfg)
38      if args.stage in ["all", "report"]:
39          from src.reporting import make_report
40          make_report(data, out, cfg)
41      manifest = environment()
42      manifest["stage"] = args.stage
43      manifest["config"] = cfg
44      manifest["files"] = sorted(str(p.relative_to(out)).replace("\\", "/") for
     p in out.rglob("*") if p.is_file() and "private" not in p.parts and p.suffix
     in {".csv", ".json", ".md", ".png"})
45      write_json(out / "run_manifest.json", manifest)
46      progress(f"Completed {args.stage}; outputs: {out.resolve()}")
47
48
49  if __name__ == "__main__":
50      main()
```

run_stage2.py

```
1   """AI-assisted research extension, OpenAI Codex / GPT-6, 2026-10-24."""
2   import argparse
3   import json
4   import os
5   from pathlib import Path
6
7   os.environ.setdefault("MPLBACKEND", "Agg")
8   os.environ.setdefault("OMP_NUM_THREADS", "2")
9   os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")
10
```

```
from src.common import environment, write_json
from src.data import find_data, load_data
from src.stage2 import run_extension

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-dir")
    parser.add_argument("--out", default="outputs/stage2")
    parser.add_argument("--base", default="outputs/stage1")
    args = parser.parse_args()
    cfg = json.loads(Path("config.json").read_text())
    ext = json.loads(Path("config_stage2.json").read_text())
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    run_extension(load_data(find_data(args.data_dir)), Path(args.base), out, cfg, ext)
    write_json(out / "run_manifest.json", {**environment(), "base_config": cfg, "extension_config": ext})
```

run__stage3.py

```
"""Run saved, explicitly scoped third-stage experiments; AI-assisted, 2026-10-25."""
import os
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")
os.environ.setdefault("MKL_NUM_THREADS", "2")
os.environ.setdefault("MPLBACKEND", "Agg")
import argparse
import json
from pathlib import Path
from src.common import environment, write_json, sha256
from src.data import load_data, find_data

if __name__ == '__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('part',choices=['q1','q2','q3'])
    parser.add_argument('--data-dir')
    parser.add_argument('--out',default='outputs/stage3')
    args=parser.parse_args()
    cfg=json.loads(Path('config_stage3.json').read_text(encoding='utf-8'))
    data=load_data(find_data(args.data_dir)); out=Path(args.out); out.mkdir(parents=True,exist_ok=True)
    expected={r['file']:r['sha256'] for r in __import__('pandas').read_csv('outputs/stage2/audit/source_manifest.csv').to_dict('records')}
    for file in ['1.csv','2.csv','3.xlsx','4.csv','8.xlsx']:
        if sha256(data.root/file)!=expected[file]: raise ValueError('Original source version changed: '+file)
    if args.part=='q1':
        from src.stage3_prediction import run_prediction
        run_prediction(data,out,cfg)
    elif args.part=='q2':
        from src.stage3_graph import run_graph
        run_graph(data,out,cfg)
    else:
        from src.stage3_generation import run_generation
        run_generation(data,out,cfg)
    write_json(out/f'{args.part}_run_manifest.json',{**environment(),'config':cfg,'config_sha256':sha256('config_stage3.json'),'sources_verified':expected})
```

run__stage4.py

```
"""Reconcile newly available raw sensory data, then evaluate absolute scores."""
from pathlib import Path
import argparse
import json
import os
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
from src.common import environment, write_json, sha256
from src.data import load_data, find_data
from src.stage4_audit import reconcile
from src.stage4_models import run_absolute, run_transfer_diagnostics, run_ordinal

ROOT = Path(__file__).resolve().parent
parser = argparse.ArgumentParser()
parser.add_argument('part', choices=['audit', 'prediction', 'ordinal', 'transfer', 'all'])
parser.add_argument('--external-dir', default=str(ROOT.parent / 'external_sources/zenodo_v4_23624915'))
parser.add_argument('--data-dir')
args = parser.parse_args()
cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8'))
data = load_data(find_data(args.data_dir)); out = ROOT / 'outputs/stage4'
raw, panel, records, concentration, fields = reconcile(data, args.external_dir, out / 'audit', cfg)
if args.part in ['prediction', 'all']:
    run_absolute(data, panel, records, concentration, out, cfg)
if args.part in ['ordinal', 'all']:
    run_ordinal(data, raw, panel, out, cfg, ROOT / 'outputs/stage3')
if args.part in ['transfer', 'all']:
    run_transfer_diagnostics(data, panel, concentration, out, cfg, ROOT / 'outputs/stage3')
write_json(out / f'{args.part}_run_manifest.json', {**environment(), 'config': cfg, 'source_identity_audit': 'audit/audit_summary.json',
                                                    'concentration_features': fields, 'part': args.part,
                                                    'code_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in [ROOT / 'run_stage4.py', ROOT / 'config_stage4.json'] + sorted((ROOT / 'src').glob('*.py'))}})
print(json.dumps({'part': args.part, 'completed': True, 'concentration_fields': len(fields)}))
```

config.json

```
{
  "seed": 20261024,
  "outer_folds": 3,
  "outer_repeats": 2,
  "inner_folds": 3,
  "ridge_alphas": [1.0, 10.0, 100.0, 1000.0],
  "pls_components": [3, 8],
  "extra_trees": 200,
  "bootstrap_repeats": 100,
  "graph_threshold": 0.7,
  "graph_ridge_alpha": 20.0,
  "graph_ridge_lambda": 5.0,
  "generation_holdout_fraction": 0.2,
  "generation_components": 10,
  "generation_samples_per_style": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "transfer_source": "Pils/Lager",
  "transfer_target": "Tripel",
  "missing_rates": [0.0, 0.05, 0.1, 0.2],
  "export_assumption_grades": true
}
```

config__stage2.json

```
{
  "seed": 20261024,
  "gbr_estimators": 100,
  "gbr_learning_rate": 0.04,
  "gbr_depth": 2,
  "permutation_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "transfer_weights": 30
}
```

config__stage3.json

```
{
  "date": "2026-10-25",
  "seed": 20261025,
  "outer_folds": 3,
  "outer_repeats": 5,
  "inner_folds": 3,
  "primary_targets": "explicit_A_and_F_prefixed_30_not_official_scope_ruling",
  "blend_penalties": [0, 0.01, 0.1, 1, 10],
  "ridge_alphas": [1, 10, 100, 1000],
  "extra_trees": 200,
  "gbr_estimators": 100,
  "graph_thresholds": [0.6, 0.7, 0.8],
  "graph_lambdas": [0, 1, 5, 20],
  "graph_null_repeats": 10,
  "bootstrap_repeats": 200,
  "permutation_repeats": 20,
```

```
  "conformal_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "generator_n": 100,
  "evaluator_n": 100,
  "real_holdout_n": 50,
  "transfer_weights": 30,
  "trust_radii": [0.25, 0.5, 1, 2],
  "transfer_target_probability_floor": 0.5,
  "transfer_min_probability_gain": 0.01,
  "claim_scope": "exploratory_internal_validation_after_stages_one_and_two_no_new_external_or_tasting_data",
  "execution_differences_from_design": [
    "Clipping and missingness ablations use fixed ExtraTrees, not all nested fusion variants; reported separately.",
    "Chemical literature evidence is graded and incomplete; no fabricated absolute-score anchors or physical validation."
  ]
}
```

config__stage3__design.json

```
{
  "stage": "stage3_design",
  "date": "2026-10-25",
  "seed": 20261025,
  "status": "protocol_and_preflight_only_not_completed_stage3_experiments",
  "validation": {
    "outer_folds": 3,
    "outer_repeats": 5,
    "inner_folds": 3,
    "stratification": "beer_style",
    "unit": "unique_beer",
    "primary_metric": "macro_MAE_on_explicitly_mapped_Aroma_Flavour_30_targets",
    "secondary_scope": "all_50_targets_and_card_sections_reported_separately",
    "paired_bootstrap_repeats": 2000,
    "bootstrap_scope": "conditional_on_saved_OOF_predictions_not_full_algorithm_uncertainty",
    "practical_relative_MAE_margin": 0.01,
    "new_external_data": "not_available_existing_data_are_exploratory"
  },
  "grading": {
    "primary_without_anchors": "training_only_ECDF_relative_rank_bins_0_to_6",
    "conformal_alpha": 0.1,
    "absolute_grade_accuracy": "not_estimable_without_observed_absolute_anchors",
    "warning_if_grade_set_size_at_least": 4,
    "anchor_analysis": "conditional_on_new_observed_same_protocol_pairs_only"
  },
  "models": {
    "baselines": ["mean", "ridge", "extra_trees", "gbr_fixed"],
    "proposed": "inner_OOF_per_target_GBR_ET_blend_shrunk_to_global_weight",
    "blend_shrinkage_grid": [0, 0.01, 0.1, 1, 10],
    "clip_quantiles": [0, 0.01, 0.025],
    "missing_rates": [0, 0.05, 0.1, 0.2],
    "mask_scenarios": ["MCAR", "preferentially_mask_low_training_quantile_values"]
  },
  "graph": {
    "thresholds": [0.6, 0.7, 0.8],
    "ridge_alphas": [1, 10, 100, 1000],
    "graph_lambdas": [0, 1, 5, 20],
    "null_graph": "permute_node_labels_preserving_unlabelled_weighted_signed_graph",
    "bootstrap_repeats": 200,
    "permutation_repeats": 20,
    "inference": "descriptive_training_selection_with_heldout_effects_no_causal_or_p_value_claim"
  },
  "generation": {
    "repeat_seeds": [20261025, 20261026, 20261027, 20261028, 20261029],
    "generator_search_training_n": 100,
    "external_evaluator_training_n": 100,
    "real_holdout_n": 50,
    "samples_per_style": 40,
    "styles": ["Tripel", "Kriek", "Lambic"],
    "primary_fields": "all_nonaggregate_fields_226",
    "unknown_aggregate_policy": "exclude_from_primary_models_until_formula_verified",
    "methods": ["bootstrap", "local_convex", "conditional_PCA_Gaussian", "constrained_local_convex"],
    "MMD_bandwidth": "fit_on_generator_training_data_once_per_split_shared_by_all_methods"
  },
  "transfer": {
    "source": "Pils/Lager",
    "target": "Tripel",
    "chemical_fields": ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"],
    "trust_region_radii": [0.25, 0.5, 1, 2],
    "objective": "Pareto_minimize_standardized_change_target_distance_and_defect_excess",
    "threshold_sensitivity_multipliers": [0.5, 1, 2],
    "missing_defect_source": "unknown_not_certified_against_median",
    "stale_hops": "multiple_model_proxy_consensus_not_real_tasting_certification",
    "failure": "preserve_source_and_record_no_feasible_nontrivial_candidate"
  }
}
```

config__stage4.json

```
{
  "date": "2026-10-25",
  "seed": 20261025,
  "outer_folds": 3,
  "outer_repeats": 5,
  "extra_trees": 200,
  "gbr_estimators": 100,
  "ridge_alphas": [1, 10, 100, 1000],
  "conformal_repeats": 5,
  "conformal_alpha": 0.1,
  "panel_bootstrap_repeats": 1000,
  "mapping_tolerance": 0.0000001,
  "external_record": "https://zenodo.org/records/23624915",
  "raw_md5": "a8f911d8c0061e096abf3b087678e164",
  "workbook_md5": "52645e38d90aafda272532e19e2fea1c",
  "primary_absolute_target": "equal_panelist_mean_after_within_beer_panelist_repeat_averaging",
  "absolute_models": ["mean", "ridge", "extra_trees", "gbr", "fixed_half_blend", "concentration_ET", "standardized_ET_train_isotonic"],
  "ordinal_target": "equal_panelist_equal_repeat_observed_rating_distribution_7_categories",
  "ordinal_decision_rules": ["mode_for_exact_match", "median_for_absolute_grade_loss", "rounded_mean_sensitivity"],
  "concentration_mixture_LP_extension": "exploratory_after_failed_single_anchor_pilot_not_pre_registered",
  "concentration_units": ["mg/L", "g/L", "ug/L", "v/v"],
  "literature_nonenal_threshold_mg_L": 0.0001,
  "threshold_sensitivity_multipliers": [0.5, 1, 2],
  "mixture_weights": [0.025, 0.05, 0.075, 0.1, 0.125, 0.15, 0.175, 0.2, 0.225, 0.25, 0.275, 0.3, 0.325, 0.35, 0.375, 0.4, 0.425, 0.45, 0.475, 0.5, 0.525, 0.55, 0.575, 0.6, 0.625, 0.65, 0.675, 0.7, 0.725, 0.75],
  "claim_scope": "Internal prediction of observed panel means; reconstructed identity mapping is audited separately; no new physical beer or tasting experiment"
}
```

scripts/review__experiments.py

```
"""Exploratory follow-up: fixed folds, ordinal objective, honest baselines, LP proof."""
from pathlib import Path
import argparse
import json
import os
import sys
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
import numpy as np
import pandas as pd
from sklearn.model_selection import RepeatedStratifiedKFold
from src.data import find_data, load_data, CHEMICAL_DEFECTS
from src.common import save_csv, write_json, sha256, environment
from src.stage3_prediction import tree_model
from src.stage4_models import ordinal_summary
from src.review_diagnostics import distribution_scores, cdf_to_probabilities, conditional_paired_interval, mixture_certificate


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-dir')
    args = parser.parse_args()
    out = ROOT / 'outputs/review_20261028'; out.mkdir(parents=True, exist_ok=True)
    old = ROOT / 'outputs/stage4'
```

```
    cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8'))
    data = load_data(find_data(args.data_dir))
    beers = data.meta.beer.to_numpy(); styles = data.meta.tasting_category_fine.to_numpy()
    targets = list(data.Y); X = data.X.to_numpy()
    Y = pd.read_csv(old / 'audit/observed_panel_means_0_6.csv').set_index('beer').loc[beers, targets].to_numpy()
    index = pd.MultiIndex.from_product([beers, targets], names=['beer', 'target'])
    table = pd.read_csv(old / 'ordinal/oof_rating_distributions.csv').set_index(['beer', 'target']).loc[index]
    q = table[[f'observed_panel_p{k}' for k in range(7)]].to_numpy().reshape(250, 50, 7)
    q /= q.sum(-1, keepdims=True)
    probability = {'frequency_ET': table[[f'predicted_p{k}' for k in range(7)]].to_numpy().reshape(250, 50, 7)}
    probability['frequency_ET'] /= probability['frequency_ET'].sum(-1, keepdims=True)
    sums = {m: np.zeros_like(q) for m in ['CDF_ET', 'population', 'style_only']}
    points = {m: np.zeros_like(Y) for m in ['training_median', 'style_median']}
    counts = np.zeros(len(beers)); fold_rows = []; role_rows = []
    scopes = {'core30': np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))]), 'all50': np.arange(50)}
    previous_roles = pd.read_csv(old / 'prediction/outer_roles.csv')
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=3, n_repeats=5, random_state=cfg['seed']).split(X, styles)):
        assert set(beers[test]) == set(previous_roles.loc[previous_roles.fold == fold, 'beer'])
        print(f'CDF and baseline comparison {fold + 1}/15', flush=True)
        model = tree_model(cfg).fit(X[train], q[train].cumsum(-1)[..., :-1].reshape(len(train), -1))
        cdf = model.predict(X[test]).reshape(len(test), 50, 6)
        predicted = {'CDF_ET': cdf_to_probabilities(cdf), 'population': np.broadcast_to(q[train].mean(0), (len(test), 50, 7)).copy(), 'style_only': np.empty((len(test), 50, 7))}
        median = np.broadcast_to(np.median(Y[train], axis=0), (len(test), 50)).copy()
        style_median = np.empty_like(median)
        for k, i in enumerate(test):
            group = train[styles[train] == styles[i]]
            if not len(group): group = train
            predicted['style_only'][k] = q[group].mean(0)
            style_median[k] = np.median(Y[group], axis=0)
        points['training_median'][test] += median; points['style_median'][test] += style_median
        for name, p in predicted.items():
            sums[name][test] += p
            for scope, selected in scopes.items():
                metrics = distribution_scores(q[test][:, selected], p[:, selected])
                fold_rows.append({'fold': fold, 'model': name, 'scope': scope, 'n_test': len(test), **{k: float(v.mean()) for k, v in metrics.items()}})
        counts[test] += 1
        role_rows += [{'fold': fold, 'beer': int(beers[i]), 'role': 'test'} for i in test]
    assert np.all(counts == 5)
    for name, p in sums.items(): probability[name] = p / counts[:, None, None]
    summary = []; by_target = []; paired = []; calibration = []
    for name, p in probability.items():
        metric = distribution_scores(q, p)
        grades = p.argmax(-1)
        for scope, selected in scopes.items():
            row = ordinal_summary(q[:, selected], p[:, selected], grades[:, selected])
            # The historical helper names squared distribution distances RPS/Brier.
            row.pop('RPS'); row.pop('Brier')
            summary.append({'model': name, 'scope': scope, **row, **{k: float(v[:, selected].mean()) for k, v in metric.items()}})
        for j, target in enumerate(targets):
            by_target.append({'model': name, 'target': target, **ordinal_summary(q[:, j:j+1], None, grades[:, j:j+1]), **{k: float(v[:, j].mean()) for k, v in metric.items()}})
        for scope, selected in scopes.items():
            x = p[:, selected, 0].ravel(); y = q[:, selected, 0].ravel()
            for b in range(10):
                mask = np.minimum((x * 10).astype(int), 9) == b
                if mask.any(): calibration.append({'model': name, 'scope': scope, 'zero_probability_bin': b, 'beer_attribute_pairs': int(mask.sum()), 'predicted_p0': float(x[mask].mean()), 'observed_p0': float(y[mask].mean())})
    for a, b in [('CDF_ET', 'frequency_ET'), ('frequency_ET', 'population'), ('CDF_ET', 'style_only')]:
        for scope, selected in scopes.items():
            d = (distribution_scores(q, probability[a])['cdf_distance'][:, selected] - distribution_scores(q, probability[b])['cdf_distance'][:, selected]).mean(1)
            paired.append({'endpoint': 'cdf_distance', 'scope': scope, 'A': a, 'B': b, **conditional_paired_interval(d, styles)})
    point_preds = {}
    for name in ['mean', 'ridge', 'extra_trees', 'gbr', 'fixed_half_blend', 'concentration_ET']:
        point_preds[name] = pd.read_csv(old / f'prediction/oof_{name}_0_6.csv').set_index('beer').loc[beers, targets].to_numpy()
    for name, v in points.items(): point_preds[name] = v / counts[:, None]
    point_rows = []; style_rows = []; graph_rows = []
    for name, p in point_preds.items():
        for scope, selected in scopes.items():
            error = np.abs(Y[:, selected] - p[:, selected]).mean(1)
            point_rows.append({'model': name, 'scope': scope, 'mae': float(error.mean())})
            for label in np.unique(styles):
                mask = styles == label
                style_rows.append({'model': name, 'scope': scope, 'style': label, 'unique_beers': int(mask.sum()), 'mae': float(error[mask].mean())})
    for name in ['mean', 'training_median', 'extra_trees', 'gbr', 'concentration_ET', 'style_median']:
        for scope, selected in scopes.items():
            d = (np.abs(Y - point_preds['fixed_half_blend']) [:, selected] - np.abs(Y - point_preds[name])[:, selected]).mean(1)
            paired.append({'endpoint': 'panel_mean_MAE', 'scope': scope, 'A': 'fixed_half_blend', 'B': name, **conditional_paired_interval(d, styles)})
    graph_preds = {kind: pd.read_csv(old / f'raw_graph/q2/oof_graph_{kind}.csv').set_index('beer').loc[beers, targets].to_numpy() for kind in ['real', 'none']}
    for scope, selected in scopes.items():
        for kind, p in graph_preds.items(): graph_rows.append({'graph': kind, 'scope': scope, 'aggregated_oof_mae': float(np.abs(Y[:, selected] - p[:, selected]).mean())})
        d = (np.abs(Y - graph_preds['real']) [:, selected] - np.abs(Y - graph_preds['none']) [:, selected]).mean(1)
        paired.append({'endpoint': 'graph_panel_mean_MAE', 'scope': scope, 'A': 'real', 'B': 'none', **conditional_paired_interval(d, styles)})
    roles = pd.read_csv(ROOT / 'outputs/stage3/q3/disjoint_roles.csv')
    concentration = data.calibrated.set_index('beer').loc[beers, CHEMICAL_DEFECTS].to_numpy(float)
    certificates = []; ablations = []
    from scipy.optimize import linprog
    for repeat in range(5):
        role = roles[roles.repeat == repeat].set_index('beer').loc[beers, 'role'].to_numpy()
        anchors = np.flatnonzero((role == 'G_search_training') & (styles == 'Tripel') & np.isfinite(concentration).all(1))
        for i in np.flatnonzero(styles == 'Pils/Lager'):
            d = (concentration[anchors] - concentration[i]).T
            cert = mixture_certificate(d)
            certificates.append({'repeat': repeat, 'beer': int(beers[i]), 'n_anchors': len(anchors), 'certified_zero_only': cert['certified_zero_only'], 'normalized_margin': cert['margin'], 'single_row_blockers': ';'.join(CHEMICAL_DEFECTS[k] for k in cert['single_row_blockers']), **{f'weight_{f}': cert['weights'][k] for k, f in enumerate(CHEMICAL_DEFECTS)}})
            for dropped in [None] + CHEMICAL_DEFECTS:
                keep = [k for k, f in enumerate(CHEMICAL_DEFECTS) if f != dropped]
                a = d[keep]; scales = np.maximum(np.abs(a).max(1), 1e-12)
                solution = linprog(-np.ones(len(anchors)), A_ub=np.vstack([a / scales[:, None], np.ones(len(anchors))]), b_ub=np.r_[np.zeros(len(keep)), .75], bounds=(0, None), method='highs')
                if not solution.success: raise RuntimeError(solution.message)
                ablations.append({'repeat': repeat, 'beer': int(beers[i]), 'dropped_constraint': dropped or 'none', 'max_target_fraction': float(solution.x.sum())})
    for name, frame in [('ordinal_metrics', summary), ('ordinal_by_target', by_target), ('ordinal_fold_metrics', fold_rows), ('zero_calibration', calibration), ('paired_comparisons', paired), ('point_metrics', point_rows), ('point_by_style', style_rows), ('graph_aggregation', graph_rows), ('mixture_certificates', certificates), ('mixture_constraint_ablation', ablations), ('outer_roles', role_rows)]:
        save_csv(pd.DataFrame(frame), out / f'{name}.csv')
    rows = table.reset_index()[['beer', 'target']].copy()
    for k in range(7): rows[f'predicted_p{k}'] = probability['CDF_ET'][..., k].ravel()
    rows['modal_grade'] = probability['CDF_ET'].argmax(-1).ravel()
    rows['median_grade'] = (probability['CDF_ET'].cumsum(-1) >= .5).argmax(-1).ravel()
    save_csv(rows, out / 'cdf_et_oof.csv')
    for name in ['training_median', 'style_median']:
        save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(point_preds[name], columns=targets)], axis=1), out / f'oof_{name}.csv')
    inputs = [old / 'ordinal/oof_rating_distributions.csv', old / 'audit/observed_panel_means_0_6.csv', old / 'prediction/outer_roles.csv', ROOT / 'outputs/stage3/q3/disjoint_roles.csv', ROOT / 'config_stage4.json']
    inputs += list((old / 'prediction').glob('oof_*_0_6.csv')) + list((old / 'raw_graph/q2').glob('oof_graph_*.csv'))
    code = [Path(__file__), ROOT / 'src/review_diagnostics.py', ROOT / 'src/stage3_prediction.py', ROOT / 'src/common.py', ROOT / 'src/stage4_models.py']
    write_json(out / 'run_manifest.json', {**environment(), 'design': 'exploratory_after_stage4_review_fixed_hyperparameters_no_test_tuning', 'config': cfg, 'resamples': 5000, 'unique_beers': 250, 'new_external_samples': 0, 'conditional_intervals': 'fixed_OOF_predictions_stratified_by_style_no_refit_or_selection_uncertainty', 'input_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in inputs}, 'code_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in code}, 'official_inputs_sha256': {p.name: sha256(
```

```
    p) for p in data.root.iterdir() if p.is_file() and p.suffix in ['.csv', '.xlsx']}, 'result_sha256': {p.name: sha256(p) for p in sorted(out.glob('*.csv'))}}))
    print(pd.DataFrame(summary)[['model', 'scope', 'cdf_distance', 'expected_individual_RPS', 'nonzero_panel_mode_accuracy']].to_string(index=False))


if __name__ == '__main__':
    main()
```

src/review__diagnostics.py

```
"""Scoring identities and certificates used in the 2026-10-28 revision."""
import numpy as np
from scipy.optimize import linprog


def distribution_scores(observed, predicted):
    """Return per-beer/per-attribute distances and expected proper scores."""
    q, p = np.asarray(observed), np.asarray(predicted)
    if q.shape != p.shape or q.shape[-1] != 7:
        raise ValueError('Expected matching seven-category distributions')
    for a in (q, p):
        if not np.isfinite(a).all() or a.min() < -1e-8 or not np.allclose(a.sum(-1), 1, atol=1e-7):
            raise ValueError('Invalid probability distribution')
    Q, P = q.cumsum(-1)[..., :-1], p.cumsum(-1)[..., :-1]
    distance = ((P - Q) ** 2).mean(-1)
    uncertainty = (Q * (1 - Q)).mean(-1)
    brier_distance = ((p - q) ** 2).sum(-1)
    return {'cdf_distance': distance, 'panel_disagreement': uncertainty,
            'expected_individual_RPS': distance + uncertainty,
            'brier_distance': brier_distance,
            'expected_individual_Brier': brier_distance + 1 - (q ** 2).sum(-1)}


def cdf_to_probabilities(cdf):
    a = np.asarray(cdf)
    if a.shape[-1] != 6 or not np.isfinite(a).all() or (np.diff(a, axis=-1) < -1e-10).any():
        raise ValueError('Six finite, monotone cumulative probabilities required')
    if a.min() < -1e-10 or a.max() > 1 + 1e-10:
        raise ValueError('CDF outside [0,1]')
    p = np.diff(np.concatenate([np.zeros_like(a[..., :1]), a, np.ones_like(a[..., :1])], axis=-1), axis=-1)
    p = np.maximum(p, 0)
    return p / p.sum(-1, keepdims=True)


def conditional_paired_interval(difference, strata, seed=20261028, resamples=5000):
    """Stratified beer bootstrap conditional on fixed OOF predictions."""
    d, strata = np.asarray(difference), np.asarray(strata)
    if d.ndim != 1 or len(d) != len(strata) or not np.isfinite(d).all():
        raise ValueError('One finite paired difference per beer required')
    rng = np.random.default_rng(seed)
    values = np.zeros(resamples)
    for label in np.unique(strata):
        v = d[strata == label]
        values += v[rng.integers(len(v), size=(resamples, len(v)))].sum(1)
    lo, hi = np.quantile(values / len(d), [.025, .975])
    return {'difference_A_minus_B': float(d.mean()), 'conditional_low95': float(lo),
            'conditional_high95': float(hi), 'unique_beers': len(d), 'resamples': resamples}


def mixture_certificate(difference):
    """D w <= 0, w>=0: exhibit v>=0 with D.T v >0 to exclude w!=0.

    Row normalization preserves inequalities and avoids mixing concentration units.
    Maximize the minimum normalized anchor excess over v on the simplex.
    """
    d = np.asarray(difference, dtype=float)
    if d.ndim != 2 or not d.size or not np.isfinite(d).all():
        raise ValueError('Finite defect-by-anchor differences required')
    scales = np.max(np.abs(d), axis=1)
    a = d / np.where(scales > 0, scales, 1)[:, None]
    m, n = a.shape
    result = linprog(np.r_[np.zeros(m), -1.],
                     A_ub=np.column_stack([-a.T, np.ones(n)]), b_ub=np.zeros(n),
                     A_eq=[np.r_[np.ones(m), 0]], b_eq=[1],
                     bounds=[(0, None)] * m + [(None, None)], method='highs')
    if not result.success:
        raise RuntimeError(result.message)
    v = result.x[:-1]
    margin = float((a.T @ v).min())
    return {'certified_zero_only': margin > 1e-8, 'margin': margin,
            'weights': v.tolist(), 'row_scales': scales.tolist(),
            'single_row_blockers': np.flatnonzero(d.min(1) > 1e-10).tolist()}
```

scripts/build__latex__revision.py

```
"""Convert the reviewed manuscript to an editable LaTeX project.

The Markdown is a text source; after generation, main.tex is independently editable.
"""
from pathlib import Path
import json
import re
import shutil

ROOT = Path(__file__).resolve().parents[1]
PROJECT = ROOT / 'paper/latex_20261028'
body = (ROOT / 'paper/revised_20261028/正文.md').read_text(encoding='utf-8')
old_md = (ROOT / 'paper/revised_absolute_20261025/论文完善稿.md').read_text(encoding='utf-8')
old_json = json.loads((ROOT / 'paper/revised_absolute_20261025/paper_content.json').read_text(encoding='utf-8'))

# Reference numbers follow their first appearance in the revised text.
mapping = {1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 16: 8, 10: 9, 11: 10, 12: 11, 15: 12, 13: 13, 14: 14}
main_text, refs_text = body.split('## 参考文献', 1)
main_text = re.sub(r'\[(\d+)-(\d+)\]', lambda m: ''.join(f'[{i}]' for i in range(int(m.group(1)), int(m.group(2)) + 1)), main_text)
main_text = re.sub(r'\[(\d+)\]', lambda m: f'[{mapping[int(m.group(1))]}]', main_text)
references = {}
for line in refs_text.splitlines():
    m = re.match(r'\[(\d+)\]\s*(.*)', line)
    if m and int(m.group(1)) in mapping:
        references[mapping[int(m.group(1))]] = m.group(2)

def escape_text(text):
    conversion = {'\\': r'\textbackslash{}', '&': r'\&', '%': r'\%', '$': r'\$', '#': r'\#', '_': r'\_', '{': r'\{', '}': r'\}', '~': r'\textasciitilde{}', '^': r'\textasciicircum{}'}
    return ''.join(conversion.get(ch, ch) for ch in text)

def inline(text):
    # Preserve math expressions, escape text, and remove Markdown emphasis/code marks.
    chunks = re.split(r'(\$[^$]+\$)', text)
    for i, chunk in enumerate(chunks):
        if not (chunk.startswith('$') and chunk.endswith('$')):
            chunk = chunk.replace('**', '').replace('`', '')
            # Preserve superscripts and Greek symbols in fonts with limited Unicode coverage.
            chunk = re.sub(r'10⁻¹⁶', r'$10^{-16}$', chunk)
            chunk = re.sub(r'10⁻⁸', r'$10^{-8}$', chunk)
            chunk = chunk.replace('μ', r'$\mu$').replace('λ', r'$\lambda$')
            parts = re.split(r'(\$[^$]+\$)', chunk)
            chunks[i] = ''.join(p if p.startswith('$') and p.endswith('$') else escape_text(p) for p in parts)
    return ''.join(chunks)

title = main_text.splitlines()[0][2:]
abstract = main_text.split('## 摘要\n', 1)[1].split('**关键词：**', 1)[0].strip()
keywords = main_text.split('**关键词：**', 1)[1].split('\n', 1)[0].strip()
section_text = main_text.split('## 1 问题分析', 1)[1]
section_text = '## 1 问题分析' + section_text

header = r'''\documentclass[UTF8,a4paper,twocolumn,10pt]{ctexart}
\usepackage{amsmath,amssymb,graphicx,longtable,booktabs,array,geometry,hyperref,listings,xcolor,multicol}
\geometry{left=2cm,right=2cm,top=2cm,bottom=2cm}
\setlength{\columnsep}{0.7cm}
\setlength{\columnseprule}{0pt}
\setlength{\tabcolsep}{2pt}
\IfFontExistsTF{LXGW WenKai GB Lite}{\setmonofont{LXGW WenKai GB Lite}}{\setmonofont{Arial Unicode MS}}
\pagestyle{plain}
\setlength{\parindent}{2em}
\setlength{\parskip}{0.3em}
\setlength{\emergencystretch}{3em}
\hypersetup{hidelinks}
\lstset{basicstyle=\ttfamily\tiny,breaklines=true,breakatwhitespace=false,breakautoindent=false,breakindent=0pt,columns=fixed,keepspaces=true,numbers=left,numberstyle=\tiny,frame=single,literate={_}{{\char95\allowbreak}}1}
\begin{document}
\twocolumn[{\begin{center}{\LARGE\bfseries TITLE}\end{center}\vspace{0.5em}
```

```
\begin{center}\textbf{摘要}\end{center}\begin{multicols}{2}
ABSTRACT

\noindent\textbf{关键词：} KEYWORDS
\end{multicols}\vspace{0.5em}}]
\thispagestyle{empty}
\clearpage
\tableofcontents
\thispagestyle{empty}
\clearpage
\setcounter{page}{1}
'''.replace('TITLE', inline(title)).replace('ABSTRACT', inline(abstract)).replace('KEYWORDS', inline(keywords))

def render_table(lines):
    rows = [[cell.strip() for cell in line.strip().strip('|').split('|')] for line in lines]
    rows = [r for r in rows if not all(re.fullmatch(r'[:\- ]+', cell or '') for cell in r)]
    n = len(rows[0]); width = 0.85 / n
    # Small chunks permit tables to break naturally between the two columns.
    blocks = []
    for first in range(1, len(rows), 4):
        out = [r'\par\noindent{\scriptsize', '\\begin{tabular}{' + ''.join(f'>{{\\raggedright\\arraybackslash}}p{{{width:.3f}\\columnwidth}}' for _ in range(n)) + '}', r'\toprule']
        out.append(' & '.join(inline(c) for c in rows[0]) + r' \\')
        out.append(r'\midrule')
        for row in rows[first:first+4]:
            out.append(' & '.join(inline(c).replace(r'\_', r'\_\allowbreak{}').replace('.', r'.\allowbreak{}') for c in row) + r' \\')
        blocks.append('\n'.join(out + [r'\bottomrule', r'\end{tabular}}\par\smallskip']))
    return '\n'.join(blocks)

def render(markdown):
    lines = markdown.splitlines(); result = []; i = 0
    while i < len(lines):
        line = lines[i].strip()
        if not line:
            result.append(''); i += 1; continue
        if line.startswith('$$'):
            formula = line[2:]
            while '$$' not in formula:
                i += 1; formula += '\n' + lines[i]
            formula = formula.split('$$', 1)[0]
            formula = re.sub(r'\\tag\{\d+\}', '', formula)
            if r'\hat g_{\mathrm{mode}}' in formula:
                formula = r'\begin{aligned}' + formula.replace(r',\qquad', r',\\') + r'\end{aligned}'
            if r'\hat B=\arg\min_B' in formula:
                formula = formula.replace(r'\left\{', r'\{').replace(r'\right\}', r'\}')
            result.append('\\begin{equation}\n' + formula + '\n\\end{equation}')
        elif line.startswith('|'):
            table = []
            while i < len(lines) and lines[i].strip().startswith('|'):
                table.append(lines[i]); i += 1
            result.append(render_table(table)); continue
        elif line.startswith('### '):
            result.append('\\subsection{' + inline(re.sub(r'^\d+(?:\.\d+)*\s*', '', line[4:])) + '}')
        elif line.startswith('## '):
            result.append('\\section{' + inline(re.sub(r'^\d+(?:\.\d+)*\s*', '', line[3:])) + '}')
        else:
            result.append(inline(line) + '\n')
        i += 1
    return '\n'.join(result)

appendix = old_md.split('## 附录A 全部50项Top-3、结果索引与运行命令', 1)[1].split('## 附录B', 1)[0]
refs = '\n'.join(r'\noindent [' + str(k) + '] ' + inline(references[k]) + r'\par' for k in sorted(references))
sources = next(b['paths'] for b in old_json['blocks'] if b['type'] == 'source_files')
for path in ('scripts/review_experiments.py', 'src/review_diagnostics.py', 'scripts/build_latex_revision.py'):
    if path not in sources: sources.append(path)
source_dir = PROJECT / 'sources'; source_dir.mkdir(exist_ok=True)
listings = []
for number, relative in enumerate(sources, 1):
    src = ROOT / relative
    if not src.is_file(): raise FileNotFoundError(src)
    dst = source_dir / f'{number:03d}_{src.name}'
    shutil.copyfile(src, dst)
    listings.append('\\subsection*{' + inline(relative) + '}\n\\lstinputlisting{' + dst.relative_to(PROJECT).as_posix() + '}')

content = (header + render(section_text) + '\n\\section*{参考文献}\n\\addcontentsline{toc}{section}{参考文献}\n' + refs
           + '\n\\appendix\n\\section{全部50项Top-3与结果索引}\n' + render(appendix)
           + '\n\\section{完整源程序}\n' + '\n'.join(listings) + '\n\\end{document}\n')
figures = PROJECT / 'figures'; figures.mkdir(exist_ok=True)
for filename in ('ordinal_review.png', 'mixture_certificate.png'):
    shutil.copyfile(ROOT / 'outputs/review_20261028/figures' / filename, figures / filename)
content = content.replace('\\subsection{面板均分预测}', r'''\begin{figure*}[t]
\centering\includegraphics[width=0.95\textwidth]{figures/ordinal_review.png}
\caption{等级分布预测的累计距离与非零众数识别比较。}
\end{figure*}
\subsection{面板均分预测}''')
content = content.replace('\\section{验证范围与应用建议}', r'''\begin{figure*}[t]
\centering\includegraphics[width=0.95\textwidth]{figures/mixture_certificate.png}
\caption{严格混配约束的不可行证书及单项约束消融。}
\end{figure*}
\section{验证范围与应用建议}''')
(PROJECT / 'main.tex').write_text(content, encoding='utf-8')
print(f'Wrote {PROJECT / "main.tex"}; included {len(sources)} source files')
```